FIELD GUIDE 001

The HTML Playbook.

Write pages you can explain.

A source-linked curriculum, reference and project collection for practical HTML authoring.

By Odejobi Abiola Samuel

OCTOBER 2026 READING EDITION / 14 OCTOBER 2026

74 chapters · 74 explained exercises · 7 static projects

Published reading curriculum and captured reference. It is not a completed whole-Standard behavioral reference. Independent technical, answer-key, accessibility and security reviews remain open. The PDF has text, links and bookmarks; it is not claimed to be a tagged accessible PDF.

Publication details

The HTML Playbook

Author: Odejobi Abiola Samuel

Publication brand: CTROTECH

Edition: October 2026 reading edition / 14 October 2026

© 2026 Odejobi Abiola Samuel. Third-party material retains its original rights and license terms.

No new open license or blanket reuse permission is granted by this edition.

Source and third-party credits

Reference summaries are derived from the captured WHATWG HTML indexes and definition pages. MDN, W3C/WAI, OWASP and other cited authorities retain their own terms. Copyright © WHATWG (Apple, Google, Mozilla, Microsoft). Extracted source summaries and declarations are arranged and linked for this book; no endorsement is implied. The WHATWG source license notice is attached to this PDF. Instrument Serif is supplied under the SIL Open Font License. The PDF also embeds DejaVu fonts. Both font notices are attached to this PDF and included with the companion files.

Preparation and review

AI-assisted editorial preparation. Author credit does not imply an exclusively human-written manuscript.

The author has approved the visual direction and confirmed the public site works. Publication does not close the remaining behavioral/value inventories, independent technical and answer-key reviews, accessibility/security checks or accessible-PDF workflow.

How to use this book

Read the chapters in order if you are starting with HTML. Use the reference when you need a definition route. Attempt each exercise before reading the explanation. Inspect the seven complete project folders to compare document structure and implementation limits.

The website and this PDF use the same chapter data. Use the companion source files for exact, unwrapped code. Syntax colours distinguish tags, attributes, strings, keywords and comments. Colours do not change the source or prove its correctness. Long lines wrap to fit the PDF; download the exact source when copying a complete example. External-media filenames in examples are illustrative unless the project includes the actual local asset. The site sandbox restricts resource loading and form submissions. It runs inline scripts; never paste secrets or run untrusted code.

A feature in a standard is not necessarily available in every browser. Index summaries are not full normative definitions. A checker pass does not establish accessibility or security. The review-status page and source package record actual check scope and remaining work.

Editorial note

Prepared with AI assistance for CTROTECH. Author credit and preparation details are recorded on the publication-details page. The source package records executed checks and open reviews separately; neither visual polish nor a successful build closes technical review.

Start here: a reading guide

Read a chapter, predict what its markup means, then test your explanation. You do not need to memorise an element list before you can build a useful page.

Choose your route

I am starting with HTML

Begin with Chapters 1 to 6 for the document and browser model. Continue through Chapters 7 to 27 for syntax, text and structure. Then attempt the profile project before looking at its reference files.

01. What HTML does · 07. The document shell · 14. Headings and paragraphs · 22. Containers and grouping · 68. Build a personal profile

I can build a page, but want stronger decisions

Work through images and media in Chapters 28 to 33, forms in 34 to 41 and interactions in 42 to 47. Move to browser behavior in 54 to 60 and the professional checks in 61 to 67. Use the earlier chapters whenever a term or content rule needs another look.

28. Images with a purpose · 34. How forms send data · 42. Buttons and commands · 54. How parsing starts · 61. Accessibility starts in the markup · 74. Capstone: the library site

I need a specific definition

Use search or the reference desk. Element and attribute summaries lead to their source definitions. The API register gives captured declarations and member routes, not a complete behavioral manual. Check the normative definition and current implementation evidence before relying on an edge case.

One useful practice loop

Predict before running

Read the objective and example. Describe the expected document relationships, submitted entries or interaction in your own words. Separate the markup rule from a styling choice.

Inspect the result

Use a complete document where a fragment needs one. Compare the source with the live DOM. Use the keyboard for interactive examples and check the console or network panel when the lesson concerns requests or loading.

Answer before opening the solution

Write a short answer and a reason. Then compare the explained answer with your reasoning, not just its vocabulary. A mismatch is a prompt to revisit the example and source, not a score.

Change one requirement

Try a longer label, a missing resource, a narrow viewport or a different control state. State what should remain true. Use fictional values, especially in forms. The book sandbox restricts resource loading and form submissions. It runs inline scripts; do not paste secrets or run code you do not trust.

Website, PDF and code

Website and PDF share the maintained chapter source. Use the exact code download for copying: some files are fragments and some media or server resources are illustrative. The seven project downloads are complete static teaching folders, not live services. Reading progress stays in this browser when storage is available; it is not an assessment or certificate.

Know what a source tells you

WHATWG defines HTML. MDN helps explain features and implementation support. WAI and WCAG provide accessibility guidance and requirements. A conformance checker, browser test and accessibility review answer different questions. Follow the source links and read the scope of each recorded check.

Scope of this edition

This is a published reading curriculum with captured reference inventories. It is not a completed whole-Standard behavioral reference. Independent technical, answer-key, accessibility and security reviews remain open. The PDF has selectable text, links and bookmarks, but is not a tagged accessible PDF.

Help correct the book

Report a correction with the page URL or chapter number, the passage, the expected result and a supporting source. For browser-dependent behavior, include the browser version and a small reproducible example. GitHub requires an account. Never include secrets or personal form data in a public issue.

Report a content correction on GitHub

October 2026 edition notes

First published reading edition, dated 14 October 2026. Release-candidate and predeployment wording have been replaced with reader-facing publication details.

Chapter 60 now distinguishes the 166 foundation interface discovery routes from the 314 WebIDL declaration owners and 1,574 direct member declarations. Declaration coverage is not described as completed behavioral review.

Expanded practical explanations of responsive-image descriptors, form entries and submission paths, dialog interactions, script loading, storage failure and discovery controls. Added an explicit fictional-data warning to the registration brief.

Added this reading guide, a correction route and consistent website/PDF publication notes. Existing independent-review states remain open.

Contents

Publication details 2

Start here: a reading guide 4

01. What HTML does 10

02. Reading a Living Standard 11

03. From source to page 13

04. Elements, tags and attributes 14

05. Think in trees 16

06. Make your first document 17

07. The document shell 19

08. html, head and body 20

09. Text, encoding and escaping 22

10. Nesting and void elements 23

11. Attributes and their values 25

12. Content categories and models 26

13. Conformance and useful errors 28

14. Headings and paragraphs 29

15. Inline meaning 31

16. Code and technical text 33

17. Links that go somewhere 35

18. Lists with a reason 37

19. Figures and captions 39

20. Dates, values and edits 41

21. Language and direction 43

22. Containers and grouping 45

23. Sections that have a subject 47

24. Articles and supporting content 48

25. Navigation and main content 50

26. Tables that explain the data 52

27. Plan a small site 54

28. Images with a purpose 56

29. Responsive image choices 58

30. Audio, video and tracks 60

31. Image maps 62

32. Embedding another document 64

33. Canvas, SVG and MathML 66

34. How forms send data 68

35. Labels and groups 70

36. Text, email and passwords 72

37. Numbers, dates and colour 74

38. Choices, files and hidden values 76

39. Menus and longer text 78

40. Validation people can recover from 80

41. Submission and server boundaries 82

42. Buttons and commands 84

43. Disclosure with details 86

44. Dialogs and popovers 88

45. Focus, editing and dragging 90

46. Templates and component boundaries 93

47. Custom data and structured data 96

48. Global attributes in context 98

49. Element-specific attributes 100

50. Metadata readers encounter 102

51. Resource relationships 104

52. Loading and policy attributes 106

53. Attributes and properties 108

54. How parsing starts 110

55. Tree construction and recovery 112

56. The document lifecycle 114

57. Scripts and execution 116

58. HTML and XML serialization 118

59. HTML and rendering 120

60. Where the wider APIs begin 122

61. Accessibility starts in the markup 126

62. Keyboard and focus review 129

63. Trust boundaries 132

64. Performance without folklore 134

65. Findable and shareable pages 136

66. Maintainable HTML 138

67. A release checklist that means something 140

68. Build a personal profile 142

69. Build a portfolio 146

70. Build a registration form 149

71. Build an article page 154

72. Build a data dashboard 158

73. Build a media gallery 162

74. Capstone: the library site 166

Element reference 171

Attributes and handlers 195

Global attributes and boundaries 235

Input states 239

Content categories 243

Obsolete features 246

API interface routes 270

API declaration register 287

Keyword, state and event routes 409

Glossary 451

Source and release notes 456

I / Understanding HTML

01. What HTML does

Give the browser a document it can understand.

By the end of this chapter

- Distinguish structure from presentation
- Choose a hyperlink for navigation

A browser needs more than words and coordinates. It needs to know which words are a heading, which image needs an alternative, and which control submits a form. HTML supplies that structure. CSS controls presentation. JavaScript can change the document and respond to events.

Those responsibilities overlap in useful ways. A details element can open without your JavaScript. A required input can stop an ordinary form submission. You still need a server to decide whether a submitted value is acceptable. Native browser behavior gives you a starting point, not an application backend.

Start with the content

Write the document before adding layout. In the example, the heading names a small lending library and the paragraph explains its purpose. The link already works with a keyboard and has a destination. If the stylesheet fails, the information remains available.

Your job is to describe what the page contains. Pick the element whose meaning fits the content, then style it. A very large paragraph does not become a heading because it looks like one.

The example

HTML / example 01

```
<main>
  <h1>Neighbourhood Library</h1>
  <p>Borrow books and find a quiet place to read.</p>
  <a href="catalogue.html">Browse the catalogue</a>
</main>
```

The main element identifies the page's dominant content. The h1 names it. The anchor is a real hyperlink because it has an href. These choices remain useful without a framework, a stylesheet or a pointer device.

A mistake to watch for

Using a clickable div for navigation adds work. It needs keyboard behavior, focus handling and semantics that a hyperlink already supplies.

Your turn

Which part of this example belongs to HTML, and which change would require CSS: renaming the heading, changing its colour, or changing the link destination?

Explained answer

The heading text and href are document content, so both are HTML changes. Colour is a presentation decision for CSS. JavaScript is unnecessary for any of these edits.

Read the source

HTML Living Standard: the language and its scope

I / Understanding HTML

02. Reading a Living Standard

Look up a rule without treating every browser as identical.

By the end of this chapter

- Distinguish authoring rules from implementation support
- Find a definition beyond the index

The HTML Living Standard changes over time. A reference to HTML5 often describes the modern language informally; it does not identify a frozen collection of features you can assume every browser supports.

The Standard contains authoring requirements, browser processing rules, explanations and indexes. The index is useful for finding a feature. Its summaries are not substitutes for the feature's actual definition. Read the definition when nesting, defaults or exceptions affect your example.

Separate three questions

Ask whether the markup conforms, whether your target browsers implement the behavior, and whether your use is accessible and appropriate. These questions need different evidence. A new element can be in the Standard while implementation remains uneven. An old presentational element can still render while being nonconforming for authors.

The example uses a normal document declaration. There is no version number to update when you use a newer conforming feature. Record the relevant source and review date in your project notes instead. Compatibility data is a dated aid, not a promise about a visitor's installed software.

How the current Standard came about

HTML changed venues and forms before it became a Living Standard. Early revisions developed at CERN and the IETF. Under W3C, HTML 3.2 and HTML4 were completed in 1997. XHTML 1.0 reformulated HTML4 in XML and was completed in 2000; that serialization was not simply a new set of HTML elements.

Apple, Mozilla and Opera formed WHATWG in 2004 to continue evolving HTML with backward compatibility and detailed browser behavior as central goals. W3C later participated in HTML5 work. In 2019, WHATWG and W3C agreed to collaborate on a single version of HTML going forward. The Standard's history section describes this evolution; it is historical explanation, not an authoring conformance rule.

For everyday authoring, this history explains why a short doctype does not select an annual release and why browser support must be checked feature by feature. It does not make old tutorials useless, but their date, serialization and assumptions matter.

The example

HTML / example 02

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Library notes</title>
</head>
<body>
  <h1>Library notes</h1>
</body>
</html>
```

The doctype is the short HTML doctype. It does not opt into a named annual release. The language, encoding and title solve separate document concerns.

A mistake to watch for

Using successful rendering as the only test of conformance. Browsers deliberately recover from many authoring errors.

Your turn

An obsolete element displays in your browser. Is that enough evidence to recommend it in new markup?

Explained answer

No. Rendering support can exist for old content. Check the authoring requirements and use a conforming replacement that preserves the intended meaning. Then check the replacement in the browsers you support.

Read the source

HTML Living Standard: conformance

HTML Living Standard: obsolete features

HTML Living Standard: history (non-normative)

I / Understanding HTML

03. From source to page

The text file and the browser document are related, but different.

By the end of this chapter

- Describe the path from bytes to nodes
- Distinguish response source from the live DOM

The server sends bytes with a media type. For a text/html response, the browser decodes those bytes and runs the HTML parser. Parsing creates nodes in a document tree. Styles and layout then determine how that tree is presented.

Scripts can change the tree after parsing. Developer Tools usually shows the current tree, not a faithful printout of the file you saved. View Source and the Elements panel answer different questions.

Follow one change

Our example gives a paragraph an id. A script can find the element and change its text. The original response still contains the old wording. A test that inspects the current DOM should expect the changed wording after that script has run.

The browser may also add implied elements and repair certain malformed structures during parsing. Do not take the repaired tree as permission to keep broken source. Find the authoring error, correct the source, and test again. Keep network failures separate from parsing failures: a valid document can still refer to an unavailable resource.

The example

HTML + JavaScript / example 03

```
<p id="opening">Opening hours are being checked.</p>
<script>
  document.getElementById("opening").textContent =
    "Open Tuesday to Saturday.";
</script>
```

The script runs after the paragraph has been parsed because it appears after that paragraph. textContent sets text, so the replacement string is not parsed as markup.

A mistake to watch for

Assuming a selector can find a later element before the parser has created it. Script placement and loading behavior affect timing.

Your turn

What will View Source show after the paragraph changes, and what will the current DOM show?

Explained answer

View Source shows the response containing the original sentence. The current DOM contains the updated text node. A later reload can produce the same change again because the script runs again.

Read the source

HTML Living Standard: documents

HTML Living Standard: parsing

DOM Standard: textContent

I / Understanding HTML

04. Elements, tags and attributes

Name the parts before debugging the page.

By the end of this chapter

- Identify tags, attributes and contents
- Keep confidential information out of source

An element is part of the document. Tags are syntax used to write many elements in HTML source. Most ordinary elements have a start tag, contents and an end tag. Some, such as img and input, are void elements and do not have end tags.

An attribute adds information to an element's start tag. In our link, href gives the destination and class provides a token for styling or scripting. Neither attribute changes the element into another kind of element.

Read the whole element

The link's accessible wording comes from its text in this simple case. A class such as button does not supply button semantics. If the control navigates, keep the anchor. If it performs an action in the current interface, consider a button.

Text between tags becomes text nodes. A comment is a different node and is not displayed as normal page content. Comments are still delivered to the browser. Never store passwords, private tokens or confidential project notes in them.

The example

HTML / example 04

```
<a class="catalogue-link" href="catalogue.html">
  Browse available books
</a>
<!-- The catalogue is a separate document. -->
```

The start tag carries two attributes. The end tag closes the anchor. Whitespace around the words can affect text nodes, but it does not create another element.

A mistake to watch for

Calling every piece of angle-bracket syntax an element. In particular, the doctype is a declaration, and an end tag is syntax rather than a second element.

Your turn

Identify the element name, attribute names, attribute values and link text in the example.

Explained answer

The element name is a. The attributes are class and href, with values catalogue-link and catalogue.html. The words Browse available books are the link text. The comment is separate from the anchor.

Read the source

HTML Living Standard: elements

HTML Living Standard: attributes

I / Understanding HTML

05. Think in trees

Parent and child relationships explain many markup rules.

By the end of this chapter

- Describe parent, sibling and descendant relationships
- Separate visual arrangement from document structure

The DOM is a tree of nodes. The document has an element child, html, which normally contains head and body. An element can contain text and other elements where its content model allows them.

Parent, child, sibling and descendant describe relationships, not screen positions. Two boxes that sit side by side can have very different places in the tree. A heading belongs to an article because of the document structure, not because it happens to appear above it visually.

Inspect one branch

Here the article is a child of main. The heading and paragraph are siblings. The em element is a child of the paragraph and a descendant of the article. A selector or a DOM method operates on these relationships.

Keep the tree understandable before styling. Deep wrappers can be appropriate, but each extra layer should have a reason. Do not use the visual layout as evidence that nesting is valid: a browser can display invalidly nested source after repairing it.

The example

HTML / example 05

```
<main>
  <article>
    <h1>A shelf for local writers</h1>
    <p>Bring a book by an <em>independent</em> author.</p>
  </article>
</main>
```

The emphasis belongs inside the sentence. Moving em around the whole article would change both the semantics and the validity question; phrasing elements cannot simply wrap arbitrary document sections.

A mistake to watch for

Assuming that any element may contain any other element as long as the end tags match.

Your turn

Is em a direct child of article? Are h1 and p siblings?

Explained answer

em is not a direct child of article; p is its parent. h1 and p share article as a parent, so they are siblings. Both are descendants of main.

Read the source

HTML Living Standard: the DOM

HTML Living Standard: content models

DOM Standard: trees and node relationships

I / Understanding HTML

06. Make your first document

Use a text editor, a browser and a small feedback loop.

By the end of this chapter

- Create a complete UTF-8 HTML document
- Verify changes in the browser and a checker

Create a folder with index.html. Save it as UTF-8, then open it in a browser. A local file is enough for the first static document. Later, use a local HTTP server for examples that depend on fetch, modules, origin behavior or server responses.

Use an editor that lets you see the actual file contents. A rich-text document renamed with an html extension is not an HTML source file. Confirm that your operating system has not quietly saved index.html.txt.

Check what you changed

Edit one heading, save, reload, and find the changed text. Open Developer Tools and inspect the heading. Change the document title and look at the browser tab. These separate checks establish where your output appears.

Run a conformance checker early. Fix errors in the smallest document before adding layout. Keep a copy you can return to, preferably in version control. Browser extensions can alter pages, so use a clean profile when investigating behavior you cannot explain.

The example

HTML / example 06

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Neighbourhood Library</title>
</head>
<body>
  <h1>A good place to start reading</h1>
  <p>Visit the library on Saturday.</p>
</body>
</html>
```

The viewport declaration supports a page sized for a device's viewport. It is not a responsive layout by itself. The page still needs CSS if its later layout should adapt to available space.

A mistake to watch for

Editing a different copy of the file from the one open in the browser. Check the address and the saved path before debugging the markup.

Your turn

Change the tab title without changing the visible heading. Which element should you edit?

Explained answer

Edit title inside head. The visible heading is h1 inside body. They may use different wording because one labels the document in browser UI while the other introduces the page content.

Read the source

HTML Living Standard: a quick introduction

II / Syntax and Document Foundations

07. The document shell

Give each document a predictable beginning.

By the end of this chapter

- Explain the job of the doctype
- Combine content without duplicating document structure

A complete HTML page normally declares the doctype, document language, encoding, title and body content. This small shell is worth understanding even if your framework generates it.

The doctype helps select standards-mode processing. It is not an instruction to fetch a document type definition. The html element is the document element. head holds document metadata; body holds the document's content.

Avoid copying a shell blindly

Set lang to the language actually used by the page. Keep the character encoding declaration early in the document; the Standard requires a meta encoding declaration to be entirely within the first 1024 bytes when that mechanism is used. A server can also communicate encoding through its response headers.

Do not duplicate head or body to combine two pages. Combine their appropriate content into a single document, resolving conflicting metadata. A stylesheet, script or template system does not excuse an invalid outer structure.

The example

HTML / example 07

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Opening hours | Neighbourhood Library</title>
</head>
<body>
  <h1>Opening hours</h1>
  <p>Tuesday to Saturday, 10:00 to 17:00.</p>
</body>
</html>
```

The title identifies this page and its site. The heading can stay shorter. Neither substitutes for the other.

A mistake to watch for

Putting a second doctype and a second body inside an existing body when pasting a complete example.

Your turn

You receive two complete HTML files and want a single page. Should you paste the second html element inside the first body?

Explained answer

No. Keep one document shell. Move the second page's relevant body content into the first body, choose the required metadata, then check heading structure, duplicate ids and links.

Read the source

HTML Living Standard: the doctype

HTML Living Standard: encoding declarations

II / Syntax and Document Foundations

08. html, head and body

Keep metadata and page content in their intended places.

By the end of this chapter

- Place metadata and visible content correctly
- Set an appropriate document language

head is not a hidden drawer for arbitrary markup. Its content model is metadata content, with additional rules for elements such as title and base. Put a visible navigation bar in body, even if it appears at the top of the screen.

body is the main container for the document's page content. The name header refers to introductory content in body; it is not a replacement for head. A page can have a document head and several meaningful headers in its content.

Set the page language

The lang attribute on html establishes the language for descendants that do not override it. Mark a phrase in another language on an appropriate containing element. Do not use a country code when you mean a language code. A region subtag can qualify a language where it helps, but it is optional when a general language tag is enough.

Optional-tag rules can allow some tags to be omitted in carefully defined situations. Explicit html, head and body tags are a useful convention for teaching and maintenance. Understand that this is a convention, not a claim that omission is always invalid.

The example

HTML / example 08

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Library membership</title>
</head>
<body>
  <header><p>Neighbourhood Library</p></header>
  <main><h1>Become a member</h1></main>
</body>
</html>
```

The header contains site introduction inside body. head contains metadata that describes the document.

A mistake to watch for

Using head where header was intended, or assuming that body must begin immediately with a heading and cannot contain a site header.

Your turn

Where should a site logo link go: head, header in body, or title?

Explained answer

Put the visible logo link in body, often inside a header. title contains text for the document title, not a visible linked logo. head is for metadata.

Read the source

HTML Living Standard: the document element

HTML Living Standard: head

HTML Living Standard: body

HTML Living Standard: language attributes

II / Syntax and Document Foundations

09. Text, encoding and escaping

Make the bytes and the intended characters agree.

By the end of this chapter

- Keep saved encoding and declared encoding consistent
- Escape markup when displaying source

UTF-8 represents characters as bytes. Your editor's save encoding and the document's declared encoding must agree. A declaration cannot repair bytes that were saved incorrectly.

Use literal readable text for ordinary characters when your workflow supports UTF-8. Character references are useful when source syntax would otherwise be ambiguous. Write < to show a less-than sign in text, and use & where an ampersand needs escaping. Escape the quote character that delimits an attribute value when it appears inside that value.

Preserve meaning when showing code

The example displays the text <button> without creating a button. The parser consumes the character references and creates text characters. This is different from inserting untrusted strings through innerHTML, which invokes markup parsing.

A non-breaking space changes line-breaking behavior. Do not scatter it through text to imitate layout. CSS is the right place for gaps between interface elements. When you copy source between tools, check for accidental invisible characters and preserve meaningful language characters.

Check the response as well as the file

If a served page displays unexpected characters, inspect its bytes and the HTTP Content-Type charset as well as the early meta declaration. Encoding detection follows ordered rules; a meta declaration cannot override every byte-order mark or transport-level decision. For new documents, save UTF-8 bytes and keep the server declaration consistent. Replacing an already misdecoded character with another declaration does not reconstruct the original data.

The example

HTML / example 09

```
<p>Write <code>&amp;lt;button&amp;gt;</code> for an action.</p>
<p>Research &amp;amp; reading</p>
<p title="She said &amp;quot;welcome&amp;quot;">A quoted greeting</p>
```

The code element supplies semantics for a code fragment; escaping stops the fragment from being parsed as an actual element. Both decisions matter.

A mistake to watch for

Escaping the same text twice. < displays the reference spelling < rather than the intended less-than sign.

Your turn

How would you show the literal source text Hello inside a code element?

Explained answer

Use <code><p>Hello</p></code> in the HTML source. The browser displays the tag spelling as text. Keep the code element so that the kind of text is still identified.

Read the source

HTML Living Standard: character references

HTML Living Standard: encoding

HTML Living Standard: character encoding declaration requirements

II / Syntax and Document Foundations

10. Nesting and void elements

Well-formed-looking tags are only the beginning.

By the end of this chapter

- Recognize void elements
- Check content rules as well as matching tags

Close nested elements in the reverse order you opened them. That keeps the source understandable, but correct nesting also depends on each element's content model. A paragraph cannot contain arbitrary flow content such as a div.

Void elements have no contents in HTML syntax. Examples include img, br, meta, link and input. Do not write an img end tag. A trailing slash on a void element is unnecessary in HTML and does not give it XML semantics.

Choose explicit source while learning

Some end tags can be omitted under specific conditions. Closing them explicitly is often easier to review. Other end tags, such as an anchor's end tag, are not optional in the same way.

Do not use a slash on a non-void element as if it closed the element in text/html. A script element still requires an end tag. If parsing produces an unexpected tree, inspect the source around the earliest error rather than adding more closing tags at the bottom.

One permitted omission, not a general shortcut

A `li` end tag may be omitted when the element is immediately followed by another `li`, or when there is no more content in its parent. In `<ul><li>One<li>Two</ul>`, the two list items are siblings. The second start tag does not make one item a child of the other.

That rule belongs to `li`. It does not allow an omitted end tag on every element, and it does not explain all the conditions for paragraphs, table sections or document-shell tags. Use the linked optional-tag rules for those cases. Self-closing syntax for foreign SVG or MathML elements is a separate rule from a trailing slash on a void HTML element.

The example

HTML / example 10

```
<p>Bring a <strong>current membership card</strong>.</p>
<img src="membership-card.svg" alt="Sample library membership card"
     width="320" height="200">
<p>Ask at the desk if you need a replacement.</p>
```

The strong element closes before its paragraph. The img stands on its own as a void element. Its width and height describe the intended intrinsic dimensions or aspect-ratio basis for layout, not a guarantee that CSS cannot resize it.

A mistake to watch for

Writing `<div />` in an HTML document and expecting later content to be outside the div.

Your turn

Why is `<p><div>Notice</div></p>` not a sound way to put a notice box inside a paragraph?

Explained answer

A div is not permitted paragraph content. The HTML parser can close the paragraph before it processes the div, producing a tree unlike the source suggests. Use a containing div or section with a paragraph inside it instead.

Read the source

HTML Living Standard: elements and tag omission

HTML Living Standard: optional tags

II / Syntax and Document Foundations

11. Attributes and their values

Presence, an empty string and the word false are different things.

By the end of this chapter

- Distinguish boolean and enumerated attributes
- Read attribute values according to their defined syntax

Quote attribute values consistently. HTML allows unquoted values only when they satisfy its restrictions; quoting is easier to maintain when a value later gains a space or punctuation.

Boolean attributes represent their state by presence or absence. disabled="false" still supplies disabled. To leave a control enabled, omit the attribute. An enumerated attribute uses defined keywords and can define missing-value, empty-value and invalid-value defaults. An absent attribute, an empty value and an unrecognized keyword are different inputs; they need not select the same state. Do not apply boolean rules to an attribute merely because its value looks like true or false.

Learn the value type

hidden, contenteditable and autocomplete do not all share the same value rules. Read the definition for the attribute and its particular element or state. Values such as ids, token lists, URLs and numbers each have their own syntax.

In HTML documents, attribute names on HTML elements are treated in an ASCII case-insensitive way during parsing. Lowercase names are a readable convention. Attribute values are not universally case-insensitive: an id and a URL path can be case-sensitive.

Separate valid source from error recovery

Conforming forms of a boolean attribute include the bare name, an empty value such as `disabled=""`, or a matching name such as `disabled="disabled"`. The strings `true` and `false` are not permitted boolean-attribute values, even though browsers treat the attribute as present. This is a useful example of browser behavior and authoring conformance giving different answers.

Removing a button's own disabled attribute does not override every surrounding condition. A disabled fieldset can also disable descendant controls under its defined rules. Check the complete form when the initial state still differs from what you expected.

The example

HTML / example 11

```
<label for="member-email">Email</label>
<input id="member-email" name="email" type="email" required>
<button type="submit" disabled>Join the waiting list</button>
```

required and disabled are boolean attributes. The id and for values establish a label relationship. The type keyword selects an input state, whose rules affect validation and interface behavior.

A mistake to watch for

Inventing a generic rule that every attribute can be turned off with the string false.

Your turn

You find `<button disabled="false">Save</button>`. How do you make it enabled in the initial HTML?

Explained answer

Remove disabled. The string false does not turn off a boolean attribute. If a script changes the state later, assign false to the element's disabled property or remove the attribute.

Read the source

HTML Living Standard: boolean attributes

HTML Living Standard: enumerated attributes

HTML Living Standard: disabling form controls

II / Syntax and Document Foundations

12. Content categories and models

Use categories to understand what can go where.

By the end of this chapter

- Use overlapping content categories
- Apply extra restrictions on transparent and interactive content

The Standard groups elements into content categories such as metadata, flow, phrasing and interactive content. Categories overlap. An element can belong to several, and membership can depend on its attributes or context.

A content model describes what an element can contain. Categories make those descriptions shorter, but they do not remove additional restrictions. A button can contain phrasing content while still prohibiting interactive descendants and descendants with tabindex.

Transparent does not mean unrestricted

An anchor has a transparent content model in relevant contexts. Its contents must fit the surrounding context, with the anchor's own restrictions still applying. Wrapping an entire card in a link does not permit putting another link or a button inside that link.

The reference includes the captured category summaries and their exceptions. Treat these as routes to the definitions, especially for media, form-related elements and transparent models. The category table contains more rows than the commonly taught core category names because it also includes form-related groupings.

The example

HTML / example 12

```
<article>
  <h2><a href="book.html">The Atlas of Small Places</a></h2>
  <p>A walking guide to overlooked streets.</p>
  <button type="button">Add to reading list</button>
</article>
```

The navigation link and the action button are separate. Neither interactive control is nested inside the other. The article can still be styled as one visual card.

A mistake to watch for

Wrapping a card containing buttons in one outer anchor and assuming transparent content permits every descendant.

Your turn

Why is a button inside an anchor a problem even if both controls can be clicked with a mouse?

Explained answer

The anchor's content restrictions disallow interactive descendants, including a button. The structure also leaves competing interactions in one region. Use separate controls with clear destinations or actions.

Read the source

HTML Living Standard: content models

HTML Living Standard: categories

HTML Living Standard: anchor content restrictions

HTML Living Standard: button content restrictions

II / Syntax and Document Foundations

13. Conformance and useful errors

Make the checker part of the feedback loop.

By the end of this chapter

- Use checker output without overclaiming
- Separate syntax, network and behavior failures

A conformance checker can identify invalid nesting, missing required attributes and many syntax errors. It cannot decide whether your alternative text is useful, whether your page is easy to navigate, or whether a server validates a submission securely.

Check the complete document as well as small examples. The containing form, table or head can change what is permitted. Fix the earliest relevant error first because a parser recovery can cause several later messages.

Classify what failed

A missing end tag, an unknown attribute and a broken resource URL are different problems. Some failures are conformance errors; some are network failures; some are mistakes in the intended behavior. Choose a check that can observe the failure you care about.

Keep intentionally invalid teaching examples separate from valid fixtures and record the expected error. A checker pass means the checked document passed that checker version and configuration. It is not a general quality certificate. If a new standardized feature triggers a message, compare the current definition with the checker's supported version before changing correct markup blindly.

The example

HTML / example 13

```
<form action="/membership" method="post">
  <label for="full-name">Full name</label>
  <input id="full-name" name="fullName" autocomplete="name" required>
  <button type="submit">Apply for membership</button>
</form>
```

A conformance check can inspect the label relationship and syntax. You still need to test actual submission handling and whether the requested name field is appropriate to the service.

A mistake to watch for

Treating a clean checker output as proof that a page meets WCAG or has no security risks.

Your turn

A document validates, but its submit button sends data to the wrong endpoint. Is the checker broken?

Explained answer

No. A syntactically valid action URL can be the wrong application destination. Test the request and server behavior in addition to conformance. Each check needs a defined responsibility.

Read the source

HTML Living Standard: conformance requirements

III / Text and Content Elements

14. Headings and paragraphs

Make the page structure readable without its styling.

By the end of this chapter

- Create a conventional heading hierarchy
- Use paragraphs, line breaks and thematic breaks for their meanings
- Group a heading and supporting subtitle without inventing a subsection

A heading names the content that follows it. Use h1 through h6 to express the hierarchy you intend. For the ordinary pages in this book, one page-level h1 and properly nested subsection headings are a practical convention.

Where a document has headings, the current Standard requires at least one computed level-1 heading and does not permit a forward level jump of more than one between consecutive headings. It does not require exactly one h1.

Choose heading levels for structure, not font size. CSS can make an h2 smaller or larger without changing its meaning. A section element does not automatically reset a conventional heading hierarchy to a new h1 in the way many old tutorials suggested.

Separate paragraphs from line breaks

Use p for paragraphs. br is a line break within content where the break itself belongs, such as an address or a poem. Empty paragraphs and repeated br elements are not layout tools. hr represents a thematic break, not merely a horizontal line to decorate a page.

The Living Standard also includes headingoffset and headingreset. These require independent compatibility and accessibility review; they are not a reason to discard a clear conventional hierarchy. This edition records them in the reference and uses ordinary heading levels in its running examples.

A heading and its supporting line

Use `hgroup` when a heading has a subtitle, alternative title or tagline that belongs with it. It contains one `h1` to `h6`, with permitted `p` elements before or after it, and allows script-supporting elements under its content model. For example, `<hgroup><h1>Library membership</h1><p>A place for every reader</p></hgroup>` keeps the tagline as a paragraph rather than pretending it is a subsection heading.

Understand offsets before relying on them

In the current Standard, `headingoffset` is an integer from 0 to 8 for authoring, and `headingreset` is boolean. Computation starts at the heading itself, adds offsets while walking its inclusive ancestors and stops at a reset boundary. The computed heading level is capped at 9. An inherited offset of 1 changes an `h2` computation to level 3; a reset boundary prevents offsets outside that boundary from contributing. When the current node has a shadow-root parent, traversal continues at that shadow root's host. An offset on the reset boundary itself still contributes before traversal stops.

These rules do not rename the DOM element or prescribe its font size. They are also not evidence that every browser and assistive-technology combination exposes the computed level correctly. Keep the ordinary hierarchy in the running example, and test any offset-based component against the actual target browsers and accessibility tools.

The example

HTML / example 14

```
<main>
  <h1>Membership</h1>
  <p>Membership is open to local residents.</p>
  <section>
    <h2>What to bring</h2>
    <p>Bring an address document and a form of identification.</p>
  </section>
</main>
```

The h2 introduces a subsection of membership. It does not depend on the browser calculating an implicit outline from section nesting.

A mistake to watch for

Picking h4 because its default size matches a design. That makes document structure depend on a browser stylesheet.

Your turn

A designer wants the main heading to look small. Should you change h1 to h5? Where should a subtitle go when it belongs with the main heading?

Explained answer

Keep the heading level that matches the document structure and change its presentation with CSS. A visual size preference does not change where the heading belongs in the hierarchy. A supporting subtitle can be a p in an hgroup with the heading; it should not become a false subsection heading.

Read the source

HTML Living Standard: headings

HTML Living Standard: paragraphs

HTML Living Standard: hgroup

HTML Living Standard: heading levels and offsets

III / Text and Content Elements

15. Inline meaning

Pick emphasis because of the sentence, not the browser font.

By the end of this chapter

- Distinguish stress, importance and presentation
- Represent titles and quotations deliberately
- Mark a defining instance and supply its definition

em marks stress emphasis. strong marks importance, seriousness or urgency. Their default italic and bold styles are not their definitions. A book title can use cite; a short quotation can use q. Use a blockquote for a quoted section rather than a short inline quotation.

small suits side comments such as a copyright or a qualification. mark can identify text relevant to the reader's current context, such as a search match. span supplies a generic phrasing container when no more specific semantic element fits.

Do not turn typography into semantics

The b and i elements also have defined uses beyond their default appearance. b can draw attention without conveying extra importance; i can mark text in an alternate voice or mood. u is for an explicitly rendered non-textual annotation, not a generic request to underline.

Quote and cite carefully. A cite attribute can record a source URL, but browsers do not normally expose it as a useful visible citation. Provide an ordinary visible source link when readers need to follow it. cite represents a work's title rather than a person's name.

Define a term, rather than just styling it

`dfn` identifies the defining instance of a term. In `<p>A <dfn>loan period</dfn> is the time before an item is due back.</p>`, the same paragraph contains the definition. The nearest containing paragraph, description-list group or section must contain the definition; merely adding `dfn` to an unexplained word is not enough. Do not nest one `dfn` inside another.

If `dfn` has a `title`, that value identifies the exact term being defined, not a longer explanatory tooltip. Without that title, if the dfn has exactly one element child, that child is an abbr with a title, and there are no child Text nodes, the abbr title gives the term. Otherwise, the dfn descendant text supplies it. Even whitespace Text nodes matter to that special-case condition. Chapter 20 explains abbreviation expansions. `s` marks wording that is no longer accurate or relevant; Chapter 20 contrasts it with `del`, which marks an edit.

A `q` quotation has phrasing content. A `blockquote` can contain a quoted section with paragraphs and other permitted structure. Neither element verifies a quotation or attributes it for you. Keep an ordinary visible source citation where readers need it.

The example

HTML / example 15

```
<p><strong>Return the key before closing time.</strong></p>
<p>You may borrow <em>two</em> reference books.</p>
<p>Our reading group chose <cite>Things Fall Apart</cite>.</p>
```

The importance of returning the key differs from the stress on two. The novel's title names a work; it does not claim emphasis in the sentence.

A mistake to watch for

Using strong around every visually bold fragment, including purely decorative interface text.

Your turn

Which element fits a short phrase that must carry stress in the sentence, and which fits an urgent warning? In the loan-period example, what makes dfn more than a styling hook?

Explained answer

Use em for stress emphasis and strong for an urgent or important warning. If the requirement is only a typeface change, use CSS on an appropriate existing element or a neutral span. It identifies the defining instance, and the containing paragraph actually states what the term means.

Read the source

HTML Living Standard: text-level semantics

HTML Living Standard: defining instances

HTML Living Standard: content no longer accurate

HTML Living Standard: inline quotations

III / Text and Content Elements

16. Code and technical text

Preserve the source when readers need to copy it.

By the end of this chapter

- Distinguish code, input and output
- Display copyable escaped source
- Distinguish meaningful subscripts and superscripts from decoration

code identifies computer code. pre preserves the significance of whitespace in preformatted text. Combine them when indentation and line breaks matter. Escape markup characters before placing HTML source inside either element.

kbd marks user input, samp marks sample output, and var marks a variable. These meanings are useful even when a stylesheet makes them look similar. A filename or a command does not need a decorative badge to be understandable.

Give examples enough context

State the file, command environment or containing element that an example needs. Readers should know whether they can paste a block into a document or need to supply surrounding code. A fragment that works in body may be invalid in head.

Avoid screenshots as the only version of code. Text can be copied, searched and read by assistive technology. If a code block scrolls horizontally, make the scroll container keyboard reachable and label it. Keep the example short enough to explain; put a longer working file in the downloads.

Subscripts and superscripts carry meaning

Use `sub` and `sup` for meaningful typographical conventions, not to move decorative text up or down. `<var>x</var><sup>2</sup>` represents a squared variable; `H<sub>2</sub>0` uses a chemical subscript. A logo that merely has raised lettering should use CSS instead. For richer mathematical structure, consider the MathML integration discussed in Chapter 33.

There is a parser detail worth checking when exact text matters: in HTML syntax, one leading newline immediately after a `pre` start tag is stripped. `<pre>` is not a raw-text escape hatch. If `code` is its first child, a newline inside that child is not the same immediate-after-`pre` case. Test the actual text and use the exact source download when copying a complete example.

Keyboard combinations can be grouped with nested `kbd`, such as `<kbd><kbd>Ctrl</kbd> + <kbd>S</kbd></kbd>`. The shortcut still depends on the application and platform. An element describing input does not register that shortcut.

The example

HTML / example 16

```
<p>Press <kbd>Ctrl</kbd> + <kbd>S</kbd> to save.</p>
<pre><code>&lt;h1&gt;Library catalogue&lt;/h1&gt;</code></pre>
<p>The server returned <samp>201 Created</samp>.</p>
```

The example shows HTML source as text. kbd identifies input rather than output. The status string is a sample response, not proof that this page made a request.

A mistake to watch for

Pasting raw <h1> markup into code and expecting it to stay visible as source.

Your turn

Why do you still need escaping inside pre and code? Should a decorative raised logo character automatically use sup?

Explained answer

Neither element turns off ordinary HTML parsing of their contents. Escape the angle brackets so the parser creates text instead of an h1 element. pre changes whitespace handling, not the basic parsing rule. No. Use CSS for a decorative position; sup is for meaningful superscript conventions.

Read the source

HTML Living Standard: preformatted text

HTML Living Standard: code

HTML Living Standard: sub and sup

HTML Living Standard: keyboard input

HTML Living Standard: sample output

HTML Living Standard: variables

III / Text and Content Elements

17. Links that go somewhere

Let the destination explain the control.

By the end of this chapter

- Choose links for navigation
- Resolve document and fragment URLs correctly

An a element with href creates a hyperlink. Give it wording that makes sense in context and a destination that does what the wording promises. A file path, a fragment and an absolute URL resolve differently.

Without a base URL that changes the target document, a fragment such as #fees points to an element with a matching id in the current document. Relative URLs resolve against the document's base URL. A base element can change that resolution across the page, which makes it easy to introduce surprising navigation bugs.

Navigation is different from an action

Use a hyperlink to go to a resource. Use a button to submit a form or perform an interface action. href="#" is not a neutral placeholder for a JavaScript button; it still has navigation behavior.

Opening a new browsing context changes the user's experience. Avoid doing it automatically without a reason. If you use target="_blank", describe the behavior where it helps and understand opener relationships. Modern anchor behavior provides implicit noopener for this target, but explicitly writing rel="noopener" can document the intention. Do not imply that this solves every trust or privacy concern.

Resolve the URL before trusting the label

If the document is `https://example.invalid/book/pages/contact.html`, `../index.html` resolves to `https://example.invalid/book/index.html`, while `/index.html` resolves at the origin root. These are different paths. A fragment-only URL also resolves against the document's base: a `base` pointing to another document can make `#fees` target that other document, not the current page.

Inspect the resolved `href` as well as its source attribute. The attribute preserves the relative spelling; the DOM property gives the resolved URL. A `download` attribute does not promise that every cross-origin link will download a file, and a mail or telephone link does not implement a communications service. Check the destination's rules and the actual user experience.

The example

HTML / example 17

```
<p><a href="catalogue.html">Browse the catalogue</a></p>
<p><a href="#fees">Read the membership fees</a></p>
<section id="fees">
  <h2>Membership fees</h2>
  <p>Ask the desk about the current membership options.</p>
</section>
```

The first URL is a sibling document. The second is a fragment in this document. The target id must be unique in the element's tree.

A mistake to watch for

Using link text such as click here repeatedly when a destination-specific phrase would be clearer.

Your turn

Would you use an anchor or a button to open the next page of the catalogue? What about removing a book from a local reading list? Can a base element make #fees target another document?

Explained answer

Use an anchor for another catalogue page. Use a button for the removal action. If the action changes server data, also design and test its request, permissions and error handling; the element choice alone does not implement those controls. Yes. Even a fragment-only reference resolves against the document base URL; inspect the resolved href before treating it as a current-document target.

Read the source

HTML Living Standard: the a element

HTML Living Standard: links

HTML Living Standard: base URLs

HTML Living Standard: downloading resources

III / Text and Content Elements

18. Lists with a reason

Order and relationships determine the markup.

By the end of this chapter

- Distinguish ordered, unordered and description lists
- Nest a list under the item it expands
- Explain what menu supplies and what it does not implement

Use ul for a group whose order is not significant, and ol when sequence or rank is part of the meaning. Each item is an li. A navigation menu is often a list of links, but not every group of adjacent items needs list markup.

A description list uses dl, dt and dd for name-value groups. A group can have multiple terms or multiple descriptions. It is not limited to dictionary definitions and does not require a table-like visual layout.

Keep nested lists inside an item

A sublist belongs inside the li it expands. A list placed beside an li is not that item's nested list. Markup should explain the relationship before CSS adds indentation.

For ordered lists, start and reversed can change the numbering behavior, and an li value can set an item's ordinal value in an ol. These are useful when the displayed order has meaning. Avoid using list markers as a substitute for actual text if readers need the text outside that presentation.

Commands and description groups

`menu` represents an unordered list of commands, a semantic alternative to `ul` for that purpose. `<menu><li><button type="button">Copy</button></li><li><button type="button">Paste</button></li></menu>` describes two command items; it does not implement either action. It is not the ARIA menu-widget pattern and does not automatically provide arrow-key navigation, a popup or a context menu.

In a `dl`, one or more `dt` names precede one or more `dd` values in each group. Keep several paragraphs belonging to one value in the same `dd`. A permitted alternative wraps each complete name-value group in a `div`, for example `<dl><div><dt>Saturday</dt><dd>10:00 to 14:00</dd></div></dl>`. Do not mix direct groups and div-wrapped groups as if the two content-model alternatives were interchangeable.

For a reversed `ol` without an explicit start, the initial ordinal is based on the number of its list-item children. A `value` on an `li` can change that item's ordinal and the sequence that follows. Check source order as well as marker styling; CSS counters are a separate presentation mechanism.

The example

HTML / example 18

```
<ol>
  <li>Choose a book.</li>
  <li>Check the lending period.
    <ul><li>Reference books have separate rules.</li></ul>
  </li>
  <li>Take the book to the desk.</li>
</ol>
```

The unordered sublist qualifies the second step. It remains inside that step's li, so the tree preserves the relationship.

A mistake to watch for

Putting the nested ul directly under the outer ol between two li elements.

Your turn

Which structure fits opening hours grouped by day names: an ordered list of steps or a description list? Does menu implement a popup and its keyboard controls?

Explained answer

A description list is a reasonable choice for day names and their hours because the groups describe name-value relationships. A table may be better when readers need to compare a regular grid of days and multiple time ranges. An ordered list communicates that order matters; it does not inherently mean a procedure. Use it if the ordered sequence itself is the point, rather than merely to pair each day with its hours. No. The element describes a command list; a popup widget and its keyboard interaction require separate implementation and testing.

Read the source

HTML Living Standard: lists

HTML Living Standard: description lists

HTML Living Standard: menu commands

HTML Living Standard: list items

III / Text and Content Elements

19. Figures and captions

Keep self-contained material with its explanation.

By the end of this chapter

- Choose a figure for self-contained content
- Separate a caption from an image alternative
- Check alternative text against the supplied teaching schematic

A figure holds self-contained content that the surrounding prose can refer to as a unit. It can contain a diagram, photograph, code listing or another suitable item. figcaption supplies its caption as the first or last child when present.

Not every image needs a figure. An icon in a button is part of that control. A photograph beside a paragraph may or may not be self-contained. Decide from the content relationship, not a desire for a styled border.

A caption and alternative text do different work

A caption explains the figure in the document. Alternative text supplies a replacement for an image according to its purpose and context. Repeating a long caption word for word as alt can add noise without helping. An image with information absent from the caption still needs that information represented appropriately.

A complex chart usually needs more than a short alt string. Provide the important conclusion and a detailed text or data alternative that readers can reach. Do not claim an image is accessible solely because alt is present.

Keep the figure's structure honest

A `figure` may have one `figcaption`, placed first or last, not inserted between arbitrary content children. The caption can describe a code listing or a quotation as well as an image. If the content is a quotation, keep attribution outside the quoted passage, for example in the figure's caption.

The supplied floor plan is an original teaching schematic, not a map of real premises. Compare its rooms with the alternative: the reading room is left of reception, and the lift is across the corridor. The caption identifies the purpose of the schematic rather than repeating those relationships. For a real plan, check every direction against the actual premises and provide another usable route to essential location information.

The example

HTML / example 19

```
<figure>
  <img src="../assets/library-floor-plan.svg"
       alt="The reading room is left of reception; the lift is opposite."
       width="640" height="360">
  <figcaption>Teaching schematic: ground-floor facilities near reception.</figcaption>
</figure>
```

The alternative conveys the directions shown by the image. The caption identifies the figure's subject in the document. A real floor plan would need its own reviewed alternative based on its actual contents. The supplied schematic is fictional, not a real-site access or evacuation plan.

A mistake to watch for

Generating alternative text from a filename without inspecting the image or its purpose.

Your turn

A chart caption says Monthly loans. Does repeating Monthly loans in alt communicate the chart's trend? In the supplied schematic, what does the alternative communicate that the caption does not?

Explained answer

No. It only repeats the subject. Describe the meaningful information, and provide an accessible data table or fuller explanation when the chart needs one. The appropriate wording depends on the actual data and the reason the chart appears. It gives the reading-room and lift relationships to reception; the caption identifies the schematic and its teaching purpose.

Read the source

HTML Living Standard: figures

HTML Living Standard: figcaption

WAI images tutorial: complex images

III / Text and Content Elements

20. Dates, values and edits

Give machines a useful value without spoiling the sentence.

By the end of this chapter

- Use machine-readable dates without invented precision
- Distinguish document edits from obsolete wording
- Expand an abbreviation visibly and distinguish edit timestamp syntax

time can pair readable date or time text with a machine-readable datetime value. Use a value that matches the defined syntax and the kind of temporal information you actually know. Do not add a timezone or a time of day just to make a date look more precise.

data associates its contents with a machine-readable value. It is useful for a product identifier or another mapping that a script or processor uses. It does not create a general validation rule for that value.

Record edits when the change matters

ins and del mark additions and deletions. Their datetime and cite attributes can record when and why a change happened, subject to their defined rules. Provide visible context when readers need it; an attribute alone is not a usable change log.

s has a different meaning from del: it marks content that is no longer accurate or relevant, without necessarily marking a document edit. An old price can be struck through while the current price remains plainly stated. Choose the meaning before choosing the decoration.

Expand an abbreviation where readers can use it

`abbr` marks an abbreviation or acronym. For example, `Hypertext Markup Language (<abbr title="Hypertext Markup Language">HTML</abbr>)` provides a visible expansion as well as optional metadata. If `title` is present on `abbr`, it must contain the expansion, and nothing else. Match the expansion's grammatical number to the abbreviation content: a plural abbreviation needs a plural expansion. Do not put an unrelated explanation there, and do not assume an expansion on one abbreviation is inherited by another occurrence.

Using `abbr` for every abbreviation is not mandatory. Expand unfamiliar terms in visible text when that helps readers, rather than making essential information available only through a tooltip whose behavior varies by device and software.

Match the date syntax to the element

`time` accepts several defined kinds of temporal values, including dates, times and durations; they are not all interchangeable timestamps. By contrast, `datetime` on `ins` and `del` uses a valid date string with optional time. If that edit value includes a time, it must be a valid global date and time with the required offset information. A bare local time such as `10:00` is not a complete edit timestamp. Keep a visible explanation of an important policy change rather than relying on metadata alone.

The example

HTML / example 20

```
<p>Next meeting: <time datetime="2026-11-07">7 November 2026</time>.</p>
<p>Book code: <data value="LIB-204">Local history collection</data>.</p>
<p>The desk opens at <del>09:00</del> <ins>10:00</ins>.</p>
```

The date supplies only a date. The collection has an explicit identifier. The changed opening time is visible in the sentence, rather than hidden in metadata.

A mistake to watch for

Writing an informal date such as 7/11/26 in datetime and assuming every parser will interpret it the same way.

Your turn

You know an event date but not its time. Should you write a midnight UTC timestamp? Is datetime="10:00" a valid edit timestamp on ins?

Explained answer

No. Use a valid date value such as 2026-11-07. A fabricated time or timezone adds information you do not have and can mislead downstream software. No. An edit needs a valid date with optional time, not a bare local time. Do not transfer every time-element value syntax to ins or del.

Read the source

HTML Living Standard: time

HTML Living Standard: edits

HTML Living Standard: abbreviations

HTML Living Standard: edit attributes

HTML Living Standard: date strings with optional time

III / Text and Content Elements

21. Language and direction

Language, writing direction and pronunciation are separate concerns.

By the end of this chapter

- Mark language changes
- Isolate mixed-direction text
- Distinguish optional word breaks from forced line breaks

Set the primary language on html, then mark language changes where they occur. A valid language tag helps software choose pronunciation, spelling and other language-sensitive behavior. It does not translate the words or determine their writing direction.

Use dir for directionality. Values such as rtl and ltr express known directions; auto lets the user agent determine direction from the content under the defined rules. For user-provided names or messages of unknown direction, isolation can stop adjacent punctuation and text from being rearranged unexpectedly.

Work with unfamiliar text

bdi isolates a run of text from the surrounding bidirectional context. bdo overrides direction and is a more specialized tool, not the normal choice for ordinary right-to-left text. Avoid inserting directional control characters casually when markup expresses the relationship more clearly.

Ruby annotations use ruby with rt and, where appropriate, rp fallback punctuation. They have language-specific uses and should be reviewed by someone who understands the writing system. Test the actual languages your product supports, including long words, mixed scripts and names that do not fit a Western given-name/surname assumption.

Breaking a line is different from changing direction

`wbr` offers a line-breaking opportunity in otherwise difficult text, such as a long identifier. In `catalogue<wbr>reference`, it supplies a possible break without inserting a hyphen or an ordinary text space. It does not mean "start a new line here" in the way `br` does. Test copying and wrapping in the actual reading interface.

`bdi` has automatic directionality by default when no `dir` is specified. Writing `dir="auto"` can make the intention explicit. `bdo` is different: it requires `dir="ltr"` or `dir="rtl"` and overrides bidirectional ordering. Do not use it as the general wrapper for names whose direction you do not know.

A small ruby example is `<ruby>□<rp>(</rp><rt>□□</rt><rp>)</rp></ruby>`. `rt` supplies the annotation; `rp` provides fallback parentheses for presentations that do not support ruby. This syntax example does not replace review of the reading and typography by someone familiar with the language.

The example

HTML / example 21

```
<p>Readers say <span lang="yo">Ẹ káàbọ̀</span> at the entrance.</p>
<p>Reserved for <bdi dir="auto">&#x645;&#x631;&#x64a;&#x645;</bdi>: shelf 4.</p>
```

The first span identifies Yoruba text. The bdi isolates the Arabic name from the English sentence and its punctuation. Language metadata does not replace a correct translation or a test with the intended text. The Arabic name uses numeric character references to keep the printed source unambiguous; the browser still renders the name as Arabic text.

A mistake to watch for

Using CSS direction as the only representation of document language, or assuming every non-Latin script is right-to-left.

Your turn

A visitor submits a name whose writing direction you do not know. What is a reasonable HTML wrapper inside an English sentence? Does wbr add a hyphen or require an immediate new line?

Explained answer

Use bdi, commonly with dir="auto", around the name. Keep the surrounding sentence language separate. Insert the untrusted name as text, not as untrusted HTML. Neither. It offers a breaking opportunity without adding a hyphen or ordinary space; br requests a line break.

Read the source

HTML Living Standard: language and direction

HTML Living Standard: bidi isolation

HTML Living Standard: word-break opportunities

HTML Living Standard: directional overrides

HTML Living Standard: ruby

HTML Living Standard: ruby annotations

HTML Living Standard: ruby fallback

IV / Semantic Website Architecture

22. Containers and grouping

Use a generic container when the content has no more specific grouping meaning.

By the end of this chapter

- Use generic wrappers deliberately
- Avoid adding semantics without a content reason

div is a generic flow container. span is its phrasing-level counterpart. Both are useful. The problem is replacing meaningful elements with them automatically, or adding layers with no clear purpose.

A wrapper for a layout grid may be a div. An independent article can be article. A quoted passage belongs in blockquote. Keep these choices distinct from class names such as card, row and panel, which describe your implementation rather than defining HTML semantics.

Group for a stated reason

Start with the text and controls, then add a wrapper only when layout, scripting or a content relationship needs one. A single element can carry several class tokens. You do not need a separate div for every CSS property.

Do not add a role to every generic container. Many wrappers should remain semantically neutral. Adding unnecessary landmarks or groups can make an interface harder to navigate with assistive technology. When you do add an explicit accessible group, give it an appropriate purpose and name and test that relationship.

A quotation needs a source, not an indentation effect

`blockquote` represents a section quoted from another source. Its `cite` attribute can record the source URL, but does not create a visible attribution. Keep an author's attribution outside the quoted content. If the quote and its attribution form a self-contained figure, the attribution can go in its `figcaption`.

Do not use `blockquote` just to obtain an indented box, and do not invent an expert quotation to give a sample credibility. A quotation can contain permitted paragraphs and other structure; a generic wrapper does not add quotation semantics.

The example

HTML / example 22

```
<div class="contact-grid">
  <section aria-labelledby="visit-title">
    <h2 id="visit-title">Visit the library</h2>
    <p>Ask at reception for directions.</p>
  </section>
  <section aria-labelledby="call-title">
    <h2 id="call-title">Contact the desk</h2>
    <p>Contact details are not configured in this teaching example.</p>
  </section>
</div>
```

The outer div is a layout wrapper. The named sections carry the content organization. A real contact page needs verified contact details; this teaching example deliberately supplies no dialable number.

A mistake to watch for

Naming a div class="section" and assuming it now has section semantics.

Your turn

A wrapper exists only to arrange two existing sections in a grid. Does it need to become a third section?

Explained answer

No. A generic div is appropriate for the layout wrapper. Another section would imply another thematic grouping and should have a content reason.

Read the source

HTML Living Standard: div

HTML Living Standard: block quotations

IV / Semantic Website Architecture

23. Sections that have a subject

A section should explain the document, not a spacing requirement.

By the end of this chapter

- Identify a thematic grouping
- Name useful regions without excessive landmarks

section represents a thematic grouping, normally with a heading. A chapter's requirements, an article's background and a service page's eligibility rules can be sections. A border or a margin does not establish a theme.

A heading can be enough to structure a straightforward document without wrapping every subsection in section. Choose wrappers when they clarify grouping or are needed for the document's relationships. Keep the conventional heading hierarchy readable either way.

Name important regions

A section with an accessible name can become a region landmark in relevant accessibility mappings. Name a region when users benefit from navigating to it, rather than naming every small block. aria-labelledby can reference an existing visible heading.

Source order should remain understandable when layout is removed. Moving columns with CSS does not change the DOM reading order. Review both the visible result and the underlying sequence, especially when the page has multiple related sections.

A heading and an accessible name are not the same mechanism

A visible heading inside a `section` helps readers understand the content, but it does not by itself always give the section a programmatic accessible name. In the example, `aria-labelledby="eligibility"` explicitly names the region from that heading. Without an accessible name, a `section` is not exposed as the same named region landmark under the HTML accessibility mapping.

Do not solve this by labelling every wrapper. Too many landmarks can obscure the useful ones. Review the resulting navigation structure with the intended accessibility tools, not only by counting section tags.

The example

HTML / example 23

```
<section aria-labelledby="eligibility">
  <h2 id="eligibility">Who can join</h2>
  <p>Membership options depend on the service you need.</p>
  <h3>Young readers</h3>
  <p>A guardian can help complete the application.</p>
</section>
```

The section has a theme and a visible heading used as its name. The h3 is a subsection under the h2, not a visual size instruction.

A mistake to watch for

Replacing every div with section as a general cleanup step.

Your turn

A block exists solely to add a background colour around two unrelated widgets. Should it automatically be a section?

Explained answer

No. A div may be the more accurate wrapper. Use section when the content forms a thematic grouping, usually introduced by a heading.

Read the source

HTML Living Standard: section

HTML Accessibility API Mappings: section

IV / Semantic Website Architecture

24. Articles and supporting content

Decide what can stand on its own.

By the end of this chapter

- Distinguish standalone and tangential content
- Use contextual headers and footers

article represents a complete or self-contained composition that could be distributed or reused independently in principle. A news story, a forum post or a self-contained review can qualify. An article can contain its own header, footer and sections.

aside holds content tangentially related to its surrounding content. The element does not mean right column. Related background could appear beneath the article on a phone and still be an aside. address provides contact information for its nearest relevant article or body ancestor; it is not a generic postal-address styling element.

Reuse header and footer carefully

header is introductory or navigational material for its relevant context. footer can contain authorship, related links or other footer information. A footer does not need to sit at the bottom of a viewport.

Keep a page's repeated site header separate from an article's own introduction. If you include dates, author details or related reading, make their relationship clear from the surrounding structure and text. Do not select article merely because a card has a title; ask whether the content is a composition in its own right.

Keep contextual landmarks and contact information distinct

A site-level `header` or `footer` can have a different accessibility mapping from one nested in an `article`, `section`, `nav`, `aside` or `main` context. Do not assume every header creates another banner landmark or every footer creates a site-level content-information landmark. Check both the document context and the mapping.

An `address` is for contact information for the nearest applicable article or body, not for any postal address mentioned in a story. A story describing a venue may simply use ordinary text for its location. The contact block also has content restrictions: it must not contain headings, sectioning content, header, footer or another address. Keep a "Contact" heading outside the address block when needed.

The example

HTML / example 24

```
<article>
  <header>
    <h2>A reading room after school</h2>
    <p>Library programme notes</p>
  </header>
  <p>The room is available for supervised study.</p>
  <footer><p>Contact the library desk for current sessions.</p></footer>
</article>
```

The article includes its own introduction and footer. Their meaning belongs to this composition; a page may also have a separate site-wide header and footer.

A mistake to watch for

Using aside because a design puts something on the right, even when it contains essential main content.

Your turn

Is the only set of instructions for submitting an application appropriate in an aside?

Explained answer

Usually not. Those instructions are part of the main task, not tangential content. Put essential instructions with the form or the relevant main section.

Read the source

HTML Living Standard: articles

HTML Living Standard: aside

HTML Living Standard: address

HTML Living Standard: header

HTML Living Standard: footer

HTML Accessibility API Mappings: header

HTML Accessibility API Mappings: footer

HTML Accessibility API Mappings: body-scoped header

HTML Accessibility API Mappings: body-scoped footer

IV / Semantic Website Architecture

25. Navigation and main content

Give recurring page regions clear jobs.

By the end of this chapter

- Create a clear main region
- Distinguish navigation regions and test a skip link
- Apply the main hidden-state authoring condition

main represents the dominant content of the document body. A normal visible page should not expose several competing main regions. Repeated site navigation and the general site footer ordinarily sit outside main.

nav identifies a major navigation section. Use it for the principal site menu or a substantial table of contents, not every isolated link. If several navigation regions need distinguishing, give them concise names such as Primary and On this page.

Make repeated content easy to pass

A skip link near the beginning can take keyboard users to main content. Its target needs a unique id and should be tested in the browsers you support. Keep the link visible on focus rather than hiding it permanently.

The search element groups controls or content used for searching or filtering. It does not implement a search service by itself. A form inside it can submit a query to a real endpoint. Keep the form's label and response behavior usable without relying on a decorative magnifying-glass icon.

State the main-content requirement precisely

The authoring rule is that a document must not have more than one `main` without a `hidden` attribute. A CSS rule that merely hides another main is not the same condition. When an application switches between main elements, manage their hidden state deliberately.

A main must also be hierarchically correct: its permitted ancestors are limited by the definition. A simple page can keep main as a direct child of body, with articles and sections inside it. Do not put the page's main inside an article or a navigation section and assume a class name repairs the relationship.

`search` can contain a site's search form or filtering controls. It supplies search-related semantics, not a query engine, and it does not replace a label for the actual input. Check current implementation support before relying on a newer element's accessibility mapping.

The example

HTML / example 25

```
<a href="#main-content">Skip to main content</a>
<nav aria-label="Primary">
  <a href="index.html">Home</a>
  <a href="catalogue.html">Catalogue</a>
</nav>
<main id="main-content" tabindex="-1">
  <h1>Library services</h1>
  <p>Choose a service from the catalogue.</p>
</main>
```

The skip destination is programmatically focusable without adding it to ordinary sequential keyboard navigation. Test actual focus and scrolling rather than assuming the fragment provides the full intended experience everywhere.

A mistake to watch for

Giving several visible main elements different class names and treating them as layout columns.

Your turn

How would you distinguish a site menu from a chapter table of contents for landmark navigation? Is CSS-only hiding enough to allow a second main without hidden?

Explained answer

Use separate nav regions and name them, for example Primary and On this page. Keep names concise and avoid repeating the word navigation when the role already communicates it. No. The authoring condition specifically concerns main elements without a hidden attribute, not simply their CSS visibility.

Read the source

HTML Living Standard: main

HTML Living Standard: navigation

HTML Living Standard: search

HTML Accessibility API Mappings: search

IV / Semantic Website Architecture

26. Tables that explain the data

Use a table when the relationships run in rows and columns.

By the end of this chapter

- Build a simple data table with headers
- Recognize when a complex table needs stronger associations
- Resolve explicit header ids and explain a zero row span

A data table relates cells along two axes. Use table, tr and the appropriate th or td cells. A caption identifies the table. thead, tbody and tfoot can group rows, but grouping alone does not create header associations for every complex layout.

For a simple table, use scope="col" on column headers and scope="row" on row headers where appropriate. More complex tables can need explicit id and headers relationships. Simplifying the data layout is often easier for readers than making a deeply spanned table navigable.

Preserve the relationships on small screens

A wide table can live in a labelled, keyboard-reachable scrolling region. Do not replace it with a screenshot. If you change its visual layout, test that header relationships and reading order still make sense.

rowspan and colspan change the cell grid. Incorrect spans can make a table model hard to reason about even when the display appears close enough. Do not use obsolete presentational attributes to recreate an old layout system. Use CSS for table styling and keep HTML focused on the data.

Know what column groups do

`colgroup` describes a group of table columns; `col` describes one or more columns within a group. For two individually described columns, `<colgroup><col><col></colgroup>` goes after any caption and before the row sections. `col` is void. A colgroup using its own `span` must not also contain col children. These elements do not replace the header cells that explain the data.

Make an explicit relationship inspectable

The example gives the Days header `id="period-days"` and the General lending header `id="collection-general"`. For a case needing explicit relationships, `<td headers="collection-general period-days">14</td>` names those two headers. Each token must identify a `th` in the same table, and ids must still be unique in the tree. The attribute records relationships, not visible header text.

`colspan` author values run from 1 to 1000. `rowspan` permits 0 to 65534; zero means all remaining rows in the row group, not an unlimited span across the whole table. Spans must not overlap cells. A cell's `cellIndex` is its position in the row's cells collection, not necessarily its visual column after spans are considered.

A correct-looking grid and valid header references are not a screen-reader test. Verify the actual header announcements and navigation in the intended browser and assistive-technology combination.

The example

HTML / example 26

```
<table>
  <caption>Lending periods</caption>
  <thead><tr><th scope="col">Collection</th><th id="period-days" scope="col">Days</th></tr><
/thead>
  <tbody>
    <tr><th id="collection-general" scope="row">General lending</th><td>14</td></tr>
    <tr><th scope="row">Reference desk copies</th><td>0</td></tr>
  </tbody>
</table>
```

Each value has a collection row header and a Days column header. The numbers are fictional teaching data, not actual library policy.

A mistake to watch for

Using a table for an entire page layout, or using bold td cells where actual th headers are required.

Your turn

Which two headers should describe the value 14 in this table? Which ids express those relationships explicitly, and does rowspan="0" cover later row groups?

Explained answer

General lending and Days. The row header supplies the category and the column header supplies the quantity. The caption identifies the table as lending-period data. headers="collection-general period-days" refers to the corresponding th ids in this table. rowspan="0" extends through the remaining rows of the current row group, not later groups.

Read the source

HTML Living Standard: tables

HTML Living Standard: column groups

HTML Living Standard: columns

HTML Living Standard: cell attributes

HTML Living Standard: header association algorithm

WAI tables tutorial: irregular headers

IV / Semantic Website Architecture

27. Plan a small site

Page structure should survive the trip between documents.

By the end of this chapter

- Assign a purpose to each page
- Test paths under a deployment subdirectory

Before writing a multi-page site, list the pages and the task each page supports. Give every document a distinct title and a useful main heading. Repeated navigation should remain consistent without making every page's content identical.

Use relative links deliberately. A path from pages/contact.html to the home page differs from the same path in index.html. A root-relative path beginning with a slash can break when the site is hosted under a repository subpath on GitHub Pages.

Work from a content map

For the library, start with Home, Catalogue, Membership and Contact. Keep policy wording in one maintained location and link to it. If pages repeat the same structure, use a build template rather than manually copying it indefinitely.

A navigation link with aria-current="page" identifies the current page. Keep that state correct on each document. Do not rely only on colour to mark it visually. Test every relative path from the built output and from the eventual deployment base path.

Distinguish a deployment prefix from an origin root

For a page at `/book/pages/contact.html`, `../index.html` targets `/book/index.html`, while `/index.html` leaves the book prefix. This is a general URL-resolution issue, not a reason to switch hosting providers. Test the actual compiled links under the chosen base path, and repeat that test after changing any base element or routing convention.

`aria-current="page"` communicates the current page; it is not a disabled state. A current-page link may remain a working link. Keep its destination, state and visible indication consistent, and do not add `aria-disabled` merely because a navigation item is current.

The example

HTML / example 27

```
<nav aria-label="Primary">
  <a href="index.html">Home</a>
  <a href="catalogue.html" aria-current="page">Catalogue</a>
  <a href="membership.html">Membership</a>
  <a href="contact.html">Contact</a>
</nav>
```

The current-page state belongs to the catalogue link on this page. A shared template needs page-specific information to put that state on the right link.

A mistake to watch for

Assuming a site that works at localhost / will work unchanged at /repository-name/ when links begin with a slash.

Your turn

From pages/contact.html, what relative path reaches index.html one directory above?

Explained answer

Use ../index.html. The double-dot segment moves to the parent directory. Check the built link rather than guessing from the source template path.

Read the source

HTML Living Standard: URLs

HTML Living Standard: hyperlinks

WAI-ARIA: aria-current

V / Images, Audio, Video and Embedded Content

28. Images with a purpose

The alternative depends on why the image is there.

By the end of this chapter

- Choose alternatives from purpose and context
- Use dimensions and loading hints deliberately
- Distinguish resource success from the image complete flag

An img embeds an image. Its src identifies a resource; alt supplies a text alternative according to the image's purpose. Useful alternative text requires the actual image and its context. A filename or a generic image description is not enough evidence.

For a purely decorative image, alt="" can keep it out of the ordinary text alternative. Do not omit alt as a casual way to say decorative. A functional image in a link needs an alternative that communicates the link's purpose if no other accessible wording provides it.

Reserve the space

Provide width and height when you know the intended image dimensions or ratio. CSS can still size the rendered image responsively. Reserving an aspect ratio helps avoid layout movement as the resource loads.

Use loading="lazy" for suitable off-screen images, not automatically for the likely largest above-the-fold image. decoding and fetchpriority are hints with defined behavior and implementation limits. Measure before claiming a performance improvement. A broken src and a valid alt are different tests: an alternative does not make a missing image resource successful.

Inspect the resource as well as the attribute

The running example now uses the supplied teaching schematic. Compare the reading room, reception and lift with the alternative before adopting its wording. A logo used as the only content of a home link needs to name that destination; a decorative divider can have an empty alternative. Those decisions come from purpose, not file format.

`complete` can be true for a broken image as well as a successfully fetched one. Check the image's actual state and natural dimensions rather than declaring success from that flag alone. `decode()` returns a promise for decoding; it can reject with an `EncodingError` when the image cannot be decoded. Handling that rejection is different from supplying useful alternative text.

`decoding` permits `sync`, `async` and `auto`; missing or invalid values select Auto. `fetchpriority` permits `high`, `low` and `auto`, with Auto as its missing and invalid default. These preferences do not guarantee a loading order or a performance gain. Lazy loading is a separate policy and should fit the image's position and purpose.

`crossorigin="anonymous"` requests CORS-enabled fetching without cross-origin credentials. It does not make a server grant access. Drawing a cross-origin image into a canvas without the required CORS access can make the bitmap unavailable for pixel readback. Chapter 33 explains that security boundary.

The example

HTML / example 28

```
<img src="../assets/library-floor-plan.svg"
     alt="The reading room is left of reception; the lift is opposite."
     width="640" height="360" decoding="async">
<p>Fictional teaching schematic, not directions to real premises.</p>
```

The supplied original schematic has a 640 by 360 view. Its alternative gives the two spatial relationships relevant here. The decoding preference is a hint, not a measured performance result. The same image could need different wording in a different context.

A mistake to watch for

Writing alt="image" or alt="photo" while leaving out the information the image contributes.

Your turn

An image repeats the adjacent text label inside the same link and adds no other information. What alt value may be appropriate? Does img.complete alone prove that its resource decoded successfully?

Explained answer

An empty alt can avoid repeating the link label, provided the visible text already supplies the required name and the image is genuinely redundant. Review the complete link and context rather than applying this to all linked images. No. A broken image can also be complete. Inspect the actual loading/decoding result and handle failures; alternative text remains a separate requirement.

Read the source

HTML Living Standard: images

WAI images tutorial

HTML Living Standard: image decoding

V / Images, Audio, Video and Embedded Content

29. Responsive image choices

Offer candidates without pretending the browser follows a simple width switch.

By the end of this chapter

- Describe candidate widths and slot size
- Separate resolution switching from art direction
- Check that the declared slot matches the implemented CSS

srcset gives the browser image candidates. Width descriptors describe candidate resource widths; sizes describes the expected rendered slot width for selecting among width candidates. A misleading sizes value can cause an unnecessarily large or blurry choice.

picture is useful for art direction or format alternatives. Its source elements appear before the fallback img. The img remains required for the actual image and its alternative text. A source inside picture has rules different from a source inside audio or video.

Match the markup to the layout

If the slot is full width on small screens and half the viewport on wider screens, describe that relationship accurately. Keep a src fallback and test actual selections in the network panel at different viewport sizes and device pixel ratios.

Do not write a fixed candidate table and promise a particular resource in every browser condition. Selection can account for implementation and device considerations. For art direction, inspect that each crop still communicates the intended content and that the single alternative text remains appropriate.

Read the descriptors literally

A `640w` descriptor says that the candidate resource has an intrinsic width of 640 pixels. It is not a breakpoint and does not mean "use this file on a 640-pixel screen." A `2x` descriptor expresses pixel density instead. Do not mix width and density descriptors in one candidate list.

With width candidates, `sizes` describes the intended image slot, not the width of the image file. It helps resource selection; it does not set the CSS layout. Check that the layout and the declared slot agree. When testing, inspect the image's `currentSrc` as well as its rendered dimensions. A previously cached resource can affect what you observe, so one resize is not a complete loading test.

Keep the two source contexts separate

A `source` in `picture` uses `srcset`, not the media-resource `src` attribute. Source elements precede the image; the image still owns the alternative. The `type` can let the browser skip an unsupported format and the `media` can select a different composition. A source is not an image on its own.

For a lazy image that meets the auto-sizes conditions, a value such as `sizes="auto, (max-width: 40rem) 100vw, 50vw"` can use the concrete rendered width, with explicit sizes as fallback. `auto` must be first and is not a general replacement for sizes on eager images. Keep dimensions or a deliberate CSS size so the

image does not depend on a misleading default box.

Resolution candidates are not an excuse to skip checking the crops. A portrait crop that removes the lift cannot keep an alternative describing the lift as if it were still visible. Use `currentSrc` to inspect selection, but remember that natural dimensions can be density-corrected rather than a direct report of the file's pixel width.

The example

HTML / example 29

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width,initial-scale=1">
  <title>Responsive schematic</title>
  <style>
    body { margin: 0; }
    img { display: block; width: 50vw; height: auto; }
    @media (max-width: 40rem) { img { width: 100vw; } }
  </style>
</head>
<body>
  <h1>Library layout: teaching schematic</h1>
  <img src="../assets/media/floor-plan-800.png"
       srcset="../assets/media/floor-plan-400.png 400w,
               ../assets/media/floor-plan-800.png 800w,
               ../assets/media/floor-plan-1200.png 1200w"
       sizes="(max-width: 40rem) 100vw, 50vw"
       alt="The reading room is left of reception; the lift is opposite."
       width="1200" height="675">
</body>
</html>
```

The three supplied raster files have the advertised intrinsic widths and the same 16:9 composition. The CSS actually implements the slot described by sizes. A browser may choose different resources under different device or caching conditions; the example does not promise an exact filename for each viewport.

A mistake to watch for

Using width descriptors with an inaccurate sizes value, or believing srcset is a guaranteed breakpoint-to-file mapping.

Your turn

When is picture more appropriate than simply adding more width candidates to one img? Does sizes set the rendered width of the image?

Explained answer

Use picture when the source itself should change, such as a different crop for a narrow layout or a supported format alternative. Use width candidates when the same image composition is available at different resource widths. No. The CSS determines layout; sizes informs resource selection. In the complete example the two descriptions agree, and all three resources exist.

Read the source

HTML Living Standard: responsive images

HTML Living Standard: picture

HTML Living Standard: source contexts

HTML Living Standard: sizes syntax

HTML Living Standard: srcset syntax

V / Images, Audio, Video and Embedded Content

30. Audio, video and tracks

Playable media needs controls and usable alternatives.

By the end of this chapter

- Provide controls and intentional loading hints
- Distinguish timed text and transcript needs
- Distinguish track-kind defaults from caption accuracy

audio and video embed media. Provide controls unless an appropriate alternative interface supplies the required operation. Autoplay is subject to browser policies and user choices; adding autoplay is not a guarantee that playback begins.

source provides alternate media resources with relevant type information. track supplies timed text such as captions, subtitles or descriptions using a supported format, commonly WebVTT. Captions and subtitles serve different purposes, so choose the kind accurately.

Plan the alternative before publishing

A transcript, captions and an audio description each address different access needs. The correct combination depends on the media and the information it conveys. A decorative silent clip and an instructional interview do not have identical requirements.

Include a poster for suitable video content and choose preload with an understanding that it is a hint. Third-party players add privacy, performance and embedding-policy concerns. Test keyboard playback controls, caption selection, mobile behavior and fallback links with the actual resources. Placeholder filenames prove nothing about media decoding or caption accuracy.

Be exact about timed text and source selection

The five track kinds are `subtitles`, `captions`, `descriptions`, `chapters` and `metadata`. Missing `kind` selects Subtitles; an invalid value selects Metadata, not Captions. `srclang` is required for a subtitle track and uses a valid language tag. A `default` track is a preference when the user's choices do not select something more appropriate, not a guarantee that it appears.

Only one subtitles-or-captions track per media element may have `default`. Separate limits apply to default descriptions and chapters. Do not mark every translation default. A loaded track and a reviewed caption are different achievements: inspect timing, speech, speaker identification and relevant non-speech sounds against the actual media.

An `audio` element can use controls and alternate source children too. It has no video picture to describe, but meaningful prerecorded speech still needs an appropriate text alternative. `muted` initializes the default-muted state; the current `muted` property can change afterward. `loop` requests repetition. `playsinline` concerns inline video presentation, not a permission to autoplay.

For media elements, a source's `media` condition is evaluated during resource selection, not continuously as a responsive picture reacts to environment changes. A nonempty media `src` takes precedence over source-child selection. Unsupported decoding needs an ordinary usable download or alternative, not only a confident MIME label.

The local fixture contains no speech and is deliberately small. Its descriptive transcript is not a claim that transcripts replace audio description for every audiovisual lesson. A real publication needs a media-specific accessibility plan and review.

The example

HTML / example 30

```
<video controls width="640" height="360" preload="metadata"
       poster="../assets/library-floor-plan.svg">
  <source src="../assets/media/schematic-demo.webm" type="video/webm">
  <source src="../assets/media/schematic-demo.mp4" type="video/mp4">
  <track kind="captions" src="../assets/media/schematic-demo.en.vtt"
         srclang="en" label="English" default>
  <p><a href="../assets/media/schematic-demo.mp4">Download the demo</a>.</p>
</video>
<p><a href="../assets/media/schematic-transcript.html">Read the descriptive transcript</a></
p>
```

The supplied four-second fixture shows the fictional schematic and sounds a short tone, with no speech. The caption labels that sound; the descriptive transcript gives the visual information. Loading the cue and decoding the video are testable, but do not certify real-world media accessibility or make a transcript sufficient for every lesson.

A mistake to watch for

Treating auto-generated captions as reviewed simply because a track file loads.

Your turn

Why put the transcript link outside the video element as well as providing fallback content? Does default on every caption translation make all of them display?

Explained answer

Fallback children are not a reliable always-visible location in browsers that support video. An ordinary external paragraph gives every reader a discoverable transcript link. No. User preferences affect activation, and only one subtitles-or-captions track may be default for that media element. Caption accuracy still needs review against the actual content.

Read the source

HTML Living Standard: media

HTML Living Standard: tracks

HTML Living Standard: source selection contexts

WAI: making audio and video accessible

V / Images, Audio, Video and Embedded Content

31. Image maps

Coordinate-based hotspots need an equivalent way to navigate.

By the end of this chapter

- Connect a map and its areas
- Review geometry and alternatives against the actual image
- Verify a hotspot against a supplied image and a real target

An image map associates areas of an image with hyperlinks. img uses usemap to refer to a map. area supplies a shape, coordinates, destination and appropriate alternative text for each linked region.

These controls are tied to the image's coordinate system. A design that resizes or changes the image needs careful testing. For many interfaces, ordinary links beside a diagram are easier to maintain and understand.

Explain the destinations

Give each linked area an alt that communicates its purpose. A geographic or building diagram should not require readers to identify a shape visually before they can follow a link. Provide visible text links when they make the choices clearer.

Check keyboard focus, activation and the relationship between the displayed image and the hotspot geometry. The existence of an area element does not prove that a visitor can find the right destination or that the coordinate mapping still matches a responsive design.

Measure coordinates in the displayed geometry

A rectangular area uses left, top, right and bottom coordinates; a circle uses centre x, centre y and radius; a polygon uses pairs of x and y values. `shape="default"` covers the image rather than taking coordinates. The coordinates are CSS-pixel distances in the defined image-map model, not a promise of automatic proportional scaling when CSS changes the image's size.

In this fixed-size example, the hotspot aligns with the supplied drawing. If you make the image fluid, verify the new geometry and activation at each relevant size rather than assuming that these coordinates adapt. The ordinary text link keeps the destination available without finding a hotspot.

A map needs a nonempty name unique among maps in the document; if it also has an id, that id must equal the name. A linked area needs destination-specific alternative text. An area without `href` is not the same hyperlink. `ismap` on an image inside a hyperlink is a different, server-side coordinate mechanism, not a replacement for these named client-side areas.

The example

HTML / example 31

```
<img src="../assets/library-floor-plan.svg"
     alt="Teaching plan: reading room left of reception, lift opposite"
     usemap="#facilities" width="640" height="360">
<map name="facilities" id="facilities">
  <area shape="rect" coords="26,240,290,322" href="#reading-room"
        alt="Reading room details">
</map>
<p><a href="#reading-room">Reading room details</a></p>
<section id="reading-room">
  <h2>Reading room</h2>
  <p>This fictional schematic places the reading room left of reception.</p>
</section>
```

The map name matches usemap, and its id matches the name. The rectangle encloses the reading-room box in the supplied 640 by 360 schematic. Both the area and the visible link reach the same real fragment target. This is not a map of actual premises.

A mistake to watch for

Copying hotspot coordinates to a different image and assuming the clickable regions still match.

Your turn

What needs to change when a replacement floor-plan image moves the reading room to another location? If CSS halves this image, may you assume the unchanged hotspot still aligns?

Explained answer

Review the area geometry against the new image, update coordinates and alternatives as necessary, and retest navigation. Also inspect the visible link wording and any directions that referred to the old layout. No. Review the displayed image-map geometry and retest pointer and keyboard activation. Keep the visible text destination usable as well.

Read the source

HTML Living Standard: image maps

V / Images, Audio, Video and Embedded Content

32. Embedding another document

An iframe introduces another browsing context and another trust decision.

By the end of this chapter

- Identify an embedding trust boundary
- Choose sandbox permissions from actual requirements
- Explain srcdoc precedence without treating it as sanitization

iframe embeds another document. Give it a useful title so readers can identify the embedded content. A title does not fix the embedded document's own accessibility problems.

sandbox restricts capabilities. The tokens you add restore particular capabilities, so start from the actual requirements and grant as little as the integration needs. Combining allow-scripts with allow-same-origin can weaken the intended isolation for same-origin content. Do not describe sandbox as a universal security wrapper.

Consider the resource owner

Third-party embeds may load tracking, cookies, scripts and further resources. Review privacy, origin policy, referrer policy and the allow attribute's Permissions Policy integration. A provider's sample embed code is a starting point, not a completed review.

object and embed serve other embedding cases and bring their own fallback and implementation concerns. param is obsolete for authoring in the captured Standard and belongs in migration guidance. Do not add it to new object examples because an old tutorial used it.

srcdoc changes the source, not the trust decision

When present, `srcdoc` takes precedence over `src` and creates an `about:srcdoc` document. Relative URLs normally use the embedding document's base unless the embedded document establishes another base. When writing srcdoc into an HTML attribute, escape attribute delimiters and ampersands appropriately. Assigning an untrusted string to the srcdoc property is still an injection-sensitive operation, not text insertion.

An empty sandbox does not itself block every resource request. Use an appropriate CSP and permissions policy for the actual boundary. `allow` delegates selected Permissions Policy features within other applicable restrictions; it does not cancel sandbox flags or make a blocked feature available automatically.

The book's editable lab deliberately blocks nested frames and external media. Run this embedding example from its standalone HTML page when inspecting the notice, rather than weakening the lab policy to make every embed render. The real example's native frame can be read without scripts.

`object` uses `data` for an embedded resource and can contain fallback content; `embed` is void and cannot contain equivalent fallback children. A useful direct link outside either remains available independently. The publication policy blocks object/plugin embedding. Do not interpret that deliberate restriction as proof that a copied object example works in every deployment.

The example

HTML / example 32

```
<iframe title="Quiet-study notice" sandbox=""
        srcdoc="<!doctype html><html lang='en'><head><title>Notice</title></head><body><h1>Q
uiet study</h1><p>Keep calls outside the reading room.</p></body></html>"
        width="640" height="200"></iframe>
```

The static notice is supplied directly with srcdoc, so no missing preview.html is required. The empty sandbox grants no opt-in capabilities. This still does not mean the iframe is a universal no-network environment or that every embedded document is accessible.

A mistake to watch for

Adding every sandbox permission until an embed works, then assuming the original security boundary remains intact.

Your turn

A widget needs only static text and images. Should you add allow-scripts and allow-same-origin by default? If src and srcdoc are both present, which supplies the iframe document?

Explained answer

No. Do not grant capabilities without a demonstrated need. Test the actual static resource under the intended restrictions and document any permissions you choose to add. srcdoc takes precedence. It still parses markup, so neither that precedence nor the sandbox makes arbitrary untrusted HTML a safe text value.

Read the source

HTML Living Standard: iframe

HTML Living Standard: object

HTML Living Standard: embed

V / Images, Audio, Video and Embedded Content

33. Canvas, SVG and MathML

Graphics need a document-level alternative strategy.

By the end of this chapter

- Distinguish bitmap drawing from document semantics
- Describe HTML integration boundaries for foreign content
- Explain why canvas export can fail for a cross-origin bitmap

canvas provides a bitmap drawing surface controlled through APIs. Drawing pixels does not automatically produce semantic headings, text or controls in the accessibility tree. Fallback content and a usable alternative interface need deliberate design.

SVG and MathML are foreign vocabularies that can be integrated into an HTML document. Their own element vocabularies and rules come from their respective specifications. HTML parsing and integration rules still matter at the boundary.

Use the right representation

For a simple diagram, an SVG with a title and a text explanation can be more maintainable than a canvas rendering. For a data chart, an accessible table may communicate the exact values better than an image alone. For meaningful mathematical notation, MathML can express structure that a screenshot loses.

This chapter introduces the HTML integration, not completed Canvas or SVG API coverage. The assigned interfaces remain in the coverage ledger; member-level behavior, conversions, exceptions and tests are still required before the full publication can be final. Do not infer that a graphic is accessible merely because it has a title element. Review its purpose, interactions and the information the alternative actually supplies.

A canvas has a bitmap size and context state

A canvas without dimensions starts with a 300 by 150 bitmap. CSS can change its displayed size without adding bitmap detail. In the 2D context, setting a dimension, even back to the same value, resets the bitmap and rendering state. Resize deliberately, then restore the needed state and redraw from maintained data.

`getContext("2d")` can return a context or null; do not assume every requested mode is supported. A canvas already bound to one context mode does not freely switch to a different one. Check the result before calling drawing methods.

An initially origin-clean bitmap can be tainted by cross-origin image drawing without appropriate CORS access. `toDataURL()` and `toBlob()` then throw `SecurityError`; drawing is not a way to bypass resource-origin restrictions. A usable export feature needs tested CORS behavior and a failure path, not advice to disable browser security.

Preserve the foreign vocabulary's structure

A small MathML example is <math xmlns="http://www.w3.org/1998/Math/MathML"><msup><mi>x</mi><mn>2</mn></msup></math>. Unlike an image of a formula, it expresses an identifier raised to a power. SVG and MathML have their own namespaces, content rules and accessibility mappings. Do not treat every foreign node as an HTMLElement, and do not infer a useful spoken result merely from a title or namespace.

The example

HTML / example 33

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 320 100"
     role="img" aria-labelledby="route-title route-desc">
  <title id="route-title">Route to the reading room</title>
  <desc id="route-desc">Reception connects directly to the reading room.</desc>
  <path d="M40 50 H280" stroke="currentColor" stroke-width="4"/>
  <circle cx="40" cy="50" r="10" fill="currentColor"/>
  <circle cx="280" cy="50" r="10" fill="currentColor"/>
</svg>
```

The graphic has a short title and a description of the relationship. The visible diagram is intentionally simple. A real route plan with turns or hazards would need a more complete alternative.

A mistake to watch for

Replacing an interactive chart with canvas pixels while removing all operable controls and data relationships.

Your turn

Your canvas chart shows exact lending totals. What alternative lets readers inspect those values without interpreting pixels? Does drawing a cross-origin image guarantee that toDataURL can read its pixels?

Explained answer

Provide a text or data-table representation with the actual values and labels. Keep it synchronized with the chart data. If the chart has controls, those also need an accessible operable interface. No. Without the required CORS access, the drawing can taint the bitmap and pixel export throws SecurityError. Provide a supported source and a useful failure path.

Read the source

HTML Living Standard: canvas

HTML Living Standard: foreign elements

MathML Core: math element

SVG 2: document structure

VI / Forms and Native Controls

34. How forms send data

Start with the request, then choose the controls.

By the end of this chapter

- Distinguish name from id
- Explain a form request without inventing a backend
- Inspect repeated entries and submitter participation

A form constructs an entry list from eligible controls and submits it according to its action, method and encoding rules. A visible input without a name usually does not contribute the expected name-value entry. An id connects labels and references; it is not the submission key.

GET commonly expresses retrieval by placing the encoded query in the URL. POST sends a request body under the selected encoding. Neither method provides encryption by itself. Use HTTPS for transport protection where appropriate and implement the server's validation, permissions and storage handling.

Make the simple path work

A search form can submit without JavaScript. Build that path first when the service supports it. Scripts can enhance the experience without becoming the only way to reach the result.

Controls outside a form can be associated through a form attribute when their element supports it. Avoid relying on visual proximity as a submission relationship. Check actual submitted entries, including unchecked controls, disabled fields and the button used to submit. A browser preview without a backend cannot prove a successful application workflow.

Predict the entry list

For an ordinary named text input, the name becomes the entry key and the current value becomes its value. A disabled control is omitted. An unchecked checkbox or radio button is omitted too; it does not automatically send a false value. A readonly text input can still contribute an entry. Repeated names can produce multiple entries, so do not assume that every form is a one-value-per-key object.

Give the server a deliberate interpretation for missing and repeated entries. Do not trust a disabled field, a hidden field or a client-side constraint to enforce a permission. Someone can construct a request without using your interface.

Observe entries instead of guessing from the controls

For a form with two checked controls both named `topic`, `new FormData(form).getAll("topic")` reads both entries. Turning the entries into an ordinary one-value object can lose that relationship. Output elements are associated with forms but do not submit their result values merely because they have names.

`new FormData(form, submitButton)` includes that eligible submitter's entry; constructing it without a submitter omits the submit buttons. A `formdata` event is fired while constructing the list, allowing a handler to inspect or amend its FormData. This is not the cancelable `submit` event and does not itself

send a request.

A named file input with no selected file still has a defined empty File entry, rather than behaving exactly like an unchecked checkbox. A hidden `_charset_` field has special encoding-value behavior. A supporting text control with `dirname="reader.dir"` can contribute an additional direction entry whose value is `ltr` or `rtl`.

The missing and invalid method defaults are GET; the usual missing or invalid enctype default is application/x-www-form-urlencoded. `method="dialog"` has a different purpose: inside a dialog it can close that dialog rather than send an application request. Chapter 41 compares the submission paths.

The FormData constructor and its entry accessors are defined by the XMLHttpRequest Standard; HTML defines how a form constructs its entries. During native construction, trying to construct another FormData from the same form inside its formdata listener hits the construction guard and throws InvalidStateError. Amend the supplied event.formData instead of starting that construction again.

The example

HTML / example 34

```
<form action="catalogue.html" method="get">
  <label for="query">Search the catalogue</label>
  <input id="query" name="q" type="search">
  <button type="submit">Search</button>
</form>
```

The query key is q because that is the name. A real catalogue endpoint must read the query and return results; a static catalogue file does not implement that search service automatically.

A mistake to watch for

Assuming that giving a control an id is enough to send its value under that key.

Your turn

Why might a filled input appear on screen but be absent from the submitted entry list? Will new FormData(form) automatically include every submit button?

Explained answer

It may lack a name, be disabled, be outside the relevant form without an association, or fall under another exclusion rule. Inspect the actual control state and form data rather than the visual layout. No. Buttons are excluded unless the relevant submitter is supplied to entry-list construction. Repeated names remain separate entries; inspect them with getAll when the data model expects several values.

Read the source

HTML Living Standard: forms

HTML Living Standard: submission

HTML Living Standard: constructing the entry list

HTML Living Standard: form methods and encodings

XMLHttpRequest Standard: FormData

VI / Forms and Native Controls

35. Labels and groups

People should know what to enter before making a mistake.

By the end of this chapter

- Associate visible labels with controls
- Group related choices with a fieldset and legend
- Check form-owner boundaries in radio grouping

A label supplies the visible prompt for a form control. An explicit label uses for to reference the control's id. Nesting an appropriate control inside its label can also establish the association under the defined rules.

A placeholder is not a durable label. It can disappear as soon as someone types and often has poor contrast or confusing wording. Use supporting text for instructions, formatting rules and examples, then connect it with aria-describedby where that relationship helps.

Group a shared question

fieldset and legend can express a group such as contact preference or membership type. Each radio still needs its own label. Keep the legend short enough that repeated announcements remain manageable.

Do not place essential instructions only in title or in a tooltip that requires a pointer. Write the instructions visibly. If a field is required, communicate that meaning consistently and explain the service's expectations. Required does not mean the requested information is necessary; review the form's data needs before adding fields.

Association has exact boundaries

An explicit label's `for` must identify a labelable control in the same tree. A hidden input is not labelable. Keep ids unique: a duplicate id can make the browser find the wrong element rather than the intended control. A label must not contain a second unrelated labelable control or another label.

For radio grouping, a shared nonempty name is not the only condition. The radios must also be in the same tree and have the same form owner, or both have no form owner. Two separate forms can each have a `contact` group without making those groups one mutual-exclusion set.

Put the fieldset's legend first. A disabled fieldset disables its descendant controls except those inside its first legend child, subject to any other disabling conditions. Its `form` attribute does not automatically associate every descendant control with that form; controls outside a form need their own applicable association. Check `control.form` rather than assuming a group caption or layout sets the owner.

The example

HTML / example 35

```
<fieldset>
  <legend>How should we contact you?</legend>
  <label><input type="radio" name="contact" value="email"> Email</label>
  <label><input type="radio" name="contact" value="phone"> Phone</label>
</fieldset>
```

Here the two radios share the same nonempty name, form owner and tree, which creates the radio group for selection behavior and submission. The legend supplies the common question. Each label names its own option.

A mistake to watch for

Using two separate names for radio options that should be mutually exclusive.

Your turn

Why does a legend not remove the need to label each radio button? Are same-named radios in two separate forms necessarily one group?

Explained answer

The legend names the group question. Each radio still needs to identify its particular answer, such as Email or Phone. The reader needs both pieces of information. No. Their form owners differ. The grouping conditions also include the same tree and a matching nonempty name; common wording on screen is not sufficient.

Read the source

HTML Living Standard: labels

HTML Living Standard: fieldset

WAI forms tutorial

HTML Living Standard: radio group conditions

HTML Living Standard: legend

VI / Forms and Native Controls

36. Text, email and passwords

Choose a state for the data, not just the keyboard.

By the end of this chapter

- Choose a text-related input state
- Distinguish syntax validation from verification
- Distinguish telephone input mechanisms from syntax enforcement

input type="text" is the general single-line text state. email and url have their own syntax and validation behavior. search can supply search-oriented interface behavior. password obscures the displayed value but does not encrypt it, protect it from scripts on the page or secure its eventual storage.

Use autocomplete tokens that describe the field's purpose. An existing-password field and a new-password field are different. Permit password managers and pasting rather than introducing unnecessary obstacles.

Do not promise more validation than you have

An email input checks its defined syntax, not whether the address belongs to someone or can receive messages. A service that needs ownership verification must implement a separate confirmation process.

inputmode is a hint about input mechanisms, not a data validator. A numeric keyboard hint does not turn a text input into an integer or stop pasted characters. Pick constraints from the service's actual requirements, and keep server-side validation independent from the client interface.

A telephone or URL is not an arbitrary number

`tel` is a single-line telephone input state with no universal telephone-number syntax constraint. It may offer a telephone-oriented keyboard. An application-specific pattern still needs visible instructions and must not pretend that one local format accepts all international numbers.

A URL-state input checks its defined absolute-URL syntax; it does not prove that a destination is trustworthy or reachable. An email input with `multiple` can accept a comma-separated list under the email-state rules. Neither syntax check verifies ownership, permission or message delivery.

For password-related autocomplete, use `current-password` for an existing credential and `new-password` for a replacement or creation field. Purpose tokens, and any applicable grouping tokens, have a defined order. `autocomplete="off"` is not a promise that every browser will suppress credential assistance.

Text-state values do not automatically trim every space. Email and URL states have their own sanitization rules, and a pasted newline is not the same thing as a multiline textarea. Inspect the resulting value before claiming that a field preserves arbitrary input exactly.

The example

HTML / example 36

```
<label for="join-email">Email address</label>
<input id="join-email" name="email" type="email" autocomplete="email"
       aria-describedby="email-help" required>
<p id="email-help">We use this address to contact you about your application.</p>
```

The helper describes this fictional form's intended use. Replace it with the actual service policy before collecting real addresses. The control cannot prove that a user controls the supplied mailbox.

A mistake to watch for

Calling a password secure because its characters appear as dots, or disabling paste to discourage password managers.

Your turn

Will type="email" prove that an address can receive your confirmation message? Does type="tel" enforce every international telephone format?

Explained answer

No. It applies the email state's syntax rules. Message delivery and ownership are application concerns that need their own process and error handling. No. It supplies a telephone-oriented control without a universal syntax validator. Define any service-specific rules visibly and validate them separately on the server.

Read the source

HTML Living Standard: input states

HTML Living Standard: telephone input

HTML Living Standard: email input

HTML Living Standard: URL input

HTML Living Standard: autofill

VI / Forms and Native Controls

37. Numbers, dates and colour

A control can normalize a value without deciding what it means.

By the end of this chapter

- Distinguish quantities from numeric-looking identifiers
- Interpret local dates without inventing timezones
- Handle numeric conversion and date-state API boundaries

number fits quantities where numeric operations make sense. Do not use it for every string made of digits. A membership identifier or postal code may need leading zeros, nonnumeric characters or a format that should not change.

min, max and step affect constraints for relevant input states. A range control suits selecting within a range when approximate adjustment is appropriate. Give the value a meaningful visible context; a bare slider can leave readers unsure what they are choosing.

Dates need a data model

date supplies a date without a timezone. datetime-local supplies a local date and time, also without an offset or timezone identifier. An appointment service needs a clear policy for interpreting that value. The displayed picker can vary across browsers while the submitted value follows defined syntax.

Colour controls and the newer alpha/colorspace attributes need their own implementation review. Do not assume every browser presents the same picker or supports every standardized extension. Keep a usable alternative when exact colour input is important.

Keep each temporal value in its own unit

`month` represents a year and month, such as `2026-10`; `week` represents an ISO week, such as `2026-W41`; `time` represents a time of day, such as `09:30`. These are not complete global timestamps. Check current picker support and provide visible format guidance rather than assuming every platform supplies the same interaction.

Step units depend on the state: a date uses days, a month uses months, a week uses weeks, and time and datetime-local use seconds. The step base comes from the applicable min, then the value content attribute, then the state's default base; week has a special default base. A valid-looking value can still have stepMismatch if it does not align with that base. `step="any"` removes the step-mismatch constraint where it applies; it is not permission for arbitrary syntax.

`valueAsNumber` returns NaN when no usable number exists, not zero. Its setter rejects infinity with TypeError, and rejects unsupported states with InvalidStateError. Setting NaN clears the applicable input. `valueAsDate` is state-specific too: date-based conversions use their defined UTC conventions, while datetime-local does not support that API. A date's calendar information is not a user's local appointment timezone.

A range control has a default range of 0 to 100 and a sanitized default value near its midpoint. It is not an empty optional number box. Keep its current value visible and label both what the quantity means and its units.

Name newer colour behavior precisely

In the current Standard, `alpha` is boolean. `colorspace` permits `limited-srgb` and `display-p3`; missing and invalid values select Limited sRGB. Without alpha, serialization makes the color opaque. The state has sanitization rules, including an opaque-black fallback when parsing fails. These are specification rules, not evidence that every tested browser implements the extensions. Do not require a new picker feature without reviewing a usable fallback.

The example

HTML / example 37

```
<label for="copies">Number of copies</label>
<input id="copies" name="copies" type="number" min="1" max="5" step="1">
<label for="visit-date">Preferred visit date</label>
<input id="visit-date" name="visitDate" type="date">
```

The quantity has integer steps within a range. The preferred date is not an instant on a global timeline and needs application interpretation before it becomes a scheduled event.

A mistake to watch for

Using number for a phone number and losing formatting or leading-zero expectations.

Your turn

Why is a phone number often better represented by type="tel" than type="number"? What does an empty number input return from valueAsNumber?

Explained answer

A phone number is an identifier with possible leading zeros, plus signs and formatting conventions. It is not a quantity to increment or average. tel can offer a useful input mechanism without pretending to validate every international number. NaN, not zero. Check for it explicitly; also handle the state-specific exceptions and do not interpret a timezone-free date as a scheduled global instant.

Read the source

HTML Living Standard: numeric and temporal controls

HTML Living Standard: month

HTML Living Standard: week

HTML Living Standard: time

HTML Living Standard: number conversion

HTML Living Standard: date conversion

HTML Living Standard: colour

VI / Forms and Native Controls

38. Choices, files and hidden values

Submission depends on the control state.

By the end of this chapter

- Predict submitted entries for checked controls
- Treat files and hidden values as untrusted input
- Distinguish checked defaults from current state and file-value restrictions

Checkboxes express independent choices. Radios with a shared name express a mutually exclusive selection. A checked checkbox or radio contributes its value under the defined entry-list rules; an unchecked one normally contributes no entry. Plan that absence on the server.

A file input lets a user choose files. accept is a hint to the chooser, not proof that a file is safe or of the declared type. File size, content, storage name and permissions need server-side checks. Do not serve uploaded files as executable content by accident.

Hidden is not private

A hidden input participates in form submission without an ordinary visible control. Its value remains in the document and can be changed by a visitor. A server must not trust a hidden price, role or authorization decision merely because the page supplied it.

Likewise, readonly and disabled are different states for supporting controls. A readonly text control can still contribute its value; a disabled control is generally excluded from the entry list. Neither state is a substitute for enforcing business rules on the server.

Predict file and checked-state behavior

If a checked checkbox has no explicit value, its default submission value is `on`, not true. `checked` in markup supplies the default checked state; the current `checked` property can differ after interaction, and `defaultChecked` records the markup default. Resetting a form restores defaults rather than proving that an application has rolled back a server action.

For a file input, read the File objects from `files`; the displayed value is not a usable path to arbitrary files on the visitor's machine. Assigning a nonempty value string programmatically throws InvalidStateError. A script can clear the selection, but cannot use that value setter to choose a private local file.

`multiple` allows more than one file; each selected file contributes an entry under the same name. The extension and File.type are not authoritative proof of the content. Apply server-side size, format, storage and access rules to the actual upload. A readonly attribute is not a valid way to make a file picker immutable; state-specific applicability matters.

The example

HTML / example 38

```
<label>
  <input type="checkbox" name="updates" value="yes">
  Email me the reading-group schedule
</label>
<label for="attachment">Supporting document</label>
<input id="attachment" name="attachment" type="file" accept=".pdf">
<input type="hidden" name="formVersion" value="1">
```

The hidden value is a routing hint for the application, not a trusted permission. A surrounding file-upload form needs the appropriate multipart encoding and a real upload handler.

A mistake to watch for

Expecting an unchecked checkbox to submit the literal value false.

Your turn

How should a server interpret the absence of an optional updates checkbox? What does a checked checkbox submit when it has no value attribute?

Explained answer

Apply the documented default for the field, commonly not opted in. Do not infer consent from a missing value. Validate any submitted value as well, since clients can send requests without using the page. Its default value is on. Also, assigning a nonempty filename to a file input value is not a file-selection API; it throws InvalidStateError.

Read the source

HTML Living Standard: checkboxes

HTML Living Standard: file upload

HTML Living Standard: hidden state

HTML Living Standard: input value modes

OWASP: file-upload defenses

VI / Forms and Native Controls

39. Menus and longer text

Choose the control that fits the number and kind of answers.

By the end of this chapter

- Use select, datalist and textarea intentionally
- Keep a native baseline for newer select customization
- Distinguish selected option content from its label metadata

select represents a choice among option elements, with optgroup available to group related choices. A native select carries behavior that a custom visual menu would otherwise need to recreate. Use a visible label and make any initial prompt consistent with required-selection rules.

textarea is for multiline text. Its initial value comes from its text content, not a value attribute. A datalist can supply suggestions for an associated input, but suggestions do not normally restrict the user to the listed values.

Customizable selects need extra review

The Living Standard includes selectedcontent and newer customizable-select behavior. Their permitted structure and browser support need separate attention. Keep a conventional select usable as the baseline rather than assuming a decorative demonstration proves full keyboard and assistive-technology support.

multiple changes both the interface and submission expectations for a select. Do not use a multiple list solely to imitate the visual height of a different widget. Explain how to choose and how the service interprets several selected values.

A placeholder option has a defined position

For the required single-select, display-size-one context used here, the placeholder label option is the first option with an empty value and no intervening optgroup ancestor. Moving the prompt into an optgroup can change whether it meets that rule. A nonempty displayed phrase alone does not establish an allowed submitted value.

Give conventional optgroups a label. The newer model also permits a first-child legend under its rules, but that needs implementation review rather than an assumption that every native picker renders arbitrary group markup. Set explicit option values when the service needs stable keys; otherwise the option's defined text processing supplies the value.

`selected` initializes an option's default selected state. The current selected property can change without rewriting that attribute. In a multiple select, several selected options produce repeated entries under the same name. For a textarea, `defaultValue` comes from its child text, while `value` is the current value and uses normalized LF line breaks. `wrap="hard"` requires cols and can insert submission line breaks; the usual Soft state does not request that hard wrapping.

Explain selectedcontent, not just its name

In a supported customizable drop-down, the select may start with a button containing <selectedcontent></selectedcontent>. The element has no authored children: the browser clones the selected option's child content into it. An option's label attribute is not that cloned content, and the update algorithm does not use this mechanism for a multiple select.

For example, <select name="shelf"><button><selectedcontent></selectedcontent></button><option value="fiction">Fiction</option><option value="history">History</option></select> shows the intended structure; add a visible associated label in a complete form. The customizable rendering opt-in also involves CSS. Support cannot be inferred from successfully retaining an unknown element in the DOM. Keep the choice values usable in a conventional native baseline, avoid ids that become duplicate relationships when content is cloned, and test real keyboard and assistive-technology behavior.

The example

HTML / example 39

```
<label for="branch">Preferred branch</label>
<select id="branch" name="branch" required>
  <option value="">Choose a branch</option>
  <option value="central">Central</option>
  <option value="east">East</option>
</select>
<label for="request">What would you like to read?</label>
<textarea id="request" name="request" rows="4"></textarea>
```

This is a conventional select baseline. Its empty initial option works with the required single-select context shown here. Check the detailed placeholder-label rules before changing the structure.

A mistake to watch for

Assuming datalist creates a strict allowed-value list or that textarea uses value like an input.

Your turn

A field must accept only Central or East. Does an input with datalist enforce that restriction? Does selectedcontent mirror an option label attribute or move its original child nodes?

Explained answer

No. A datalist provides suggestions. Use a suitable select for this choice and validate the submitted value on the server. A client can send any value regardless of the visible controls. Neither. Its defined update clones the selected option child content, not the label attribute. A retained unknown tag alone is not proof that the browser implements those updates.

Read the source

HTML Living Standard: select

HTML Living Standard: selectedcontent

HTML Living Standard: textarea

HTML Living Standard: options

HTML Living Standard: option groups

HTML Living Standard: datalist

VI / Forms and Native Controls

40. Validation people can recover from

An error message should help someone finish the task.

By the end of this chapter

- Apply state-specific constraints
- Design and test the correction path
- Choose output, progress and meter without inventing a running service

Native constraints include required, appropriate type rules and state-specific min, max, step, pattern and length restrictions. Their applicability differs by input state. A constraint in the wrong state may do nothing.

Constraint validation helps guide ordinary interaction. It does not secure a request. A server must validate the submitted data, enforce permissions and return useful errors even when JavaScript or browser validation has been bypassed.

Keep the correction near the field

When application validation finds an error, identify the field and explain how to fix it. Preserve the submitted non-sensitive values when appropriate. Connect useful error text with aria-describedby and update aria-invalid when the application knows the value is invalid. Do not mark a field invalid before there is a reason to do so.

setCustomValidity can add an application message to supporting controls; a nonempty message keeps the control invalid until cleared. A mismatch between custom validity and visible errors can trap someone in a form. Test correction, not just failure. Native error UI varies between browsers, so do not base instructions on one screenshot.

Know what the constraints actually measure

Pattern matching uses the JavaScript Unicode-sets `v` mode and checks the whole nonempty value, not a substring. Do not include slash delimiters. An invalid pattern has no compiled expression, so testing one successful string is not enough to prove that the intended rule is enforced. `required` handles the otherwise-empty value separately.

Length constraints use the defined API value length, measured in UTF-16 code units rather than user-perceived characters. Their tooShort and tooLong conditions have user-edit and dirty-value requirements; a script assignment is not an equivalent way to test all typing constraints. Test real edits and the correction path.

`checkValidity()` checks applicable constraints and can fire invalid events on failing controls. `reportValidity()` additionally uses the browser's reporting behavior. The invalid event does not bubble like an ordinary input event; a form-level listener can use capture when it needs those events. Keep error messages meaningful instead of promising identical native popup text everywhere.

Results and gauges are not extra submission fields

`output` represents a calculation or action result. For example, `<output for="copies" name="copyResult">2 copies selected</output>` relates a result to the copies input from Chapter 37. Its for value is a unique-token list of element ids in the same tree, not the label element's one-control rule. An output value is not submitted merely because name is present, and it is barred from constraint validation.

`progress` represents task completion: `<label for="catalogue-progress">Records indexed</label><progress id="catalogue-progress" max="10" value="3">3 of 10</progress>` has a determinate value. Remove value for indeterminate progress. The default maximum is 1; an explicit maximum must be positive, and a determinate value must be within the permitted range.

`meter` is a measurement within a known range, not a task's completion. `<label for="seats">Seats occupied</label><meter id="seats" min="0" max="20" value="12">12 of 20 seats</meter>` expresses a gauge. Its value is required; low, high and optimum are optional range points with ordering constraints. Give a visible value and units where useful, rather than relying on color alone. None of these elements implements a calculation, upload, indexing job or occupancy sensor by itself.

The example

HTML / example 40

```
<label for="card-number">Membership number</label>
<input id="card-number" name="cardNumber" inputmode="numeric"
       pattern="[0-9]{6}" aria-describedby="card-format" required>
<p id="card-format">Enter the six digits printed on your card.</p>
```

The pattern applies to this text-state input. inputmode is a keyboard hint. The six-digit rule is fictional teaching policy and needs a matching server-side rule in a real service.

A mistake to watch for

Adding a pattern without visible formatting guidance, or showing a generic Something went wrong message for a correctable field error.

Your turn

After setting a custom validity message, a script sees the corrected value. What must it do to let the field become valid? Should seats occupied out of 20 be represented as task progress?

Explained answer

Clear the custom message with setCustomValidity("") when the custom condition passes. Other native constraints may still apply, so inspect validity rather than assuming clearing one message proves every rule passed. No. That is a bounded measurement suited to meter, not progress. A computed result can

use output, but its value is not automatically a submitted form entry.

Read the source

HTML Living Standard: constraint validation

HTML Living Standard: pattern syntax

HTML Living Standard: length constraints

HTML Living Standard: output

HTML Living Standard: progress

HTML Living Standard: meter

VI / Forms and Native Controls

41. Submission and server boundaries

A finished form includes what happens after the request.

By the end of this chapter

- Choose an encoding for the actual data
- Separate a static form interface from its service
- Predict submitter exceptions and validation-bypass behavior

Form submission produces an entry list and a request under defined rules. application/x-www-form-urlencoded is the common default encoding. File uploads through a conventional POST form need multipart/form-data. Do not choose text/plain as a general production data format merely because it looks readable.

A submit button can override supported submission attributes for that submission. The button used can also contribute its own name and value. Test these details when one form supports several actions.

Protect and explain the result

Use server-side validation, appropriate authorization and protections suited to the application, including CSRF defenses where the authentication and request model need them. HTTPS protects transport; it does not decide whether the requester may change a record.

A successful HTTP response is not necessarily a successful business operation. Show whether the application accepted, rejected or queued the action, and explain the next step. Handle retries and duplicate submissions according to the service's data model. Static hosting serves the interface; a working submission service needs a separately implemented handler.

Choose the submission path deliberately

Calling `form.requestSubmit()` runs the applicable constraint validation and, if submission proceeds, dispatches a `submit` event. Calling `form.submit()` directly bypasses that validation step and does not fire the `submit` event. Do not treat those methods as interchangeable shortcuts. Pass the intended submit button to `requestSubmit()` when its name, value or overrides matter.

For a multipart upload sent with `fetch` and `FormData`, let the browser generate the `Content-Type` boundary rather than hand-writing a bare multipart header. Read the service's expected request format and response contract before adapting a markup-only example. Use fictional data in local demonstrations. GET query values appear in the URL, and switching to POST does not replace HTTPS or careful logging and storage policies.

Defaults, overrides and exceptions belong in the contract

The accepted authoring value of accept-charset is UTF-8, matched case-insensitively. This is not a menu of legacy encodings to copy into a new form. Submission overrides belong to an eligible submit button; a hidden text field with the same spelling does not override the method or endpoint.

`requestSubmit(submitter)` throws TypeError if the argument is not a submit button and NotFoundError if its form owner is different. Omitting the argument does not select every named button. A control named or identified as submit can also mask the form's submit method through named-property behavior; avoid such collisions rather than silently switching to a validation-bypassing path.

`novalidate` on a form and `formnovalidate` on an applicable submit button skip the interactive validation stage; they do not remove server-side obligations. A canceled submit event prevents that submission, while a formdata handler can amend the entry list. Constructing a FormData object alone sends nothing.

For a dialog form, method="dialog" closes the containing dialog and can set its return value from the submitter instead of issuing the usual request. Keep that separate from a form advertised as saving an application. For real native requests, inspect the method, target, encoded entries and response; an ordinary local fixture is evidence of browser behavior, not a deployed business service.

The example

HTML / example 41

```
<form action="/applications" method="post" enctype="multipart/form-data">
  <label for="proof">Address document</label>
  <input id="proof" name="proof" type="file" required>
  <button type="submit" name="action" value="apply">Send application</button>
</form>
```

The endpoint is illustrative and is not implemented by this static book. A real service needs a handler, storage rules, privacy information and testable success and failure states.

A mistake to watch for

Claiming a contact or application form works because the browser renders the fields and button.

Your turn

What else is required before this form can be advertised as accepting real applications? What happens if requestSubmit receives a submit button owned by another form?

Explained answer

A working endpoint, validated and authorized handling, appropriate upload limits and storage controls, a clear privacy policy, and tested user-visible results for success, error and retry cases. Confirm deployment and transport behavior too. It throws NotFoundError. A non-submit button produces TypeError. Handle the intended form/submitter relationship rather than treating either method as a generic submit shortcut.

Read the source

HTML Living Standard: form submission

HTML Living Standard: constructing the entry list

HTML Living Standard: the form element and its methods

MDN: using FormData objects

OWASP: file-upload defenses

OWASP: CSRF prevention

VII / Interaction and Modern HTML

42. Buttons and commands

A button should say what will happen.

By the end of this chapter

- Choose a button type deliberately
- Distinguish actions from persistent toggles
- Distinguish reset defaults from clearing and image-button payloads

Use button for an action. Give it a clear name and choose type deliberately. A button associated with a form can submit by default in relevant states, so write type="button" for a local interface action and type="submit" for an intended submission.

A toggle button needs a represented state when the interaction is a persistent toggle. aria-pressed is appropriate for that pattern, not for every button that can be clicked. Keep the state synchronized with the result.

Prefer a native action when it fits

The Standard includes command and commandfor for invoker commands, including relationships to dialogs and popovers under their defined rules. This can reduce script, but each command and target needs current compatibility review. Custom command names and their handling do not become application behavior automatically.

Avoid nesting another interactive control inside a button. An icon can accompany text, but a tooltip should not be the only way to discover an otherwise unnamed action. Disabled buttons are also a communication decision: if someone needs to know why they cannot continue, provide that explanation outside the unavailable control.

Defaults are contextual, not a slogan

For button, missing or invalid type selects the Auto state. A button in that state is a submit button when neither command nor commandfor is present and its parent is not select. An explicit type="submit" is also a submit button. An invoker inside a form should deliberately use type="button"; suppressing default submission with command attributes is not the same as implementing the intended action.

The built-in command keywords are toggle-popover, show-popover, hide-popover, close, request-close and show-modal. commandfor names a target id in the same tree. A custom keyword starts with two hyphens, such as `--pin`; it dispatches a command event rather than saving a pinned item automatically. The event exposes its command and source. Test implementation and cancellation before replacing a compatible scripted path.

Four input button states have different payloads

`<input type="submit" value="Send">` is a submit control, while reset restores defaults and button has no default action. A reset event can be canceled. Resetting a form does not reverse a server transaction or necessarily clear every custom application message.

An image submit control uses src for its image and alt for its purpose. Its activated coordinates contribute entries such as `apply.x` and `apply.y` for name="apply", not an ordinary value under apply. Image buttons are also a historical exception to the form.elements collection. Prefer an ordinary text button when an image adds no useful information.

The example

HTML / example 42

```
<form action="catalogue.html" method="get">
  <label for="button-query">Book title</label>
  <input id="button-query" name="q" type="search">
  <button type="submit">Find books</button>
  <button type="reset">Restore search defaults</button>
</form>
```

The submit button requests submission; the reset button restores the controls to their defaults, which are empty here. Reset is not a general erase or server-undo operation. The static catalogue filename does not implement a search service.

A mistake to watch for

Leaving a local interaction button without a type inside a form, then being surprised by submission.

Your turn

Does a button that opens a dialog always need aria-pressed? Does reset necessarily erase every value?

Explained answer

No. aria-pressed represents a toggle-button state. A dialog opener is not automatically a toggle button. Give it an appropriate name and implement the dialog interaction and focus behavior correctly. No. It restores defaults; a nonempty default returns. An image submitter also has coordinate entries rather than the ordinary name/value payload of a text submit button.

Read the source

HTML Living Standard: button

HTML Living Standard: input submit

HTML Living Standard: image submit

HTML Living Standard: reset

HTML Living Standard: plain button

VII / Interaction and Modern HTML

43. Disclosure with details

Use a native disclosure before rebuilding one.

By the end of this chapter

- Use a native disclosure
- Control a boolean open state correctly
- Apply same-tree exclusive-group constraints

details represents a disclosure. Its first summary supplies the label. The open attribute controls whether the contents are shown. This is a boolean attribute, so open="false" is still open.

A disclosure suits supplementary information that can reasonably remain collapsed. Do not hide the only instruction someone needs to complete a required field or understand a price behind an obscure label.

Keep the summary useful

Write the summary as a question or a short description of the hidden information. Clicking it and keyboard activation should work without your own click handler. Avoid putting competing interactive controls in a summary without a specific reviewed reason.

The name attribute can associate details elements in an exclusive group under its defined behavior. Compatibility still matters. A group of disclosures is not automatically a tab interface or a full application accordion with every optional keyboard interaction from another pattern.

Test the disclosure with a keyboard and inspect whether its content is understandable when expanded. If you add animation, respect reduced-motion preferences and avoid delaying access to the actual information.

Exclusive groups have authoring rules

A details name group uses the same nonempty name in the same tree. Its members need not be adjacent, although keeping related disclosures together makes their relationship easier to understand. At most one member may have open, and a member must not contain another member of that same group. Browser enforcement does not make misleading initial markup conforming.

Opening a second member closes the first. That may frustrate someone comparing both answers, so use exclusivity only when it fits the task. A native details group does not promise the arrow-key interactions of every custom accordion pattern.

Place summary first in the details content model. The first summary child is the disclosure control; adding several summaries does not create several independent controls. Toggle notifications are queued and can be coalesced after rapid changes. Do not treat each attribute mutation as a guaranteed separate user action.

The example

HTML / example 43

```
<details>
  <summary>Can I renew a borrowed book?</summary>
  <p>Ask the desk before the due date. Some collections have separate rules.</p>
</details>
```

The summary remains the control. The paragraph is ordinary document content when the disclosure is open. The library rule is an illustrative example, not a real lending policy.

A mistake to watch for

Adding role="button" and a second keyboard listener to a summary, creating duplicate or conflicting behavior.

Your turn

How do you make this disclosure initially expanded, and how do you make it initially collapsed? May two same-named members both be authored open, or nested in that same group?

Explained answer

Add open to details for an initially expanded disclosure. Omit open for the collapsed state. The string false is not a way to turn off the boolean attribute. No. Both arrangements violate the group authoring rules, even if a browser repairs their eventual open state.

Read the source

HTML Living Standard: details and summary

HTML Living Standard: summary

VII / Interaction and Modern HTML

44. Dialogs and popovers

Choose the interaction before choosing the container.

By the end of this chapter

- Distinguish modal dialog and popover behavior
- Provide a keyboard-usable close path
- Distinguish cancelable close requests and popover defaults

A dialog can be shown modally with showModal or non-modally with show. A modal dialog changes interaction with the rest of the page. Opening it requires an intentional focus destination, a usable close path and sensible focus restoration when it closes.

Writing open in source displays a dialog in its non-modal open state; it does not reproduce every part of showModal. Use the dialog API or an appropriately supported invoker relationship for the intended behavior.

A popover is not automatically a dialog

popover controls display and dismissal behavior for a popover element. It does not assign the semantics of a menu, dialog or tooltip. Choose markup and accessibility behavior for the actual content. The auto, manual and newer hint behaviors need their own defined handling and compatibility review.

closedby and declarative commands are newer parts of this area. Keep a compatible close mechanism and test Escape, focus, pointer dismissal and small-screen scrolling. Avoid using a modal merely to deliver a short message that could be shown in the page.

Inspect the whole interaction

A modal dialog opened with `showModal()` makes the surrounding document inert under the defined modal-dialog rules. A popover does not make the rest of the document inert merely by being open. Neither feature supplies an appropriate accessible name for your content automatically. Give a dialog a meaningful name, choose its initial focus from the task and verify that closing leaves focus somewhere useful.

In the example, a valid form submission with `method="dialog"` closes its containing dialog rather than sending a network request. The activated submit button can supply the dialog's return value. That is a local interaction result, not confirmation that a server has saved anything.

Closing and requesting closure are different

close() closes an open dialog and can set its returnValue. requestClose() first fires a cancel event; preventDefault can reject that request. If it proceeds, closure also fires close. Calling requestClose ignores closedby="none"; that attribute configures the relevant user dismissal actions, not a veto on every script operation.

closedby accepts any, closerequest and none. Missing or invalid values select Auto, which behaves as Close Request for a modal dialog and None otherwise. Keep a usable close control and do not rely only on a newer light-dismiss affordance.

showModal throws InvalidStateError when the dialog is already open non-modally, while show throws when it is already modal. Use the intended lifecycle instead of manually removing open: that can skip close events and leave the document's modal blocking state wrong. Do not add tabindex to dialog; choose an appropriate autofocus destination under the dialog rules.

Popover defaults are not boolean defaults

Missing popover means no popover; an empty value means Auto; an invalid value means Manual. Manual popovers do not light-dismiss like auto popovers. Hint has its own stack behavior and must be tested separately. Neither state gives the content a menu or dialog role.

beforetoggle can cancel opening under the defined rules, while toggle reports state changes through queued notifications. showPopover on a non-popover throws NotSupportedError; disconnected or otherwise invalid state can produce InvalidStateError. A working supported invoker has its own target/action relationship; storing those attributes alone does not prove the behavior.

The example

HTML + JavaScript / example 44

```
<button type="button" id="open-help">Membership help</button>
<dialog id="help-dialog" aria-labelledby="help-title">
  <h2 id="help-title">Membership help</h2>
  <p>Ask at the desk if you cannot supply an address document.</p>
  <form method="dialog"><button type="submit" autofocus>Close help</button></form>
</dialog>
<script>
  const help = document.getElementById("help-dialog");
  document.getElementById("open-help").addEventListener("click", () => {
    help.showModal();
  });
</script>
<button type="button" popovertarget="reading-tip">Reading tip</button>
<div id="reading-tip" popover>
  <h2>A note about renewals</h2>
  <p>This is a fictional teaching note, not a real lending policy.</p>
  <button type="button" popovertarget="reading-tip"
          popovertargetaction="hide">Dismiss reading tip</button>
</div>
```

The close button submits the method="dialog" form, closing the dialog rather than posting to a server. Test the actual focus behavior and decide whether a more specific initial focus target is needed for longer content. The short help dialog puts initial focus on its close control. The separate note uses an auto popover with an explicit dismiss button, not a modal dialog or menu. Test support and actual focus restoration.

A mistake to watch for

Assuming that adding the popover attribute supplies menu keyboard navigation or a dialog role.

Your turn

Why is toggling the dialog open attribute not equivalent to calling showModal? Will closedby="none" prevent a script from calling requestClose?

Explained answer

showModal establishes modal behavior, including its relationship with the top layer and the rest of the page. The open attribute alone shows an open dialog without that same modal operation. No. The method still requests closure and fires cancel. A canceled request can keep the dialog open; the attribute is not a general script-access restriction.

Read the source

HTML Living Standard: dialog

HTML Living Standard: popovers

VII / Interaction and Modern HTML

45. Focus, editing and dragging

An interaction needs a path that does not depend on a pointer.

By the end of this chapter

- Explain sequential and programmatic focus
- Provide a non-drag path for important actions
- Test actual reordering and focus rather than a decorative button

Focusable and sequentially focusable are related but distinct. tabindex="0" can place an otherwise suitable element in sequential focus order. tabindex="-1" commonly allows programmatic focus without adding an ordinary tab stop. Positive tabindex values create a separate ordering problem and are usually best avoided.

contenteditable enables editing under its defined states. It does not provide a complete rich-text editor, accessible toolbar, sanitation pipeline or save system. Editing behavior and supported input methods need real-device testing.

Offer alternatives to dragging

draggable participates in the drag-and-drop model. A drag gesture should not be the only way to perform an important operation. Provide buttons or another keyboard-usable mechanism for moving an item, and keep the outcome understandable.

Use focus to support a task, not to surprise someone whenever content changes. When removing a focused item, decide where focus goes next. Avoid trapping focus in a non-modal widget. Mobile input, speech control and assistive technology can expose assumptions that mouse-only testing misses.

Editing and dragging use enumerated states

contenteditable accepts true, an empty true value, false and plaintext-only; missing or invalid values inherit. isContentEditable reports the resulting editing condition, not merely the stored attribute spelling. Plaintext-only is not an HTML sanitizer or a persistent storage service.

draggable is enumerated, not boolean. Its true and false values differ from missing or invalid Auto, under which images, links and selections have their own default drag behavior. Do not write draggable="false" and assume it follows the boolean-attribute rule from disabled.

During drag-and-drop, DataTransfer has mode and phase restrictions. Write the intended data during dragstart, allow an intended drop through the relevant dragover handling, and read/validate the data in the allowed drop phase. Treat incoming text or HTML as untrusted. A synthetic DragEvent or constructed DataTransfer is not proof that native pointer, touch or assistive-technology dragging works.

The reordering controls are both a keyboard path and a single-pointer non-drag path. Those are separate accessibility considerations: keyboard operation alone is not the entire dragging-movements requirement. Test the actual pointer and keyboard controls, focus after a move and the readable result.

The example

HTML + JavaScript / example 45

```
<section aria-labelledby="list-title">
  <h2 id="list-title" tabindex="-1">Your fictional reading list</h2>
  <ol id="reading-list">
    <li tabindex="-1"><span>Local histories</span>
      <button type="button" data-shift="-1" disabled>Move up</button>
      <button type="button" data-shift="1" disabled>Move down</button></li>
    <li tabindex="-1"><span>Poetry</span>
      <button type="button" data-shift="-1" disabled>Move up</button>
      <button type="button" data-shift="1" disabled>Move down</button></li>
    <li tabindex="-1"><span>Fiction</span>
      <button type="button" data-shift="-1" disabled>Move up</button>
      <button type="button" data-shift="1" disabled>Move down</button></li>
  </ol>
  <p id="move-status" role="status">Changes stay in this page only.</p>
</section>
<script>
  const list = document.getElementById("reading-list");
  const status = document.getElementById("move-status");
  function updateMoves() {
    [...list.children].forEach((item, index, items) => {
      const title = item.querySelector("span").textContent;
      item.querySelectorAll("button").forEach(button => {
        const up = button.dataset.shift === "-1";
        button.disabled = up ? index === 0 : index === items.length - 1;
        button.setAttribute("aria-label", (up ? "Move up: " : "Move down: ") + title);
      });
    });
  }
  list.addEventListener("click", event => {
    const button = event.target.closest("button[data-shift]");
    if (!button || button.disabled) return;
    const item = button.closest("li");
    const shift = Number(button.dataset.shift);
    if (shift !== -1 && shift !== 1) return;
    const peer = shift === -1 ? item.previousElementSibling : item.nextElementSibling;
    if (!peer) return;
    list.insertBefore(item, shift === -1 ? peer : peer.nextElementSibling);
    updateMoves();
    item.focus();
    status.textContent = item.querySelector("span").textContent + " moved " +
      (shift === -1 ? "up." : "down.");
  });
  updateMoves();
</script>
```

The controls now perform real local reordering. Boundary controls are disabled; focus goes to the moved item and visible status text explains the result. Controls begin disabled so a script failure does not leave apparently working actions. The list is not saved to a backend, and a status element is not proof of screen-reader announcements.

A mistake to watch for

Adding tabindex="0" to every paragraph and wrapper, making keyboard navigation unnecessarily long.

Your turn

An item can only be moved by dragging. What should you add before treating the interaction as broadly usable? Does a keyboard-only drag alternative necessarily cover every single-pointer non-drag requirement?

Explained answer

Provide an operable non-drag mechanism, such as Move up and Move down controls, then implement focus management and meaningful feedback for the result. Test with a keyboard and the actual input methods you support. No. Review both input paths. The example provides clickable buttons as well as keyboard activation, performs the move and leaves an inspectable focus and status result.

Read the source

HTML Living Standard: focus

HTML Living Standard: drag and drop

HTML Living Standard: editing states

HTML Living Standard: draggable

WCAG: dragging movements

VII / Interaction and Modern HTML

46. Templates and component boundaries

A template stores a fragment without rendering it as ordinary page content.

By the end of this chapter

- Clone inert template content
- Distinguish templates, slots and custom-element registration
- Distinguish slot assignment from moving nodes and root security

template holds inert template contents that a script can clone and insert. Its content is a DocumentFragment. Querying the template element's ordinary descendants is not the same as querying its content fragment.

slot is a placeholder within a shadow tree for assigned content. The slot attribute on an element identifies the named slot it should be assigned to under the relevant rules. This attribute is different from a slot element.

Separate the pieces of a component

Autonomous custom elements use valid custom-element names, including the required hyphen and other naming restrictions, and are defined through the custom-elements machinery. A custom tag name alone does not register a working component. Customized built-in elements have separate is behavior and implementation considerations.

Declarative shadow DOM uses supported template shadowroot attributes to establish a shadow root during parsing under defined conditions. The evolving set includes mode, focus, cloning, serialization and registry-related controls. Review each independently. A shadow boundary is not a security sandbox or an excuse to ignore accessible names and keyboard behavior.

A shadow root is created under parsing rules

A supported parser can consume `<template shadowrootmode="open"><slot name="heading"></slot><slot></slot></template>` inside an eligible host. A light-DOM child such as `<h2 slot="heading">Library notice</h2>` can be assigned to that named slot. The host still owns that light-DOM child: slotting does not move it into the shadow tree. Unassigned slots display their fallback content.

shadowrootmode permits open and closed; missing or invalid means no declarative shadow root. delegates-focus, cloning and serialization are controlled by their respective boolean shadowroot attributes, not by arbitrary strings such as false. shadowrootslotassignment permits named or manual, with Named as its missing and invalid default. The registry-related attribute needs its own scoped-registry review and fallback; presence in the Standard is not universal implementation evidence.

Ordinary innerHTML insertion does not generally consume declarative shadow templates as the document parser does. Use a suitable parsing API only after reviewing its injection and sanitization contract. A closed root changes ordinary lookup access; it does not protect against all same-page code, including code that retains a root reference.

The current Standard also defines template for-based streaming content updates with processing-instruction markers. That newer mechanism is part of the open feature reconciliation, not silently covered by cloneNode in this introductory example.

Registration and assignment have separate APIs

Defining the same custom-element name twice produces NotSupportedError, and an invalid name produces SyntaxError under the registration rules. A constructor being valid JavaScript is not enough to satisfy custom-element construction requirements. whenDefined concerns registry definition, not proof that every instance is rendered, upgraded successfully or accessible.

slotchange concerns changes to assignment, not every text edit within an already assigned node. Test assignedNodes or assignedElements for the relevant slot and keep a visible fallback where useful.

The example

HTML + JavaScript / example 46

```
<template id="book-template">
  <article class="book">
    <h2></h2>
    <p></p>
  </article>
</template>
<div id="book-results"></div>
<script>
  const template = document.getElementById("book-template");
  const card = template.content.cloneNode(true);
  card.querySelector("h2").textContent = "Local histories";
  card.querySelector("p").textContent = "A collection of neighbourhood stories.";
  document.getElementById("book-results").append(card);
</script>
```

The example clones template content and inserts plain text into it. This is a template example, not a registered custom element or a shadow tree.

A mistake to watch for

Treating a shadow root as protection against hostile same-page code, or treating a template string as safely sanitized HTML.

Your turn

Why does the script use template.content rather than expecting the article to be a normal child rendered in the page? Does a named slot transfer ownership of a light-DOM child into the shadow tree?

Explained answer

Template contents are held in a separate fragment and are not rendered as ordinary template children. Clone that fragment, fill it safely and insert it where the page should display it. No. The child remains in light DOM and is assigned for presentation. Neither slotting nor a closed root is a general security sandbox.

Read the source

HTML Living Standard: template

HTML Living Standard: custom elements

HTML Living Standard: slot

DOM Standard: shadow trees

VII / Interaction and Modern HTML

47. Custom data and structured data

Store page-specific values without inventing semantics.

By the end of this chapter

- Read custom data with explicit type handling
- Distinguish page-specific data from public structured annotations
- Validate dataset values and distinguish assignments from deletion

data-* attributes store page-specific data where no more appropriate attribute fits. Scripts can access them through dataset with the defined name conversion. Values are strings; a value such as "false" is not automatically a false boolean.

Custom data is visible in the document. Do not place credentials or trusted authorization decisions there. A visitor can inspect and change it. Do not use a data attribute as a substitute for a real href, label or disabled state when the platform already defines one.

Microdata serves a different purpose

itemscope, itemtype, itemprop, itemid and itemref participate in microdata. A vocabulary supplies meaning to properties; arbitrary names do not create a recognized public schema. Search and other consumers have their own supported formats and policies.

Keep visible content and structured annotations consistent. Do not claim that adding structured data guarantees a search result treatment. Choose a supported vocabulary and validate both its syntax and the content it represents using the relevant consumer's current documentation.

Conversion and deletion have defined rules

A dash followed by an ASCII lowercase letter becomes an uppercase letter in dataset: data-book-id becomes bookId. Other permitted characters are not all transformed by that same rule. Assigning null writes the string "null"; delete removes an entry. Assigning through a key with a dash followed by an ASCII lowercase letter, such as dataset["loan-days"], throws SyntaxError rather than acting like dataset.loanDays.

A test with `in` can see properties on a prototype as well as this map's own entries. Use a suitable own-property check where that distinction matters. Convert only the format the application actually accepts; Number("") becoming zero is not proof that an empty quantity is valid.

Microdata needs a real item model

itemscope starts an item, itemtype identifies vocabulary types, and itemprop names properties. itemid supplies a global item identifier under the vocabulary and authoring rules; itemref can refer to additional property-bearing elements by id in the same tree. It is not a general DOM ownership or permission relationship.

A Book item can use name for its visible title and an author property containing a nested Person item with its own name. Do not invent a credential, review or affiliation simply to fill a structured-data property. Element-specific value rules matter: meta uses content, time can use datetime, and URL-bearing elements have their own value extraction. A parsed graph alone does not guarantee a search treatment.

The example

HTML + JavaScript / example 47

```
<article id="book-record" data-book-id="LIB-204" data-copy-count="3">
  <h2>Local histories</h2>
  <p id="copy-note">This is a fictional record.</p>
</article>
<script>
  const record = document.getElementById("book-record");
  const rawCount = record.dataset.copyCount;
  const count = Number(rawCount);
  const valid = /^[0-9]+$/.test(rawCount) && Number.isSafeInteger(count);
  document.getElementById("copy-note").textContent = valid
    ? count + " copies listed in this fictional record."
    : "No usable quantity in this fictional record.";
</script>
```

The record now demonstrates string-to-number validation and a real text update instead of an unimplemented Save button. It deliberately rejects malformed or unsafe numeric values and never treats this local record as an authorization decision or real stock availability.

A mistake to watch for

Adding data-disabled="true" and expecting native disabled behavior.

Your turn

What JavaScript type do you get from element.dataset.count when the source has data-count="3"? What does assigning null to a dataset entry do?

Explained answer

You get the string "3". Convert and validate it deliberately if your application needs a number. Do not trust the value merely because it appeared in your own initial markup. It stores the string "null". Use delete to remove the entry, and validate any parsed value deliberately rather than treating stored strings as trusted typed data.

Read the source

HTML Living Standard: custom data

HTML Living Standard: microdata

Schema.org: Book vocabulary

VIII / Attributes and Metadata

48. Global attributes in context

Global availability does not make every use a good idea.

By the end of this chapter

- Apply global attributes with their own constraints
- Keep id relationships unique and meaningful
- Distinguish hidden rendering from form eligibility and selector syntax

The global attributes section lists attributes common to HTML elements. DOM also owns requirements for id, class and slot. Other attributes and families integrate specifications such as ARIA and CSS Shadow Parts. Keep ownership and namespace boundaries clear.

An id must satisfy its defined authoring constraints and be unique among ids in the element's tree. class is a set of tokens. title supplies advisory information, but it is not a dependable sole label or instruction because exposure varies with device and interaction.

Hidden and inert differ

hidden participates in rendering and relevance rules, including its until-found state in supporting implementations. inert prevents interaction and affects focus and accessibility exposure under its rules. Neither is simply a synonym for a CSS property.

Use lang and dir for their own meanings. accesskey can conflict with browser, operating-system or assistive-technology commands, so do not invent a keyboard shortcut scheme casually. Newer autocorrect, writingsuggestions and heading-level attributes need compatibility review. The reference keeps them visible rather than silently dropping them because they are unfamiliar.

Rendering, interaction and serialization differ

hidden is enumerated: missing means Not Hidden, empty or invalid means Hidden, and until-found has its own discovery behavior. Supported fragment navigation or find-in-page can reveal until-found content and fire beforematch. Its layout and CSS conditions matter; do not force display:none and expect the same discovery path.

CSS can override ordinary hidden rendering. More importantly, hidden content can still contain scripts or eligible form controls; hiding is not disabling, withholding a secret or revoking a permission. inert acts on the relevant interaction/focus subtree. A modal dialog has a defined inertness escape under its own rules, so do not generalize that exception to every nested widget.

An id must be nonempty and contain no ASCII whitespace. It is not limited to alphanumeric characters: reader:name can be a valid id, while a CSS selector referring to it needs suitable escaping. getElementById does not parse its argument as a CSS selector. Keep relationship references unique within their actual tree.

Hints retain their own keywords

translate uses yes or an empty value for Yes and no for No; omission or invalid values use inheritance. spellcheck uses true/empty, false and its defined default behavior, not a blanket boolean presence rule. autocapitalize offers off/none, on/sentences, words and characters. autocorrect uses on and off. writingsuggestions has true/false and default behavior. These affect eligible editing behavior and do not validate the field or promise an identical keyboard.

enterkeyhint includes enter, done, go, next, previous, search and send; inputmode has its separate input-mechanism hints. Neither causes a server submission just because a key's label suggests one. accesskey uses its defined single-character token rules and platform modifier behavior; avoid conflicts and do not claim that it is a universal shortcut.

Styling hooks cross a separate specification boundary

part names exposed shadow parts; exportparts forwards named parts through another component boundary. A part="badge" can be styled through a supported ::part(badge) selector on the host. These are CSS Shadow Parts hooks, not data-* names or permission grants. Review CSS ownership and implementation separately.

The example

HTML / example 48

```
<section id="renewal" class="notice compact" lang="en">
  <h2>Renew your membership</h2>
  <p>Use the same name as your current library card.</p>
</section>
```

id is a stable reference target. The two classes are separate tokens, not two independent class attributes. The section's language is explicit here even though it may already inherit English from html.

A mistake to watch for

Relying on title as the only label for an icon-only control or the only explanation of a field error.

Your turn

Can two unrelated controls safely share an id just because they are on different visual panels? Does hiding a form control necessarily remove its submitted entry?

Explained answer

Not within the same element tree. Visual separation does not establish a new id scope. Use unique ids and update all for, fragment and ARIA references consistently. No. Hidden rendering and entry-list eligibility are different. A valid id with punctuation can also need CSS escaping even though getElementById can use that id directly.

Read the source

HTML Living Standard: global attributes

HTML Living Standard: hidden

HTML Living Standard: inert

HTML Living Standard: spelling

HTML Living Standard: capitalization

HTML Living Standard: autocorrection

HTML Living Standard: writing suggestions

HTML Living Standard: enter key hints

CSS Shadow Parts specification

VIII / Attributes and Metadata

49. Element-specific attributes

The same spelling can have different rules.

By the end of this chapter

- Interpret attributes in element and state context
- Distinguish stored markup from implemented behavior
- Treat index rows as navigation and handler cancellation as event-specific

An attribute name is not a complete definition. type means different things on input, button, script and other elements. value differs between a text input, an option and a progress element. Read the element and state alongside the name.

The reference retains contextual rows rather than merging everything into one alphabetical name list. An input's type can determine whether min, maxlength or another constraint applies. A parser accepting an attribute does not make the use conforming or effective.

Check the default as well as the explicit value

Missing-value defaults and invalid-value defaults can differ. Enumerated attributes have defined keywords and states. Boolean attributes depend on presence. Token lists have their own constraints. When a script reflects an attribute through a property, the property can normalize or interpret it rather than exposing the raw source string.

For a new attribute, verify its canonical definition, supported elements, permitted values, side effects and implementation data. Preserve compatibility notes and fallback behavior. Do not infer support merely because getAttribute returns the text you set; the DOM can store unknown attributes too.

An index is a route to the definition

The attribute index is a navigation aid, not the entire normative contract. Follow each context's definition for value syntax, applicable states, side effects and exceptions. A shared spelling does not merge input, option and progress values into one behavior.

Event-handler attributes such as onclick integrate JavaScript handler processing. They are not boolean switches. Assigning a function to an onclick property differs from storing text in an inline attribute, and returning false from an addEventListener callback is not a replacement for preventDefault. Follow the applicable event's cancellation rules and CSP restrictions.

Nonce handling is a further warning against a generic getAttribute-based support detector: a connected nonce-bearing element can have an empty content attribute while its nonce property exposes the internal nonce. Chapter 53 shows the distinction. API reflection, parsing and behavior remain separate checks.

The example

HTML / example 49

```
<label for="age-years">Age in years</label>
<input id="age-years" name="age" type="number" min="0" max="120">
<label for="reader-name">Reader name</label>
<input id="reader-name" name="readerName" type="text" maxlength="100">
```

The numeric bounds suit the number state. The text length constraint suits the text state. These are illustrative service limits, not universal requirements for personal data.

A mistake to watch for

Adding maxlength to a number input and assuming it enforces the number of digits.

Your turn

Why does reading an unfamiliar attribute with getAttribute fail to prove native support? Does returning false from every addEventListener callback cancel the browser action?

Explained answer

getAttribute reads stored attribute text. It does not demonstrate that the browser implements the attribute's specified behavior. Test the behavior and consult dated implementation evidence. No. Use the event cancellation mechanism where the event is cancelable. Inline-handler processing and addEventListener are not interchangeable conventions.

Read the source

HTML Living Standard: attribute index

HTML Living Standard: input applicability

HTML Living Standard: event handlers

VIII / Attributes and Metadata

50. Metadata readers encounter

A good title helps someone find the right tab again.

By the end of this chapter

- Write a specific document title
- Describe metadata without search-result guarantees
- Keep metadata consistent with visible content and base authoring

title identifies the document in browser UI and other contexts. Make it specific enough to distinguish this page from the rest of the site. A visible h1 and the document title have related but separate jobs.

meta can declare encoding or named metadata. A description is useful page-summary metadata, but its presence does not guarantee that a search engine will display it. Do not promise rankings from a tag.

Keep head deliberate

Use link for supported resource relationships such as stylesheets and icons. A favicon is not the same as a visible logo. style holds embedded CSS when permitted. base changes URL resolution and target defaults, so use it only when you understand the effects on every relative reference.

Metadata names and link types have defined or registered meanings. Do not invent a plausible name and assume a consumer understands it. Social sharing formats also need their own current documentation. Keep canonical URLs accurate, and avoid adding several conflicting declarations through independently configured tools.

One mechanism does not impersonate every response header

Choose the appropriate meta form: encoding, named metadata or a defined pragma. http-equiv is not an arbitrary way to set every HTTP response header. A meta-delivered CSP has important limitations, including directives that require response-header delivery; it is not identical to a header policy. Automatic refresh also has navigation and accessibility consequences.

A document has at most one base, with href, target or both. Put its href before other URL-bearing elements and its target before relevant hyperlinks. Browser handling that ignores later duplicate bases does not authorize duplicate authoring. An explicit link target can override the default target, while a fragment-only URL can still resolve against the changed base.

Style and stylesheet relationships need permitted contexts and real CSS resources. Named metadata, consumer-specific social formats and the document title have different contracts. Keep the body and annotations aligned before optimizing for a hypothetical search display.

The example

HTML / example 50

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="description" content="A teaching page with fictional opening hours and a conta
ct-status notice.">
  <title>Visit and contact | Teaching library</title>
</head>
<body>
  <h1>Visit and contact</h1>
  <h2>Example hours</h2>
  <p>Tuesday to Saturday, 09:00 to 17:00. Fictional hours for teaching.</p>
  <h2>Contact status</h2>
  <p>Real contact details are not configured in this example.</p>
</body>
</html>
```

The description now matches the actual fictional hours and contact-status notice in the body. Neither the metadata nor the page advertises a real library service or promises a particular search snippet.

A mistake to watch for

Giving every page the same title, or treating a meta description as a guaranteed search-result advertisement.

Your turn

Which title better identifies the contact page: Welcome or Visit and contact | Neighbourhood Library? Does browser handling of a second base mean two base elements are conforming?

Explained answer

The second identifies both the page purpose and the site. It remains useful in a tab list, bookmark collection or history entry where surrounding page content is absent. No. The authoring rule permits at most one. Metadata must also describe actual content rather than promise details the body does not supply.

Read the source

HTML Living Standard: metadata

HTML Living Standard: meta

HTML Living Standard: base

HTML Living Standard: style

VIII / Attributes and Metadata

51. Resource relationships

rel tells consumers how a link relates to its context.

By the end of this chapter

- Read rel as a defined relationship
- Evaluate hints using actual requests
- Distinguish a reusable early request from a hint count

A link relation gives a relationship meaning rather than merely a destination. link rel="stylesheet" requests a stylesheet relationship. An anchor's rel can annotate a hyperlink. Their allowed contexts and effects depend on the particular link type.

preload, preconnect and related hints should correspond to a real loading need. A preload has request details such as as and, when relevant, type or crossorigin. A mismatch with the eventual request can waste work rather than helping.

Avoid hints by habit

Measure the loading problem before adding hints. A preconnect to a third party can create network and privacy costs even when the visitor never uses that service. A resource hint is not proof of a faster page.

Canonical, alternate, author and other relations have defined uses and consumer expectations. Keep the reference's link-type inventory separate from a list of attribute names. The rel attribute's existence does not enumerate all its possible relationship tokens or guarantee a particular consumer's action.

Request matching matters more than the word preload

For preload, as supplies the appropriate destination, such as script or style, and CORS/credentials and integrity details need to match the eventual use. An early request to the wrong resource or under a mismatched mode is not guaranteed reusable. Inspect real requests and cache behavior instead of counting link elements.

Modulepreload has module-specific fetching and processing behavior, not merely the semantics of preload with a different name. Preconnect concerns connection preparation, not downloading every future resource. Browser discretion and the workload still matter.

An alternate relation can identify an alternative representation or language under its applicable rules; a canonical relation communicates a preferred identifier to consumers, not an HTTP redirect. supported rel tokens and allowed contexts differ between link, a, area and form. relList.supports can be a scoped vocabulary probe, not proof that a consumer honors a canonical or that a hint improved performance.

The example

HTML / example 51

```
<link rel="stylesheet" href="../assets/site.css">
<link rel="icon" href="../assets/favicon.svg" type="image/svg+xml">
```

Both publication-relative resources are supplied. This is head-context markup, and the standalone example can fetch the real stylesheet and icon. The editable lab deliberately blocks external resource loading; do not weaken that policy to test a resource relationship.

A mistake to watch for

Preloading every image, font and script on a page without checking which resources are actually needed early.

Your turn

Why might a preload download fail to benefit the later resource request? Does a canonical link redirect the visitor automatically?

Explained answer

Its request parameters can differ from the later request, the resource may be unused, or the early work may contend with something more important. Inspect actual network behavior and request matching. No. It expresses a relationship for consumers. Likewise, a preload tag is not evidence of reuse or a measured loading improvement.

Read the source

HTML Living Standard: link types

HTML Living Standard: link element

HTML Living Standard: preload

HTML Living Standard: modulepreload

HTML Living Standard: preconnect

VIII / Attributes and Metadata

52. Loading and policy attributes

A hint and a security policy do not do the same job.

By the end of this chapter

- Distinguish loading hints from policy enforcement
- Trace integration claims to the owning specification
- Distinguish integrity enforcement from trust and nonce generation

loading and fetchpriority influence eligible loading decisions. They are not strict scheduling commands. crossorigin participates in the resource's fetching and CORS behavior; it does not grant your server permission to expose private data.

integrity integrates Subresource Integrity for supported resource uses. The hash must match the actual bytes, and cross-origin requirements matter. A placeholder digest is not a security control. nonce participates in a Content Security Policy arrangement and should be generated and delivered according to that policy's requirements.

Follow the actual authority

HTML defines where these attributes integrate, while Fetch, CORS, CSP, Referrer Policy and SRI own important parts of their behavior. Read the relevant specification and deployment documentation for a security assertion.

referrerpolicy controls referrer information according to the defined policy, not all information a remote service can learn. Avoid presenting one attribute as a complete privacy solution. Test response headers and real requests as well as the source markup.

Integrity is byte checking, not a trust certificate

SRI metadata can contain supported algorithm/digest tokens. When supported algorithms of different strengths are present, the strongest applicable algorithm governs matching; a matching weaker digest does not rescue a wrong stronger one. The check can block execution after a request has still occurred.

Cross-origin integrity use also needs the required CORS-enabled response. Adding crossorigin="anonymous" alone does not make a server grant that access. Changing even one script byte requires the corresponding digest to change. Review the code and distribution path, not merely the fact that a digest matches.

A CSP nonce should be unpredictable and fresh for the relevant response, with matching policy and element values. Hard-coding the same value in a static example is not the production generation mechanism. Nonce hiding moves the connected element's value out of the ordinary content attribute and exposes it through the nonce property under the defined rules.

Referrer Policy controls referrer information, not whether a destination receives a request or learns other request information. A no-referrer link can still navigate successfully while omitting the Referer header. Fetch/CORS, CSP, Referrer Policy and SRI remain distinct authorities and require actual header/request checks.

The example

HTML + JavaScript / example 52

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>A supplied integrity-check example</title>
  <script src="../assets/integrity-demo.js"
          integrity="sha384-PYqdPK9Xnp/sfr1PXqd/y/pWZibvmdfPUzshLt7bF4U5H6NHlaHycsosnMoI3qin
" defer></script>
</head>
<body>
  <h1>Integrity-check teaching example</h1>
  <p id="integrity-status">The supplied script has not run.</p>
</body>
</html>
```

The SHA-384 digest matches the actual supplied script bytes, not a placeholder. Its only action is to update the visible local status. Run the standalone example to inspect that resource; the editable lab blocks external scripts. Matching bytes does not prove that arbitrary code is safe.

A mistake to watch for

Copying a static nonce into every deployment and describing it as a complete CSP configuration.

Your turn

Can adding crossorigin make a remote server return private data to your script when that server does not allow the origin? Will a matching weaker digest necessarily rescue a mismatched stronger digest?

Explained answer

No. The server's CORS policy and the request mode and credentials behavior still apply. The attribute is part of a fetching configuration, not a bypass for a server's access policy. No. SRI uses its strongest applicable algorithm rules. Also, a matching digest verifies bytes rather than certifying the code or its source as safe.

Read the source

HTML Living Standard: CORS settings

HTML Living Standard: nonce

Subresource Integrity specification

Content Security Policy specification

Referrer Policy specification

Fetch Standard

VIII / Attributes and Metadata

53. Attributes and properties

Source text, defaults and current state can diverge.

By the end of this chapter

- Distinguish raw attributes and current state
- Read the correct layer when debugging controls
- Trace dirty/default/reset behavior and special reflection cases

getAttribute reads an attribute's text or returns null when absent. A corresponding DOM property can expose a parsed, normalized or stateful value. Do not assume they always remain equal.

For a text input, the value attribute is associated with its default value, while the value property describes the current value under the control's rules. User edits can change the current value without rewriting the source attribute. checked and defaultChecked have a related distinction for appropriate controls.

Read the right layer

Boolean reflection commonly maps property true to attribute presence and false to absence. URL properties can return resolved absolute URLs while the attribute contains a relative string. Numeric reflection has its own rules, defaults and limits.

When debugging, ask whether the question concerns the original source, the current attribute, or the current control state. Reset behavior, script assignment and user interaction can affect different layers. Avoid a generic helper that treats every attribute and property as interchangeable strings.

Default state follows its own update rules

Before a text input has a dirty current value, changing its value attribute can update the current value as well. After an edit or a value-property assignment, that synchronization changes; defaultValue can change without replacing the current text. A form reset restores the relevant default under its rules. Inspect the actual state instead of assuming every setAttribute call has the same result.

Nonce is a deliberate special case: the connected element's getAttribute("nonce") can be empty while its nonce property retains the internal nonce. Source text, current content attribute and internal state are not three interchangeable strings.

Collections can have different result shapes

form.elements includes listed controls associated with that form, including eligible external controls, but has the historical image-button exception. namedItem can return a single element, a RadioNodeList when several match, or null. A RadioNodeList value concerns its checked radio, not a generic serialization of every checkbox. Use the form entry-list APIs for repeated submitted values.

select.options is a live options collection with its own indexed/add/remove behavior. selectedOptions has a separate current-selection purpose. Do not freeze a live collection assumption into a mutation loop without checking which nodes remain.

document.all is a legacy compatibility object with unusual falsiness and typeof behavior. It is not a recommended feature-detection API. A member inventory must retain that legacy behavior honestly, without encouraging it in new application code.

The example

HTML + JavaScript / example 53

```
<label for="nickname">Reading-group name</label>
<input id="nickname" name="nickname" value="Guest">
<script>
  const field = document.getElementById("nickname");
  field.value = "Ada";
  console.log(field.value);                 // Ada
  console.log(field.getAttribute("value")); // Guest
</script>
```

The script changes the current input value. The attribute retains Guest in this example. This is a specific input-state behavior, not a universal rule for every property named value.

A mistake to watch for

Using getAttribute("value") to read a text field after the user has edited it.

Your turn

Which should an ordinary script read for the current text input value after editing? Can setting the value attribute affect a pristine input differently from one already edited?

Explained answer

Read the input's value property. If you need the default value or the raw attribute, ask for that specifically instead of assuming it equals the current state. Yes. The dirty-value rules matter. Reset uses the relevant default, and nonce hiding is another reason not to assume getAttribute always reproduces the original source spelling.

Read the source

HTML Living Standard: reflection

HTML Living Standard: input value

HTML Living Standard: form control collections

HTML Living Standard: radio node lists

HTML Living Standard: options collections

HTML Living Standard: legacy all collection

HTML Living Standard: nonce hiding

IX / Browser Processing and Advanced Concepts

54. How parsing starts

The parser has states, not a general find-and-replace rule.

By the end of this chapter

- Describe tokenizer context
- Choose text insertion for plain untrusted text
- Distinguish parser contexts and fragment destinations

HTML parsing includes tokenization and tree construction. The tokenizer recognizes text, tags, attributes, comments and other tokens while its state affects how characters are interpreted. A less-than sign inside an ordinary text context does not have the same surrounding rules as text in script or style.

Character references are consumed in the contexts defined by the parser. An apparent tag inside a JavaScript string can still interact with the HTML script-data rules, which is why serializing arbitrary data into inline script needs careful escaping and a reviewed approach.

Do not sanitize with a tag regex

Untrusted HTML needs a context-appropriate safe handling strategy. Plain text should usually be inserted as text. If a product truly accepts markup, use a maintained sanitizer configured for the allowed content and review how it interacts with the destination.

Encoding for one context does not automatically encode for another. An HTML attribute, JavaScript string, URL and CSS value have different grammar. The examples here are educational text; they are not a complete sanitizer implementation.

The surrounding grammar changes the token

Normal text, RCDATA in title/textarea, raw text in style and the script-data states do not all recognize the same markup. Character references can be consumed in ordinary text and RCDATA, but an `&` spelling inside script is not converted by that same rule into JavaScript source. Attribute values also have context-dependent reference rules.

The parser operates on decoded characters after encoding detection. NULs, numeric references and malformed input have specified error/replacement behavior. A parse error is not a general stop signal for text/html, and recovery does not make the original source conforming.

A fragment has a destination context

The same `<tr><td>Fiction</td></tr>` string does not produce the same nodes when parsed for a div and for a tbody. Fragment parsing uses a context element; it is not just document parsing with the outer tags removed. DOMParser.parseFromString with text/html creates a separate HTML document. Unsupported MIME-type arguments produce TypeError; XML inputs have their own well-formedness/error results.

An innerHTML-inserted script generally does not run through ordinary script execution, but that fact is not an XSS defense: event-handler attributes and other active content still matter. A separately parsed

document is not a sanitizer either. Do not adopt arbitrary parsed nodes into the live page without reviewing the trust boundary.

Trusted Types can constrain covered injection sinks where implemented and enforced. It does not certify that a policy's transformation is safe. Sanitization support, policy enforcement and parsing behavior each need their own checks.

The example

HTML + JavaScript / example 54

```
<p id="message"></p>
<script>
  const submittedText = "<strong>Recommended</strong>";
  document.getElementById("message").textContent = submittedText;
</script>
```

The literal string appears as text because the script assigns textContent. It does not create a strong element. Use this route when the product accepts text rather than markup.

A mistake to watch for

Assuming that stripping text between angle brackets with a regular expression produces safe HTML.

Your turn

If a comment field is intended to accept plain text, why use textContent rather than innerHTML? Will the same row string necessarily create a row inside a div and a tbody?

Explained answer

textContent creates text without interpreting the submitted string as markup. innerHTML parses markup and can introduce dangerous behavior if the data is not handled with a correct sanitization strategy. No. The fragment context changes tree construction. Also, a script not running after innerHTML insertion does not establish that the inserted HTML is safe.

Read the source

HTML Living Standard: tokenization

HTML Living Standard: dynamic insertion

HTML Living Standard: fragment parsing

DOM Parsing and Serialization

OWASP: XSS prevention

IX / Browser Processing and Advanced Concepts

55. Tree construction and recovery

Inspect what the browser built, then fix the source.

By the end of this chapter

- Observe a repaired tree without recommending invalid source
- Reduce parsing bugs to a testable fixture
- Assert parsed relationships without endorsing recovery

The tree builder processes tokens using insertion modes and special rules. Table parsing, implied end tags and misnested formatting are examples where a browser can build a tree that differs from an author's naive reading of the source.

Error recovery is specified so browsers can process existing pages consistently. It is not a list of recommended authoring shortcuts. A source file can appear usable while the parsed relationships break labels, selectors or accessibility semantics.

Make a small failing case

Reduce unexpected markup to the shortest document that still produces the surprising tree. Compare View Source with the live DOM. Remove scripts and extensions that might be modifying the result before blaming parsing.

Keep invalid examples explicitly labelled. When a checker reports a problem, record the intended correction and verify the resulting tree. Add a regression test for the relationship that mattered, not just a screenshot of the page.

Three useful reduced observations

A table containing `<tr>` directly can gain a tbody in the parsed tree. A selector that requires table > tr will therefore miss a row even though the source visually suggests that relationship. Write the appropriate tbody explicitly when the teaching contract depends on it.

A div start tag inside a p can close the paragraph under tree-construction rules. Later end tags can create additional unexpected structure. Misnested formatting uses the adoption-agency rules; unexpected table content can be foster-parented. These are distinct algorithms, not one generic "the browser fixes it" step.

Keep the invalid source fixture and the corrected source separate. Check the exact relationship you rely on, such as whether a section is outside a paragraph or a row belongs to the expected row group. A DOM shape observed after recovery is not permission to publish the broken source.

Avoid stream-writing as an enhancement strategy

Document.open/write/writeln/close manipulate document input streams under special rules. Calling write after ordinary parsing can replace the document, and intervening parser/network/execution conditions affect behavior. There are security and Trusted Types integration constraints as well. Prefer creating the intended nodes or a reviewed content-insertion path for a new interface; a short write call is not a general safe DOM append.

The example

HTML / example 55

```
<div class="notice">
  <p>Applications are open.</p>
  <section>
    <h2>How to apply</h2>
    <p>Collect a form at the desk.</p>
  </section>
</div>
```

This corrected structure keeps the section outside the paragraph. A paragraph cannot be used as a generic container for the whole notice, even if recovery makes the output look plausible.

A mistake to watch for

Testing a CSS selector against the intended source hierarchy rather than the hierarchy the parser actually created.

Your turn

How should you investigate a table row that appears in an unexpected place in Developer Tools? Does an automatically inserted tbody prove that the author explicitly supplied it?

Explained answer

Create a reduced complete document, inspect its actual DOM and check the source against table content rules. Parsing can relocate or imply nodes. Verify the correction with a checker and a DOM assertion. No. Record source and parsed DOM separately. Correct the authoring source and assert the intended relationship instead of relying only on a screenshot.

Read the source

HTML Living Standard: tree construction

IX / Browser Processing and Advanced Concepts

56. The document lifecycle

Wait for the condition your code actually needs.

By the end of this chapter

- Choose a readiness condition
- Separate document lifecycle from application data loading
- Separate lifecycle signals from operation promises

The document moves through loading states. DOMContentLoaded relates to parsing and eligible deferred/module script execution. The load event waits for the document's relevant dependent resources. Neither event proves that every later asynchronous operation has completed.

A script placed after a target element can often use that element immediately. A deferred external classic script runs after parsing under its ordering rules. Code that can run after DOMContentLoaded has already fired may need to inspect readyState before deciding whether to attach a listener.

Separate readiness from data

A catalogue fetch completing is an application condition, not the same as the initial DOM being ready. Give the result area explicit loading, empty, error and success states where needed. Do not leave a spinner indefinitely because a request failed.

Document navigation, visibility changes, page restoration and the back-forward cache introduce further lifecycle cases. Avoid using unload as a dependable save system. Design persistence and recovery around the actual platform behavior and test reload and back navigation.

Readiness changes do not describe every operation

readyState progresses through loading, interactive and complete under the document algorithms. At interactive, parsing has finished but the relevant deferred/module work and dependent resource lifecycle can still have work to do. readystatechange, DOMContentLoaded and load are different observation points.

The readyState guard avoids missing an already-fired DOMContentLoaded event. A module that resumes after an await or a script dynamically loaded later can need that guard too. Represent application completion with the promise or state for that operation, rather than declaring it finished when the document becomes interactive.

Use pageshow/pagehide and the persisted indication when investigating restoration. visibilitychange describes visibility state, not reliable proof that a session has ended. A pending async operation, a background timer and a back-forward-cache restoration each need their own analysis. Do not rely on unload to complete an important save or payment.

Tasks and microtasks are different work

Promise reactions and queueMicrotask use microtask processing; timers schedule tasks under their own rules. A checkpoint can drain additional microtasks queued by earlier ones. A long microtask chain can postpone other work rather than making it "background" work. Rendering is not guaranteed after every task, and requestAnimationFrame is not a fixed-frequency clock.

The example

HTML + JavaScript / example 56

```
<p id="status-message">Preparing the catalogue view.</p>
<script>
  function initialize() {
    document.getElementById("status-message").textContent = "Catalogue view ready.";
  }
  if (document.readyState === "loading") {
    document.addEventListener("DOMContentLoaded", initialize, { once: true });
  } else {
    initialize();
  }
</script>
```

This pattern handles running before or after initial parsing has finished. It announces only that this local view initialization ran, not that remote catalogue data was fetched.

A mistake to watch for

Waiting for load for every DOM task, or assuming DOMContentLoaded waits for every image and application request.

Your turn

What should your code wait for before rendering data from a specific fetch request? Does a zero-delay timer prove that the page was painted before its callback?

Explained answer

Wait for that request and its response processing, then render or show its error state. Document readiness alone does not prove the data is available. No. Scheduling and rendering have separate rules. Wait for the actual operation and measure rendering if that is the question.

Read the source

HTML Living Standard: document readiness

HTML Living Standard: stop parsing

HTML Living Standard: event loops

HTML Living Standard: animation frames

IX / Browser Processing and Advanced Concepts

57. Scripts and execution

Order, dependencies and origin rules affect the result.

By the end of this chapter

- Choose a loading strategy for dependencies
- Distinguish disabled scripting from script failure
- Test disabled and failed scripting as separate cases

An ordinary external classic script without async or defer can block parsing while it is fetched and executed. defer allows eligible external classic scripts to run after parsing, preserving their deferred execution order. async allows independent loading and execution without that dependency ordering guarantee.

Module scripts have their own loading, dependency, scope and CORS behavior. They are deferred by default in the usual parser-inserted case. Do not assume that adding type="module" to an existing global script preserves every behavior.

Give dependency rules a home

If one script relies on another, represent the dependency through an appropriate module graph or a deliberately ordered loading strategy. Import maps define supported module-specifier mappings under their own processing rules. Newer script types, including speculation rules, need their own review.

noscript provides content for defined scripting-disabled cases. It is not a catch-all for script download failures or runtime exceptions. A progressively enhanced page should keep useful content and a clear failure path even when one script fails.

Check which rules apply

For a classic script, `defer` is useful on an external script with `src`; adding it to an inline classic script does not defer that inline code. An async script must tolerate running when its own download is ready rather than relying on document order. Keep shared setup explicit instead of hoping the network will deliver files in the right order.

Module code runs in strict mode and has module scope. Top-level declarations do not become window properties merely because the module is loaded by a page. Check any code that expects globals, and test a failed dependency as well as the successful path.

Dynamic script insertion has its own flags

A dynamically inserted external classic script can run asynchronously by default. If an application deliberately needs insertion order, set the relevant async property before insertion and review the loading/error contract. Merely changing an attribute after execution does not rerun a script.

An import map uses its defined JSON imports/scopes rules and affects module-specifier resolution. It does not load a library by itself, and it does not remap every classic script src or fetch URL. Place the mapping

before dependent resolution and review current multiple-map/merging rules rather than repeating an old universal one-map claim.

A module must also satisfy fetching, MIME-type, CORS and dependency rules. Failed imports need a usable baseline. Top-level await and application promises add completion conditions beyond assuming that every statement finished at DOMContentLoaded. nomodule addresses module-capable processing, not a promise that a non-module fallback is current or accessible.

The supplied catalogue is an actual progressive enhancement: its list exists in HTML and the external script adds a local filter. Disabled controls are enabled only after their listeners are installed. A resource failure does not turn an apparently working search into a fake service.

The example

HTML + JavaScript / example 57

```
<section aria-labelledby="catalogue-heading">
  <h2 id="catalogue-heading">Supplied fictional titles</h2>
  <form id="local-filter">
    <label for="catalogue-query">Filter supplied titles</label>
    <input id="catalogue-query" name="q" type="search" disabled>
    <button type="submit" disabled>Filter titles</button>
    <button type="reset" disabled>Show all titles</button>
  </form>
  <p id="catalogue-result" role="status">All three teaching titles are listed below.</p>
  <ul id="catalogue-list">
    <li>Local histories</li>
    <li>Poetry</li>
    <li>Fiction</li>
  </ul>
  <p><a href="#catalogue-list">Browse the supplied list</a></p>
</section>
<script src="../assets/catalogue-demo.js" defer></script>
<noscript><p>Filtering needs JavaScript; the supplied list remains readable.</p></noscript>
```

The supplied deferred script enables and implements local filtering and reset. The list is meaningful before enhancement, while failed or disabled scripting leaves its controls disabled. noscript is not the runtime-failure path. The editor blocks external scripts, so use the standalone example for the filter. No server search or real stock is implied.

A mistake to watch for

Using async on two dependent classic scripts and relying on their tag order for execution order.

Your turn

Would noscript automatically display when a script downloads successfully but throws an exception? Does an import map automatically fetch every mapped library?

Explained answer

No. Scripting remains enabled. Handle the failure in the application design and keep a useful underlying document instead of relying on noscript for runtime errors. No. It supplies resolution mappings for the

relevant module loading. Test imports, failures and the non-enhanced baseline separately.

Read the source

HTML Living Standard: script

HTML Living Standard: noscript

IX / Browser Processing and Advanced Concepts

58. HTML and XML serialization

The media type changes the parser and its rules.

By the end of this chapter

- Distinguish HTML and XML processing
- Test the media type rather than guessing from source style
- Observe served media type, case and empty-element differences

HTML syntax and XML syntax are not interchangeable decorations around the same file. A response's media type determines the relevant processing path. Renaming a file or adding an XHTML-looking namespace does not make a text/html response use XML parsing.

In HTML, element and attribute processing includes defined case handling and error recovery. XML has different well-formedness, namespace and case rules. An XML parse error is not handled like an ordinary recoverable HTML authoring error.

Do not infer a mode from a slash

A trailing slash on br in HTML is not the XML mechanism applied to every element. A non-void HTML element still needs its end tag. Conversely, XML serialization has its own namespace and empty-element rules.

If you intentionally serve application/xhtml+xml, test the actual HTTP Content-Type and XML requirements. For normal HTML authoring in this book, use the short HTML doctype, UTF-8 and text/html. Preserve namespace-sensitive behavior when integrating SVG or MathML.

Namespaces and well-formedness are substantive

XML preserves case distinctions in element and attribute names. A namespaced html root is needed for the intended XHTML vocabulary, and attributes such as xmlns are not a switch that changes an existing text/html response into XML. An XML serialization must also satisfy well-formedness, including proper quoting and balanced tags.

For HTML elements in text/html, a slash before a start-tag close does not turn an ordinary div into an XML empty element. An unquoted attribute value immediately followed by slash can absorb that slash into the value, which is another reason not to treat visual slash style as mode evidence.

DOMParser accepts defined MIME-type arguments. An XML syntax error can be represented by a document containing parsererror, rather than an HTML-style recovered tree. The detailed error

presentation can differ by engine. XMLSerializer serialization is not a conversion that silently sanitizes arbitrary data or guarantees that the result can be safely reparsed in any chosen context.

Check served Content-Type and the resulting DOM together. The controlled tests use the same namespace-bearing bytes under text/html and application/xhtml+xml to demonstrate that difference; they do not change this publication into an XHTML service.

The example

HTML / example 58

```
<!doctype html>
<html lang="en">
<head><meta charset="utf-8"><title>HTML serialization</title></head>
<body><p>Neighbourhood Library<br>Reading room entrance</p></body>
</html>
```

This is intended as text/html. The br element is void and does not have an end tag. A local browser file test is not proof of the headers used by a deployed server.

A mistake to watch for

Adding xmlns to html and assuming it switches the HTML parser to XML mode.

Your turn

Which matters more for choosing the HTML or XML parsing path: an XHTML-looking source style or the actual response media type? Does adding xmlns to HTML source override a text/html response?

Explained answer

The actual processing context and response media type matter. Confirm the server behavior and follow the rules for that syntax; visual resemblance to XHTML is insufficient. No. The processing context still matters. XML parsing preserves case and has different empty-element and well-formedness behavior.

Read the source

HTML Living Standard: XML syntax

HTML Living Standard: HTML syntax

DOM Parsing and Serialization

IX / Browser Processing and Advanced Concepts

59. HTML and rendering

Default styles are a starting point, not the meaning.

By the end of this chapter

- Distinguish default appearance from meaning
- Keep source and interaction order understandable
- Keep actual navigation destinations and reading order coherent

Browsers have default rendering rules and styles. A heading often appears bold and large; an anchor often appears underlined. Those defaults make an unstyled document usable, but they do not define every semantic or accessibility behavior.

Changing display can alter layout without changing the element's underlying HTML definition. It can also interact with accessibility mappings and implementation quirks. Test the actual result rather than assuming a CSS property is semantically harmless everywhere.

Keep reading order deliberate

Layout can visually reorder items while the DOM order stays the same. A keyboard or screen reader may encounter a sequence unlike the visual arrangement. Put the logical order in HTML and use layout to adapt it without creating competing stories.

Use HTML attributes with their intended behavior rather than recreating them through presentation alone. CSS hiding, hidden and inert have different consequences. Contrast, focus visibility, reflow and zoom are also part of the usable page, even though their implementation often lives in CSS.

Visual position does not rewrite source order

Flex/grid order can change the visual sequence without rewriting the DOM's ordinary sequence. Do not repair a visual mismatch with a chain of positive tabindex values. Fix the meaningful source order, then check both the layout and the actual keyboard/reading experience.

Browser default styles are rendering guidance with implementation variation. Removing a default outline, suppressing an underline or forcing colours can remove a useful affordance. Test forced-colour settings, zoom and narrow layouts with actual content.

A computed CSS value does not prove an accessible-name or role mapping. HTML-AAM and the accessible-name computation address different questions from a screenshot. Display changes and native control styling need implementation and assistive-technology evidence beyond the semantic source definition.

The example

HTML / example 59

```
<article>
  <h1>Visit the fictional reading room</h1>
  <p>These are teaching values, not a real service timetable.</p>
  <a href="#example-hours">Read the example hours</a>
  <section id="example-hours" aria-labelledby="hours-heading">
    <h2 id="hours-heading">Example opening hours</h2>
    <p>Tuesday to Saturday, <time datetime="09:00">09:00</time>
       to <time datetime="17:00">17:00</time>.</p>
  </section>
</article>
```

The link now reaches actual in-document teaching content. The DOM gives the context before the destination and remains useful without layout. These fictional hours are not a claim about a real library.

A mistake to watch for

Choosing an element because its browser default margin looks right, then compensating for the wrong semantics with ARIA.

Your turn

Can CSS make an h2 look larger than an h1 without changing their heading levels? Can a reversed flex layout prove that DOM or keyboard order was reversed too?

Explained answer

Yes. Presentation and heading rank are different. Keep the semantic hierarchy appropriate and use CSS for the visual scale, then check that the overall design still communicates the structure. No. Test each sequence deliberately. Prefer one meaningful source order over a separate positive-tabindex patch.

Read the source

HTML Living Standard: rendering

HTML accessibility mappings

IX / Browser Processing and Advanced Concepts

60. Where the wider APIs begin

Know the boundary before calling the reference exhaustive.

By the end of this chapter

- Identify wider API responsibilities
- Keep API integration separate from member-level completeness
- Execute storage recovery and a bounded worker with failure paths

The HTML Standard includes more than markup. It defines navigation and history behavior, messaging, workers, worklets, storage, event loops and many interfaces. A language guide can introduce their integration, but a named-interface list does not document every member or algorithm.

localStorage is synchronous, origin-scoped storage with its own access and failure behavior. It is not a suitable place for secrets simply because it is local. Workers run script in a different execution environment, not a second DOM you can manipulate directly. Messaging needs explicit origin and data validation where relevant.

Treat integration as a design decision

A history update needs a corresponding content and navigation strategy. A cross-document message needs an expected sender and validated payload. A background task needs cancellation and failure handling. These responsibilities do not disappear because the API call is short.

What the catalogue actually contains

The foundation catalogue retains 166 named interface discovery routes. The expanded declaration register separately records 314 named WebIDL declaration owners and 1,574 direct member declarations in the captured source. Owners include more than interfaces, and direct declarations do not expand every inherited member. These are different inventories, not competing counts of the same thing.

Use the declaration register for signatures and links to definitions. A declaration alone does not explain an algorithm, an exception, an event sequence or a security boundary. Some member links deliberately fall back to their owning definition and are labelled accordingly. Whole-Standard behavioral coverage, imported cross-specification APIs and independent review remain open.

Storage can fail. Access may be restricted by the document's origin or browser policy, and writes can fail when storage limits are reached. Keep the page usable when a preference cannot be saved. Do not assume that a successful write is a durable backup or that any script running in the same origin is barred from reading the stored data.

Storage methods have different contracts

Storage values are strings. getItem returns null when the key is absent; key(index) can return null; ordering is not a portable insertion-order guarantee. setItem can throw QuotaExceededError, and obtaining storage for an ineligible origin or under policy can throw SecurityError. localStorage and sessionStorage have different lifetime/partition rules. Use removeItem for your own key instead of clear when unrelated same-origin settings must remain.

A storage event is delivered to the relevant other documents, not as a notification to the writing document itself. A no-change write is not a new-value event. Same-origin access is not a user-account permission boundary, and persistence can be cleared or restricted by browser policy.

History and messages need more than a method call

pushState and replaceState do not by themselves load a new view. Their serializable state and same-origin URL constraints can produce DataCloneError or SecurityError. Read popstate state and restoration needs when designing traversal. Newer Navigation APIs require separate lifecycle/compatibility work.

postMessage serializes data using the defined structured-clone rules. An uncloneable function can produce DataCloneError. Use a specific target origin for normal cross-document delivery, then validate event.origin, event.source and the payload shape at the receiver. A wildcard or a familiar-looking message string is not sender authentication. Transferring an ArrayBuffer changes ownership and detaches it under the transfer rules; it is not just copying JSON.

MessageChannel/MessagePort and BroadcastChannel have different addressing and lifecycle contracts. Start or close ports/channels as appropriate, handle messageerror where relevant and avoid assuming that a channel name is a secret permission token.

Worker and scheduling scope remains explicit

A dedicated worker has its own global scope, not document access. Construction/loading, CSP, origin restrictions, serialization, errors and termination all matter. The supplied worker is a tiny bounded calculation, not a performance benchmark or shared/service-worker lifecycle demonstration.

A timeout is a minimum-delay scheduling request, not an exact alarm. Nested timers have the defined clamping rules, and background scheduling can differ. requestAnimationFrame runs under rendering opportunities; it can be paused or throttled. The full owner/member register, worklets, shared/service workers and all event-loop/timing/navigation algorithms remain open, not silently excluded.

The example

HTML + JavaScript / example 60

```
<form id="example-preference">
  <label for="example-view">Example view preference</label>
  <select id="example-view" name="view">
    <option value="comfortable">Comfortable</option>
    <option value="compact">Compact</option>
  </select>
  <button type="submit" disabled>Store example preference</button>
  <button type="button" id="remove-preference" disabled>Remove stored preference</button>
</form>
<p id="preference-result" role="status">No storage result yet.</p>
<button type="button" id="run-worker" disabled>Run background sum</button>
<p id="worker-result" role="status">No calculation requested.</p>
<p><a id="worker-source" href="../assets/worker-demo.js" download="worker-demo.js">Download
the supplied worker source</a></p>
<script>
  const form = document.getElementById("example-preference");
  const choice = document.getElementById("example-view");
  const note = document.getElementById("preference-result");
  const key = "ctrotech.teaching.view";
  try {
    const stored = localStorage.getItem(key);
    if (stored === "compact" || stored === "comfortable") {
      choice.value = stored;
      note.textContent = "Stored example preference restored.";
    } else {
      note.textContent = "No recognized stored preference; choose a local value.";
    }
  } catch {
    note.textContent = "Storage unavailable; the selection still works locally.";
  }
  form.addEventListener("submit", event => {
    event.preventDefault();
    try {
      localStorage.setItem(key, choice.value);
      note.textContent = "Example preference stored in this browser.";
    } catch {
      note.textContent = "Not stored; the selection still works locally.";
    }
  });
  const remove = document.getElementById("remove-preference");
  remove.addEventListener("click", () => {
    try {
      localStorage.removeItem(key);
      note.textContent = "The example's stored preference was removed.";
    } catch {
      note.textContent = "Storage unavailable; no removal confirmed.";
    }
  });
  form.querySelector("button[type=submit]").disabled = false;
  remove.disabled = false;
  const run = document.getElementById("run-worker");
```

Code continues on the next page

HTML + JavaScript / example 60 / continued

```
  const workerNote = document.getElementById("worker-result");
  run.addEventListener("click", () => {
    run.disabled = true;
    workerNote.textContent = "Calculating the teaching sum.";
    let worker;
    let finished = false;
    let timer;
    const finish = text => {
      if (finished) return;
      finished = true;
      clearTimeout(timer);
      workerNote.textContent = text;
      if (worker) worker.terminate();
      run.disabled = false;
    };
    timer = setTimeout(() => finish("Calculation timed out; try again."), 5000);
    try {
      worker = new Worker(document.getElementById("worker-source").href);
      worker.addEventListener("message", event => {
        const data = event.data;
        finish(data && data.kind === "sum" && Number.isFinite(data.total)
          ? "Background sum: " + data.total
          : "The worker returned an unusable result.");
      }, { once: true });
      worker.addEventListener("error", event => {
        event.preventDefault();
        finish("Background calculation unavailable.");
      }, { once: true });
      worker.addEventListener("messageerror", () => finish("Worker message could not be read
."), { once: true });
      worker.postMessage([2, 3, 5]);
    } catch {
      finish("Background calculation unavailable.");
    }
  });
  run.disabled = false;
</script>
```

The storage controls restore, store and remove only a namespaced teaching key, with truthful failure messages. The supplied worker validates a short finite-number payload, computes 10 and is terminated after completion, reported failure or the stated five-second teaching timeout. Its URL comes from an actual source link, so the compiled hashed asset stays resolvable. The editor restricts storage/workers; use the standalone example for successful paths. This is not a durable backup, a view-layout implementation or an exhaustive worker application.

A mistake to watch for

Calling a reference complete because every interface name appears somewhere in it.

Your turn

Why does the example catch a storage failure rather than assuming localStorage is always writable? Does a storage event notify the writing document of its own setItem call?

Explained answer

Access restrictions, browser settings and storage failures can prevent writing. A preference control should remain usable and explain the actual result. Do not report a successful save when the write failed. No. Use an explicit local update and the specified events for other documents. Transfer, history and worker methods also have their own serialization, origin and lifecycle constraints.

Read the source

HTML Living Standard: web application APIs

HTML Living Standard: web storage

HTML Living Standard: messages

HTML Living Standard: workers

HTML Living Standard: history

HTML Living Standard: timers

HTML Living Standard: animation frames

X / Professional HTML Engineering

61. Accessibility starts in the markup

Semantic HTML helps, but it does not certify the whole experience.

By the end of this chapter

- Use native semantics with accurate names
- Separate automated checks from accessibility conformance
- Test the actual local filter across success, empty and reset states

Use native elements that fit the task, then review the name, role, state and behavior actually exposed to users. A button needs an understandable name and an operable action. A heading needs a useful position in the document hierarchy. A table needs its data relationships.

ARIA can supplement semantics where needed. It can also introduce wrong or conflicting information. Follow ARIA in HTML restrictions rather than adding roles to every element. Prefer an existing visible label through a native label relationship or a suitable reference before inventing an unrelated hidden name.

Review the entire task

A successful automated scan does not demonstrate WCAG conformance. Test keyboard operation, visible focus, text alternatives, error correction, zoom, reflow and appropriate assistive-technology behavior. Include the actual data and content states, not just a polished empty page.

WCAG describes requirements at the level of the delivered experience. HTML-AAM, AccName, WAI-ARIA and ARIA in HTML help answer more specific mapping and authoring questions. This edition links to them but has not completed an independent accessibility audit. Record that limitation instead of turning a few semantic examples into a certification badge.

A usable name must still describe the visible task

Accessible-name computation follows its rules, including labelledby references, aria-label, host-language labels and other context-specific sources. An aria-label can override a useful visible label in relevant cases. That is not automatically an improvement: preserve the visible label text in the accessible name where the applicable Label in Name requirement calls for it.

ARIA roles/states do not implement a custom widget's keyboard behavior, validation or permission checks. Adding role="button" to a span does not turn it into a native button. Use ARIA in HTML to check permitted overrides, and HTML-AAM/AccName for mapping/name questions rather than equating a DOM attribute with an OS accessibility result.

Follow one task across its states

For the supplied filter, test label/name, query entry, Enter activation, matching results, no matches, reset and failed scripting. Verify status communication with the actual assistive technology. Do not move focus to the status after every query merely to force an announcement.

Record zoom/reflow, contrast, visible focus, target sizing, error recovery and text alternatives as separate applicable checks. WCAG 2.2 has requirements beyond a clean semantic snippet; no automated result or role query closes that review.

The example

HTML + JavaScript / example 61

```
<section aria-labelledby="catalogue-heading">
  <h2 id="catalogue-heading">Supplied fictional titles</h2>
  <form id="local-filter">
    <label for="catalogue-query">Filter supplied titles</label>
    <input id="catalogue-query" name="q" type="search" disabled>
    <button type="submit" disabled>Filter titles</button>
    <button type="reset" disabled>Show all titles</button>
  </form>
  <p id="catalogue-result" role="status">All three teaching titles are listed below.</p>
  <ul id="catalogue-list">
    <li>Local histories</li>
    <li>Poetry</li>
    <li>Fiction</li>
  </ul>
  <p><a href="#catalogue-list">Browse the supplied list</a></p>
</section>
<script src="../assets/catalogue-demo.js" defer></script>
<noscript><p>Filtering needs JavaScript; the supplied list remains readable.</p></noscript>
```

The visible label, native controls and actual local filter support a complete small task. Enter submits, reset restores the three titles and a no-match query gets a visible status. No live catalogue or availability service is implied. The external script is supplied; the editor blocks it, so the standalone example is the enhancement test target. role=status is not proof that every assistive-technology combination announces the update.

A mistake to watch for

Adding an aria-label that contradicts the visible wording, or describing role-based automated tests as a complete accessibility review.

Your turn

A scanner reports no issues. What should you do before claiming that the search experience is accessible? Does overriding a visible label with an unrelated aria-label necessarily improve usability?

Explained answer

Complete the relevant manual and assistive-technology checks across the actual task and states, assess applicable WCAG requirements, and document results and limitations. A scanner result is one piece of evidence. No. Preserve task meaning and review name computation, permitted ARIA and Label in Name. Test the actual task and assistive-technology output, not just the attribute.

Read the source

HTML Living Standard: HTML semantics and ARIA

WCAG 2.2

ARIA in HTML

WAI tutorials

Accessible name computation

HTML accessibility mappings

WCAG: label in name

X / Professional HTML Engineering

62. Keyboard and focus review

Follow the task without touching the mouse.

By the end of this chapter

- Run a task-based keyboard check
- Plan focus after dynamic changes
- Execute removal/restoration with predictable focus destinations

Begin at the top of the page and use Tab, Shift+Tab, Enter, Space and the keys appropriate to each native control. Confirm that every needed action is reachable, the order is sensible and focus remains visible.

Do not require hover to reveal the only route to an action. A control that appears on pointer hover should have an equivalent usable focus or interaction path. Keep focus indications strong enough to locate rather than removing outlines without a replacement.

Check transitions, not just tab stops

Open and close a dialog. Submit an invalid form. Remove a focused list item. Load more results. These transitions need deliberate focus and feedback behavior. Moving focus on every update can be as disruptive as losing it.

A disabled control and an aria-disabled control do not automatically behave the same way. ARIA communicates semantics but does not implement all interaction restrictions. Choose the appropriate element/state and test the actual behavior.

Record the browser, operating system and assistive technology used. A successful keyboard pass in one environment is useful evidence, not a claim about all combinations.

Focus must survive the state transition

A predictable focus order is about usable relationships, not merely counting Tab presses. If a focused row disappears, select the next or previous relevant action, or a meaningful heading when the list is empty. Restore should also have a deliberate destination. The working example makes that policy inspectable.

Sequential focusability, programmatic focus, disabled state and aria-disabled remain different. A custom aria-disabled action still needs its click/keyboard suppression implemented. A sticky overlay can obscure the focused item even when the item is technically focused; review Focus Not Obscured alongside Focus Order and Focus Visible.

Native control keys and platform tab preferences can vary. Record actual key actions and results, not a universal assumption that every native widget exposes exactly the same tab stops. Focus tests should cover pointer-triggered transitions too, and never claim OS screen-reader output from document.activeElement alone.

The example

HTML + JavaScript / example 62

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Local reading-list focus review</title>
<style>
  .skip-link { position: absolute; left: .5rem; top: .5rem;
    transform: translateY(-180%); background: white; color: black;
    padding: .5rem; z-index: 2; }
  .skip-link:focus { transform: none; }
  a:focus-visible, button:focus-visible, [tabindex]:focus-visible {
    outline: 3px solid #163fc2; outline-offset: 3px;
  }
</style>
</head>
<body>
<a href="#keyboard-main" class="skip-link">Skip to main content</a>
<main id="keyboard-main" tabindex="-1">
  <h1 id="list-heading" tabindex="-1">Fictional reading list</h1>
  <ul id="removable-list">
    <li><span>Local histories</span> <button type="button" disabled>Remove Local histories</
button></li>
    <li><span>Poetry</span> <button type="button" disabled>Remove Poetry</button></li>
    <li><span>Fiction</span> <button type="button" disabled>Remove Fiction</button></li>
  </ul>
  <p id="remove-status" role="status">Changes stay in this page only.</p>
  <button id="restore-list" type="button" disabled>Restore initial titles</button>
</main>
<script>
  const list = document.getElementById("removable-list");
  const note = document.getElementById("remove-status");
  const heading = document.getElementById("list-heading");
  function enableItems() {
    list.querySelectorAll("button").forEach(button => button.disabled = false);
  }
  list.addEventListener("click", event => {
    if (!(event.target instanceof Element)) return;
    const button = event.target.closest("button");
    if (!button || button.disabled) return;
    const item = button.closest("li");
    const next = item.nextElementSibling || item.previousElementSibling;
    const destination = next ? next.querySelector("button") : heading;
    const title = item.querySelector("span").textContent;
    item.remove();
    destination.focus();
    note.textContent = title + " removed from this local list.";
  });
  const restore = document.getElementById("restore-list");
  restore.addEventListener("click", () => {
    list.replaceChildren();
    for (const title of ["Local histories", "Poetry", "Fiction"]) {
```

Code continues on the next page

HTML + JavaScript / example 62 / continued

```
      const item = document.createElement("li");
      const text = document.createElement("span");
      const button = document.createElement("button");
      text.textContent = title;
      button.type = "button";
      button.textContent = "Remove " + title;
      item.append(text, " ", button);
      list.append(item);
    }
    heading.focus();
    note.textContent = "The initial teaching list was restored.";
  });
  enableItems();
  restore.disabled = false;
</script>
</body>
</html>
```

The skip link has a supplied focus-visible style, and removal now performs the actual local action. Focus moves to the next item, otherwise the previous item, otherwise the heading. Restore rebuilds the teaching list safely and returns focus to its heading. This is not saved to a server; status announcements and focus visibility still need actual OS AT and user review.

A mistake to watch for

Setting a positive tabindex sequence to match a visual layout instead of fixing the source order.

Your turn

After deleting the focused row in a list, where should focus go? What should happen when the final item is removed in this example?

Explained answer

There is no universal target. Choose a predictable nearby control or a relevant list heading based on the task, then test the behavior. Do not let focus disappear into the document body accidentally. The meaningful list heading receives focus. The visible result explains the local removal; actual assistive-technology feedback must still be checked.

Read the source

HTML Living Standard: focus model

WCAG: focus order

WCAG: focus not obscured

X / Professional HTML Engineering

63. Trust boundaries

Treat browser input as something the server must verify.

By the end of this chapter

- Identify client-controlled data
- Choose controls for specific trust boundaries
- Distinguish text insertion, serialization and origin validation

A browser can display constraints and hidden values, but a requester can send a different request. Validate on the server, enforce the relevant authentication and authorization rules, and apply abuse controls appropriate to the service.

For content insertion, distinguish plain text from accepted markup. textContent fits text. Markup acceptance needs a maintained, context-appropriate sanitization strategy. A handmade list of forbidden strings is not a dependable general sanitizer.

Review each boundary

A form crosses into a server handler. An iframe crosses into another document. A script can introduce a remote execution dependency. A file upload crosses into storage and later retrieval. Document the expected origin, allowed capabilities and validation for each boundary.

Security policies and headers belong to the deployment as well as the document. CSP, CORS, SRI, referrer policy and Permissions Policy address different concerns. Do not describe any one of them as a complete defense. Test the intended failure behavior, and never place private API credentials in HTML, client JavaScript, data attributes or a downloadable environment file.

An origin is a browser boundary, not the whole service policy

Origin comparison uses the relevant scheme/host/port rules, not a substring test on a URL. Sandboxing without allow-same-origin can create an opaque origin; a null message origin is not enough to identify a trusted sender. Capability allowances and destination validation need deliberate design.

CSP limits covered execution/resource behavior, CORS governs specified cross-origin response sharing, SRI checks resource bytes, Referrer Policy controls referrer information and Permissions Policy controls eligible features. None substitutes for authenticated server authorization. A blocked script request and a refused form action are different failure paths.

Encode for the actual output and review the sanitizer

For embedded JSON in script text, JSON.stringify alone is not a general HTML-context encoder: a raw closing script delimiter can end the element before JavaScript processes it. A reviewed serializer can escape the relevant less-than characters as JavaScript/JSON escapes, then preserve the data on decoding. Do not reuse that technique indiscriminately for HTML attributes, CSS or URLs.

A Trusted Types policy whose callback simply returns input does not create a sanitizer. Browser enforcement can reject a string at a covered sink, while unsafe allowed transformations can still be unsafe. Prefer plain text where markup is not a product requirement, keep maintained sanitizer versions/configuration visible and test both rejected input and allowed output.

For servers, require the intended method/content handling, identity, authorization, CSRF protection where applicable, upload constraints and log/privacy controls. Hidden fields and a correctly validated client form are not substitutes. This static publication does not add a collection service.

The example

HTML + JavaScript / example 63

```
<p id="reader-comment"></p>
<script>
  const commentFromAReader = "A useful guide <script>example</scr" + "ipt>";
  document.getElementById("reader-comment").textContent = commentFromAReader;
</script>
```

The comment is rendered as text. Splitting the closing script spelling here avoids accidentally terminating the surrounding HTML script element. Real serialization should use a reviewed data-embedding strategy.

A mistake to watch for

Thinking that a value is trusted because it originated in a hidden input or a front-end build variable.

Your turn

Why must the server ignore a submitted role="admin" value when deciding a requester's permissions? Does a Trusted Types policy that returns every input string establish safe HTML?

Explained answer

The requester controls submitted data. Authorization must use trusted server-side identity and permission information, not a role string supplied by the client. No. Enforcement and the safety of the policy transformation are separate. A client-side origin or role string is not trusted server authorization either.

Read the source

HTML Living Standard: dynamic markup insertion

OWASP: input validation guidance

OWASP: authorization guidance

OWASP: XSS prevention

OWASP: CSP guidance

X / Professional HTML Engineering

64. Performance without folklore

Find the slow resource before adding an optimization.

By the end of this chapter

- Relate HTML loading choices to real resources
- Report measured results with their conditions
- Use a supplied dimensioned resource without fabricating a speed result

HTML influences resource discovery and loading. An early stylesheet link, a delayed image source, script loading attributes and image candidates can affect the page's critical work. Measure the actual page on representative conditions before prescribing changes.

Provide image dimensions and appropriate resources. Keep likely important above-the-fold media from being needlessly delayed. Lazy loading is useful when deferring a resource is appropriate; it is not a rule to apply to every image.

Test a before and an after

Record the page version, device or emulation, network conditions and measurement method. Repeat enough to recognize variation. Do not invent percentage improvements or treat one warm-cache result as a stable performance guarantee.

A fast initial response can coexist with a slow interaction. Loading less script helps only when it addresses actual work. Keep the underlying document usable while enhancements load. Review the network waterfall and interaction path together rather than optimizing a score in isolation.

Attribute hints have limited states

fetchpriority accepts high, low and auto, with Auto for missing or invalid values. It influences the relevant fetch, not a guaranteed queue position. loading uses its defined eager/lazy states; missing/invalid follows its eager default. Deferred lazy resources also have scripting-related conditions. Verify the actual resource request, decoding and layout instead of treating attribute reflection as a speed measurement.

Match a metric to the user question

LCP concerns an eligible largest-content paint candidate, CLS concerns qualifying unexpected layout shifts and INP concerns the relevant interaction responsiveness measurement. Those metrics have their own definitions, aggregation and implementation boundaries; a request count is none of them. Field data and a controlled lab run answer different questions.

Performance entries may expose limited cross-origin details without the appropriate Timing-Allow-Origin configuration. Warm caches, service workers, throttling and device work can change results. Record the conditions and repeat comparisons; no percentage or score improvement is claimed by this example.

The example

HTML / example 64

```
<figure>
  <img src="../assets/media/floor-plan-800.png"
       alt="Fictional schematic showing entrance, reception and reading room."
       width="800" height="450" style="max-width: 100%; height: auto"
       loading="eager" fetchpriority="high">
  <figcaption>A supplied teaching schematic, not a real entrance photograph.</figcaption>
</figure>
```

The resource is actually supplied and decodes to 800 by 450 pixels, so the dimensions reserve its ratio. The CSS also constrains the displayed width to the available space while retaining that ratio. This is a single likely important figure in a tiny teaching page. A high priority hint does not prove earlier delivery or an improved performance score; compare measured alternatives before generalizing.

A mistake to watch for

Claiming that fetchpriority="high" on every image makes the page faster.

Your turn

What evidence should accompany a claim that an HTML loading change improved performance? Does a reflected fetchPriority value of high prove that an image downloaded first?

Explained answer

A recorded before/after measurement under stated comparable conditions, with the tested page version and enough detail to reproduce it. Explain variability and avoid generalizing beyond the measured scenario. No. It is a hint under the fetch rules. Record actual comparable measurements, not a property value presented as a performance result.

Read the source

HTML Living Standard: fetch priority

HTML Living Standard: lazy loading

Resource Timing specification

Web performance metrics

X / Professional HTML Engineering

65. Findable and shareable pages

Describe the page accurately before chasing a search treatment.

By the end of this chapter

- Write task-specific titles and links
- Qualify metadata and search-consumer claims
- Keep discovery links and fictional metadata honest

Use descriptive titles, meaningful headings and links, useful body content and accurate metadata. A crawler cannot discover a resource through a private thought or a button whose only destination exists in opaque application state.

Canonical URLs should identify the intended canonical document, not point every page to the home page. Alternate-language relationships need correct URLs and language relationships. Structured data should match the visible content and an actual supported vocabulary.

Keep promises narrow

A meta description can help summarize a page but does not force a search snippet. Structured data does not guarantee a rich result. Social-card previews depend on platform behavior, caching and the accessible resource URLs.

Test the deployed output, not only templates. A robots restriction, a wrong status code, an inaccessible asset or a repository subpath error can undo otherwise tidy head markup. Do not add a measurement claim or an SEO guarantee to sales copy without actual evidence.

Discovery is not access control

A crawler needs to fetch a page to read its `noindex` instruction. Blocking that fetch in `robots.txt` is not a reliable way to make the crawler process the page's noindex metadata. Neither directive protects private information. Use actual authentication and authorization for private content.

An `llms.txt` file can offer a plain-text route through a publication, but it is not a universal crawler protocol or a promise of inclusion in AI answers. Keep any discovery export consistent with the visible book. Search visibility, snippet choice and model use remain outside the publisher's control.

Relationships must agree with reachable content

Use real href destinations for crawlable navigation. A canonical is a consumer signal, not a redirect. Language alternatives need the correct language tags, corresponding reachable variants and consumer-specific rules; a tag alone does not translate a page. Redirect/status handling, relative-base resolution and content consistency all need deployed checks.

Consumer-specific social-card and structured-data formats are not automatically HTML normative behavior. Keep image URLs and visible content accurate, and do not invent an author credential, rating or service policy to fill a field.

robots and noindex are discovery/indexing instructions, not secrets protection. A sitemap is a route suggestion, not an indexing guarantee. Optional llms exports should match the actual publication and its rights; they neither grant a new reuse license nor guarantee model retrieval or citation.

The example

HTML / example 65

```
<article>
  <h1>Example membership-renewal guide</h1>
  <p>This fictional checklist is for teaching, not a real library policy.</p>
  <a href="#sample-document-list">Read the sample document checklist</a>
  <section id="sample-document-list" aria-labelledby="document-heading">
    <h2 id="document-heading">Sample document checklist</h2>
    <ul>
      <li>A fictional practice membership card.</li>
      <li>A completed practice form containing no real personal details.</li>
    </ul>
  </section>
</article>
```

The descriptive native link reaches actual supplied fictional content. It no longer advertises an absent accepted-document page or a real eligibility policy. Search indexing, rich results and ranking are not established by this snippet.

A mistake to watch for

Using the same generic summary and title on every page, then adding unsupported ranking claims.

Your turn

Does a valid canonical link prove that a search engine will choose your preferred URL? Can robots restrictions safely protect an unlisted private page?

Explained answer

No. It supplies a signal that consumers interpret under their own rules. Keep the signal accurate and consistent with redirects, links and actual content, then inspect the deployed behavior. No. Use real authorization. Discovery instructions and consumer signals must not be represented as an access-control system.

Read the source

HTML Living Standard: document metadata

HTML Living Standard: link types

Google Search Central: block indexing with noindex

Google Search Central: AI features and your website

Google: crawlable links

X / Professional HTML Engineering

66. Maintainable HTML

A useful baseline makes later changes less fragile.

By the end of this chapter

- Keep a useful baseline where feasible
- Review content variation and maintenance responsibilities
- Test the actual no-script and failed-resource baseline

Keep the content, relationships and primary task available before adding optional enhancements where the service permits it. Progressive enhancement is a design approach, not a promise that every application can offer identical behavior without scripting.

Use consistent naming, a small number of clear components and maintained sources for repeated structures. Avoid editing separate website and PDF copies of the same explanation. A single-source build reduces drift, but the generated outputs still need independent checks.

Plan for changed content

Test a long name, a translated label, no results, a server error and more content than the design screenshot contained. A layout that only works with one short English phrase is not ready for real users.

Keep source review dates, compatibility notes and meaningful changelog entries. A Living Standard and browser implementations can change independently. Assign an actual maintainer before promising updates. Do not advertise an automated maintenance process that has not been configured.

Enhancement must fail honestly

A form aimed at a static filename does not acquire a server query handler merely because its query string looks plausible. Supply a real handler or clearly describe the enhancement as local. The revised example keeps all fictional titles in HTML, disables the unavailable filter initially and enables it only after setup.

Test no scripting, blocked/missing resources and runtime failures separately. noscript only addresses its defined scripting-disabled conditions. Keep the primary route usable where feasible; do not display an action as ready before its handler is available.

Make the change traceable through every output

Keep one maintained manuscript, rebuild website/PDF/code exports and identify the source/artifact fingerprints for the actual check. A single-source pipeline reduces drift but does not prove that typography, links, accessibility tags and examples survive transformation.

When a resource is hashed or moved, test the compiled request, not just the unbuilt source URL. Store compatibility/review dates alongside a correction and distinguish a new Standard definition from stable implementation support. Backups, repository history and reproducible build instructions are different maintenance assets, not interchangeable claims.

The example

HTML + JavaScript / example 66

```
<section aria-labelledby="catalogue-heading">
  <h2 id="catalogue-heading">Supplied fictional titles</h2>
  <form id="local-filter">
    <label for="catalogue-query">Filter supplied titles</label>
    <input id="catalogue-query" name="q" type="search" disabled>
    <button type="submit" disabled>Filter titles</button>
    <button type="reset" disabled>Show all titles</button>
  </form>
  <p id="catalogue-result" role="status">All three teaching titles are listed below.</p>
  <ul id="catalogue-list">
    <li>Local histories</li>
    <li>Poetry</li>
    <li>Fiction</li>
  </ul>
  <p><a href="#catalogue-list">Browse the supplied list</a></p>
</section>
<script src="../assets/catalogue-demo.js" defer></script>
<noscript><p>Filtering needs JavaScript; the supplied list remains readable.</p></noscript>
```

The supplied list is usable without JavaScript. An actual external script adds local filtering and reset; disabled or failed scripting leaves an honest browse-only baseline. This is not a server-side search endpoint. Test the standalone example because the restricted editor blocks external scripts.

A mistake to watch for

Calling a page progressively enhanced when every visible route silently depends on one script succeeding.

Your turn

What content states should you test besides the short, successful example shown in a design file? Does a static catalogue.html file implement search just because a form submits q to it?

Explained answer

Include empty, loading, error, long-content, translated and high-density states as relevant to the task. Test the actual interaction and recovery path, not just their appearance. No. Implement a real endpoint or disclose the local-only behavior. In this example the list remains readable and unavailable enhancement controls remain disabled.

Read the source

HTML Living Standard: extensibility and authoring

X / Professional HTML Engineering

67. A release checklist that means something

An unchecked box is an open task, not a smaller font.

By the end of this chapter

- Keep independent review states
- Tie release claims to specific evidence
- Separate gate evidence, tagged structure and real deployment

Check the built documents with a recorded checker version. Test internal links and referenced assets under the real deployment base path. Inspect the browser console and run behavior tests for the examples that make behavior claims.

Review keyboard navigation, focus, names, text alternatives, form errors, reflow and relevant assistive-technology use. Record what was tested and what was not. Security and accessibility reviews need named responsibilities and evidence rather than decorative badges.

Separate authoring from approval

An answer being written does not mean it has been independently checked. A chapter existing does not mean every assigned feature is fully documented. Keep drafting, example conformance, browser behavior, compatibility and editorial approval as separate states.

For publication, verify the PDF's text, links, navigation and accessibility workflow; do not call an untagged export an accessible PDF. Check downloads in a logged-out browser and test the promised delivery method. The published reading edition keeps its executed checks and open reviews separate. Being available on the web does not mean every whole-Standard coverage gate has closed.

Record an observation, not a badge

A useful release note identifies the artifact, the date, the tool or browser, the action performed and the observed result. "The homepage loaded" is useful delivery evidence, but it says nothing about an untested registration handler. "Zero automated accessibility violations" describes that scan, not a complete keyboard or screen-reader review. Keep the original limits beside the result so the next reader knows what still needs testing.

For this book, report corrections with the chapter or reference URL, the passage, a reproducible example when relevant and the source that supports the proposed change. Do not include personal form data, private credentials or unpublished security details in a public issue.

Close the gate the evidence actually covers

For each promised element, attribute, keyword/default, member and algorithm, retain its owner, definition, teaching location, test/review evidence and unresolved conditions. A captured signature or a nearby mention is not completed behavioral coverage. No scope reduction for this publication has been approved.

Use a release matrix that separates manuscript/explained answers, exhaustive coverage, accessibility/security and the accessible tagged PDF. A build pass may update the build row without closing any of those four gates. Record artifact hashes, source commit, tool versions and the actual action/result for each claim.

A tagged PDF requires semantic structure

Selectable text, bookmarks and a document-language field do not supply a tag tree. Headings, paragraphs, lists, tables, figures, links, code and reading order need appropriate semantic treatment; decorative material and repeating furniture need appropriate artifact handling. Font/Unicode mapping, link associations and language changes also require review.

Check the generated tag tree and logical reading order, then use suitable validation and actual assistive-technology inspection. A four-page prototype does not certify a full book, and machine validation alone does not establish user experience or an independent approval. The full-book tagged edition remains an explicit open gate.

Verify delivery separately

A successful Git push means the remote commit was updated; read it back. A hosted CI pass means its defined jobs completed, not that Vercel deployed the current site. Check actual production routes, downloads, headers and rollback/recovery conditions before claiming delivery of a new edition. Keep private credentials out of source, logs and public issue reports.

The example

HTML / example 67

```
<section aria-labelledby="release-heading">
  <h2 id="release-heading">Before publishing</h2>
  <ul>
    <li>Validate the built documents and record the output.</li>
    <li>Test the main tasks with a keyboard.</li>
    <li>Check paths under the deployment subdirectory.</li>
    <li>Record remaining content and technical review work.</li>
  </ul>
</section>
```

This is a visible checklist, not an assertion that its tasks passed. A project should link each claimed result to the relevant command output, environment or review note.

A mistake to watch for

Renaming a draft final and carrying its unchecked review states into the release unchanged.

Your turn

What should you do if examples validate but the API-member inventory is incomplete? Do selectable PDF text and bookmarks establish an accessible tag tree?

Explained answer

Report the example result separately, keep API coverage open, and limit the publication promise or complete the missing inventory before making the broader claim. One passing check cannot close unrelated work. No. Semantic tags, associations, reading order and accessibility review are separate work. Build, repository delivery, hosted CI and production deployment must also be evidenced separately.

Read the source

HTML Living Standard: conformance

WAI PDF technique: reading and tab order

XI / Practical Projects

68. Build a personal profile

Build a truthful volunteer profile whose headings and contact-status link work without scripts.

By the end of this chapter

- Organise a fictional person's introduction and experience into a useful heading structure.
- Make a descriptive contact-status link resolve to real content.
- Distinguish a complete static profile from a working contact service.

Before you start

Review Chapters 7, 14, 17, 18, 23, 25. Create a document shell, choose headings, make a list and connect a same-page link.

Working terms

A fragment is the part of a URL after # that can identify a document destination. A landmark identifies a major page region through native HTML or an appropriate role. Contact status explains whether a contact route actually exists; it is not itself a messaging service.

Build contract

Create a one-page profile for Ada, a fictional library volunteer. Include one page-level title, a short introduction, three relevant practice activities and a contact-status section. Make the fictional context visible rather than relying on a README. Do not invent a real address, availability claim or endorsement. The reference has no portrait because no visual information is needed to complete this task.

Implementation route

Begin with the content, then choose the structure. Put the whole page topic in the main heading. Experience and contact status are separate sections with second-level headings. Use a list for the three activities, not a table or a row of decorative cards. A link to `#contact` takes the reader to the status section. It is navigation, so it remains an anchor even if styled prominently.

Acceptance and recovery

Open the file without JavaScript. Read it in document order, activate the contact-status link with Enter and confirm that the destination explains the absence of a contact service. Try a 320 CSS-pixel viewport and enlarge text. The profile should remain readable without page-wide horizontal scrolling. No-script operation is a property of this static design, not a claim that the reader has tested it.

Reference walkthrough

The download adds a language declaration, title, viewport metadata, a keyboard skip link and a local stylesheet around the subject content. Its activity statements are explicitly practice tasks. The contact section gives no coordinator instruction that might be mistaken for a real service. Replacing that status with a real route is a separate publishing decision requiring verified details and the person's agreement.

Assessment rubric: 28 observable points

Award one point for each demonstrated check below, four per area. Record the evidence next to the check; untested checks receive no point. These points are an authoring assessment aid, not WCAG conformance or security certification. A broken primary task, a misleading service claim, unsafe data collection or a dead required destination blocks readiness regardless of the score.

Content and truth (4 points)

- Fictional context is visible.
- Ada is named.
- Introduction identifies the practice role.
- No unverifiable service claim appears.

Structure (4 points)

- One main landmark is present.
- The page topic has an h1.

- Experience has an h2.
- Contact status has an h2.

Experience (4 points)

- Three activities are supplied.
- Activities form a native list.
- Each activity is specific.
- No table is used for layout.

Contact navigation (4 points)

- The link says what it opens.
- Its fragment matches a unique ID.
- Keyboard activation reaches the section.
- The section states no contact route is supplied.

Baseline (4 points)

- No script is needed to read.
- No script is needed to navigate.
- Stylesheet failure leaves logical content.
- No fabricated form is present.

Presentation (4 points)

- Text remains readable at 320 CSS pixels.
- The page has no horizontal overflow.
- Keyboard focus is visible.
- Text enlargement does not hide the contact section.

Delivery (4 points)

- The HTML has a useful document title.
- The language is declared.
- All local links resolve.
- The README explains how to open the page.

The example

HTML / example 68

```
<main>
  <h1>Ada, a fictional library volunteer</h1>
  <p>A made-up profile for practising HTML, not a real volunteer service.</p>
  <p>In this exercise, Ada helps readers explore a small collection.</p>
  <a href="#contact">Read the contact status</a>
  <section>
    <h2>Practice experience</h2>
    <ul><li>Preparing a fictional reading-group notice.</li>
      <li>Organising a sample shelf list.</li>
      <li>Writing a catalogue entry as a learning task.</li></ul>
  </section>
  <section id="contact"><h2>Contact status</h2>
    <p>No personal contact details or messaging service are supplied.</p>
  </section>
</main>
```

The fragment teaches the profile content, not a complete document shell. The downloadable index.html supplies that shell and keeps the contact destination on the same page. A section heading identifies the contact topic; a link is appropriate because activation navigates rather than submitting data. The list expresses a collection of activities. Fictional status is part of the interface, not hidden assessment metadata.

A mistake to watch for

A prominent Contact button with no destination promises an action the page cannot provide. Use a descriptive link to the actual contact-status section, and state the absence of a service.

Your turn

A learner replaces the contact-status anchor with a button that does nothing and removes the fictional notice. Which checks fail, and what is the smallest truthful repair?

Explained answer

The contact-navigation checks fail because the button has neither a destination nor an implemented action. Content truth also fails: a reader can mistake the volunteer and contact route for real offerings. Restore an anchor whose href is #contact, retain a unique contact section that states no route is supplied, and restore the visible fictional notice. Test Enter activation and the narrow page again. Styling alone cannot repair the broken task.

Complete project folder: project-profile.zip in the companion downloads.

Read the source

HTML Living Standard: heading elements

HTML Living Standard: links

WCAG 2.2 Understanding: Reflow

WCAG 2.2 Understanding: Focus Order

XI / Practical Projects

69. Build a portfolio

Build a three-page learning portfolio with two distinct project notes and honest contact status.

By the end of this chapter

- Connect two project summaries to separate working detail pages.
- Keep primary navigation and current-page information consistent.
- Describe static learning work without inventing clients, outcomes or a contact backend.

Before you start

Review Chapters 17, 24, 25, 27, 61. Resolve sibling-relative links, structure independent project notes and plan navigation across files.

Working terms

A project summary introduces one item of work; a detail page supplies its substantive explanation. A current-page marker describes the document currently being viewed. A relative URL is resolved against a base URL, so moving files can change its destination.

Build contract

Deliver `index.html`, `project.html` and `reading-list.html`. The index introduces a fictional front-end learner's work, summarises two different studies and links each to its own detail page. The notice-board note explains a document-structure study; the reading-list note explains why its collection is a list. Include a contact-status section on the index. No contact form or real personal information is required.

Implementation route

Write the two summaries before the navigation. Use articles for independently understandable project summaries, with third-level headings under Selected work. Give each link a destination-specific name. Reuse a primary navigation landmark across the three documents and mark only the current document's link with `aria-current="page"`. This attribute supplies current-page information; visible styling and actual destinations still matter.

Acceptance and recovery

Open each detail page directly, not only through the index. Confirm that its title identifies its own subject and that its primary links still work. Return to `index.html#contact` from each detail. Remove scripting and check again. A browser reload, an extracted ZIP and a narrow viewport must not turn either project into an empty placeholder.

Reference walkthrough

The supplied files have real relative links, consistent local CSS and no client or revenue claims. Each detail page includes actual sample content and a short design decision. The contact section states that the site does not collect messages. This meets the fictional exercise contract. A real professional portfolio would require the owner's own work, permission to show it, evidence for outcomes and an appropriate verified contact route.

Assessment rubric: 28 observable points

Award one point for each demonstrated check below, four per area. Record the evidence next to the check; untested checks receive no point. These points are an authoring assessment aid, not WCAG conformance or security certification. A broken primary task, a misleading service claim, unsafe data collection or a dead required destination blocks readiness regardless of the score.

Portfolio content (4 points)

- The fictional status is visible.
- An introduction is supplied.
- Two distinct studies are summarised.
- No unsupported client or outcome claim appears.

Detail destinations (4 points)

- The notice-board link opens project.html.
- The reading-list link opens reading-list.html.
- Both detail documents contain substantive content.
- The two links do not lead to the same empty page.

Structure (4 points)

- Selected work has a heading.
- Each project summary has a subordinate heading.
- Each page has a main landmark.
- Document titles distinguish the three pages.

Navigation (4 points)

- All three pages expose primary navigation.
- All primary destinations resolve.
- Exactly one link per primary navigation is current.
- Both detail pages can reach the contact status.

Contact truth (4 points)

- Contact status is present on the index.
- Its ID is unique.
- No fake message submission is offered.
- No real personal details are invented.

Recovery (4 points)

- Each detail opens directly.
- Links work without JavaScript.
- Links work after ZIP extraction.
- 320-pixel reading does not cause page-wide overflow.

Delivery (4 points)

- Each page declares its language.
- Focus styling is visible.
- All stylesheet references are local and resolve.
- Assessment notes identify static limitations.

The example

HTML / example 69

```
<section aria-labelledby="work-title">
  <h2 id="work-title">Selected work</h2>
  <article><h3>Library notice board</h3>
    <p>A static notice-page structure study.</p>
    <a href="../projects/portfolio/project.html">Read the notice-board note</a>
  </article>
  <article><h3>Reading list</h3>
    <p>A semantic list study, not a reservation service.</p>
    <a href="../projects/portfolio/reading-list.html">Read the reading-list note</a>
  </article>
</section>
```

These are two separate studies with separate destinations. The chapter fragment uses paths from the wrapped examples directory to the supplied reference documents. Inside the extracted project folder, the index uses sibling-relative URLs instead. The reference primary navigation marks only the active document with aria-current="page". A current-page marker cannot fix a wrong href. Both detail pages include a return route to the index contact status, which is deliberately not a message service.

A mistake to watch for

Repeating one project description behind two different link labels makes the portfolio look more complete than it is. Build both detail pages and inspect each directly.

Your turn

Both project links currently open project.html, while reading-list.html contains only a title. Is the three-file count enough? Explain the repair and how to verify it.

Explained answer

No. A file count proves neither distinct work nor usable navigation. Write the reading-list sample and its list-design explanation, point the corresponding summary to reading-list.html, and keep the notice-board summary pointing to project.html. Open each detail directly, compare the content, inspect the document title and current-page link, then follow its contact-status return link. Repeat from the extracted ZIP with JavaScript disabled.

Complete project folder: project-portfolio.zip in the companion downloads.

Read the source

HTML Living Standard: the article element

HTML Living Standard: the nav element

HTML Living Standard: links

WCAG 2.2 Understanding: Focus Order

XI / Practical Projects

70. Build a registration form

Build an explicitly local form demonstration with linked errors, reset recovery and disabled no-script controls.

By the end of this chapter

- Associate required labels, instructions and field errors with three practice controls.
- Keep a no-script or failed-script form disabled instead of falling back to GET collection.
- Test linked error correction, reset and honest local-success messaging.

Before you start

Review Chapters 34, 35, 36, 38, 40, 41, 53, 66. Build labelled controls, inspect browser validity, distinguish form submission from local script and preserve a safe failure baseline.

Working terms

Constraint validation checks applicable browser-side rules, not server trust. An error summary collects correction links above a form. Custom validity is a script-supplied message that makes a control invalid while the message is nonempty. A disabled baseline keeps this practice task unavailable until its local handlers are installed.

Build contract

Collect only fictional practice values for name, one email and a session choice. This is a browser-side interface study, not a registration service. The baseline fieldset is disabled. No control has a submission name and there is no application endpoint. Tell readers not to enter personal information. The local script must install its handlers before enabling fields, cancel submit, validate the fields and display only a local result.

Implementation route

Use explicit labels and visible required indicators. Leave native `required` and `type="email"` constraints on the controls. After setup, `form.noValidate = true` selects the custom-summary submit path; it does not remove ValidityState or make the values trusted. The script inspects each control's validity and adds a non-whitespace rule for the practice name with `setCustomValidity()`. The browser email syntax check cannot prove that an address exists or belongs to the user.

Place a summary before the form with links to invalid fields. Reveal inline error text, set `aria-invalid="true"` only on failed fields and focus the summary after an unsuccessful submit. Use stable description IDs for the field messages and email hint. When the learner edits a failed field, revalidate it and remove that error when corrected. Clear any previous success message as soon as input changes. The script builds message nodes using text content, not HTML interpolation.

Acceptance and recovery

Submit all fields empty, then a whitespace-only name and malformed email, then a valid practice set. Follow every summary link with the keyboard. Confirm that no form navigation or application request occurs and the URL does not acquire query values. Reset after an error and after success; values, errors, invalid states and custom name validity must return to their initial state. With JavaScript disabled or `demo.js` blocked, the form remains disabled and its explanation stays readable. This is failure containment for this static reference, not a claim of universal browser privacy.

Reference walkthrough

The chapter's marked form content is reused in both the registration and capstone downloads. One maintained local script supplies both copies. The form has no named application controls, but that omission is not the primary no-submit mechanism: the disabled baseline and installed submit handler are essential. `autocomplete="off"` is only a browser hint and cannot guarantee that a browser stores nothing. The demonstration does not write application data to cookies or Web Storage. A real service needs a real endpoint, server validation, an appropriate transport and request-security design, data minimisation and a truthful retention notice; enabling this static form is not that implementation.

Assessment rubric: 28 observable points

Award one point for each demonstrated check below, four per area. Record the evidence next to the check; untested checks receive no point. These points are an authoring assessment aid, not WCAG conformance or security certification. A broken primary task, a misleading service claim, unsafe data collection or a dead required destination blocks readiness regardless of the score.

Control contract (4 points)

- The page visibly says practice only.
- Exactly name/email/session data are requested.
- All controls have explicit labels.
- Required state is visible and programmatic.

Disabled baseline (4 points)

- The initial fieldset is disabled.
- No application endpoint is claimed.
- Blocked scripting leaves the fields disabled.
- Instructions remain readable without scripts.

Validation (4 points)

- Empty values produce three errors.
- Whitespace-only name is rejected.
- Malformed single-email syntax is rejected.
- A valid practice set produces only local success.

Error navigation (4 points)

- The summary precedes the form.
- The failed-submit summary receives focus.
- Each error link focuses its field.
- Each invalid field has an associated visible message.

Correction (4 points)

- Invalid fields expose aria-invalid.
- Correcting a failed field removes its error.
- Editing removes an earlier success message.
- Hidden summary content is not left visibly stale.

Reset (4 points)

- Name/email/session values reset.
- Custom name validity resets.
- Inline and summary errors clear.
- Invalid attributes and earlier success clear.

Boundary evidence (4 points)

- Submit does not change the URL.
- No application request is emitted.
- The script writes no application storage.
- Real-service requirements are clearly distinguished.

The example

HTML + JavaScript / example 70

```
<link rel="stylesheet" href="../assets/project-reference.css">
<main><h1>Reading-group form study</h1>
<!-- registration:start -->
<p class="note" id="demo-note">Practice only. Use made-up details such as reader@example.inv
alid. This page checks fields locally; no application is sent or saved by its script. Browse
r autofill and history are separate browser features. JavaScript must load successfully to e
nable the fields. This is not a registration service.</p>
<section id="demo-errors" class="errors" aria-labelledby="demo-errors-title" tabindex="-1" h
idden>
  <h2 id="demo-errors-title">Correct these fields</h2>
  <ul id="demo-error-list"></ul>
</section>
<form id="demo-form" aria-describedby="demo-note" autocomplete="off">
  <fieldset id="demo-fields" disabled>
    <legend>Fictional practice details</legend>
    <label for="demo-name">Practice name (required)</label>
    <input id="demo-name" required aria-describedby="demo-name-error">
    <p id="demo-name-error" class="field-error" hidden></p>
    <label for="demo-email">Practice email (required)</label>
    <input id="demo-email" type="email" required aria-describedby="demo-email-help demo-emai
l-error">
    <p id="demo-email-help">Use one made-up address, for example reader@example.invalid.</p>
    <p id="demo-email-error" class="field-error" hidden></p>
    <label for="demo-session">Practice session (required)</label>
    <select id="demo-session" required aria-describedby="demo-session-error">
      <option value="">Choose a practice session</option>
      <option value="morning">Morning study</option>
      <option value="afternoon">Afternoon study</option>
    </select>
    <p id="demo-session-error" class="field-error" hidden></p>
    <button type="submit">Check this demonstration</button>
    <button type="reset">Reset example fields</button>
  </fieldset>
</form>
<p id="demo-result" role="status"></p>
<!-- registration:end -->
</main>
<script src="../assets/project-registration.js" defer></script>
```

The download and wrapped chapter example load the same maintained local behaviour. Disabled fieldset descendants cannot be used for the application task before successful setup. Once handlers are installed, the script enables them and suppresses form navigation with preventDefault(). The summary supplies a

focused overview and links that explicitly focus each failed control. A polite status reports only that the browser checks passed. No confirmation of membership, delivery or server acceptance is justified. See assets/project-registration.js in the site source, or demo.js in the ZIP, for the complete validation and reset implementation.

A mistake to watch for

A form with method="get" and an action ending in #note is not a harmless no-backend fallback. Named successful controls can still enter a URL query. A success message must not imply that local validity created an application.

Your turn

Why must this static reference not use a GET-to-fragment fallback? Describe the failed-script behaviour, then explain what reset must clear after a whitespace-name error.

Explained answer

A fragment names a destination within a resource; it does not cancel form submission. A GET form with named controls can place values in the URL and navigate even without an application backend. This reference therefore starts with disabled fields and enables them only after the local submit/error/reset handlers are installed. If the script is blocked, the instructions remain and the fields stay disabled. Reset clears the controls, the summary list, inline messages, aria-invalid attributes and the custom validity message on the name. The result reports reset, not application acceptance. Test reset again after a valid local check, since stale success text can otherwise survive.

Complete project folder: project-registration.zip in the companion downloads.

Read the source

HTML Living Standard: the form element

HTML Living Standard: constraint validation

HTML Living Standard: the fieldset element

WAI Forms Tutorial: user notifications

WCAG 2.2 Understanding: Focus Order

XI / Practical Projects

71. Build an article page

Publish a complete fictional lending article with three destinations, a defined policy and a machine-readable review date.

By the end of this chapter

- Write consistent borrowing, renewal and return explanations rather than policy placeholders.
- Connect an on-page contents list to three unique section IDs.
- Encode a displayed review date using the time element.

Before you start

Review Chapters 14, 17, 20, 24, 25. Organise a self-contained article, make fragment links and encode a date that matches its displayed text.

Working terms

A policy model is the explicit fictional rule set used for this exercise. A review date identifies the stated editorial check date, not the date of every loan. Calendar-day arithmetic here follows the convention stated in the article; it does not infer a real organisation's counting rules.

Build contract

Write an article titled How lending works with an introduction, an on-page contents navigation, Borrowing, Renewal and Returns sections, and a review date. Label the policy fictional on the page. Give enough information to evaluate the model without contacting a nonexistent desk. This exercise supplies a 14-calendar-day loan, one eligible 14-calendar-day renewal before the due date, and a paper-log practice return. The page itself processes none of these actions.

Implementation route

Use an article because the guide is a self-contained subject, and give each substantive section its own heading and stable ID. The contents landmark needs a name distinct from any site-wide primary navigation. Describe the loan date arithmetic explicitly: a loan starting 1 October 2026 is due 15 October; one eligible renewal extends that due date to 29 October. Define this convention in prose instead of asking the reader to infer inclusivity from a bare number. Mark the review date with `time datetime="2026-10-15"` while displaying 15 October 2026.

Acceptance and recovery

Activate all three contents links, inspect their fragments and verify each destination exists once. Read just the headings and confirm the policy sequence makes sense. Check the date attribute against the displayed review date. Disable scripts and repeat navigation. At narrow widths, the contents links must wrap without covering the article. A valid `time` value does not verify that an editor actually reviewed a policy; review provenance remains editorial evidence.

Reference walkthrough

The supplied article contains the entire practice policy rather than an instruction to ask someone for it. It distinguishes a possible model extension from an actual renewal operation. Returns are represented as a local paper exercise, not a real address or live record. The article footer belongs to that article's review information; the outer project footer carries publication credit. Adaptation to a real library requires replacing both the fictional policy and the unverified service context.

Assessment rubric: 28 observable points

Award one point for each demonstrated check below, four per area. Record the evidence next to the check; untested checks receive no point. These points are an authoring assessment aid, not WCAG conformance or security certification. A broken primary task, a misleading service claim, unsafe data collection or a dead required destination blocks readiness regardless of the score.

Article contract (4 points)

- The article has the required title.
- Fictional policy status is visible.
- An introduction identifies the model.
- No real service is implied.

Contents navigation (4 points)

- The nav has a descriptive accessible name.
- Borrowing links to its section.
- Renewal links to its section.
- Returns links to its section.

Borrowing (4 points)

- The 14-calendar-day rule is stated.
- A start date example is supplied.
- Its due date is 15 October 2026.
- The model date convention is understandable.

Renewal (4 points)

- Only one renewal is allowed in the model.
- Eligibility and before-due-date scope are stated.
- The extension is 14 calendar days.
- The sample extended due date is 29 October.

Returns (4 points)

- A return procedure is actually described.
- The procedure is identified as practice.
- No real branch address is invented.
- No live return recording is claimed.

Review and structure (4 points)

- Review text displays 15 October 2026.
- datetime contains 2026-10-15.
- All three section IDs are unique.
- The article headings form a coherent hierarchy.

Resilience (4 points)

- Contents links work without scripts.
- All fragments resolve.
- The page reads at 320 CSS pixels.
- README and solution distinguish static policy from service.

The example

HTML / example 71

```
<article>
  <h1>How lending works</h1>
  <p>Fictional practice policy, not a real lending service.</p>
  <nav aria-label="On this page"><a href="#borrowing">Borrowing</a>
    <a href="#renewal">Renewal</a> <a href="#returns">Returns</a></nav>
  <section id="borrowing"><h2>Borrowing</h2>
    <p>A practice loan lasts 14 calendar days after its start date:
      1 October 2026 gives a 15 October 2026 due date.</p></section>
  <section id="renewal"><h2>Renewal</h2>
    <p>One eligible renewal before the due date adds 14 calendar days
      to that date: 15 October becomes 29 October.</p></section>
  <section id="returns"><h2>Returns</h2>
    <p>Mark the practice return in a local paper log.
      This page records no real return.</p></section>
  <footer>Reviewed <time datetime="2026-10-15">15 October 2026</time>.</footer>
</article>
```

All contents links lead to real, distinct sections. The sample borrowing and renewal dates follow the explicitly supplied calendar-day convention; they are not claims about a real library policy. The time element encodes the same date that readers see. The reference supplies a complete explanation of the practice return, while making clear that HTML content does not update a lending system. Section headings, navigation and an article footer express different relationships rather than acting as interchangeable layout containers.

A mistake to watch for

A policy heading followed by “ask at the desk” does not meet a brief requiring the policy itself. A review date without a matching valid datetime also loses machine-readable precision.

Your turn

An article says loans last 14 days after 1 October and gives a 14 October due date. Its displayed review date is 15 October but datetime is 2026-10-14. What should change?

Explained answer

Use the stated model convention consistently: 14 calendar days after 1 October 2026 is 15 October, and the eligible 14-day extension is 29 October. If a real policy uses another counting convention, explain and apply that convention instead of silently changing the arithmetic. Match the reviewed date attribute to the displayed date as 2026-10-15. Verify all three section destinations after editing; date syntax and working links still do not prove that a real library approved the policy.

Complete project folder: project-article.zip in the companion downloads.

Read the source

HTML Living Standard: the article element

HTML Living Standard: the nav element

HTML Living Standard: the time element

HTML Living Standard: links

XI / Practical Projects

72. Build a data dashboard

Make a static dashboard whose loan total, units, period and accessible table agree.

By the end of this chapter

- Reconcile a displayed total with all three fictional collection counts.
- Provide a caption and explicit row and column headers for a simple data table.
- Keep wide data readable in a labelled keyboard-focusable scroll region.

Before you start

Review Chapters 26, 61, 62. Create a captioned simple table, distinguish headers from data and test a keyboard-accessible scrolling region.

Working terms

A measure is the quantity counted, here loan entries. A unit tells readers how to interpret a value. Reconciliation checks that summary and rows describe the same dataset and arithmetic. A table header association connects a data cell with its relevant row or column heading.

Build contract

Build a static October 2026 dashboard for three fictional loan counts: Fiction 42, Local history 18 and Reference 9. Display their total, 69 loans, and the count of represented collections, three. Label the values as fictional loan entries, not unique people, stock levels or live availability. There is no comparison period, so do not invent a growth arrow or percentage.

Implementation route

Use a description list for the named summary values and a table for the cross-row data relationships. Give the table a caption that identifies the subject and period. Mark column headers with `scope="col"` and each collection's row header with `scope="row"`. Include a period column so that a copied row retains useful context. Scope is suitable for this simple grid; complex multi-level tables need an explicit association review rather than a mechanical copy of this example.

Acceptance and recovery

Recalculate the total from the rows, then compare every displayed summary against the table. At 320 CSS pixels, the document itself should fit the viewport. The wide table may scroll inside its labelled region; preserve the two-dimensional relationships instead of hiding columns. Tab to that region, check its visible focus and inspect horizontal scroll. Reflow permits exceptions for content that requires two-dimensional layout, but the exception does not excuse page-wide overflow or an inaccessible scrolling surface.

Reference walkthrough

The supplied dashboard is all static HTML. It exposes the period, units and fiction notice without hover, script or colour interpretation. Its summary cannot remain correct if someone changes a row without updating the total, so reconciliation is an explicit acceptance check. The scroll wrapper is focusable and named. Native markup provides associations for this simple table; confirming its actual presentation in a screen reader still requires an assistive-technology review.

Assessment rubric: 28 observable points

Award one point for each demonstrated check below, four per area. Record the evidence next to the check; untested checks receive no point. These points are an authoring assessment aid, not WCAG conformance or security certification. A broken primary task, a misleading service claim, unsafe data collection or a dead required destination blocks readiness regardless of the score.

Data truth (4 points)

- Fictional status is visible.
- The period is October 2026.
- Units mean loan entries.
- No live feed or unsupported comparison is claimed.

Rows (4 points)

- Fiction is 42.
- Local history is 18.
- Reference is 9.
- All three rows carry the period.

Summary (4 points)

- Total is visibly 69 loans.
- The total equals 42 + 18 + 9.
- Three collections are reported.
- Summary labels explain their quantities.

Table structure (4 points)

- A subject-and-period caption is present.
- A thead groups column headings.
- Each column heading has scope=col.
- Each collection heading has scope=row.

Narrow layout (4 points)

- The 320-pixel document fits its viewport.
- The table is inside a bounded scroll region.
- No data column is hidden.
- The scroll region has an accessible name.

Keyboard and baseline (4 points)

- The table region can receive keyboard focus.
- Focus is visible.
- Data remain readable without scripts.
- Reading order retains caption/headings/rows.

Delivery (4 points)

- Local CSS resolves.
- Document title identifies the dashboard.
- No unexplained chart substitutes for data.
- Solution notes require recalculation after edits.

The example

HTML / example 72

```
<p>Fictional October 2026 loan entries, not a live feed.</p>
<dl><dt>Total across these collections</dt><dd>69 loans</dd></dl>
<table>
  <caption>Fictional loans by collection, October 2026</caption>
  <thead><tr><th scope="col">Collection</th><th scope="col">Loans (count)</th>
    <th scope="col">Period</th></tr></thead>
  <tbody><tr><th scope="row">Fiction</th><td>42</td><td>October 2026</td></tr>
    <tr><th scope="row">Local history</th><td>18</td><td>October 2026</td></tr>
    <tr><th scope="row">Reference</th><td>9</td><td>October 2026</td></tr></tbody>
</table>
```

The total and rows now describe the same three collections: 42 + 18 + 9 = 69. No reader count or percentage can be inferred from those values. The caption supplies table context, while scoped headers

express the simple row/column relationships. The fragment focuses on the data model; the complete reference adds the named scroll wrapper, focus styling and second summary value. A passing conformance checker does not establish that those associations were understood through a particular assistive technology.

A mistake to watch for

A summary showing 128 active readers above a table of 69 loan entries mixes incompatible quantities and has no supporting data. Reconcile the measure as well as the arithmetic.

Your turn

Someone changes Reference from 9 to 12 but leaves 69 loans in the summary and adds “69 readers.” What needs correction and why?

Explained answer

The revised loan total is 42 + 18 + 12 = 72. Update the summary and any explanatory arithmetic to 72 loans. Do not relabel it readers: one reader may have multiple loan entries and there is no distinct-person dataset. Keep the period and collection count explicit, then recheck all rows, scope attributes and the narrow scroll region. A matching number alone would not repair the unit error.

Complete project folder: project-dashboard.zip in the companion downloads.

Read the source

HTML Living Standard: table header cells

WAI Tables Tutorial: two headers

WCAG 2.2 Understanding: Reflow

WCAG 2.2 Understanding: Focus Order

XI / Practical Projects

73. Build a media gallery

Present three original local SVG drawings with accurate alternatives, matching dimensions and honest loading choices.

By the end of this chapter

- Describe the supplied drawings accurately rather than inventing photographic detail.
- Pair each image with a caption that contributes context instead of duplicating its alternative.
- Keep local image assets usable offline and reserve a matching aspect ratio.

Before you start

Review Chapters 19, 28, 29, 33. Choose alternatives from an image's purpose, pair figures with captions and compare an asset's dimensions with its display.

Working terms

A text alternative supplies image-equivalent meaning for its context. A caption adds related visible context. An intrinsic aspect ratio comes from the resource geometry; the supplied drawings are 640 by 400. Lazy loading is a browser loading hint, not a promise about a request time.

Build contract

Deliver three self-contained drawings named `shelves.svg`, `window.svg` and `tables.svg`. State that these are illustrations, not photographs of a real library. Use a list of figures with an image and caption in each. The reference drawings are made from simple geometric shapes and need no external font, network image or attribution to an unknown photographer. Retain the publication credit and rights notice when using the supplied package; do not assume the whole book has an open licence.

Implementation route

The shelves drawing has three rows of six book shapes. The window has six panes. The tables drawing shows two tables from above, each with a book shape. Write alternatives from those visible facts. Captions provide the learning context, such as organising a collection, rather than repeating the full alternative. These images carry information in this exercise, so an empty alternative would remove part of the intended content. A genuinely decorative image in another context could require a different decision.

Acceptance and recovery

Set both HTML dimensions and SVG viewBox to the same 640 by 400 ratio, then constrain display size with CSS without distorting the image. The first image uses eager loading; later images use lazy loading as a hint. This is an authoring choice, not a timing guarantee or a measured performance improvement. Check actual rendering after scrolling the lazy images into view. Extract the ZIP and confirm all three local files load without network access. Block image loading once and confirm the text alternatives and captions still preserve meaningful information.

Reference walkthrough

The generated SVGs contain their own drawing instructions and no remote URLs or scripts. The img elements supply alternatives in the embedding HTML. Width and height reserve the intended ratio even when display width changes. There is no lightbox or synthetic button because viewing these figures requires neither a custom interaction nor a modal. The layout uses a responsive list grid and keeps source order intact when it collapses to one column.

Assessment rubric: 28 observable points

Award one point for each demonstrated check below, four per area. Record the evidence next to the check; untested checks receive no point. These points are an authoring assessment aid, not WCAG conformance or security certification. A broken primary task, a misleading service claim, unsafe data collection or a dead required destination blocks readiness regardless of the score.

Asset delivery (4 points)

- All three SVG files are supplied.
- SVGs contain no remote resources.
- The page says they are drawings.
- No real venue photography is claimed.

Accurate alternatives (4 points)

- The shelves alternative describes three rows.
- The shelves alternative matches six shapes per row.
- The window alternative matches six panes.
- The tables alternative matches two tables with book shapes.

Figure relationships (4 points)

- The gallery is a native list.
- Each list item contains a figure.
- Each image has its caption in that figure.
- Captions add context instead of duplicating alternatives.

Geometry (4 points)

- HTML width is 640 for every image.
- HTML height is 400 for every image.
- Each SVG viewBox is 0 0 640 400.
- Narrow rendering preserves the ratio.

Loading (4 points)

- The first image is not lazy.
- Later images carry the lazy hint.
- All images render after scrolling into view.
- No measured speed claim is invented.

Fallback (4 points)

- Local files load after ZIP extraction.
- Images require no network resource.
- Blocked images leave meaningful alternatives.
- Captions remain readable with images blocked.

Presentation and rights (4 points)

- The 320-pixel page has no horizontal overflow.
- The list reading order is meaningful.
- No unnecessary lightbox control is offered.
- Publication credit and rights limits are retained.

The example

HTML / example 73

```
<ul>
  <li><figure>
    <img src="../projects/gallery/shelves.svg"
      alt="Three shelf rows, each holding six book shapes"
      width="640" height="400" style="max-width:100%;height:auto" loading="eager" decoding="
async">
    <figcaption>Organising a small collection.</figcaption>
  </figure></li>
  <li><figure>
    <img src="../projects/gallery/window.svg"
      alt="A rectangular window divided into six panes"
      width="640" height="400" style="max-width:100%;height:auto" loading="lazy" decoding="a
sync">
    <figcaption>Light in a reading space.</figcaption>
  </figure></li>
  <li><figure>
    <img src="../projects/gallery/tables.svg"
      alt="Two reading tables seen from above, each with a book shape"
      width="640" height="400" style="max-width:100%;height:auto" loading="lazy" decoding="a
sync">
    <figcaption>A study in shared space.</figcaption>
  </figure></li>
</ul>
```

The three alternatives match the actual geometric drawings and do not describe people, colours or photographic settings that the files do not show. Captions explain why each subject appears in the gallery. The wrapped chapter example links to the supplied project image files; the extracted gallery uses sibling-relative paths. HTML width/height and SVG viewBox all share a 1.6:1 ratio. Local assets and explicit dimensions make the reference self-contained, but loading="lazy" remains a browser hint rather than proof of faster loading.

A mistake to watch for

Describing a generated drawing as a photograph, or using an alternative with more objects than the asset contains, makes the text fallback inaccurate. Inspect the delivered image, not a remembered prompt.

Your turn

The window SVG has six panes, but its alternative says four and its HTML height is changed to 640 while width stays 640. What should be repaired?

Explained answer

Restore an accurate six-pane alternative. Keep the supplied 640 by 400 width/height pair so its reserved ratio matches the 0 0 640 400 SVG viewBox; CSS may scale the display while height remains automatic. Check the actual loaded image and the blocked-image state, then verify all three files from the extracted ZIP. Neither a plausible alternative nor successful image loading proves the other requirement.

Complete project folder: project-gallery.zip in the companion downloads.

Read the source

HTML Living Standard: the img element

WAI Images Tutorial: informative images

WCAG 2.2 Understanding: Reflow

XI / Practical Projects

74. Capstone: the library site

Integrate four honest library pages, then verify navigation, local form recovery, data and offline delivery as one system.

By the end of this chapter

- Integrate Home, Catalogue, Membership and Visit with consistent navigation and truthful service boundaries.
- Reuse and retest the disabled-baseline local form instead of implying a real membership backend.
- Assess the whole extracted site against observable functional, accessibility and privacy checks.

Before you start

Review Chapters 27, 61, 62, 66, 68, 69, 70, 71, 72, 73. Complete the six smaller projects, plan file routes and test both successful enhancement and failed-script recovery.

Working terms

Integration testing checks connected components as one task flow. A service boundary identifies what the static interface cannot do. An acceptance blocker is a critical failure that prevents readiness even when other rubric points are earned. Offline delivery means the supplied local documents and resources can be used without fetching a remote dependency.

Build contract

Deliver four documents: `index.html`, `catalogue.html`, `membership.html` and `visit.html`. The fictional context must be visible throughout the site. Home links readers to the other tasks. Catalogue contains three fictional titles and their collections. Membership reuses the Chapter 70 local demonstration, not a collection endpoint. Visit supplies fictional practice hours and explicitly says that no actual destination, access conditions or contact route have been verified.

Implementation route

Use the same primary-navigation labels and destinations on every page, with exactly one current-page marker. Give each document its own title and main heading. Keep the catalogue caption and headers meaningful. Bound its wide table inside a labelled, focusable scroll region rather than hiding its second column on a phone. Reuse the form markup and maintained script from Chapter 70, then test that the integration still preserves all error, correction, reset and failed-script behaviours.

Truthful visit information

The practice timetable is Tuesday to Saturday, 09:00 to 17:00, closed Sunday and Monday. These are example hours, not a real timetable. Do not infer step-free access from an invented building or publish an imaginary address as a destination. The reference clearly states that entrance routes, parking and facilities are unverified. A real service needs verified location and relevant access details, not a generic accessible badge.

Acceptance and recovery

Enter the site through every page directly and follow every primary link. Check all four at 320 CSS pixels, with keyboard-only navigation and without scripting. Use valid and invalid fictional values on Membership and inspect requests and the URL. Re-run the form suite rather than assuming component reuse proves correct integration. Extract the ZIP and inspect the same tasks offline. Styles or images failing should not remove the page's content, and script failure must leave the form disabled.

Reference walkthrough

The supplied site offers only reading and local validation tasks. It cannot reserve a title, accept an application, search a catalogue or record a visit. Catalogue entries are static HTML, not a backend disguised by a polished layout. The Visit page makes its missing real-service details explicit. README, BRIEF and SOLUTION describe these limits from the same chapter source. This reference is a teaching implementation, not independent accessibility approval, a security audit or a production service.

Assessment rubric: 28 observable points

Award one point for each demonstrated check below, four per area. Record the evidence next to the check; untested checks receive no point. These points are an authoring assessment aid, not WCAG conformance or security certification. A broken primary task, a misleading service claim, unsafe data collection or a dead required destination blocks readiness regardless of the score.

Four-page contract (4 points)

- All four required documents exist.
- Every page has its own document title.
- Every page has a main heading.
- Fictional static-service status is visible.

Navigation (4 points)

- All primary labels are consistent.
- All primary destinations resolve from every page.
- Exactly one current-page link appears per page.
- The skip link reaches each main landmark.

Catalogue (4 points)

- Three fictional entries are supplied.
- The caption describes the data.
- Row and column headers identify relationships.
- The table scrolls inside a named keyboard-focusable region.

Membership safety (4 points)

- The no-script baseline is disabled.
- Setup enables only local validation.
- Invalid submit provides linked field errors.
- Valid submit claims no sent application.

Membership recovery (4 points)

- Correcting failed fields removes their errors.
- Editing clears prior success.
- Reset clears values/errors/custom validity.
- Blocked script leaves instructions and disabled fields.

Visit truth (4 points)

- Practice hours are explicitly fictional.
- Sunday/Monday closure is stated.
- No real address or contact is invented.
- Access conditions are explicitly unverified.

Integrated evidence (4 points)

- Every page opens directly after extraction.
- All four pages fit a 320-pixel viewport outside tables.
- Keyboard focus remains visible.
- Submit emits no application request or URL values.

The example

HTML / example 74

```
<nav aria-label="Primary">
  <a href="../projects/capstone/index.html" aria-current="page">Home</a>
  <a href="../projects/capstone/catalogue.html">Catalogue</a>
  <a href="../projects/capstone/membership.html">Membership</a>
  <a href="../projects/capstone/visit.html">Visit</a>
</nav>
<main><h1>Neighbourhood Library</h1>
  <p>A fictional static reference site, not a real library service.</p>
  <p>No books can be reserved and no applications are collected here.</p>
  <ul><li><a href="../projects/capstone/catalogue.html">Browse fictional book data</a></li>
    <li><a href="../projects/capstone/membership.html">Inspect the demonstration form</a></l
i>
    <li><a href="../projects/capstone/visit.html">Read the fictional visit information</a></
li></ul>
</main>
```

The chapter fragment previews Home and links into the four-document reference; it is not the entire capstone. Within the download, the four pages use sibling-relative links and each updates its own current-page marker. The generated membership form comes from Chapter 70 and uses the same script, so its privacy and error-recovery boundaries stay aligned. Catalogue and Visit contain actual explanatory content, not headings awaiting a backend. A static reference can be complete against this contract while remaining insufficient as a real public library service.

A mistake to watch for

A working Home page does not prove the capstone works. Dead catalogue links, an enabled no-script membership form or unverified step-free claims are critical failures even if the visual design scores well.

Your turn

Home passes a visual check, but Membership submits named values with GET when its script is blocked and Visit claims step-free access without evidence. Can a high rubric total make this ready?

Explained answer

No. Privacy/task and truthful-service blockers prevent readiness regardless of points. Restore the disabled fieldset baseline and install the local submit/error/reset handlers before enabling it; confirm the blocked-script test leaves it unusable for collection and emits no application query. Remove the unsupported access claim and state what has not been verified. Then test all four pages directly, all navigation routes, membership errors and reset, narrow table scrolling and the extracted offline package. Do not convert same-assistant checks into independent accessibility or security approval.

Complete project folder: project-capstone.zip in the companion downloads.

Read the source

HTML Living Standard: the nav element

HTML Living Standard: links

HTML Living Standard: the form element

WAI Forms Tutorial: user notifications

HTML Living Standard: table header cells

WCAG 2.2 Understanding: Reflow

WCAG 2.2 Understanding: Focus Order

Element reference

Index-derived summaries from the captured foundation snapshot. Read the full definition for restrictions, defaults and exceptions. Standard presence does not imply universal implementation. API member-level completeness is still open.

a

Hyperlink

Categories: flow ; phrasing *; interactive ; palpable

Parents: phrasing

Content: transparent *

Attributes: globals ; href ; target ; download ; ping ; rel ; hreflang ; type ; referrerpolicy

Open the source definition or discovery route

abbr

Abbreviation

Categories: flow ; phrasing ; palpable

Parents: phrasing

Content: phrasing

Attributes: globals

Open the source definition or discovery route

address

Contact information for a page or article element

Categories: flow ; palpable

Parents: flow

Content: flow *

Attributes: globals

Open the source definition or discovery route

area

Hyperlink or dead area on an image map

Categories: flow ; phrasing

Parents: phrasing *

Content: empty

Attributes: globals ; alt ; coords ; shape ; href ; target ; download ; ping ; rel ; referrerpolicy

Open the source definition or discovery route

article

Self-contained syndicatable or reusable composition

Categories: flow ; sectioning ; palpable

Parents: flow

Content: flow

Attributes: globals

Open the source definition or discovery route

aside

Sidebar for tangentially related content

Categories: flow ; sectioning ; palpable

Parents: flow

Content: flow

Attributes: globals

Open the source definition or discovery route

audio

Audio player

Categories: flow ; phrasing ; embedded ; interactive ; palpable *

Parents: phrasing

Content: source *; track *; transparent *

Attributes: globals ; src ; crossorigin ; preload ; autoplay ; loading ; loop ; muted ; controls

Open the source definition or discovery route

b

Keywords

Categories: flow ; phrasing ; palpable

Parents: phrasing

Content: phrasing

Attributes: globals

Open the source definition or discovery route

base

Base URL and default target navigable for hyperlinks and forms

Categories: metadata

Parents: head

Content: empty

Attributes: globals ; href ; target

Open the source definition or discovery route

bdi

Text directionality isolation

Categories: flow ; phrasing ; palpable

Parents: phrasing

Content: phrasing

Attributes: globals

Open the source definition or discovery route

bdo

Text directionality formatting

Categories: flow ; phrasing ; palpable

Parents: phrasing

Content: phrasing

Attributes: globals

Open the source definition or discovery route

blockquote

A section quoted from another source

Categories: flow ; palpable

Parents: flow

Content: flow

Attributes: globals ; cite

Open the source definition or discovery route

body

Document body

Categories: none

Parents: html

Content: flow

Attributes: globals ; onafterprint ; onbeforeprint ; onbeforeunload ; onhashchange ; onlanguagechange ; onmessage ; onmessageerror ; onoffline ; ononline ; onpageswap ; onpagehide ; onpagereveal ; onpageshow ; onpopstate ; onrejectionhandled ; onstorage ; onunhandledrejection ; onunload

Open the source definition or discovery route

br

Line break, e.g. in poem or postal address

Categories: flow ; phrasing

Parents: phrasing

Content: empty

Attributes: globals

Open the source definition or discovery route

button

Button control

Categories: flow ; phrasing ; interactive ; listed ; labelable ; submittable ; form-associated ; palpable

Parents: phrasing ; select *

Content: phrasing *

Attributes: globals ; command ; commandfor ; disabled ; form ; formaction ; formenctype ; formmethod ; formnovalidate ; formtarget ; name ; popovertarget ; popovertargetaction ; type ; value

Open the source definition or discovery route

canvas

Scriptable bitmap canvas

Categories: flow ; phrasing ; embedded ; palpable

Parents: phrasing

Content: transparent

Attributes: globals ; width ; height

Open the source definition or discovery route

caption

Table caption

Categories: none

Parents: table

Content: flow *

Attributes: globals

Open the source definition or discovery route

cite

Title of a work

Categories: flow ; phrasing ; palpable

Parents: phrasing

Content: phrasing

Attributes: globals

Open the source definition or discovery route

code

Computer code

Categories: flow ; phrasing ; palpable

Parents: phrasing

Content: phrasing

Attributes: globals

Open the source definition or discovery route

col

Table column

Categories: none

Parents: colgroup

Content: empty

Attributes: globals ; span

Open the source definition or discovery route

colgroup

Group of columns in a table

Categories: none

Parents: table

Content: col *; template *

Attributes: globals ; span

Open the source definition or discovery route

data

Machine-readable equivalent

Categories: flow ; phrasing ; palpable

Parents: phrasing

Content: phrasing

Attributes: globals ; value

Open the source definition or discovery route

datalist

Container for options for combo box control

Categories: flow ; phrasing

Parents: phrasing

Content: phrasing *; option *; script-supporting elements *

Attributes: globals

Open the source definition or discovery route

dd

Content for corresponding dt element(s)

Categories: none

Parents: dl ; div *

Content: flow

Attributes: globals

Open the source definition or discovery route

del

A removal from the document

Categories: flow ; phrasing *; palpable

Parents: phrasing

Content: transparent

Attributes: globals ; cite ; datetime

Open the source definition or discovery route

details

Disclosure control for hiding details

Categories: flow ; interactive ; palpable

Parents: flow

Content: summary *; flow

Attributes: globals ; name ; open

Open the source definition or discovery route

dfn

Defining instance

Categories: flow ; phrasing ; palpable

Parents: phrasing

Content: phrasing *

Attributes: globals

Open the source definition or discovery route

dialog

Dialog box or window

Categories: flow

Parents: flow

Content: flow

Attributes: globals ; open

Open the source definition or discovery route

div

Generic flow container, or container for name-value groups in dl elements

Categories: flow ; palpable

Parents: flow ; dl ; option ; optgroup ; select

Content: flow *

Attributes: globals

Open the source definition or discovery route

dl

Association list consisting of zero or more name-value groups

Categories: flow ; palpable

Parents: flow

Content: dt *; dd *; div *; script-supporting elements

Attributes: globals

Open the source definition or discovery route

dt

Legend for corresponding dd element(s)

Categories: none

Parents: dl ; div *

Content: flow *

Attributes: globals

Open the source definition or discovery route

em

Stress emphasis

Categories: flow ; phrasing ; palpable

Parents: phrasing

Content: phrasing

Attributes: globals

Open the source definition or discovery route

embed

Plugin

Categories: flow ; phrasing ; embedded ; interactive ; palpable

Parents: phrasing

Content: empty

Attributes: globals ; src ; type ; width ; height ; any*

Open the source definition or discovery route

fieldset

Group of form controls

Categories: flow ; listed ; form-associated ; palpable

Parents: flow

Content: legend *; flow

Attributes: globals ; disabled ; form ; name

Open the source definition or discovery route

figcaption

Caption for figure

Categories: none

Parents: figure

Content: flow

Attributes: globals

Open the source definition or discovery route

figure

Figure with optional caption

Categories: flow ; palpable

Parents: flow

Content: figcaption *; flow

Attributes: globals

Open the source definition or discovery route

footer

Footer for a page or section

Categories: flow ; palpable

Parents: flow

Content: flow *

Attributes: globals

Open the source definition or discovery route

form

User-submittable form

Categories: flow ; palpable

Parents: flow

Content: flow *

Attributes: globals ; accept-charset ; action ; autocomplete ; enctype ; method ; name ; novalidate ; rel ; target

Open the source definition or discovery route

hl

Heading

Categories: flow ; heading ; palpable

Parents: legend ; summary ; flow

Content: phrasing

Attributes: globals

Open the source definition or discovery route

h2

Heading

Categories: flow ; heading ; palpable

Parents: legend ; summary ; flow

Content: phrasing

Attributes: globals

Open the source definition or discovery route

h3

Heading

Categories: flow ; heading ; palpable

Parents: legend ; summary ; flow

Content: phrasing

Attributes: globals

Open the source definition or discovery route

h4

Heading

Categories: flow ; heading ; palpable

Parents: legend ; summary ; flow

Content: phrasing

Attributes: globals

Open the source definition or discovery route

h5

Heading

Categories: flow ; heading ; palpable

Parents: legend ; summary ; flow

Content: phrasing

Attributes: globals

Open the source definition or discovery route

h6

Heading

Categories: flow ; heading ; palpable

Parents: legend ; summary ; flow

Content: phrasing

Attributes: globals

Open the source definition or discovery route

head

Container for document metadata

Categories: none

Parents: html

Content: metadata content *

Attributes: globals

Open the source definition or discovery route

header

Introductory or navigational aids for a page or section

Categories: flow ; palpable

Parents: flow

Content: flow *

Attributes: globals

Open the source definition or discovery route

hgroup

Heading container

Categories: flow ; palpable

Parents: legend ; summary ; flow

Content: h1 ; h2 ; h3 ; h4 ; h5 ; h6 ; p ; script-supporting elements

Attributes: globals

Open the source definition or discovery route

hr

Thematic break

Categories: flow

Parents: flow ; select

Content: empty

Attributes: globals

Open the source definition or discovery route

html

Root element

Categories: none

Parents: none*

Content: head *; body *

Attributes: globals

Open the source definition or discovery route

i

Alternate voice

Categories: flow ; phrasing ; palpable

Parents: phrasing

Content: phrasing

Attributes: globals

Open the source definition or discovery route

iframe

Child navigable

Categories: flow ; phrasing ; embedded ; interactive ; palpable

Parents: phrasing

Content: empty

Attributes: globals ; src ; srcdoc ; name ; sandbox ; allow ; allowfullscreen ; width ; height ; referrerpolicy ; loading

Open the source definition or discovery route

img

Image

Categories: flow ; phrasing ; embedded ; interactive *; form-associated ; palpable

Parents: phrasing ; picture

Content: empty

Attributes: globals ; alt ; src ; srcset ; sizes ; crossorigin ; usemap ; ismap ; controls ; width ; height ; referrerpolicy ; decoding ; loading ; fetchpriority

Open the source definition or discovery route

input

Form control

Categories: flow ; phrasing ; interactive *; listed ; labelable ; submittable ; resettable ; form-associated ; palpable *

Parents: phrasing

Content: empty

Attributes: globals ; accept ; alpha ; alt ; autocomplete ; checked ; colorspace ; dirname ; disabled ; form ; formaction ; formenctype ; formmethod ; formnovalidate ; formtarget ; height ; list ; max ; maxlength ; min ; minlength ; multiple ; name ; pattern ; placeholder ; popovertarget ; popovertargetaction ; readonly ; required ; size ; src ; step ; type ; value ; width

Open the source definition or discovery route

ins

An addition to the document

Categories: flow ; phrasing *; palpable

Parents: phrasing

Content: transparent

Attributes: globals ; cite ; datetime

Open the source definition or discovery route

kbd

User input

Categories: flow ; phrasing ; palpable

Parents: phrasing

Content: phrasing

Attributes: globals

Open the source definition or discovery route

label

Caption for a form control

Categories: flow ; phrasing ; interactive ; palpable

Parents: phrasing

Content: phrasing *

Attributes: globals ; for

Open the source definition or discovery route

legend

Caption for fieldset

Categories: none

Parents: fieldset ; optgroup

Content: phrasing *; heading content

Attributes: globals

Open the source definition or discovery route

li

List item

Categories: none

Parents: ol ; ul ; menu *

Content: flow

Attributes: globals ; value *

Open the source definition or discovery route

link

Link metadata

Categories: metadata ; flow *; phrasing *

Parents: head ; noscript *; phrasing *

Content: empty

Attributes: globals ; href ; crossorigin ; rel ; as ; media ; hreflang ; type ; sizes ; imagesrcset ; imagesizes ; referrerpolicy ; integrity ; blocking ; color ; disabled ; fetchpriority

Open the source definition or discovery route

main

Container for the dominant contents of the document

Categories: flow ; palpable

Parents: flow *

Content: flow

Attributes: globals

Open the source definition or discovery route

map

Image map

Categories: flow ; phrasing *; palpable

Parents: phrasing

Content: transparent ; area *

Attributes: globals ; name

Open the source definition or discovery route

mark

Highlight

Categories: flow ; phrasing ; palpable

Parents: phrasing

Content: phrasing

Attributes: globals

Open the source definition or discovery route

menu

Menu of commands

Categories: flow ; palpable *

Parents: flow

Content: li ; script-supporting elements

Attributes: globals

Open the source definition or discovery route

meta

Text metadata

Categories: metadata ; flow *; phrasing *

Parents: head ; noscript *; phrasing *

Content: empty

Attributes: globals ; name ; http-equiv ; content ; charset ; media

Open the source definition or discovery route

meter

Gauge

Categories: flow ; phrasing ; labelable ; palpable

Parents: phrasing

Content: phrasing *

Attributes: globals ; value ; min ; max ; low ; high ; optimum

Open the source definition or discovery route

nav

Section with navigational links

Categories: flow ; sectioning ; palpable

Parents: flow

Content: flow

Attributes: globals

Open the source definition or discovery route

noscript

Fallback content for script

Categories: metadata ; flow ; phrasing

Parents: head *; phrasing *; select ; optgroup

Content: varies*

Attributes: globals

Open the source definition or discovery route

object

Image, child navigable , or plugin

Categories: flow ; phrasing ; embedded ; interactive *; listed ; form-associated ; palpable

Parents: phrasing

Content: transparent

Attributes: globals ; data ; type ; name ; form ; width ; height

Open the source definition or discovery route

ol

Ordered list

Categories: flow ; palpable *

Parents: flow

Content: li ; script-supporting elements

Attributes: globals ; reversed ; start ; type

Open the source definition or discovery route

optgroup

Group of options in a list box

Categories: none

Parents: select ; div *

Content: option *; script-supporting elements *; noscript *; div *; legend *

Attributes: globals ; disabled ; label

Open the source definition or discovery route

option

Option in a list box or combo box control

Categories: none

Parents: select ; datalist ; optgroup ; div *

Content: text *; div *; phrasing *

Attributes: globals ; disabled ; label ; selected ; value

Open the source definition or discovery route

output

Calculated output value

Categories: flow ; phrasing ; listed ; labelable ; resettable ; form-associated ; palpable

Parents: phrasing

Content: phrasing

Attributes: globals ; for ; form ; name

Open the source definition or discovery route

p

Paragraph

Categories: flow ; palpable

Parents: flow

Content: phrasing

Attributes: globals

Open the source definition or discovery route

picture

Image

Categories: flow ; phrasing ; embedded ; palpable

Parents: phrasing

Content: source *; one img ; script-supporting elements

Attributes: globals

Open the source definition or discovery route

pre

Block of preformatted text

Categories: flow ; palpable

Parents: flow

Content: phrasing

Attributes: globals

Open the source definition or discovery route

progress

Progress bar

Categories: flow ; phrasing ; labelable ; palpable

Parents: phrasing

Content: phrasing *

Attributes: globals ; value ; max

Open the source definition or discovery route

q

Quotation

Categories: flow ; phrasing ; palpable

Parents: phrasing

Content: phrasing

Attributes: globals ; cite

Open the source definition or discovery route

rp

Parenthesis for ruby annotation text

Categories: none

Parents: ruby

Content: text

Attributes: globals

Open the source definition or discovery route

rt

Ruby annotation text

Categories: none

Parents: ruby

Content: phrasing

Attributes: globals

Open the source definition or discovery route

ruby

Ruby annotation(s)

Categories: flow ; phrasing ; palpable

Parents: phrasing

Content: phrasing ; rt ; rp *

Attributes: globals

Open the source definition or discovery route

s

Inaccurate text

Categories: flow ; phrasing ; palpable

Parents: phrasing

Content: phrasing

Attributes: globals

Open the source definition or discovery route

samp

Computer output

Categories: flow ; phrasing ; palpable

Parents: phrasing

Content: phrasing

Attributes: globals

Open the source definition or discovery route

script

Embedded script

Categories: metadata ; flow ; phrasing ; script-supporting

Parents: head ; phrasing ; script-supporting

Content: script, data, or script documentation*

Attributes: globals ; src ; type ; nomodule ; async ; defer ; crossorigin ; integrity ; referrerpolicy ; blocking ; fetchpriority

Open the source definition or discovery route

search

Container for search controls

Categories: flow ; palpable

Parents: flow

Content: flow

Attributes: globals

Open the source definition or discovery route

section

Generic document or application section

Categories: flow ; sectioning ; palpable

Parents: flow

Content: flow

Attributes: globals

Open the source definition or discovery route

select

List box control

Categories: flow ; phrasing ; interactive ; listed ; labelable ; submittable ; resettable ; form-associated ; palpable

Parents: phrasing

Content: option *; optgroup *; hr *; script-supporting elements *; noscript *; div *; button *

Attributes: globals ; autocomplete ; disabled ; form ; multiple ; name ; required ; size

Open the source definition or discovery route

selectedcontent

Mirrors content from an option

Categories: none

Parents: button

Content: empty

Attributes: globals

Open the source definition or discovery route

slot

Shadow tree slot

Categories: flow ; phrasing

Parents: phrasing

Content: transparent

Attributes: globals ; name

Open the source definition or discovery route

small

Side comment

Categories: flow ; phrasing ; palpable

Parents: phrasing

Content: phrasing

Attributes: globals

Open the source definition or discovery route

source

Image source for img or media source for video or audio

Categories: none

Parents: picture ; video ; audio

Content: empty

Attributes: globals ; type ; media ; src ; srcset ; sizes ; width ; height

Open the source definition or discovery route

span

Generic phrasing container

Categories: flow ; phrasing ; palpable

Parents: phrasing

Content: phrasing

Attributes: globals

Open the source definition or discovery route

strong

Importance

Categories: flow ; phrasing ; palpable

Parents: phrasing

Content: phrasing

Attributes: globals

Open the source definition or discovery route

style

Embedded styling information

Categories: metadata ; flow

Parents: head ; noscript *; flow *

Content: text*

Attributes: globals ; media ; blocking

Open the source definition or discovery route

sub

Subscript

Categories: flow ; phrasing ; palpable

Parents: phrasing

Content: phrasing

Attributes: globals

Open the source definition or discovery route

summary

Caption for details

Categories: none

Parents: details

Content: phrasing ; heading content

Attributes: globals

Open the source definition or discovery route

sup

Superscript

Categories: flow ; phrasing ; palpable

Parents: phrasing

Content: phrasing

Attributes: globals

Open the source definition or discovery route

table

Table

Categories: flow ; palpable

Parents: flow

Content: caption *; colgroup *; thead *; tbody *; tfoot *; tr *; script-supporting elements

Attributes: globals

Open the source definition or discovery route

tbody

Group of rows in a table

Categories: none

Parents: table

Content: tr ; script-supporting elements

Attributes: globals

Open the source definition or discovery route

td

Table cell

Categories: none

Parents: tr

Content: flow

Attributes: globals ; colspan ; rowspan ; headers

Open the source definition or discovery route

template

Template

Categories: metadata ; flow ; phrasing ; script-supporting

Parents: metadata ; phrasing ; script-supporting ; colgroup *

Content: empty

Attributes: globals ; for ; shadowrootmode ; shadowrootdelegatesfocus ; shadowrootserializable ; shadowrootslotassignment ; shadowrootclonable ; shadowrootcustomelementregistry

Open the source definition or discovery route

textarea

Multiline text controls

Categories: flow ; phrasing ; interactive ; listed ; labelable ; submittable ; resettable ; form-associated ; palpable

Parents: phrasing

Content: text

Attributes: globals ; autocomplete ; cols ; dirname ; disabled ; form ; maxlength ; minlength ; name ; placeholder ; readonly ; required ; rows ; wrap

Open the source definition or discovery route

tfoot

Group of footer rows in a table

Categories: none

Parents: table

Content: tr ; script-supporting elements

Attributes: globals

Open the source definition or discovery route

th

Table header cell

Categories: interactive *

Parents: tr

Content: flow *

Attributes: globals ; colspan ; rowspan ; headers ; scope ; abbr

Open the source definition or discovery route

thead

Group of heading rows in a table

Categories: none

Parents: table

Content: tr ; script-supporting elements

Attributes: globals

Open the source definition or discovery route

time

Machine-readable equivalent of date- or time-related data

Categories: flow ; phrasing ; palpable

Parents: phrasing

Content: phrasing

Attributes: globals ; datetime

Open the source definition or discovery route

title

Document title

Categories: metadata

Parents: head

Content: text *

Attributes: globals

Open the source definition or discovery route

tr

Table row

Categories: none

Parents: table ; thead ; tbody ; tfoot

Content: th *; td ; script-supporting elements

Attributes: globals

Open the source definition or discovery route

track

Timed text track

Categories: none

Parents: audio ; video

Content: empty

Attributes: globals ; default ; kind ; label ; src ; srclang

Open the source definition or discovery route

u

Unarticulated annotation

Categories: flow ; phrasing ; palpable

Parents: phrasing

Content: phrasing

Attributes: globals

Open the source definition or discovery route

ul

List

Categories: flow ; palpable *

Parents: flow

Content: li ; script-supporting elements

Attributes: globals

Open the source definition or discovery route

var

Variable

Categories: flow ; phrasing ; palpable

Parents: phrasing

Content: phrasing

Attributes: globals

Open the source definition or discovery route

video

Video player

Categories: flow ; phrasing ; embedded ; interactive ; palpable

Parents: phrasing

Content: source *; track *; transparent *

Attributes: globals ; src ; crossorigin ; poster ; preload ; autoplay ; playsinline ; loading ; loop ; muted ; controls ; width ; height

Open the source definition or discovery route

wbr

Line breaking opportunity

Categories: flow ; phrasing

Parents: phrasing

Content: empty

Attributes: globals

Open the source definition or discovery route

math

MathML root

Categories: flow ; phrasing ; embedded ; palpable

Parents: phrasing

Content: per [MATHML]

Attributes: per [MATHML]

Open the source definition or discovery route

svg

SVG root

Categories: flow ; phrasing ; embedded ; palpable

Parents: phrasing

Content: per [SVG]

Attributes: per [SVG]

Open the source definition or discovery route

autonomous custom elements

Author-defined elements

Categories: flow ; phrasing ; palpable

Parents: flow ; phrasing

Content: transparent

Attributes: globals ; any, as decided by the element's author

Open the source definition or discovery route

Attributes and handlers

Index-derived summaries from the captured foundation snapshot. Read the full definition for restrictions, defaults and exceptions. Standard presence does not imply universal implementation. API member-level completeness is still open.

abbr

Alternative label to use for the header cell when referencing the cell in other contexts

Applies to: th

Value: Text *

Open the source definition or discovery route

accept

Hint for expected file type in file upload controls

Applies to: input

Value: Set of comma-separated tokens * consisting of valid MIME type strings with no parameters or audio/* , video/* , or image/*

Open the source definition or discovery route

accept-charset

Character encodings to use for form submission

Applies to: form

Value: ASCII case-insensitive match for " UTF-8 "

Open the source definition or discovery route

accesskey

Keyboard shortcut to activate or focus element

Applies to: HTML elements

Value: Ordered set of unique space-separated tokens , none of which are identical to another, each consisting of one code point in length

Open the source definition or discovery route

action

URL to use for form submission

Applies to: form

Value: Valid non-empty URL potentially surrounded by spaces

Open the source definition or discovery route

allow

Permissions policy to be applied to the iframe 's contents

Applies to: iframe

Value: Serialized permissions policy

Open the source definition or discovery route

allowfullscreen

Whether to allow the iframe 's contents to use requestFullscreen()

Applies to: iframe

Value: Boolean attribute

Open the source definition or discovery route

alpha

Allow the color's alpha component to be set

Applies to: input

Value: Boolean attribute

Open the source definition or discovery route

alt

Replacement text for use when images are not available

Applies to: area ; img ; input

Value: Text *

Open the source definition or discovery route

as

Destination for a preload request (for rel =" preload " and rel =" modulepreload ")

Applies to: link

Value: Preload destination , for rel =" preload "; module preload destination , for rel =" modulepreload "

Open the source definition or discovery route

async

Execute script when available, without blocking while fetching

Applies to: script

Value: Boolean attribute

Open the source definition or discovery route

autocapitalize

Recommended autocapitalization behavior (for supported input methods)

Applies to: HTML elements

Value: " on "; " off "; " none "; " sentences "; " words "; " characters "

Open the source definition or discovery route

autocomplete

Default setting for autofill feature for controls in the form

Applies to: form

Value: " on "; " off "

Open the source definition or discovery route

autocomplete

Hint for form autofill feature

Applies to: input ; select ; textarea

Value: Autofill field name and related tokens*

Open the source definition or discovery route

autocorrect

Recommended autocorrection behavior (for supported input methods)

Applies to: HTML elements

Value: " on "; " off "; the empty string

Open the source definition or discovery route

autofocus

Automatically focus the element when the page is loaded

Applies to: HTML elements

Value: Boolean attribute

Open the source definition or discovery route

autoplay

Hint that the media resource can be started automatically when the page is loaded

Applies to: audio ; video

Value: Boolean attribute

Open the source definition or discovery route

blocking

Whether the element is potentially render-blocking

Applies to: link ; script ; style

Value: Unordered set of unique space-separated tokens *

Open the source definition or discovery route

charset

Character encoding declaration

Applies to: meta

Value: " utf-8 "

Open the source definition or discovery route

checked

Whether the control is checked

Applies to: input

Value: Boolean attribute

Open the source definition or discovery route

cite

Link to the source of the quotation or more information about the edit

Applies to: blockquote ; del ; ins ; q

Value: Valid URL potentially surrounded by spaces

Open the source definition or discovery route

class

Classes to which the element belongs

Applies to: HTML elements

Value: Set of space-separated tokens

Open the source definition or discovery route

closedby

Which user actions will close the dialog

Applies to: dialog

Value: " any "; " closerequest "; " none ";

Open the source definition or discovery route

color

Color to use when customizing a site's icon (for rel =" mask-icon ")

Applies to: link

Value: CSS `<color>`

Open the source definition or discovery route

colorspace

The color space of the serialized color

Applies to: input

Value: " limited-srgb "; " display-p3 "

Open the source definition or discovery route

cols

Maximum number of characters per line

Applies to: textarea

Value: Valid non-negative integer greater than zero

Open the source definition or discovery route

colspan

Number of columns that the cell is to span

Applies to: td ; th

Value: Valid non-negative integer greater than zero

Open the source definition or discovery route

command

Indicates to the targeted element which action to take.

Applies to: button

Value: " toggle-popover "; " show-popover "; " hide-popover "; " close "; " request-close "; " show-modal "; a custom command keyword

Open the source definition or discovery route

commandfor

Targets another element to be invoked.

Applies to: button

Value: ID *

Open the source definition or discovery route

content

Value of the element

Applies to: meta

Value: Text *

Open the source definition or discovery route

contenteditable

Whether the element is editable

Applies to: HTML elements

Value: " true "; " false "; " plaintext-only "; the empty string

Open the source definition or discovery route

controls

Show user agent controls

Applies to: audio ; video img ;

Value: Boolean attribute

Open the source definition or discovery route

coords

Coordinates for the shape to be created in an image map

Applies to: area

Value: Valid list of floating-point numbers *

Open the source definition or discovery route

crossorigin

How the element handles crossorigin requests

Applies to: audio ; img ; link ; script ; video

Value: " anonymous "; " use-credentials "; the empty string

Open the source definition or discovery route

data

Address of the resource

Applies to: object

Value: Valid non-empty URL potentially surrounded by spaces

Open the source definition or discovery route

datetime

Date and (optionally) time of the change

Applies to: del ; ins

Value: Valid date string with optional time

Open the source definition or discovery route

datetime

Machine-readable value

Applies to: time

Value: Valid month string , valid date string , valid yearless date string , valid time string , valid local date and time string , valid time-zone offset string , valid global date and time string , valid week string , valid non-negative integer , or valid duration string

Open the source definition or discovery route

decoding

Decoding hint to use when processing this image for presentation

Applies to: img

Value: " sync "; " async "; " auto "

Open the source definition or discovery route

default

Enable the track if no other text track is more suitable

Applies to: track

Value: Boolean attribute

Open the source definition or discovery route

defer

Defer script execution

Applies to: script

Value: Boolean attribute

Open the source definition or discovery route

dir

The text directionality of the element

Applies to: HTML elements

Value: " ltr "; " rtl "; " auto "

Open the source definition or discovery route

dir

The text directionality of the element

Applies to: bdo

Value: " ltr "; " rtl "

Open the source definition or discovery route

dirname

Name of form control to use for sending the element's directionality in form submission

Applies to: input ; textarea

Value: Text *

Open the source definition or discovery route

disabled

Whether the form control is disabled

Applies to: button ; input ; optgroup ; option ; select ; textarea ; form-associated custom elements

Value: Boolean attribute

Open the source definition or discovery route

disabled

Whether the descendant form controls, except any inside legend , are disabled

Applies to: fieldset

Value: Boolean attribute

Open the source definition or discovery route

disabled

Whether the link is disabled

Applies to: link

Value: Boolean attribute

Open the source definition or discovery route

download

Whether to download the resource instead of navigating to it, and its filename if so

Applies to: a ; area

Value: Text

Open the source definition or discovery route

draggable

Whether the element is draggable

Applies to: HTML elements

Value: " true "; " false "

Open the source definition or discovery route

enctype

Entry list encoding type to use for form submission

Applies to: form

Value: " application/x-www-form-urlencoded "; " multipart/form-data "; " text/plain "

Open the source definition or discovery route

enterkeyhint

Hint for selecting an enter key action

Applies to: HTML elements

Value: " enter "; " done "; " go "; " next "; " previous "; " search "; " send "

Open the source definition or discovery route

fetchpriority

Sets the priority for fetches initiated by the element

Applies to: img ; link ; script

Value: " auto "; " high "; " low "

Open the source definition or discovery route

for

Associate the label with form control

Applies to: label

Value: ID *

Open the source definition or discovery route

for

Specifies controls from which the output was calculated

Applies to: output

Value: Unordered set of unique space-separated tokens consisting of IDs*

Open the source definition or discovery route

for

Updates existing content

Applies to: template

Value: Text

Open the source definition or discovery route

form

Associates the element with a form element

Applies to: button ; fieldset ; input ; object ; output ; select ; textarea ; form-associated custom elements

Value: ID *

Open the source definition or discovery route

formaction

URL to use for form submission

Applies to: button ; input

Value: Valid non-empty URL potentially surrounded by spaces

Open the source definition or discovery route

formenctype

Entry list encoding type to use for form submission

Applies to: button ; input

Value: " application/x-www-form-urlencoded "; " multipart/form-data "; " text/plain "

Open the source definition or discovery route

formmethod

Variant to use for form submission

Applies to: button ; input

Value: " get "; " post "; " dialog "

Open the source definition or discovery route

formnovalidate

Bypass form control validation for form submission

Applies to: button ; input

Value: Boolean attribute

Open the source definition or discovery route

formtarget

Navigable for form submission

Applies to: button ; input

Value: Valid navigable target name or keyword

Open the source definition or discovery route

headers

The header cells for this cell

Applies to: td ; th

Value: Unordered set of unique space-separated tokens consisting of IDs*

Open the source definition or discovery route

headingoffset

Offsets heading levels for descendants

Applies to: HTML elements

Value: Valid non-negative integer between 0 and 8

Open the source definition or discovery route

headingreset

Prevents a heading offset computation from traversing beyond the element with the attribute

Applies to: HTML elements

Value: Boolean attribute

Open the source definition or discovery route

height

Vertical dimension

Applies to: canvas ; embed ; iframe ; img ; input ; object ; source (in picture); video

Value: Valid non-negative integer

Open the source definition or discovery route

hidden

Whether the element is relevant

Applies to: HTML elements

Value: " until-found "; " hidden "; the empty string

Open the source definition or discovery route

high

Low limit of high range

Applies to: meter

Value: Valid floating-point number *

Open the source definition or discovery route

href

Address of the hyperlink

Applies to: a ; area

Value: Valid URL potentially surrounded by spaces

Open the source definition or discovery route

href

Address of the hyperlink

Applies to: link

Value: Valid non-empty URL potentially surrounded by spaces

Open the source definition or discovery route

href

Document base URL

Applies to: base

Value: Valid URL potentially surrounded by spaces

Open the source definition or discovery route

hreflang

Language of the linked resource

Applies to: a ; link

Value: Valid BCP 47 language tag

Open the source definition or discovery route

http-equiv

Pragma directive

Applies to: meta

Value: " content-type "; " default-style "; " refresh "; " x-ua-compatible "; " content-security-policy "

Open the source definition or discovery route

id

The element's ID

Applies to: HTML elements

Value: Text *

Open the source definition or discovery route

imagesizes

Image sizes for different page layouts (for rel =" preload ")

Applies to: link

Value: Valid source size list

Open the source definition or discovery route

imagesrcset

Images to use in different situations, e.g., high-resolution displays, small monitors, etc. (for rel =" preload ")

Applies to: link

Value: Comma-separated list of image candidate strings

Open the source definition or discovery route

inert

Whether the element is inert .

Applies to: HTML elements

Value: Boolean attribute

Open the source definition or discovery route

inputmode

Hint for selecting an input modality

Applies to: HTML elements

Value: " none "; " text "; " tel "; " email "; " url "; " numeric "; " decimal "; " search "

Open the source definition or discovery route

integrity

Integrity metadata used in Subresource Integrity checks [SRI]

Applies to: link ; script

Value: Text

Open the source definition or discovery route

is

Creates a customized built-in element

Applies to: HTML elements

Value: Valid custom element name of a defined customized built-in element

Open the source definition or discovery route

ismap

Whether the image is a server-side image map

Applies to: img

Value: Boolean attribute

Open the source definition or discovery route

itemid

Global identifier for a microdata item

Applies to: HTML elements

Value: Valid URL potentially surrounded by spaces

Open the source definition or discovery route

itemprop

Property names of a microdata item

Applies to: HTML elements

Value: Unordered set of unique space-separated tokens consisting of valid absolute URLs , defined property names , or text*

Open the source definition or discovery route

itemref

Referenced elements

Applies to: HTML elements

Value: Unordered set of unique space-separated tokens consisting of IDs*

Open the source definition or discovery route

itemscope

Introduces a microdata item

Applies to: HTML elements

Value: Boolean attribute

Open the source definition or discovery route

itemtype

Item types of a microdata item

Applies to: HTML elements

Value: Unordered set of unique space-separated tokens consisting of valid absolute URLs *

Open the source definition or discovery route

kind

The type of text track

Applies to: track

Value: " subtitles "; " captions "; " descriptions "; " chapters "; " metadata "

Open the source definition or discovery route

label

User-visible label

Applies to: optgroup ; option ; track

Value: Text

Open the source definition or discovery route

lang

Language of the element

Applies to: HTML elements

Value: Valid BCP 47 language tag or the empty string

Open the source definition or discovery route

list

List of autocomplete options

Applies to: input

Value: ID *

Open the source definition or discovery route

loading

Used when determining loading deferral

Applies to: iframe ; img ; audio ; video

Value: " lazy "; " eager "

Open the source definition or discovery route

loop

Whether to loop the media resource

Applies to: audio ; video

Value: Boolean attribute

Open the source definition or discovery route

low

High limit of low range

Applies to: meter

Value: Valid floating-point number *

Open the source definition or discovery route

max

Maximum value

Applies to: input

Value: Varies*

Open the source definition or discovery route

max

Upper bound of range

Applies to: meter ; progress

Value: Valid floating-point number *

Open the source definition or discovery route

maxlength

Maximum length of value

Applies to: input ; textarea

Value: Valid non-negative integer

Open the source definition or discovery route

media

Applicable media

Applies to: link ; meta ; source ; style

Value: Valid media query list

Open the source definition or discovery route

method

Variant to use for form submission

Applies to: form

Value: " get "; " post "; " dialog "

Open the source definition or discovery route

min

Minimum value

Applies to: input

Value: Varies*

Open the source definition or discovery route

min

Lower bound of range

Applies to: meter

Value: Valid floating-point number *

Open the source definition or discovery route

minlength

Minimum length of value

Applies to: input ; textarea

Value: Valid non-negative integer

Open the source definition or discovery route

multiple

Whether to allow multiple values

Applies to: input ; select

Value: Boolean attribute

Open the source definition or discovery route

muted

Whether to mute the media resource by default

Applies to: audio ; video

Value: Boolean attribute

Open the source definition or discovery route

name

Name of the element to use for form submission and in the form.elements API

Applies to: button ; fieldset ; input ; output ; select ; textarea ; form-associated custom elements

Value: Text *

Open the source definition or discovery route

name

Name of group of mutually-exclusive details elements

Applies to: details

Value: Text *

Open the source definition or discovery route

name

Name of form to use in the document.forms API

Applies to: form

Value: Text *

Open the source definition or discovery route

name

Name of content navigable

Applies to: iframe ; object

Value: Valid navigable target name or keyword

Open the source definition or discovery route

name

Name of image map to reference from the usemap attribute

Applies to: map

Value: Text *

Open the source definition or discovery route

name

Metadata name

Applies to: meta

Value: Text *

Open the source definition or discovery route

name

Name of shadow tree slot

Applies to: slot

Value: Text

Open the source definition or discovery route

nomodule

Prevents execution in user agents that support module scripts

Applies to: script

Value: Boolean attribute

Open the source definition or discovery route

nonce

Cryptographic nonce used in Content Security Policy checks [CSP]

Applies to: HTML elements

Value: Text

Open the source definition or discovery route

novalidate

Bypass form control validation for form submission

Applies to: form

Value: Boolean attribute

Open the source definition or discovery route

open

Whether the details are visible

Applies to: details

Value: Boolean attribute

Open the source definition or discovery route

open

Whether the dialog box is showing

Applies to: dialog

Value: Boolean attribute

Open the source definition or discovery route

optimum

Optimum value in gauge

Applies to: meter

Value: Valid floating-point number *

Open the source definition or discovery route

pattern

Pattern to be matched by the form control's value

Applies to: input

Value: Regular expression matching the JavaScript Pattern production

Open the source definition or discovery route

ping

URLs to ping

Applies to: a ; area

Value: Set of space-separated tokens consisting of valid non-empty URLs

Open the source definition or discovery route

placeholder

User-visible label to be placed within the form control

Applies to: input ; textarea

Value: Text *

Open the source definition or discovery route

playsinline

Encourage the user agent to display video content within the element's playback area

Applies to: video

Value: Boolean attribute

Open the source definition or discovery route

popover

Makes the element a popover element

Applies to: HTML elements

Value: " auto "; " manual "; " hint "; the empty string

Open the source definition or discovery route

popovertarget

Targets a popover element to toggle, show, or hide

Applies to: button ; input

Value: ID *

Open the source definition or discovery route

popovertargetaction

Indicates whether a targeted popover element is to be toggled, shown, or hidden

Applies to: button ; input

Value: " toggle "; " show "; " hide "

Open the source definition or discovery route

poster

Poster frame to show prior to video playback

Applies to: video

Value: Valid non-empty URL potentially surrounded by spaces

Open the source definition or discovery route

preload

Hints how much buffering the media resource will likely need

Applies to: audio ; video

Value: " none "; " metadata "; " auto "; the empty string

Open the source definition or discovery route

readonly

Whether to allow the value to be edited by the user

Applies to: input ; textarea

Value: Boolean attribute

Open the source definition or discovery route

readonly

Affects willValidate , plus any behavior added by the custom element author

Applies to: form-associated custom elements

Value: Boolean attribute

Open the source definition or discovery route

referrerpolicy

Referrer policy for fetches initiated by the element

Applies to: a ; area ; iframe ; img ; link ; script

Value: Referrer policy

Open the source definition or discovery route

rel

Relationship between the location in the document containing the hyperlink and the destination resource

Applies to: a ; area

Value: Unordered set of unique space-separated tokens *

Open the source definition or discovery route

rel

Relationship between the document containing the hyperlink and the destination resource

Applies to: link

Value: Unordered set of unique space-separated tokens *

Open the source definition or discovery route

required

Whether the control is required for form submission

Applies to: input ; select ; textarea

Value: Boolean attribute

Open the source definition or discovery route

reversed

Number the list backwards

Applies to: ol

Value: Boolean attribute

Open the source definition or discovery route

rows

Number of lines to show

Applies to: textarea

Value: Valid non-negative integer greater than zero

Open the source definition or discovery route

rowspan

Number of rows that the cell is to span

Applies to: td ; th

Value: Valid non-negative integer

Open the source definition or discovery route

sandbox

Security rules for nested content

Applies to: iframe

Value: Unordered set of unique space-separated tokens , ASCII case-insensitive , consisting of " allow-downloads " " allow-forms " " allow-modals " " allow-orientation-lock " " allow-pointer-lock " " allow-popups " " allow-popups-to-escape-sandbox " " allow-presentation " " allow-same-origin " " allow-scripts " " allow-top-navigation " " allow-top-navigation-by-user-activation " " allow-top-navigation-to-custom-protocols "

Open the source definition or discovery route

scope

Specifies which cells the header cell applies to

Applies to: th

Value: " row "; " col "; " rowgroup "; " colgroup "

Open the source definition or discovery route

selected

Whether the option is selected by default

Applies to: option

Value: Boolean attribute

Open the source definition or discovery route

shadowrootclonable

Sets clonable on a declarative shadow root

Applies to: template

Value: Boolean attribute

Open the source definition or discovery route

shadowrootcustomelementregistry

Enables declarative shadow roots to indicate they will use a custom element registry

Applies to: template

Value: Boolean attribute

Open the source definition or discovery route

shadowrootdelegatesfocus

Sets delegates focus on a declarative shadow root

Applies to: template

Value: Boolean attribute

Open the source definition or discovery route

shadowrootmode

Enables streaming declarative shadow roots

Applies to: template

Value: " open "; " closed "

Open the source definition or discovery route

shadowrootserializable

Sets serializable on a declarative shadow root

Applies to: template

Value: Boolean attribute

Open the source definition or discovery route

shadowrootslotassignment

Sets slot assignment on a declarative shadow root

Applies to: template

Value: " named "; " manual "

Open the source definition or discovery route

shape

The kind of shape to be created in an image map

Applies to: area

Value: " circle "; " default "; " poly "; " rect "

Open the source definition or discovery route

size

Size of the control

Applies to: input ; select

Value: Valid non-negative integer greater than zero

Open the source definition or discovery route

sizes

Sizes of the icons (for rel =" icon ")

Applies to: link

Value: Unordered set of unique space-separated tokens , ASCII case-insensitive , consisting of sizes*

Open the source definition or discovery route

sizes

Image sizes for different page layouts

Applies to: img ; source

Value: Valid source size list

Open the source definition or discovery route

slot

The element's desired slot

Applies to: HTML elements

Value: Text

Open the source definition or discovery route

span

Number of columns spanned by the element

Applies to: col ; colgroup

Value: Valid non-negative integer greater than zero

Open the source definition or discovery route

spellcheck

Whether the element is to have its spelling and grammar checked

Applies to: HTML elements

Value: " true "; " false "; the empty string

Open the source definition or discovery route

src

Address of the resource

Applies to: audio ; embed ; iframe ; img ; input ; script ; source (in video or audio); track ; video

Value: Valid non-empty URL potentially surrounded by spaces

Open the source definition or discovery route

srcdoc

A document to render in the iframe

Applies to: iframe

Value: The source of an iframe srcdoc document *

Open the source definition or discovery route

srclang

Language of the text track

Applies to: track

Value: Valid BCP 47 language tag

Open the source definition or discovery route

srcset

Images to use in different situations, e.g., high-resolution displays, small monitors, etc.

Applies to: img ; source

Value: Comma-separated list of image candidate strings

Open the source definition or discovery route

start

Starting value of the list

Applies to: ol

Value: Valid integer

Open the source definition or discovery route

step

Granularity to be matched by the form control's value

Applies to: input

Value: Valid floating-point number greater than zero, or " any "

Open the source definition or discovery route

style

Presentational and formatting instructions

Applies to: HTML elements

Value: CSS declarations*

Open the source definition or discovery route

tabindex

Whether the element is focusable and sequentially focusable , and the relative order of the element for the purposes of sequential focus navigation

Applies to: HTML elements

Value: Valid integer

Open the source definition or discovery route

target

Navigable for hyperlink navigation

Applies to: a ; area

Value: Valid navigable target name or keyword

Open the source definition or discovery route

target

Default navigable for hyperlink navigation and form submission

Applies to: base

Value: Valid navigable target name or keyword

Open the source definition or discovery route

target

Navigable for form submission

Applies to: form

Value: Valid navigable target name or keyword

Open the source definition or discovery route

title

Advisory information for the element

Applies to: HTML elements

Value: Text

Open the source definition or discovery route

title

Full term or expansion of abbreviation

Applies to: abbr ; dfn

Value: Text

Open the source definition or discovery route

title

Description of pattern (when used with pattern attribute)

Applies to: input

Value: Text

Open the source definition or discovery route

title

Title of the link

Applies to: link

Value: Text

Open the source definition or discovery route

title

CSS style sheet set name

Applies to: link ; style

Value: Text

Open the source definition or discovery route

translate

Whether the element is to be translated when the page is localized

Applies to: HTML elements

Value: " yes "; " no "; the empty string

Open the source definition or discovery route

type

Hint for the type of the referenced resource

Applies to: a ; link

Value: Valid MIME type string

Open the source definition or discovery route

type

Type of button

Applies to: button

Value: " submit "; " reset "; " button "

Open the source definition or discovery route

type

Type of embedded resource

Applies to: embed ; object ; source

Value: Valid MIME type string

Open the source definition or discovery route

type

Type of form control

Applies to: input

Value: input type keyword

Open the source definition or discovery route

type

Kind of list marker

Applies to: ol

Value: " 1 "; " a "; " A "; " i "; " I "

Open the source definition or discovery route

type

Type of script

Applies to: script

Value: " module "; " importmap "; " speculationrules "; a valid MIME type string that is not a JavaScript MIME type essence match

Open the source definition or discovery route

usemap

Name of image map to use

Applies to: img

Value: Valid hash-name reference *

Open the source definition or discovery route

value

Value to be used for form submission

Applies to: button ; option

Value: Text

Open the source definition or discovery route

value

Machine-readable value

Applies to: data

Value: Text *

Open the source definition or discovery route

value

Value of the form control

Applies to: input

Value: Varies*

Open the source definition or discovery route

value

Ordinal value of the list item

Applies to: li

Value: Valid integer

Open the source definition or discovery route

value

Current value of the element

Applies to: meter ; progress

Value: Valid floating-point number

Open the source definition or discovery route

width

Horizontal dimension

Applies to: canvas ; embed ; iframe ; img ; input ; object ; source (in picture); video

Value: Valid non-negative integer

Open the source definition or discovery route

wrap

How the value of the form control is to be wrapped for form submission

Applies to: textarea

Value: " soft "; " hard "

Open the source definition or discovery route

writingsuggestions

Whether the element can offer writing suggestions or not.

Applies to: HTML elements

Value: " true "; " false "; the empty string

Open the source definition or discovery route

onafterprint

afterprint event handler for Window object

Applies to: body

Value: Event handler content attribute

Open the source definition or discovery route

onauxclick

auxclick event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onbeforeinput

beforeinput event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onbeforematch

beforematch event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onbeforeprint

beforeprint event handler for Window object

Applies to: body

Value: Event handler content attribute

Open the source definition or discovery route

onbeforeunload

beforeunload event handler for Window object

Applies to: body

Value: Event handler content attribute

Open the source definition or discovery route

onbeforetoggle

beforetoggle event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onblur

blur event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

oncancel

cancel event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

oncanplay

canplay event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

oncanplaythrough

canplaythrough event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onchange

change event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onclick

click event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onclose

close event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

oncommand

command event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

oncontextlost

contextlost event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

oncontextmenu

contextmenu event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

oncontextrestored

contextrestored event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

oncopy

copy event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

oncuechange

cuechange event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

oncut

cut event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

ondblclick

dblclick event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

ondrag

drag event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

ondragend

dragend event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

ondragenter

dragenter event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

ondragleave

dragleave event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

ondragover

dragover event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

ondragstart

dragstart event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

ondrop

drop event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

ondurationchange

durationchange event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onemptied

emptied event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onended

ended event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onerror

error event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onfocus

focus event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onformdata

formdata event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onhashchange

hashchange event handler for Window object

Applies to: body

Value: Event handler content attribute

Open the source definition or discovery route

oninput

input event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

oninvalid

invalid event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onkeydown

keydown event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onkeypress

keypress event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onkeyup

keyup event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onlanguagechange

languagechange event handler for Window object

Applies to: body

Value: Event handler content attribute

Open the source definition or discovery route

onload

load event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onloadeddata

loadeddata event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onloadedmetadata

loadedmetadata event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onloadstart

loadstart event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onmessage

message event handler for Window object

Applies to: body

Value: Event handler content attribute

Open the source definition or discovery route

onmessageerror

messageerror event handler for Window object

Applies to: body

Value: Event handler content attribute

Open the source definition or discovery route

onmousedown

mousedown event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onmouseenter

mouseenter event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onmouseleave

mouseleave event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onmousemove

mousemove event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onmouseout

mouseout event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onmouseover

mouseover event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onmouseup

mouseup event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onoffline

offline event handler for Window object

Applies to: body

Value: Event handler content attribute

Open the source definition or discovery route

ononline

online event handler for Window object

Applies to: body

Value: Event handler content attribute

Open the source definition or discovery route

onpagehide

pagehide event handler for Window object

Applies to: body

Value: Event handler content attribute

Open the source definition or discovery route

onpagereveal

pagereveal event handler for Window object

Applies to: body

Value: Event handler content attribute

Open the source definition or discovery route

onpageshow

pageshow event handler for Window object

Applies to: body

Value: Event handler content attribute

Open the source definition or discovery route

onpageswap

pageswap event handler for Window object

Applies to: body

Value: Event handler content attribute

Open the source definition or discovery route

onpaste

paste event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onpause

pause event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onplay

play event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onplaying

playing event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onpopstate

popstate event handler for Window object

Applies to: body

Value: Event handler content attribute

Open the source definition or discovery route

onprogress

progress event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onratechange

ratechange event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onreset

reset event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onresize

resize event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onrejectionhandled

rejectionhandled event handler for Window object

Applies to: body

Value: Event handler content attribute

Open the source definition or discovery route

onscroll

scroll event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onscrollend

scrollend event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onsecuritypolicyviolation

securitypolicyviolation event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onseeked

seeked event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onseeking

seeking event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onselect

select event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onslotchange

slotchange event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onstalled

stalled event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onstorage

storage event handler for Window object

Applies to: body

Value: Event handler content attribute

Open the source definition or discovery route

onsubmit

submit event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onsuspend

suspend event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

ontimeupdate

timeupdate event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

ontoggle

toggle event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onunhandledrejection

unhandledrejection event handler for Window object

Applies to: body

Value: Event handler content attribute

Open the source definition or discovery route

onunload

unload event handler for Window object

Applies to: body

Value: Event handler content attribute

Open the source definition or discovery route

onvolumechange

volumechange event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onwaiting

waiting event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

onwheel

wheel event handler

Applies to: HTML elements

Value: Event handler content attribute

Open the source definition or discovery route

Global attributes and boundaries

Index-derived summaries from the captured foundation snapshot. Read the full definition for restrictions, defaults and exceptions. Standard presence does not imply universal implementation. API member-level completeness is still open.

accesskey

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

autocapitalize

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

autocorrect

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

autofocus

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

contenteditable

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

dir

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

draggable

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

enterkeyhint

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

headingoffset

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

headingreset

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

hidden

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

inert

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

inputmode

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

is

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

itemid

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

itemprop

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

itemref

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

itemscope

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

itemtype

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

lang

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

nonce

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

popover

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

spellcheck

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

style

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

tabindex

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

title

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

translate

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

writingsuggestions

Authority: HTML (may integrate external definitions)

Open the source definition or discovery route

id

Authority: DOM requirements plus HTML authoring requirements

Open the source definition or discovery route

class

Authority: DOM requirements plus HTML authoring requirements

Open the source definition or discovery route

slot

Authority: DOM requirements plus HTML authoring requirements

Open the source definition or discovery route

data-*

HTML elements; names and values have restrictions

Authority: HTML

Open the source definition or discovery route

role

Element-specific restrictions apply; external normative inventory pending

Authority: WAI-ARIA + ARIA in HTML

Open the source definition or discovery route

aria-*

Not an unrestricted wildcard; individual attribute audit pending

Authority: WAI-ARIA + ARIA in HTML

Open the source definition or discovery route

part

External normative definition review pending

Authority: CSS Shadow Parts, not a locally defined HTML attribute

Open the source definition or discovery route

exportparts

External normative definition review pending

Authority: CSS Shadow Parts, not a locally defined HTML attribute

Open the source definition or discovery route

xmlns

Conditional serialization/value requirements; not a general HTML namespace switch

Authority: HTML authoring allowance / XML namespace mechanism

Open the source definition or discovery route

xml:lang

HTML spelling and namespaced XML attribute have different rules

Authority: HTML and XML

Open the source definition or discovery route

xml:space

XML context; cannot be serialized as a namespaced attribute in text/html

Authority: XML

Open the source definition or discovery route

Input states

Index-derived summaries from the captured foundation snapshot. Read the full definition for restrictions, defaults and exceptions. Standard presence does not imply universal implementation. API member-level completeness is still open.

hidden

State: Hidden

Data: An arbitrary string

Control: n/a

Open the source definition or discovery route

text

State: Text

Data: Text with no line breaks

Control: A text control

Open the source definition or discovery route

search

State: Search

Data: Text with no line breaks

Control: Search control

Open the source definition or discovery route

tel

State: Telephone

Data: Text with no line breaks

Control: A text control

Open the source definition or discovery route

url

State: URL

Data: An absolute URL

Control: A text control

Open the source definition or discovery route

email

State: Email

Data: An email address or list of email addresses

Control: A text control

Open the source definition or discovery route

password

State: Password

Data: Text with no line breaks (sensitive information)

Control: A text control that obscures data entry

Open the source definition or discovery route

date

State: Date

Data: A date (year, month, day) with no time zone

Control: A date control

Open the source definition or discovery route

month

State: Month

Data: A date consisting of a year and a month with no time zone

Control: A month control

Open the source definition or discovery route

week

State: Week

Data: A date consisting of a week-year number and a week number with no time zone

Control: A week control

Open the source definition or discovery route

time

State: Time

Data: A time (hour, minute, seconds, fractional seconds) with no time zone

Control: A time control

Open the source definition or discovery route

datetime-local

State: Local Date and Time

Data: A date and time (year, month, day, hour, minute, second, fraction of a second) with no time zone

Control: A date and time control

Open the source definition or discovery route

number

State: Number

Data: A numerical value

Control: A text control or spinner control

Open the source definition or discovery route

range

State: Range

Data: A numerical value, with the extra semantic that the exact value is not important

Control: A slider control or similar

Open the source definition or discovery route

color

State: Color

Data: An sRGB color with 8-bit red, green, and blue components

Control: A color picker

Open the source definition or discovery route

checkbox

State: Checkbox

Data: A set of zero or more values from a predefined list

Control: A checkbox

Open the source definition or discovery route

radio

State: Radio Button

Data: An enumerated value

Control: A radio button

Open the source definition or discovery route

file

State: File Upload

Data: Zero or more files each with a MIME type and optionally a filename

Control: A label and a button

Open the source definition or discovery route

submit

State: Submit Button

Data: An enumerated value, with the extra semantic that it must be the last value selected and initiates form submission

Control: A button

Open the source definition or discovery route

image

State: Image Button

Data: A coordinate, relative to a particular image's size, with the extra semantic that it must be the last value selected and initiates form submission

Control: Either a clickable image, or a button

Open the source definition or discovery route

reset

State: Reset Button

Data: n/a

Control: A button

Open the source definition or discovery route

button

State: Button

Data: n/a

Control: A button

Open the source definition or discovery route

Content categories

Index-derived summaries from the captured foundation snapshot. Read the full definition for restrictions, defaults and exceptions. Standard presence does not imply universal implementation. API member-level completeness is still open.

Metadata content

Members: base ; link ; meta ; noscript ; script ; style ; template ; title

Exceptions: ,

Open the source definition or discovery route

Flow content

Members: a ; abbr ; address ; article ; aside ; audio ; b ; bdi ; bdo ; blockquote ; br ; button ; canvas ; cite ; code ; data ; datalist ; del ; details ; dfn ; dialog ; div ; dl ; em ; embed ; fieldset ; figure ; footer ; form ; h1 ; h2 ; h3 ; h4 ; h5 ; h6 ; header ; hgroup ; hr ; i ; iframe ; img ; input ; ins ; kbd ; label ; map ; mark ; MathML math ; menu ; meter ; nav ; noscript ; object ; ol ; output ; p ; picture ; pre ; progress ; q ; ruby ; s ; samp ; script ; search ; section ; select ; slot ; small ; span ; strong ; style ; sub ; sup ; SVG svg ; table ; template ; textarea ; time ; u ; ul ; var ; video ; wbr ; autonomous custom elements ; Text

Exceptions: area (if it is a descendant of a map element); link (if it is allowed in the body); main (if it is a hierarchically correct main element); meta (if the itemprop attribute is present)

Open the source definition or discovery route

Sectioning content

Members: article ; aside ; nav ; section

Exceptions: ,

Open the source definition or discovery route

Heading content

Members: h1 ; h2 ; h3 ; h4 ; h5 ; h6 ; hgroup

Exceptions: ,

Open the source definition or discovery route

Phrasing content

Members: a ; abbr ; audio ; b ; bdi ; bdo ; br ; button ; canvas ; cite ; code ; data ; datalist ; del ; dfn ; em ; embed ; i ; iframe ; img ; input ; ins ; kbd ; label ; map ; mark ; MathML math ; meter ; noscript ; object ; output ; picture ; progress ; q ; ruby ; s ; samp ; script ; select ; selectedcontent ; slot ; small ; span ; strong ; sub ; sup ; SVG svg ; template ; textarea ; time ; u ; var ; video ; wbr ; autonomous custom elements ; Text

Exceptions: area (if it is a descendant of a map element); link (if it is allowed in the body); meta (if the itemprop attribute is present)

Open the source definition or discovery route

Embedded content

Members: audio ; canvas ; embed ; iframe ; img ; MathML math ; object ; picture ; SVG svg ; video

Exceptions: ,

Open the source definition or discovery route

Interactive content

Members: button ; details ; embed ; iframe ; label ; select ; textarea

Exceptions: a (if the href attribute is present); audio (if the controls attribute is present); img (if the usemap or controls attribute is present); input (if the type attribute is not in the Hidden state); video (if the controls attribute is present)

Open the source definition or discovery route

Form-associated elements

Members: button ; fieldset ; input ; label ; object ; output ; select ; textarea ; img ; form-associated custom elements

Exceptions: ,

Open the source definition or discovery route

Listed elements

Members: button ; fieldset ; input ; object ; output ; select ; textarea ; form-associated custom elements

Exceptions: ,

Open the source definition or discovery route

Submittable elements

Members: button ; input ; select ; textarea ; form-associated custom elements

Exceptions: ,

Open the source definition or discovery route

Resettable elements

Members: input ; output ; select ; textarea ; form-associated custom elements

Exceptions: ,

Open the source definition or discovery route

Autocapitalize-and-autocorrect inheriting elements

Members: button ; fieldset ; input ; output ; select ; textarea

Exceptions: ,

Open the source definition or discovery route

Labelable elements

Members: button ; input ; meter ; output ; progress ; select ; textarea ; form-associated custom elements

Exceptions: ,

Open the source definition or discovery route

Palpable content

Members: a ; abbr ; address ; article ; aside ; b ; bdi ; bdo ; blockquote ; button ; canvas ; cite ; code ; data ; del ; details ; dfn ; div ; em ; embed ; fieldset ; figure ; footer ; form ; h1 ; h2 ; h3 ; h4 ; h5 ; h6 ; header ; hgroup ; i ; iframe ; img ; ins ; kbd ; label ; main ; map ; mark ; MathML math ; meter ; nav ; object ; output ; p ; picture ; pre ; progress ; q ; ruby ; s ; samp ; search ; section ; select ; small ; span ; strong ; sub ; sup ; SVG svg ; table ; textarea ; time ; u ; var ; video ; autonomous custom elements

Exceptions: audio (if the controls attribute is present); dl (if the element's children include at least one name-value group); input (if the type attribute is not in the Hidden state); menu (if the element's children include at least one li element); ol (if the element's children include at least one li element); ul (if the element's children include at least one li element); Text that

is not inter-element whitespace

Open the source definition or discovery route

Script-supporting elements

Members: script ; template

Exceptions: ,

Open the source definition or discovery route

Obsolete features

Index-derived summaries from the captured foundation snapshot. Read the full definition for restrictions, defaults and exceptions. Standard presence does not imply universal implementation. API member-level completeness is still open.

applet

Use embed or object instead.

Open the source definition or discovery route

acronym

Use abbr instead.

Open the source definition or discovery route

bgsound

Use audio instead.

Open the source definition or discovery route

dir

Use ul instead.

Open the source definition or discovery route

frame

Either use iframe and CSS instead, or use server-side includes to generate complete pages with the various invariant parts merged in.

Open the source definition or discovery route

frameset

Either use iframe and CSS instead, or use server-side includes to generate complete pages with the various invariant parts merged in.

Open the source definition or discovery route

noframes

Either use iframe and CSS instead, or use server-side includes to generate complete pages with the various invariant parts merged in.

Open the source definition or discovery route

isindex

Use an explicit form and text control combination instead.

Open the source definition or discovery route

keygen

For enterprise device management use cases, use native on-device management capabilities. For certificate enrollment use cases, use the Web Cryptography API to generate a keypair for the certificate, and then export the certificate and key to allow the user to install them manually. [WEBCRYPTO]

Open the source definition or discovery route

listing

Use pre and code instead.

Open the source definition or discovery route

menuitem

To implement a custom context menu, use script to handle the contextmenu event.

Open the source definition or discovery route

nextid

Use GUIDs instead.

Open the source definition or discovery route

noembed

Use object instead of embed when fallback is necessary.

Open the source definition or discovery route

param

Use the data attribute of the object element to set the URL of the external resource.

Open the source definition or discovery route

plaintext

Use the " text/plain " MIME type instead.

Open the source definition or discovery route

rb

Providing the ruby base directly inside the ruby element or using nested ruby elements is sufficient.

Open the source definition or discovery route

rtc

Providing the ruby base directly inside the ruby element or using nested ruby elements is sufficient.

Open the source definition or discovery route

strike

Use del instead if the element is marking an edit, otherwise use s instead.

Open the source definition or discovery route

xmp

Use pre and code instead, and escape " < " and " & " characters as " < " and " & " respectively.

Open the source definition or discovery route

basefont

Use appropriate elements or CSS instead. Where the tt element would have been used for marking up keyboard input, consider the kbd element; for variables, consider the var element; for computer code, consider the code element; and for computer output, consider the samp element. Similarly, if the big element is being used to denote a heading, consider using the h1 element; if it is being used for marking up important passages, consider the strong element; and if it is being used for highlighting text for reference purposes, consider the mark element. See also the text-level semantics usage summary for more suggestions with examples.

Open the source definition or discovery route

big

Use appropriate elements or CSS instead. Where the tt element would have been used for marking up keyboard input, consider the kbd element; for variables, consider the var element; for computer code, consider the code element; and for computer output, consider the samp element. Similarly, if the big element is being used to denote a heading, consider using the h1 element; if it is being used for marking up important passages, consider the strong element; and if it is being used for highlighting text for reference purposes, consider the mark element. See also the text-level semantics usage summary for more suggestions with examples.

Open the source definition or discovery route

blink

Use appropriate elements or CSS instead. Where the tt element would have been used for marking up keyboard input, consider the kbd element; for variables, consider the var element; for computer code, consider the code element; and for computer output, consider the samp element. Similarly, if the big element is being used to denote a heading, consider using the h1 element; if it is being used for marking up important passages, consider the strong element; and if it is being used for highlighting text for reference purposes, consider the mark element. See also the text-level semantics usage summary for more suggestions with examples.

Open the source definition or discovery route

center

Use appropriate elements or CSS instead. Where the tt element would have been used for marking up keyboard input, consider the kbd element; for variables, consider the var element; for computer code, consider the code element; and for computer output, consider the samp element. Similarly, if the big element is being used to denote a heading, consider using the h1 element; if it is being used for marking up important passages, consider the strong element; and if it is being used for highlighting text for reference purposes, consider the mark element. See also the text-level semantics usage summary for more suggestions with examples.

Open the source definition or discovery route

font

Use appropriate elements or CSS instead. Where the tt element would have been used for marking up keyboard input, consider the kbd element; for variables, consider the var element; for computer code, consider the code element; and for computer output, consider the samp element. Similarly, if the big element is being used to denote a heading, consider using the h1 element; if it is being used for marking up important passages, consider the strong element; and if it is being used for highlighting text for reference purposes, consider the mark element. See also the text-level semantics usage summary for more suggestions with examples.

Open the source definition or discovery route

marquee

Use appropriate elements or CSS instead. Where the tt element would have been used for marking up keyboard input, consider the kbd element; for variables, consider the var element; for computer code, consider the code element; and for computer output, consider the samp element. Similarly, if the big element is being used to denote a heading, consider using the h1 element; if it is being used for marking up important passages, consider the strong element; and if it is being used for highlighting text for reference purposes, consider the mark element. See also the text-level semantics usage summary for more suggestions with examples.

Open the source definition or discovery route

multicol

Use appropriate elements or CSS instead. Where the tt element would have been used for marking up keyboard input, consider the kbd element; for variables, consider the var element; for computer code, consider the code element; and for computer output, consider the samp element. Similarly, if the big element is being used to denote a heading, consider using the h1 element; if it is being used for marking up important passages, consider the strong element; and if it is being used for highlighting text for reference purposes, consider the mark element. See also the text-level semantics usage summary for more suggestions with examples.

Open the source definition or discovery route

nobr

Use appropriate elements or CSS instead. Where the tt element would have been used for marking up keyboard input, consider the kbd element; for variables, consider the var element; for computer code, consider the code element; and for computer output, consider the samp element. Similarly, if the big element is being used to denote a heading, consider using the h1 element; if it is being used for marking up important passages, consider the strong element; and if it is being used for highlighting text for reference purposes, consider the mark element. See also the text-level semantics usage summary for more suggestions with examples.

Open the source definition or discovery route

spacer

Use appropriate elements or CSS instead. Where the tt element would have been used for marking up keyboard input, consider the kbd element; for variables, consider the var element; for computer code, consider the code element; and for computer output, consider the samp element. Similarly, if the big element is being used to denote a heading, consider using the h1 element; if it is being used for marking up important passages, consider the strong element; and if it is being used for highlighting text for reference purposes, consider the mark element. See also the text-level semantics usage summary for more suggestions with examples.

Open the source definition or discovery route

tt

Use appropriate elements or CSS instead. Where the tt element would have been used for marking up keyboard input, consider the kbd element; for variables, consider the var element; for computer code, consider the code element; and for computer output, consider the samp element. Similarly, if the big element is being used to denote a heading, consider using the h1 element; if it is being used for marking up important passages, consider the strong element; and if it is being used for highlighting text for reference purposes, consider the mark element. See also the text-level semantics usage summary for more suggestions with examples.

Open the source definition or discovery route

charset

Use an HTTP `Content-Type` header on the linked resource instead.

Applies to: charset on a elements

Open the source definition or discovery route

charset

Use an HTTP `Content-Type` header on the linked resource instead.

Applies to: charset on link elements

Open the source definition or discovery route

charset

Omit the attribute. Both documents and scripts are required to use UTF-8 , so it is redundant to specify it on the script element since it inherits from the document.

Applies to: charset on script elements (except as noted in the previous section)

Open the source definition or discovery route

coords

Use area instead of a for image maps.

Applies to: coords on a elements

Open the source definition or discovery route

shape

Use area instead of a for image maps.

Applies to: shape on a elements

Open the source definition or discovery route

methods

Use the HTTP OPTIONS feature instead.

Applies to: methods on a elements

Open the source definition or discovery route

methods

Use the HTTP OPTIONS feature instead.

Applies to: methods on link elements

Open the source definition or discovery route

name

Use the id attribute instead.

Applies to: name on a elements (except as noted in the previous section)

Open the source definition or discovery route

name

Use the id attribute instead.

Applies to: name on embed elements

Open the source definition or discovery route

name

Use the id attribute instead.

Applies to: name on img elements

Open the source definition or discovery route

name

Use the id attribute instead.

Applies to: name on option elements

Open the source definition or discovery route

rev

Use the rel attribute instead, with an opposite term. (For example, instead of rev="made" , use rel="author" .)

Applies to: rev on a elements

Open the source definition or discovery route

rev

Use the rel attribute instead, with an opposite term. (For example, instead of rev="made" , use rel="author" .)

Applies to: rev on link elements

Open the source definition or discovery route

urn

Specify the preferred persistent identifier using the href attribute instead.

Applies to: urn on a elements

Open the source definition or discovery route

urn

Specify the preferred persistent identifier using the href attribute instead.

Applies to: urn on link elements

Open the source definition or discovery route

accept

Use the accept attribute directly on the input elements instead.

Applies to: accept on form elements

Open the source definition or discovery route

hreflang

These attributes do not do anything useful, and for historical reasons there are no corresponding IDL attributes on area elements. Omit them altogether.

Applies to: hreflang on area elements

Open the source definition or discovery route

type

These attributes do not do anything useful, and for historical reasons there are no corresponding IDL attributes on area elements. Omit them altogether.

Applies to: type on area elements

Open the source definition or discovery route

nohref

Omitting the href attribute is sufficient; the nohref attribute is unnecessary. Omit it altogether.

Applies to: nohref on area elements

Open the source definition or discovery route

profile

Unnecessary. Omit it altogether.

Applies to: profile on head elements

Open the source definition or discovery route

manifest

Use service workers instead. [SW]

Applies to: manifest on html elements

Open the source definition or discovery route

version

Unnecessary. Omit it altogether.

Applies to: version on html elements

Open the source definition or discovery route

ismap

Unnecessary. Omit it altogether. All input elements with a type attribute in the Image Button state are processed as server-side image maps.

Applies to: ismap on input elements

Open the source definition or discovery route

usemap

Use the img element for image maps.

Applies to: usemap on input elements

Open the source definition or discovery route

usemap

Use the img element for image maps.

Applies to: usemap on object elements

Open the source definition or discovery route

longdesc

Use a regular a element to link to the description, or (in the case of images) use an image map to provide a link from the image to the image's description.

Applies to: longdesc on iframe elements

Open the source definition or discovery route

longdesc

Use a regular a element to link to the description, or (in the case of images) use an image map to provide a link from the image to the image's description.

Applies to: longdesc on img elements

Open the source definition or discovery route

lowsrc

Use a progressive JPEG image (given in the src attribute), instead of using two separate images.

Applies to: lowsrc on img elements

Open the source definition or discovery route

target

Unnecessary. Omit it altogether.

Applies to: target on link elements

Open the source definition or discovery route

type

To implement a custom context menu, use script to handle the contextmenu event. For toolbar menus, omit the attribute.

Applies to: type on menu elements

Open the source definition or discovery route

label

To implement a custom context menu, use script to handle the contextmenu event.

Applies to: label on menu elements

Open the source definition or discovery route

contextmenu

To implement a custom context menu, use script to handle the contextmenu event.

Applies to: contextmenu on all elements

Open the source definition or discovery route

onshow

To implement a custom context menu, use script to handle the contextmenu event.

Applies to: onshow on all elements

Open the source definition or discovery route

scheme

Use only one scheme per field, or make the scheme declaration part of the value.

Applies to: scheme on meta elements

Open the source definition or discovery route

archive

Use the data and type attributes to invoke plugins .

Applies to: archive on object elements

Open the source definition or discovery route

classid

Use the data and type attributes to invoke plugins .

Applies to: classid on object elements

Open the source definition or discovery route

code

Use the data and type attributes to invoke plugins .

Applies to: code on object elements

Open the source definition or discovery route

codebase

Use the data and type attributes to invoke plugins .

Applies to: codebase on object elements

Open the source definition or discovery route

codetype

Use the data and type attributes to invoke plugins .

Applies to: codetype on object elements

Open the source definition or discovery route

declare

Repeat the object element completely each time the resource is to be reused.

Applies to: declare on object elements

Open the source definition or discovery route

standby

Optimize the linked resource so that it loads quickly or, at least, incrementally.

Applies to: standby on object elements

Open the source definition or discovery route

typemustmatch

Avoid using object elements with untrusted resources.

Applies to: typemustmatch on object elements

Open the source definition or discovery route

language

Omit the attribute for JavaScript; for data blocks , use the type attribute instead.

Applies to: language on script elements (except as noted in the previous section)

Open the source definition or discovery route

event

Use DOM events mechanisms to register event listeners. [DOM]

Applies to: event on script elements

Open the source definition or discovery route

for

Use DOM events mechanisms to register event listeners. [DOM]

Applies to: for on script elements

Open the source definition or discovery route

type

Omit the attribute for CSS; for data blocks , use script as the container instead of style .

Applies to: type on style elements (except as noted in the previous section)

Open the source definition or discovery route

datapagesize

Unnecessary. Omit it altogether.

Applies to: datapagesize on table elements

Open the source definition or discovery route

summary

Use one of the techniques for describing tables given in the table section instead.

Applies to: summary on table elements

Open the source definition or discovery route

abbr

Use text that begins in an unambiguous and terse manner, and include any more elaborate text after that. The title attribute can also be useful in including more detailed text, so that the cell's contents can be made terse. If it's a heading, use th (which has an abbr attribute).

Applies to: abbr on td elements

Open the source definition or discovery route

axis

Use the scope attribute on the relevant th .

Applies to: axis on td and th elements

Open the source definition or discovery route

scope

Use th elements for heading cells.

Applies to: scope on td elements

Open the source definition or discovery route

datasrc

Use script and a mechanism such as XMLHttpRequest to populate the page dynamically. [XHR]

Applies to: datasrc on a , button , div , frame , iframe , img , input , label , legend , marquee , object , option , select , span , table , and textarea elements

Open the source definition or discovery route

datafld

Use script and a mechanism such as XMLHttpRequest to populate the page dynamically. [XHR]

Applies to: datafld on a , button , div , fieldset , frame , iframe , img , input , label , legend , marquee , object , select , span , and textarea elements

Open the source definition or discovery route

dataformatas

Use script and a mechanism such as XMLHttpRequest to populate the page dynamically. [XHR]

Applies to: dataformatas on button , div , input , label , legend , marquee , object , option , select , span , and table elements

Open the source definition or discovery route

dropzone

Use script to handle the dragenter and dragover events instead.

Applies to: dropzone on all elements

Open the source definition or discovery route

alink

Use CSS instead.

Applies to: alink on body elements

Open the source definition or discovery route

bgcolor

Use CSS instead.

Applies to: bgcolor on body elements

Open the source definition or discovery route

bottommargin

Use CSS instead.

Applies to: bottommargin on body elements

Open the source definition or discovery route

leftmargin

Use CSS instead.

Applies to: leftmargin on body elements

Open the source definition or discovery route

link

Use CSS instead.

Applies to: link on body elements

Open the source definition or discovery route

marginheight

Use CSS instead.

Applies to: marginheight on body elements

Open the source definition or discovery route

marginwidth

Use CSS instead.

Applies to: marginwidth on body elements

Open the source definition or discovery route

rightmargin

Use CSS instead.

Applies to: rightmargin on body elements

Open the source definition or discovery route

text

Use CSS instead.

Applies to: text on body elements

Open the source definition or discovery route

topmargin

Use CSS instead.

Applies to: topmargin on body elements

Open the source definition or discovery route

vlink

Use CSS instead.

Applies to: vlink on body elements

Open the source definition or discovery route

clear

Use CSS instead.

Applies to: clear on br elements

Open the source definition or discovery route

align

Use CSS instead.

Applies to: align on caption elements

Open the source definition or discovery route

align

Use CSS instead.

Applies to: align on col elements

Open the source definition or discovery route

char

Use CSS instead.

Applies to: char on col elements

Open the source definition or discovery route

charoff

Use CSS instead.

Applies to: charoff on col elements

Open the source definition or discovery route

valign

Use CSS instead.

Applies to: valign on col elements

Open the source definition or discovery route

width

Use CSS instead.

Applies to: width on col elements

Open the source definition or discovery route

align

Use CSS instead.

Applies to: align on div elements

Open the source definition or discovery route

compact

Use CSS instead.

Applies to: compact on dl elements

Open the source definition or discovery route

align

Use CSS instead.

Applies to: align on embed elements

Open the source definition or discovery route

hspace

Use CSS instead.

Applies to: hspace on embed elements

Open the source definition or discovery route

vspace

Use CSS instead.

Applies to: vspace on embed elements

Open the source definition or discovery route

align

Use CSS instead.

Applies to: align on hr elements

Open the source definition or discovery route

color

Use CSS instead.

Applies to: color on hr elements

Open the source definition or discovery route

noshade

Use CSS instead.

Applies to: noshade on hr elements

Open the source definition or discovery route

size

Use CSS instead.

Applies to: size on hr elements

Open the source definition or discovery route

width

Use CSS instead.

Applies to: width on hr elements

Open the source definition or discovery route

align

Use CSS instead.

Applies to: align on h1 , h6 elements

Open the source definition or discovery route

align

Use CSS instead.

Applies to: align on iframe elements

Open the source definition or discovery route

allowtransparency

Use CSS instead.

Applies to: allowtransparency on iframe elements

Open the source definition or discovery route

frameborder

Use CSS instead.

Applies to: frameborder on iframe elements

Open the source definition or discovery route

framespacing

Use CSS instead.

Applies to: framespacing on iframe elements

Open the source definition or discovery route

hspace

Use CSS instead.

Applies to: hspace on iframe elements

Open the source definition or discovery route

marginheight

Use CSS instead.

Applies to: marginheight on iframe elements

Open the source definition or discovery route

marginwidth

Use CSS instead.

Applies to: marginwidth on iframe elements

Open the source definition or discovery route

scrolling

Use CSS instead.

Applies to: scrolling on iframe elements

Open the source definition or discovery route

vspace

Use CSS instead.

Applies to: vspace on iframe elements

Open the source definition or discovery route

align

Use CSS instead.

Applies to: align on input elements

Open the source definition or discovery route

border

Use CSS instead.

Applies to: border on input elements

Open the source definition or discovery route

hspace

Use CSS instead.

Applies to: hspace on input elements

Open the source definition or discovery route

vspace

Use CSS instead.

Applies to: vspace on input elements

Open the source definition or discovery route

align

Use CSS instead.

Applies to: align on img elements

Open the source definition or discovery route

border

Use CSS instead.

Applies to: border on img elements (except as noted in the previous section)

Open the source definition or discovery route

hspace

Use CSS instead.

Applies to: hspace on img elements

Open the source definition or discovery route

vspace

Use CSS instead.

Applies to: vspace on img elements

Open the source definition or discovery route

align

Use CSS instead.

Applies to: align on legend elements

Open the source definition or discovery route

type

Use CSS instead.

Applies to: type on li elements

Open the source definition or discovery route

compact

Use CSS instead.

Applies to: compact on menu elements

Open the source definition or discovery route

align

Use CSS instead.

Applies to: align on object elements

Open the source definition or discovery route

border

Use CSS instead.

Applies to: border on object elements

Open the source definition or discovery route

hspace

Use CSS instead.

Applies to: hspace on object elements

Open the source definition or discovery route

vspace

Use CSS instead.

Applies to: vspace on object elements

Open the source definition or discovery route

compact

Use CSS instead.

Applies to: compact on ol elements

Open the source definition or discovery route

align

Use CSS instead.

Applies to: align on p elements

Open the source definition or discovery route

width

Use CSS instead.

Applies to: width on pre elements

Open the source definition or discovery route

align

Use CSS instead.

Applies to: align on table elements

Open the source definition or discovery route

bgcolor

Use CSS instead.

Applies to: bgcolor on table elements

Open the source definition or discovery route

border

Use CSS instead.

Applies to: border on table elements

Open the source definition or discovery route

bordercolor

Use CSS instead.

Applies to: bordercolor on table elements

Open the source definition or discovery route

cellpadding

Use CSS instead.

Applies to: cellpadding on table elements

Open the source definition or discovery route

cellspacing

Use CSS instead.

Applies to: cellspacing on table elements

Open the source definition or discovery route

frame

Use CSS instead.

Applies to: frame on table elements

Open the source definition or discovery route

height

Use CSS instead.

Applies to: height on table elements

Open the source definition or discovery route

rules

Use CSS instead.

Applies to: rules on table elements

Open the source definition or discovery route

width

Use CSS instead.

Applies to: width on table elements

Open the source definition or discovery route

align

Use CSS instead.

Applies to: align on tbody , thead , and tfoot elements

Open the source definition or discovery route

char

Use CSS instead.

Applies to: char on tbody , thead , and tfoot elements

Open the source definition or discovery route

charoff

Use CSS instead.

Applies to: charoff on tbody , thead , and tfoot elements

Open the source definition or discovery route

height

Use CSS instead.

Applies to: height on thead , tbody , and tfoot elements

Open the source definition or discovery route

valign

Use CSS instead.

Applies to: valign on tbody , thead , and tfoot elements

Open the source definition or discovery route

align

Use CSS instead.

Applies to: align on td and th elements

Open the source definition or discovery route

bgcolor

Use CSS instead.

Applies to: bgcolor on td and th elements

Open the source definition or discovery route

char

Use CSS instead.

Applies to: char on td and th elements

Open the source definition or discovery route

charoff

Use CSS instead.

Applies to: charoff on td and th elements

Open the source definition or discovery route

height

Use CSS instead.

Applies to: height on td and th elements

Open the source definition or discovery route

nowrap

Use CSS instead.

Applies to: nowrap on td and th elements

Open the source definition or discovery route

valign

Use CSS instead.

Applies to: valign on td and th elements

Open the source definition or discovery route

width

Use CSS instead.

Applies to: width on td and th elements

Open the source definition or discovery route

align

Use CSS instead.

Applies to: align on tr elements

Open the source definition or discovery route

bgcolor

Use CSS instead.

Applies to: bgcolor on tr elements

Open the source definition or discovery route

char

Use CSS instead.

Applies to: char on tr elements

Open the source definition or discovery route

charoff

Use CSS instead.

Applies to: charoff on tr elements

Open the source definition or discovery route

height

Use CSS instead.

Applies to: height on tr elements

Open the source definition or discovery route

valign

Use CSS instead.

Applies to: valign on tr elements

Open the source definition or discovery route

compact

Use CSS instead.

Applies to: compact on ul elements

Open the source definition or discovery route

type

Use CSS instead.

Applies to: type on ul elements

Open the source definition or discovery route

background

Use CSS instead.

Applies to: background on body , table , thead , tbody , tfoot , tr , td , and th elements

Open the source definition or discovery route

Condition 1

Authors should not specify a border attribute on an img element. If the attribute is present, its value must be the string " 0 ". CSS should be used instead.

Open the source definition or discovery route

Condition 2

Authors should not specify a charset attribute on a script element. If the attribute is present, its value must be an ASCII case-insensitive match for " utf-8 ". (This has no effect in a document that conforms to the requirements elsewhere in this standard of being encoded as UTF-8 .)

Open the source definition or discovery route

Condition 3

Authors should not specify a language attribute on a script element. If the attribute is present, its value must be an ASCII case-insensitive match for the string " JavaScript " and either the type attribute must be omitted or its value must be an ASCII case-insensitive match for the string " text/javascript ". The attribute should be entirely omitted instead (with the value " JavaScript ", it has no effect), or replaced with use of the type attribute.

Open the source definition or discovery route

Condition 4

Authors should not specify a value for the type attribute on script elements that is the empty string or a JavaScript MIME type essence match . Instead, they should omit the attribute, which has the same effect.

Open the source definition or discovery route

Condition 5

Authors should not specify a type attribute on a style element. If the attribute is present, its value must be an ASCII case-insensitive match for " text/css ".

Open the source definition or discovery route

Condition 6

Authors should not specify the name attribute on a elements. If the attribute is present, its value must not be the empty string and must neither be equal to the value of any of the IDs in the element's tree other than the element's own ID , if any, nor be equal to the value of any of the other name attributes on a elements in the element's tree . If this attribute is present and the element has an ID , then the attribute's value must be equal to the element's ID . In earlier versions of the language, this attribute was intended as a way to specify possible targets for fragments in URLs . The id attribute should be used instead.

Open the source definition or discovery route

Condition 7

Authors should not, but may despite requirements to the contrary elsewhere in this specification, specify the maxlength and size attributes on input elements whose type attributes are in the Number state. One valid reason for using these attributes regardless is to help legacy user agents that do not support input elements with type="number" to still render the text control with a useful width.

Open the source definition or discovery route

API interface routes

Index-derived summaries from the captured foundation snapshot. Read the full definition for restrictions, defaults and exceptions. Standard presence does not imply universal implementation. API member-level completeness is still open.

AudioTrack

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

AudioTrackList

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

BarProp

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

BeforeUnloadEvent

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

BroadcastChannel

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

CanvasGradient

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

CanvasPattern

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

CanvasRenderingContext2D

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

CloseWatcher

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

CommandEvent

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

CustomElementRegistry

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

CustomStateSet

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

DOMParser

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

DOMStringList

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

DOMStringMap

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

DataTransfer

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

DataTransferItem

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

DataTransferItemList

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

DedicatedWorkerGlobalScope

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

Document

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

DragEvent

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

Element

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

ElementInternals

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

ErrorEvent

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

EventSource

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

External

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

FormDataEvent

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLAllCollection

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLAnchorElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLAreaElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLAudioElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLBRElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLBaseElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLBodyElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLButtonElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLCanvasElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLDListElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLDataElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLDataListElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLDetailsElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLDialogElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLDirectoryElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLDivElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLEmbedElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLFieldSetElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLFontElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLFormControlsCollection

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLFormElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLFrameElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLFrameSetElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLHRElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLHeadElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLHeadingElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLHtmlElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLIFrameElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLImageElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLInputElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLLIElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLLabelElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLLegendElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLLinkElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLMapElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLMarqueeElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLMediaElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLMenuElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLMetaElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLMeterElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLModElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLOListElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLObjectElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLOptGroupElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLOptionElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLOptionsCollection

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLOutputElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLParagraphElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLParamElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLPictureElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLPreElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLProgressElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLQuoteElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLScriptElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLSelectElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLSelectedContentElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLSlotElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLSourceElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLSpanElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLStyleElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLTableCaptionElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLTableCellElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLTableColElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLTableElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLTableRowElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLTableSectionElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLTemplateElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLTextAreaElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLTimeElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLTitleElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLTrackElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLUListElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLUnknownElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HTMLVideoElement

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

HashChangeEvent

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

History

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

ImageBitmap

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

ImageBitmapRenderingContext

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

ImageData

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

Location

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

MediaError

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

MessageChannel

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

MessageEvent

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

MessagePort

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

MimeType

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

MimeTypeArray

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

NavigateEvent

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

Navigation

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

NavigationActivation

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

NavigationCurrentEntryChangeEvent

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

NavigationDestination

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

NavigationHistoryEntry

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

NavigationPrecommitController

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

NavigationTransition

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

Navigator

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

NotRestoredReasonDetails

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

NotRestoredReasons

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

OffscreenCanvas

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

OffscreenCanvasRenderingContext2D

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

Origin

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

PageRevealEvent

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

PageSwapEvent

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

PageTransitionEvent

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

Path2D

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

Plugin

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

PluginArray

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

PopStateEvent

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

PromiseRejectionEvent

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

RadioNodeList

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

Range

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

Sanitizer

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

ShadowRoot

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

SharedWorker

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

SharedWorkerGlobalScope

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

Storage

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

StorageEvent

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

SubmitEvent

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

TextMetrics

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

TextTrack

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

TextTrackCue

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

TextTrackCueList

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

TextTrackList

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

TimeRanges

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

ToggleEvent

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

TrackEvent

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

UserActivation

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

ValidityState

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

VideoTrack

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

VideoTrackList

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

VisibilityStateEntry

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

Window

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

Worker

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

WorkerGlobalScope

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

WorkerLocation

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

WorkerNavigator

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

Worklet

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

WorkletGlobalScope

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

XMLSerializer

Named interfaces in the non-normative index; individual members not yet extracted

Open the source definition or discovery route

API declaration register

314 named declarations and 1,574 direct member declarations, extracted from the captured HTML WebIDL through W3C Webref. Partial declarations are consolidated. Inherited and mixed-in members remain on their declaring owner. This is a signature register, not completed behavioral documentation. The website supplies direct member links, match labels and searchable owner pages.

AbstractWorker

interface mixin

WebIDL / AbstractWorker

```
interface mixin AbstractWorker {
  attribute EventHandler onerror;
};
```

Normative owner definition

AnimationFrameProvider

interface mixin

WebIDL / AnimationFrameProvider

```
interface mixin AnimationFrameProvider {
  unsigned long requestAnimationFrame(FrameRequestCallback callback);
  undefined cancelAnimationFrame(unsigned long handle);
};
```

Normative owner definition

AssignedNodesOptions

dictionary

WebIDL / AssignedNodesOptions

```
dictionary AssignedNodesOptions {
  boolean flatten = false;
};
```

Normative owner definition

AudioTrack

interface

WebIDL / AudioTrack

```
[Exposed=Window]
interface AudioTrack {
  readonly attribute DOMString id;
  readonly attribute DOMString kind;
  readonly attribute DOMString label;
  readonly attribute DOMString language;
  attribute boolean enabled;
};
```

Normative owner definition

AudioTrackList

interface / inherits EventTarget

WebIDL / AudioTrackList

```
[Exposed=Window]
interface AudioTrackList : EventTarget {
  readonly attribute unsigned long length;
  getter AudioTrack (unsigned long index);
  AudioTrack? getTrackById(DOMString id);

  attribute EventHandler onchange;
  attribute EventHandler onaddtrack;
  attribute EventHandler onremovetrack;
};
```

Normative owner definition

BarProp

interface

WebIDL / BarProp

```
[Exposed=Window]
interface BarProp {
  readonly attribute boolean visible;
};
```

Normative owner definition

BeforeUnloadEvent

interface / inherits Event

WebIDL / BeforeUnloadEvent

```
[Exposed=Window]
interface BeforeUnloadEvent : Event {
  attribute DOMString returnValue;
};
```

Normative owner definition

BlobCallback

callback

WebIDL / BlobCallback

```
callback BlobCallback = undefined (Blob? blob);
```

Normative owner definition

BroadcastChannel

interface / inherits EventTarget

WebIDL / BroadcastChannel

```
[Exposed=(Window,Worker)]
interface BroadcastChannel : EventTarget {
  constructor(DOMString name);

  readonly attribute DOMString name;
  undefined postMessage(any message);
  undefined close();
  attribute EventHandler onmessage;
  attribute EventHandler onmessageerror;
};
```

Normative owner definition

CanPlayTypeResult

enum

WebIDL / CanPlayTypeResult

```
enum CanPlayTypeResult { "" /* empty string */, "maybe", "probably" };
```

Normative owner definition

CanvasColorType

enum

WebIDL / CanvasColorType

```
enum CanvasColorType { "unorm8", "float16" };
```

Normative owner definition

CanvasCompositing

interface mixin

WebIDL / CanvasCompositing

```
interface mixin CanvasCompositing {
  // compositing
  attribute unrestricted double globalAlpha; // (default 1.0)
  attribute DOMString globalCompositeOperation; // (default "source-over")
};
```

Normative owner definition

CanvasDirection

enum

WebIDL / CanvasDirection

```
enum CanvasDirection { "ltr", "rtl", "inherit" };
```

Normative owner definition

CanvasDrawImage

interface mixin

WebIDL / CanvasDrawImage

```
interface mixin CanvasDrawImage {
  // drawing images
  undefined drawImage(CanvasImageSource image, unrestricted double dx, unrestricted double d
y);
  undefined drawImage(CanvasImageSource image, unrestricted double dx, unrestricted double d
y, unrestricted double dw, unrestricted double dh);
  undefined drawImage(CanvasImageSource image, unrestricted double sx, unrestricted double s
```

Code continues on the next page

WebIDL / CanvasDrawImage / continued

```
y, unrestricted double sw, unrestricted double sh, unrestricted double dx, unrestricted doub
le dy, unrestricted double dw, unrestricted double dh);
};
```

Normative owner definition

CanvasDrawPath

interface mixin

WebIDL / CanvasDrawPath

```
interface mixin CanvasDrawPath {
  // path API (see also CanvasPath)
  undefined beginPath();
  undefined fill(optional CanvasFillRule fillRule = "nonzero");
  undefined fill(Path2D path, optional CanvasFillRule fillRule = "nonzero");
  undefined stroke();
  undefined stroke(Path2D path);
  undefined clip(optional CanvasFillRule fillRule = "nonzero");
  undefined clip(Path2D path, optional CanvasFillRule fillRule = "nonzero");
  boolean isPointInPath(unrestricted double x, unrestricted double y, optional CanvasFillRul
e fillRule = "nonzero");
  boolean isPointInPath(Path2D path, unrestricted double x, unrestricted double y, optional
CanvasFillRule fillRule = "nonzero");
  boolean isPointInStroke(unrestricted double x, unrestricted double y);
  boolean isPointInStroke(Path2D path, unrestricted double x, unrestricted double y);
};
```

Normative owner definition

CanvasFillRule

enum

WebIDL / CanvasFillRule

```
enum CanvasFillRule { "nonzero", "evenodd" };
```

Normative owner definition

CanvasFillStrokeStyles

interface mixin

WebIDL / CanvasFillStrokeStyles

```
interface mixin CanvasFillStrokeStyles {
  // colors and styles (see also the CanvasPathDrawingStyles and CanvasTextDrawingStyles int
erfaces)
  attribute (DOMString or CanvasGradient or CanvasPattern) strokeStyle; // (default black)
  attribute (DOMString or CanvasGradient or CanvasPattern) fillStyle; // (default black)
  CanvasGradient createLinearGradient(double x0, double y0, double x1, double y1);
  CanvasGradient createRadialGradient(double x0, double y0, double r0, double x1, double y1,
 double r1);
  CanvasGradient createConicGradient(double startAngle, double x, double y);
  CanvasPattern? createPattern(CanvasImageSource image, [LegacyNullToEmptyString] DOMString
repetition);

};
```

Normative owner definition

CanvasFilters

interface mixin

WebIDL / CanvasFilters

```
interface mixin CanvasFilters {
  // filters
  attribute DOMString filter; // (default "none")
};
```

Normative owner definition

CanvasFontKerning

enum

WebIDL / CanvasFontKerning

```
enum CanvasFontKerning { "auto", "normal", "none" };
```

Normative owner definition

CanvasFontStretch

enum

WebIDL / CanvasFontStretch

```
enum CanvasFontStretch { "ultra-condensed", "extra-condensed", "condensed", "semi-condensed"
, "normal", "semi-expanded", "expanded", "extra-expanded", "ultra-expanded" };
```

Normative owner definition

CanvasFontVariantCaps

enum

WebIDL / CanvasFontVariantCaps

```
enum CanvasFontVariantCaps { "normal", "small-caps", "all-small-caps", "petite-caps", "all-p
etite-caps", "unicase", "titling-caps" };
```

Normative owner definition

CanvasGradient

interface

WebIDL / CanvasGradient

```
[Exposed=(Window,Worker)]
interface CanvasGradient {
  // opaque object
  undefined addColorStop(double offset, DOMString color);
};
```

Normative owner definition

CanvasImageData

interface mixin

WebIDL / CanvasImageData

```
interface mixin CanvasImageData {
  // pixel manipulation
  ImageData createImageData([EnforceRange] long sw, [EnforceRange] long sh, optional ImageDa
taSettings settings = {});
  ImageData createImageData(ImageData imageData);
  ImageData getImageData([EnforceRange] long sx, [EnforceRange] long sy, [EnforceRange] long
 sw, [EnforceRange] long sh, optional ImageDataSettings settings = {});
  undefined putImageData(ImageData imageData, [EnforceRange] long dx, [EnforceRange] long dy
);
```

Code continues on the next page

WebIDL / CanvasImageData / continued

```
  undefined putImageData(ImageData imageData, [EnforceRange] long dx, [EnforceRange] long dy
, [EnforceRange] long dirtyX, [EnforceRange] long dirtyY, [EnforceRange] long dirtyWidth, [E
nforceRange] long dirtyHeight);
};
```

Normative owner definition

CanvasImageSmoothing

interface mixin

WebIDL / CanvasImageSmoothing

```
interface mixin CanvasImageSmoothing {
  // image smoothing
  attribute boolean imageSmoothingEnabled; // (default true)
  attribute ImageSmoothingQuality imageSmoothingQuality; // (default low)

};
```

Normative owner definition

CanvasImageSource

typedef

WebIDL / CanvasImageSource

```
typedef (HTMLOrSVGImageElement or
         HTMLVideoElement or
         HTMLCanvasElement or
         ImageBitmap or
         OffscreenCanvas or
         VideoFrame) CanvasImageSource;
```

Normative owner definition

CanvasLineCap

enum

WebIDL / CanvasLineCap

```
enum CanvasLineCap { "butt", "round", "square" };
```

Normative owner definition

CanvasLineJoin

enum

WebIDL / CanvasLineJoin

```
enum CanvasLineJoin { "round", "bevel", "miter" };
```

Normative owner definition

CanvasPath

interface mixin

WebIDL / CanvasPath

```
interface mixin CanvasPath {
  // shared path API methods
  undefined closePath();
  undefined moveTo(unrestricted double x, unrestricted double y);
  undefined lineTo(unrestricted double x, unrestricted double y);
  undefined quadraticCurveTo(unrestricted double cpx, unrestricted double cpy, unrestricted
double x, unrestricted double y);
  undefined bezierCurveTo(unrestricted double cp1x, unrestricted double cp1y, unrestricted d
ouble cp2x, unrestricted double cp2y, unrestricted double x, unrestricted double y);
  undefined arcTo(unrestricted double x1, unrestricted double y1, unrestricted double x2, un
restricted double y2, unrestricted double radius);
  undefined rect(unrestricted double x, unrestricted double y, unrestricted double w, unrest
ricted double h);
  undefined roundRect(unrestricted double x, unrestricted double y, unrestricted double w, u
nrestricted double h, optional (unrestricted double or DOMPointInit or sequence<(unrestricte
d double or DOMPointInit)>) radii = 0);
  undefined arc(unrestricted double x, unrestricted double y, unrestricted double radius, un
restricted double startAngle, unrestricted double endAngle, optional boolean counterclockwis
e = false);
  undefined ellipse(unrestricted double x, unrestricted double y, unrestricted double radius
X, unrestricted double radiusY, unrestricted double rotation, unrestricted double startAngle
, unrestricted double endAngle, optional boolean counterclockwise = false);
};
```

Normative owner definition

CanvasPathDrawingStyles

interface mixin

WebIDL / CanvasPathDrawingStyles

```
interface mixin CanvasPathDrawingStyles {
  // line caps/joins
  attribute unrestricted double lineWidth; // (default 1)
  attribute CanvasLineCap lineCap; // (default "butt")
  attribute CanvasLineJoin lineJoin; // (default "miter")
  attribute unrestricted double miterLimit; // (default 10)

  // dashed lines
  undefined setLineDash(sequence<unrestricted double> segments); // default empty
  sequence<unrestricted double> getLineDash();
  attribute unrestricted double lineDashOffset;
};
```

Normative owner definition

CanvasPattern

interface

WebIDL / CanvasPattern

```
[Exposed=(Window,Worker)]
interface CanvasPattern {
  // opaque object
  undefined setTransform(optional DOMMatrix2DInit transform = {});
};
```

Normative owner definition

CanvasRect

interface mixin

WebIDL / CanvasRect

```
interface mixin CanvasRect {
  // rects
  undefined clearRect(unrestricted double x, unrestricted double y, unrestricted double w, u
nrestricted double h);
  undefined fillRect(unrestricted double x, unrestricted double y, unrestricted double w, un
restricted double h);
  undefined strokeRect(unrestricted double x, unrestricted double y, unrestricted double w,
unrestricted double h);
};
```

Normative owner definition

CanvasRenderingContext2D

interface / includes CanvasSettings, CanvasState, CanvasTransform, CanvasCompositing, CanvasImageSmoothing, CanvasFillStrokeStyles, CanvasShadowStyles, CanvasFilters, CanvasRect, CanvasDrawPath, CanvasUserInterface, CanvasText, CanvasDrawImage, CanvasImageData, CanvasPathDrawingStyles, CanvasTextDrawingStyles, CanvasPath

WebIDL / CanvasRenderingContext2D

```
[Exposed=Window]
interface CanvasRenderingContext2D {
  // back-reference to the canvas
  readonly attribute HTMLCanvasElement canvas;
};
```

Normative owner definition

CanvasRenderingContext2DSettings

dictionary

WebIDL / CanvasRenderingContext2DSettings

```
dictionary CanvasRenderingContext2DSettings {
  boolean alpha = true;
  boolean desynchronized = false;
  PredefinedColorSpace colorSpace = "srgb";
  CanvasColorType colorType = "unorm8";
  boolean willReadFrequently = false;
};
```

Normative owner definition

CanvasSettings

interface mixin

WebIDL / CanvasSettings

```
interface mixin CanvasSettings {
  // settings
  CanvasRenderingContext2DSettings getContextAttributes();
};
```

Normative owner definition

CanvasShadowStyles

interface mixin

WebIDL / CanvasShadowStyles

```
interface mixin CanvasShadowStyles {
  // shadows
  attribute unrestricted double shadowOffsetX; // (default 0)
  attribute unrestricted double shadowOffsetY; // (default 0)
  attribute unrestricted double shadowBlur; // (default 0)
  attribute DOMString shadowColor; // (default transparent black)
};
```

Normative owner definition

CanvasState

interface mixin

WebIDL / CanvasState

```
interface mixin CanvasState {
  // state
  undefined save(); // push state on state stack
  undefined restore(); // pop state stack and restore state
  undefined reset(); // reset the rendering context to its default state
  boolean isContextLost(); // return whether context is lost
};
```

Normative owner definition

CanvasText

interface mixin

WebIDL / CanvasText

```
interface mixin CanvasText {
  // text (see also the CanvasPathDrawingStyles and CanvasTextDrawingStyles interfaces)
  undefined fillText(DOMString text, unrestricted double x, unrestricted double y, optional
unrestricted double maxWidth);
  undefined strokeText(DOMString text, unrestricted double x, unrestricted double y, optiona
l unrestricted double maxWidth);
  TextMetrics measureText(DOMString text);
};
```

Normative owner definition

CanvasTextAlign

enum

WebIDL / CanvasTextAlign

```
enum CanvasTextAlign { "start", "end", "left", "right", "center" };
```

Normative owner definition

CanvasTextBaseline

enum

WebIDL / CanvasTextBaseline

```
enum CanvasTextBaseline { "top", "hanging", "middle", "alphabetic", "ideographic", "bottom"
};
```

Normative owner definition

CanvasTextDrawingStyles

interface mixin

WebIDL / CanvasTextDrawingStyles

```
interface mixin CanvasTextDrawingStyles {
  // text
  attribute DOMString lang; // (default: "inherit")
  attribute DOMString font; // (default 10px sans-serif)
  attribute CanvasTextAlign textAlign; // (default: "start")
  attribute CanvasTextBaseline textBaseline; // (default: "alphabetic")
  attribute CanvasDirection direction; // (default: "inherit")
  attribute DOMString letterSpacing; // (default: "0px")
  attribute CanvasFontKerning fontKerning; // (default: "auto")
  attribute CanvasFontStretch fontStretch; // (default: "normal")
  attribute CanvasFontVariantCaps fontVariantCaps; // (default: "normal")
  attribute CanvasTextRendering textRendering; // (default: "auto")
  attribute DOMString wordSpacing; // (default: "0px")
};
```

Normative owner definition

CanvasTextRendering

enum

WebIDL / CanvasTextRendering

```
enum CanvasTextRendering { "auto", "optimizeSpeed", "optimizeLegibility", "geometricPrecisio
n" };
```

Normative owner definition

CanvasTransform

interface mixin

WebIDL / CanvasTransform

```
interface mixin CanvasTransform {
  // transformations (default transform is the identity matrix)
  undefined scale(unrestricted double x, unrestricted double y);
  undefined rotate(unrestricted double angle);
  undefined translate(unrestricted double x, unrestricted double y);
  undefined transform(unrestricted double a, unrestricted double b, unrestricted double c, u
nrestricted double d, unrestricted double e, unrestricted double f);

  [NewObject] DOMMatrix getTransform();
  undefined setTransform(unrestricted double a, unrestricted double b, unrestricted double c
, unrestricted double d, unrestricted double e, unrestricted double f);
  undefined setTransform(optional DOMMatrix2DInit transform = {});
  undefined resetTransform();

};
```

Normative owner definition

CanvasUserInterface

interface mixin

WebIDL / CanvasUserInterface

```
interface mixin CanvasUserInterface {
  undefined drawFocusIfNeeded(Element element);
  undefined drawFocusIfNeeded(Path2D path, Element element);
};
```

Normative owner definition

CloseWatcher

interface / inherits EventTarget

WebIDL / CloseWatcher

```
[Exposed=Window]
interface CloseWatcher : EventTarget {
  constructor(optional CloseWatcherOptions options = {});

  undefined requestClose();
```

Code continues on the next page

WebIDL / CloseWatcher / continued

```
  undefined close();
  undefined destroy();

  attribute EventHandler oncancel;
  attribute EventHandler onclose;
};
```

Normative owner definition

CloseWatcherOptions

dictionary

WebIDL / CloseWatcherOptions

```
dictionary CloseWatcherOptions {
  AbortSignal signal;
};
```

Normative owner definition

ColorSpaceConversion

enum

WebIDL / ColorSpaceConversion

```
enum ColorSpaceConversion { "none", "default" };
```

Normative owner definition

CommandEvent

interface / inherits Event

WebIDL / CommandEvent

```
[Exposed=Window]
interface CommandEvent : Event {
  constructor(DOMString type, optional CommandEventInit eventInitDict = {});
  readonly attribute Element? source;
  readonly attribute DOMString command;
};
```

Normative owner definition

CommandEventInit

dictionary / inherits EventInit

WebIDL / CommandEventInit

```
dictionary CommandEventInit : EventInit {
  Element? source = null;
  DOMString command = "";
};
```

Normative owner definition

CustomElementConstructor

callback

WebIDL / CustomElementConstructor

```
callback CustomElementConstructor = HTMLElement ();
```

Normative owner definition

CustomElementRegistry

interface

WebIDL / CustomElementRegistry

```
[Exposed=Window]
interface CustomElementRegistry {
  constructor();

  [CEReactions] undefined define(DOMString name, CustomElementConstructor constructor, optio
nal ElementDefinitionOptions options = {});
  (CustomElementConstructor or undefined) get(DOMString name);
  DOMString? getName(CustomElementConstructor constructor);
  Promise<CustomElementConstructor> whenDefined(DOMString name);
  [CEReactions] undefined upgrade(Node root);
  [CEReactions] undefined initialize(Node root);
};
```

Normative owner definition

CustomStateSet

interface

WebIDL / CustomStateSet

```
[Exposed=Window]
interface CustomStateSet {
  setlike<DOMString>;
};
```

Normative owner definition

DataTransfer

interface

WebIDL / DataTransfer

```
[Exposed=Window]
interface DataTransfer {
  constructor();

  attribute DOMString dropEffect;
  attribute DOMString effectAllowed;

  [SameObject] readonly attribute DataTransferItemList items;

  undefined setDragImage(Element image, long x, long y);

  /* old interface */
  readonly attribute FrozenArray<DOMString> types;
  DOMString getData(DOMString format);
  undefined setData(DOMString format, DOMString data);
  undefined clearData(optional DOMString format);
  [SameObject] readonly attribute FileList files;
};
```

Normative owner definition

DataTransferItem

interface

WebIDL / DataTransferItem

```
[Exposed=Window]
interface DataTransferItem {
  readonly attribute DOMString kind;
  readonly attribute DOMString type;
  undefined getAsString(FunctionStringCallback? _callback);
  File? getAsFile();
};
```

Normative owner definition

DataTransferItemList

interface

WebIDL / DataTransferItemList

```
[Exposed=Window]
interface DataTransferItemList {
  readonly attribute unsigned long length;
  getter DataTransferItem (unsigned long index);
  DataTransferItem? add(DOMString data, DOMString type);
  DataTransferItem? add(File data);
  undefined remove(unsigned long index);
  undefined clear();
};
```

Normative owner definition

DedicatedWorkerGlobalScope

interface / inherits WorkerGlobalScope / includes AnimationFrameProvider, MessageEventTarget

WebIDL / DedicatedWorkerGlobalScope

```
[Global=(Worker,DedicatedWorker),Exposed=DedicatedWorker]
interface DedicatedWorkerGlobalScope : WorkerGlobalScope {
  [Replaceable] readonly attribute DOMString name;

  undefined postMessage(any message, sequence<object> transfer);
  undefined postMessage(any message, optional StructuredSerializeOptions options = {});

  undefined close();
};
```

Normative owner definition

Document

interface / includes GlobalEventHandlers

WebIDL / Document

```
[LegacyOverrideBuiltIns]
partial interface Document {
  static Document parseHTMLUnsafe((TrustedHTML or DOMString) html, optional (ParseHTMLUnsafe
Options or TrustedHTMLParserOptions) options = {});
  static Document parseHTML(DOMString html, optional SetHTMLOptions options = {});

  // resource metadata management
  [PutForwards=href, LegacyUnforgeable] readonly attribute Location? location;
```

Code continues on the next page

WebIDL / Document / continued

```
  attribute USVString domain;
  readonly attribute USVString referrer;
  attribute USVString cookie;
  readonly attribute DOMString lastModified;
  readonly attribute DocumentReadyState readyState;

  // DOM tree accessors
  getter object (DOMString name);
  [CEReactions] attribute DOMString title;
  [CEReactions] attribute DOMString dir;
  [CEReactions] attribute HTMLElement? body;
  readonly attribute HTMLHeadElement? head;
  [SameObject] readonly attribute HTMLCollection images;
  [SameObject] readonly attribute HTMLCollection embeds;
  [SameObject] readonly attribute HTMLCollection plugins;
  [SameObject] readonly attribute HTMLCollection links;
  [SameObject] readonly attribute HTMLCollection forms;
  [SameObject] readonly attribute HTMLCollection scripts;
  NodeList getElementsByName(DOMString elementName);
  readonly attribute HTMLOrSVGScriptElement? currentScript; // classic scripts in a document
 tree only

  // dynamic markup insertion
  [CEReactions] Document open(optional DOMString unused1, optional DOMString unused2); // bo
th arguments are ignored
  WindowProxy? open(USVString url, DOMString name, DOMString features);
  [CEReactions] undefined close();
  [CEReactions] undefined write((TrustedHTML or DOMString)... text);
  [CEReactions] undefined writeln((TrustedHTML or DOMString)... text);

  // user interaction
  readonly attribute WindowProxy? defaultView;
  boolean hasFocus();
  [CEReactions] attribute DOMString designMode;
  [CEReactions] boolean execCommand(DOMString commandId, optional boolean showUI = false, op
tional DOMString value = "");
  boolean queryCommandEnabled(DOMString commandId);
  boolean queryCommandIndeterm(DOMString commandId);
  boolean queryCommandState(DOMString commandId);
  boolean queryCommandSupported(DOMString commandId);
  DOMString queryCommandValue(DOMString commandId);
  readonly attribute boolean hidden;
  readonly attribute DocumentVisibilityState visibilityState;

  // special event handler IDL attributes that only apply to Document objects
  [LegacyLenientThis] attribute EventHandler onreadystatechange;
  attribute EventHandler onvisibilitychange;

  // also has obsolete members
};
```

WebIDL / Document

```
partial interface Document {
  [CEReactions] attribute [LegacyNullToEmptyString] DOMString fgColor;
  [CEReactions] attribute [LegacyNullToEmptyString] DOMString linkColor;
  [CEReactions] attribute [LegacyNullToEmptyString] DOMString vlinkColor;
  [CEReactions] attribute [LegacyNullToEmptyString] DOMString alinkColor;
  [CEReactions] attribute [LegacyNullToEmptyString] DOMString bgColor;

  [SameObject] readonly attribute HTMLCollection anchors;
  [SameObject] readonly attribute HTMLCollection applets;

  undefined clear();
  undefined captureEvents();
  undefined releaseEvents();

  [SameObject] readonly attribute HTMLAllCollection all;
};
```

Normative owner definition

DocumentOrShadowRoot

interface mixin

WebIDL / DocumentOrShadowRoot

```
partial interface mixin DocumentOrShadowRoot {
  readonly attribute Element? activeElement;
};
```

Normative owner definition

DocumentReadyState

enum

WebIDL / DocumentReadyState

```
enum DocumentReadyState { "loading", "interactive", "complete" };
```

Normative owner definition

DocumentVisibilityState

enum

WebIDL / DocumentVisibilityState

```
enum DocumentVisibilityState { "visible", "hidden" };
```

Normative owner definition

DOMParser

interface

WebIDL / DOMParser

```
[Exposed=Window]
interface DOMParser {
  constructor();

  [NewObject] Document parseFromString((TrustedHTML or DOMString) string, DOMParserSupported
Type type);
};
```

Normative owner definition

DOMParserSupportedType

enum

WebIDL / DOMParserSupportedType

```
enum DOMParserSupportedType {
  "text/html",
  "text/xml",
  "application/xml",
  "application/xhtml+xml",
  "image/svg+xml"
};
```

Normative owner definition

DOMStringList

interface

WebIDL / DOMStringList

```
[Exposed=(Window,Worker)]
interface DOMStringList {
  readonly attribute unsigned long length;
```

Code continues on the next page

WebIDL / DOMStringList / continued

```
  getter DOMString? item(unsigned long index);
  boolean contains(DOMString string);
};
```

Normative owner definition

DOMStringMap

interface

WebIDL / DOMStringMap

```
[Exposed=Window,
 LegacyOverrideBuiltIns]
interface DOMStringMap {
  getter DOMString (DOMString name);
  [CEReactions] setter undefined (DOMString name, DOMString value);
  [CEReactions] deleter undefined (DOMString name);
};
```

Normative owner definition

DragEvent

interface / inherits MouseEvent

WebIDL / DragEvent

```
[Exposed=Window]
interface DragEvent : MouseEvent {
  constructor(DOMString type, optional DragEventInit eventInitDict = {});

  readonly attribute DataTransfer? dataTransfer;
};
```

Normative owner definition

DragEventInit

dictionary / inherits MouseEventInit

WebIDL / DragEventInit

```
dictionary DragEventInit : MouseEventInit {
  DataTransfer? dataTransfer = null;
};
```

Normative owner definition

Element

interface

WebIDL / Element

```
partial interface Element {
  [CEReactions] undefined setHTML(DOMString html, optional SetHTMLOptions options = {});
  [CEReactions] undefined appendHTML(DOMString html, optional SetHTMLOptions options = {});
  [CEReactions] undefined prependHTML(DOMString html, optional SetHTMLOptions options = {});
  [CEReactions] undefined setHTMLUnsafe((TrustedHTML or DOMString) html, optional (SetHTMLUn
safeOptions or TrustedHTMLParserOptions) options = {});
  [CEReactions] undefined appendHTMLUnsafe((TrustedHTML or DOMString) html, optional (SetHTM
LUnsafeOptions or TrustedHTMLParserOptions) options = {});
  [CEReactions] undefined prependHTMLUnsafe((TrustedHTML or DOMString) html, optional (SetHT
MLUnsafeOptions or TrustedHTMLParserOptions) options = {});
  DOMString getHTML(optional GetHTMLOptions options = {});

  [CEReactions] attribute (TrustedHTML or [LegacyNullToEmptyString] DOMString) innerHTML;
  [CEReactions] attribute (TrustedHTML or [LegacyNullToEmptyString] DOMString) outerHTML;
  [CEReactions] undefined insertAdjacentHTML(DOMString position, (TrustedHTML or DOMString)
string);
};
```

Normative owner definition

ElementContentEditable

interface mixin

WebIDL / ElementContentEditable

```
interface mixin ElementContentEditable {
  [CEReactions] attribute DOMString contentEditable;
  [CEReactions] attribute DOMString enterKeyHint;
  readonly attribute boolean isContentEditable;
  [CEReactions] attribute DOMString inputMode;
};
```

Normative owner definition

ElementDefinitionOptions

dictionary

WebIDL / ElementDefinitionOptions

```
dictionary ElementDefinitionOptions {
  DOMString extends;
};
```

Normative owner definition

ElementInternals

interface / includes ARIAMixin

WebIDL / ElementInternals

```
[Exposed=Window]
interface ElementInternals {
  // Shadow root access
  readonly attribute ShadowRoot? shadowRoot;

  // Form-associated custom elements
  undefined setFormValue((File or USVString or FormData)? value,
                         optional (File or USVString or FormData)? state);

  readonly attribute HTMLFormElement? form;

  undefined setValidity(optional ValidityStateFlags flags = {},
                        optional DOMString message,
                        optional HTMLElement anchor);
  readonly attribute boolean willValidate;
  readonly attribute ValidityState validity;
  readonly attribute DOMString validationMessage;
  boolean checkValidity();
  boolean reportValidity();

  readonly attribute NodeList labels;

  // Custom state pseudo-class
  [SameObject] readonly attribute CustomStateSet states;
};
```

Normative owner definition

ErrorEvent

interface / inherits Event

WebIDL / ErrorEvent

```
[Exposed=*]
interface ErrorEvent : Event {
  constructor(DOMString type, optional ErrorEventInit eventInitDict = {});

  readonly attribute DOMString message;
  readonly attribute USVString filename;
  readonly attribute unsigned long lineno;
  readonly attribute unsigned long colno;
  readonly attribute any error;
};
```

Normative owner definition

ErrorEventInit

dictionary / inherits EventInit

WebIDL / ErrorEventInit

```
dictionary ErrorEventInit : EventInit {
  DOMString message = "";
  USVString filename = "";
  unsigned long lineno = 0;
  unsigned long colno = 0;
  any error;
};
```

Normative owner definition

EventHandler

typedef

WebIDL / EventHandler

```
typedef EventHandlerNonNull? EventHandler;
```

Normative owner definition

EventHandlerNonNull

callback

WebIDL / EventHandlerNonNull

```
[LegacyTreatNonObjectAsNull]
callback EventHandlerNonNull = any (Event event);
```

Normative owner definition

EventSource

interface / inherits EventTarget

WebIDL / EventSource

```
[Exposed=(Window,Worker)]
interface EventSource : EventTarget {
  constructor(USVString url, optional EventSourceInit eventSourceInitDict = {});

  readonly attribute USVString url;
  readonly attribute boolean withCredentials;

  // ready state
  const unsigned short CONNECTING = 0;
  const unsigned short OPEN = 1;
  const unsigned short CLOSED = 2;
  readonly attribute unsigned short readyState;

  // networking
  attribute EventHandler onopen;
  attribute EventHandler onmessage;
  attribute EventHandler onerror;
  undefined close();
};
```

Normative owner definition

EventSourceInit

dictionary

WebIDL / EventSourceInit

```
dictionary EventSourceInit {
  boolean withCredentials = false;
};
```

Normative owner definition

External

interface

WebIDL / External

```
[Exposed=Window]
interface External {
  undefined AddSearchProvider();
  undefined IsSearchProviderInstalled();
};
```

Normative owner definition

FocusOptions

dictionary

WebIDL / FocusOptions

```
dictionary FocusOptions {
  boolean preventScroll = false;
  boolean focusVisible;
};
```

Normative owner definition

FormDataEvent

interface / inherits Event

WebIDL / FormDataEvent

```
[Exposed=Window]
interface FormDataEvent : Event {
  constructor(DOMString type, FormDataEventInit eventInitDict);

  readonly attribute FormData formData;
};
```

Normative owner definition

FormDataEventInit

dictionary / inherits EventInit

WebIDL / FormDataEventInit

```
dictionary FormDataEventInit : EventInit {
  required FormData formData;
};
```

Normative owner definition

FrameRequestCallback

callback

WebIDL / FrameRequestCallback

```
callback FrameRequestCallback = undefined (DOMHighResTimeStamp time);
```

Normative owner definition

FunctionStringCallback

callback

WebIDL / FunctionStringCallback

```
callback FunctionStringCallback = undefined (DOMString data);
```

Normative owner definition

GetHTMLOptions

dictionary

WebIDL / GetHTMLOptions

```
dictionary GetHTMLOptions {
  boolean serializableShadowRoots = false;
  sequence<ShadowRoot> shadowRoots = [];
};
```

Normative owner definition

GlobalEventHandlers

interface mixin

WebIDL / GlobalEventHandlers

```
interface mixin GlobalEventHandlers {
  attribute EventHandler onabort;
  attribute EventHandler onauxclick;
  attribute EventHandler onbeforeinput;
  attribute EventHandler onbeforematch;
  attribute EventHandler onbeforetoggle;
```

Code continues on the next page

WebIDL / GlobalEventHandlers / continued

```
  attribute EventHandler onblur;
  attribute EventHandler oncancel;
  attribute EventHandler oncanplay;
  attribute EventHandler oncanplaythrough;
  attribute EventHandler onchange;
  attribute EventHandler onclick;
  attribute EventHandler onclose;
  attribute EventHandler oncommand;
  attribute EventHandler oncontextlost;
  attribute EventHandler oncontextmenu;
  attribute EventHandler oncontextrestored;
  attribute EventHandler oncopy;
  attribute EventHandler oncuechange;
  attribute EventHandler oncut;
  attribute EventHandler ondblclick;
  attribute EventHandler ondrag;
  attribute EventHandler ondragend;
  attribute EventHandler ondragenter;
  attribute EventHandler ondragleave;
  attribute EventHandler ondragover;
  attribute EventHandler ondragstart;
  attribute EventHandler ondrop;
  attribute EventHandler ondurationchange;
  attribute EventHandler onemptied;
  attribute EventHandler onended;
  attribute OnErrorEventHandler onerror;
  attribute EventHandler onfocus;
  attribute EventHandler onformdata;
  attribute EventHandler oninput;
  attribute EventHandler oninvalid;
  attribute EventHandler onkeydown;
  attribute EventHandler onkeypress;
  attribute EventHandler onkeyup;
  attribute EventHandler onload;
  attribute EventHandler onloadeddata;
  attribute EventHandler onloadedmetadata;
  attribute EventHandler onloadstart;
  attribute EventHandler onmousedown;
  [LegacyLenientThis] attribute EventHandler onmouseenter;
  [LegacyLenientThis] attribute EventHandler onmouseleave;
  attribute EventHandler onmousemove;
  attribute EventHandler onmouseout;
  attribute EventHandler onmouseover;
  attribute EventHandler onmouseup;
  attribute EventHandler onpaste;
  attribute EventHandler onpause;
  attribute EventHandler onplay;
  attribute EventHandler onplaying;
  attribute EventHandler onprogress;
  attribute EventHandler onratechange;
  attribute EventHandler onreset;
  attribute EventHandler onresize;
  attribute EventHandler onscroll;
  attribute EventHandler onscrollend;
```

Code continues on the next page

WebIDL / GlobalEventHandlers / continued

```
  attribute EventHandler onsecuritypolicyviolation;
  attribute EventHandler onseeked;
  attribute EventHandler onseeking;
  attribute EventHandler onselect;
  attribute EventHandler onslotchange;
  attribute EventHandler onstalled;
  attribute EventHandler onsubmit;
  attribute EventHandler onsuspend;
  attribute EventHandler ontimeupdate;
  attribute EventHandler ontoggle;
  attribute EventHandler onvolumechange;
  attribute EventHandler onwaiting;
  attribute EventHandler onwebkitanimationend;
  attribute EventHandler onwebkitanimationiteration;
  attribute EventHandler onwebkitanimationstart;
  attribute EventHandler onwebkittransitionend;
  attribute EventHandler onwheel;
};
```

Normative owner definition

HashChangeEvent

interface / inherits Event

WebIDL / HashChangeEvent

```
[Exposed=Window]
interface HashChangeEvent : Event {
  constructor(DOMString type, optional HashChangeEventInit eventInitDict = {});

  readonly attribute USVString oldURL;
  readonly attribute USVString newURL;
};
```

Normative owner definition

HashChangeEventInit

dictionary / inherits EventInit

WebIDL / HashChangeEventInit

```
dictionary HashChangeEventInit : EventInit {
  USVString oldURL = "";
  USVString newURL = "";
};
```

Normative owner definition

History

interface

WebIDL / History

```
[Exposed=Window]
interface History {
  readonly attribute unsigned long length;
  attribute ScrollRestoration scrollRestoration;
  readonly attribute any state;
  undefined go(optional long delta = 0);
  undefined back();
  undefined forward();
  undefined pushState(any data, DOMString unused, optional USVString? url = null);
  undefined replaceState(any data, DOMString unused, optional USVString? url = null);
};
```

Normative owner definition

HTMLAllCollection

interface

WebIDL / HTMLAllCollection

```
// GENERATED CONTENT - DO NOT EDIT
// Content was automatically extracted by Reffy into webref
// (https://github.com/w3c/webref)
// Source: HTML Standard (https://html.spec.whatwg.org/multipage/)

[Exposed=Window,
 LegacyUnenumerableNamedProperties]
interface HTMLAllCollection {
  readonly attribute unsigned long length;
  getter Element (unsigned long index);
  getter (HTMLCollection or Element)? namedItem(DOMString name);
  (HTMLCollection or Element)? item(optional DOMString nameOrIndex);

  // Note: HTMLAllCollection objects have a custom [[Call]] internal method and an [[IsHTMLD
DA]] internal slot.
};
```

Normative owner definition

HTMLAnchorElement

interface / inherits HTMLElement / includes HyperlinkElementUtils, HTMLHyperlinkElementUtils

WebIDL / HTMLAnchorElement

```
[Exposed=Window]
interface HTMLAnchorElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, Reflect] attribute DOMString download;
  [CEReactions, Reflect] attribute USVString ping;
  [CEReactions, Reflect] attribute DOMString rel;
  [SameObject, PutForwards=value, Reflect="rel"] readonly attribute DOMTokenList relList;

  [CEReactions] attribute DOMString text;

  [CEReactions] attribute DOMString referrerPolicy;

  // also has obsolete members
};
```

WebIDL / HTMLAnchorElement

```
partial interface HTMLAnchorElement {
  [CEReactions, Reflect] attribute DOMString coords;
  [CEReactions, Reflect] attribute DOMString charset;
  [CEReactions, Reflect] attribute DOMString name;
  [CEReactions, Reflect] attribute DOMString rev;
  [CEReactions, Reflect] attribute DOMString shape;
};
```

Normative owner definition

HTMLAreaElement

interface / inherits HTMLElement / includes HyperlinkElementUtils, HTMLHyperlinkElementUtils

WebIDL / HTMLAreaElement

```
[Exposed=Window]
interface HTMLAreaElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, Reflect] attribute DOMString alt;
  [CEReactions, Reflect] attribute DOMString coords;
  [CEReactions, Reflect] attribute DOMString shape;
  [CEReactions, Reflect] attribute DOMString download;
  [CEReactions, Reflect] attribute USVString ping;
  [CEReactions, Reflect] attribute DOMString rel;
  [SameObject, PutForwards=value, Reflect="rel"] readonly attribute DOMTokenList relList;
  [CEReactions] attribute DOMString referrerPolicy;

  // also has obsolete members
};
```

WebIDL / HTMLAreaElement

```
partial interface HTMLAreaElement {
  [CEReactions, Reflect] attribute boolean noHref;
};
```

Normative owner definition

HTMLAudioElement

interface / inherits HTMLMediaElement

WebIDL / HTMLAudioElement

```
[Exposed=Window,
 LegacyFactoryFunction=Audio(optional DOMString src)]
interface HTMLAudioElement : HTMLMediaElement {
  [HTMLConstructor] constructor();
};
```

Normative owner definition

HTMLBaseElement

interface / inherits HTMLElement

WebIDL / HTMLBaseElement

```
[Exposed=Window]
interface HTMLBaseElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, ReflectSetter] attribute USVString href;
  [CEReactions, Reflect] attribute DOMString target;
};
```

Normative owner definition

HTMLBodyElement

interface / inherits HTMLElement / includes WindowEventHandlers

WebIDL / HTMLBodyElement

```
[Exposed=Window]
interface HTMLBodyElement : HTMLElement {
  [HTMLConstructor] constructor();
```

Code continues on the next page

WebIDL / HTMLBodyElement / continued

```
  // also has obsolete members
};
```

WebIDL / HTMLBodyElement

```
partial interface HTMLBodyElement {
  [CEReactions, Reflect] attribute [LegacyNullToEmptyString] DOMString text;
  [CEReactions, Reflect] attribute [LegacyNullToEmptyString] DOMString link;
  [CEReactions, Reflect] attribute [LegacyNullToEmptyString] DOMString vLink;
  [CEReactions, Reflect] attribute [LegacyNullToEmptyString] DOMString aLink;
  [CEReactions, Reflect] attribute [LegacyNullToEmptyString] DOMString bgColor;
  [CEReactions, Reflect] attribute DOMString background;
};
```

Normative owner definition

HTMLBRElement

interface / inherits HTMLElement

WebIDL / HTMLBRElement

```
[Exposed=Window]
interface HTMLBRElement : HTMLElement {
  [HTMLConstructor] constructor();

  // also has obsolete members
};
```

WebIDL / HTMLBRElement

```
partial interface HTMLBRElement {
  [CEReactions, Reflect] attribute DOMString clear;
};
```

Normative owner definition

HTMLButtonElement

interface / inherits HTMLElement / includes PopoverTargetAttributes

WebIDL / HTMLButtonElement

```
[Exposed=Window]
interface HTMLButtonElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, ReflectSetter] attribute DOMString command;
  [CEReactions, Reflect="commandfor"] attribute Element? commandForElement;
  [CEReactions, Reflect] attribute boolean disabled;
  readonly attribute HTMLFormElement? form;
  [CEReactions, ReflectSetter] attribute USVString formAction;
  [CEReactions] attribute DOMString formEnctype;
  [CEReactions] attribute DOMString formMethod;
  [CEReactions, Reflect] attribute boolean formNoValidate;
  [CEReactions, Reflect] attribute DOMString formTarget;
  [CEReactions, Reflect] attribute DOMString name;
  [CEReactions, ReflectSetter] attribute DOMString type;
  [CEReactions, Reflect] attribute DOMString value;

  readonly attribute boolean willValidate;
  readonly attribute ValidityState validity;
  readonly attribute DOMString validationMessage;
  boolean checkValidity();
  boolean reportValidity();
  undefined setCustomValidity(DOMString error);

  readonly attribute NodeList labels;
};
```

Normative owner definition

HTMLCanvasElement

interface / inherits HTMLElement

WebIDL / HTMLCanvasElement

```
[Exposed=Window]
interface HTMLCanvasElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions] attribute unsigned long width;
  [CEReactions] attribute unsigned long height;

  RenderingContext? getContext(DOMString contextId, optional any options = null);

  USVString toDataURL(optional DOMString type = "image/png", optional any quality);
  undefined toBlob(BlobCallback _callback, optional DOMString type = "image/png", optional a
ny quality);
  OffscreenCanvas transferControlToOffscreen();
};
```

Normative owner definition

HTMLDataElement

interface / inherits HTMLElement

WebIDL / HTMLDataElement

```
[Exposed=Window]
interface HTMLDataElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, Reflect] attribute DOMString value;
};
```

Normative owner definition

HTMLDataListElement

interface / inherits HTMLElement

WebIDL / HTMLDataListElement

```
[Exposed=Window]
interface HTMLDataListElement : HTMLElement {
  [HTMLConstructor] constructor();

  [SameObject] readonly attribute HTMLCollection options;
};
```

Normative owner definition

HTMLDetailsElement

interface / inherits HTMLElement

WebIDL / HTMLDetailsElement

```
[Exposed=Window]
interface HTMLDetailsElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, Reflect] attribute DOMString name;
  [CEReactions, Reflect] attribute boolean open;
};
```

Normative owner definition

HTMLDialogElement

interface / inherits HTMLElement

WebIDL / HTMLDialogElement

```
[Exposed=Window]
interface HTMLDialogElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, Reflect] attribute boolean open;
  attribute DOMString returnValue;
  [CEReactions, ReflectSetter] attribute DOMString closedBy;
  [CEReactions] undefined show();
  [CEReactions] undefined showModal();
  [CEReactions] undefined close(optional DOMString returnValue);
  [CEReactions] undefined requestClose(optional DOMString returnValue);
};
```

Normative owner definition

HTMLDirectoryElement

interface / inherits HTMLElement

WebIDL / HTMLDirectoryElement

```
[Exposed=Window]
interface HTMLDirectoryElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, Reflect] attribute boolean compact;
};
```

Normative owner definition

HTMLDivElement

interface / inherits HTMLElement

WebIDL / HTMLDivElement

```
[Exposed=Window]
interface HTMLDivElement : HTMLElement {
  [HTMLConstructor] constructor();

  // also has obsolete members
};
```

WebIDL / HTMLDivElement

```
partial interface HTMLDivElement {
  [CEReactions, Reflect] attribute DOMString align;
};
```

Normative owner definition

HTMLDListElement

interface / inherits HTMLElement

WebIDL / HTMLDListElement

```
[Exposed=Window]
interface HTMLDListElement : HTMLElement {
  [HTMLConstructor] constructor();

  // also has obsolete members
};
```

WebIDL / HTMLDListElement

```
partial interface HTMLDListElement {
  [CEReactions, Reflect] attribute boolean compact;
};
```

Normative owner definition

HTMLElement

interface / inherits Element / includes GlobalEventHandlers, ElementContentEditable, HTMLOrSVGOrMathMLElement

WebIDL / HTMLElement

```
[Exposed=Window]
interface HTMLElement : Element {
  [HTMLConstructor] constructor();

  // metadata attributes
  [CEReactions, Reflect] attribute DOMString title;
  [CEReactions, Reflect] attribute DOMString lang;
  [CEReactions] attribute boolean translate;
  [CEReactions] attribute DOMString dir;

  // user interaction
  [CEReactions] attribute (boolean or unrestricted double or DOMString)? hidden;
```

Code continues on the next page

WebIDL / HTMLElement / continued

```
  [CEReactions, Reflect] attribute boolean inert;
  undefined click();
  [CEReactions, Reflect] attribute DOMString accessKey;
  readonly attribute DOMString accessKeyLabel;
  [CEReactions] attribute boolean draggable;
  [CEReactions] attribute boolean spellcheck;
  [CEReactions, ReflectSetter] attribute DOMString writingSuggestions;
  [CEReactions, ReflectSetter] attribute DOMString autocapitalize;
  [CEReactions] attribute boolean autocorrect;

  [CEReactions] attribute [LegacyNullToEmptyString] DOMString innerText;
  [CEReactions] attribute [LegacyNullToEmptyString] DOMString outerText;

  ElementInternals attachInternals();

  // The popover API
  undefined showPopover(optional ShowPopoverOptions options = {});
  undefined hidePopover();
  boolean togglePopover(optional (TogglePopoverOptions or boolean) options = {});
  [CEReactions] attribute DOMString? popover;

  [CEReactions, Reflect, ReflectRange=(0, 8)] attribute unsigned long headingOffset;
  [CEReactions, Reflect] attribute boolean headingReset;
};
```

Normative owner definition

HTMLEmbedElement

interface / inherits HTMLElement

WebIDL / HTMLEmbedElement

```
[Exposed=Window]
interface HTMLEmbedElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, ReflectURL] attribute USVString src;
  [CEReactions, Reflect] attribute DOMString type;
  [CEReactions, Reflect] attribute DOMString width;
  [CEReactions, Reflect] attribute DOMString height;
  Document? getSVGDocument();

  // also has obsolete members
};
```

WebIDL / HTMLEmbedElement

```
partial interface HTMLEmbedElement {
  [CEReactions, Reflect] attribute DOMString align;
  [CEReactions, Reflect] attribute DOMString name;
};
```

Normative owner definition

HTMLFieldSetElement

interface / inherits HTMLElement

WebIDL / HTMLFieldSetElement

```
[Exposed=Window]
interface HTMLFieldSetElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, Reflect] attribute boolean disabled;
  readonly attribute HTMLFormElement? form;
  [CEReactions, Reflect] attribute DOMString name;

  readonly attribute DOMString type;

  [SameObject] readonly attribute HTMLCollection elements;

  readonly attribute boolean willValidate;
  [SameObject] readonly attribute ValidityState validity;
  readonly attribute DOMString validationMessage;
  boolean checkValidity();
  boolean reportValidity();
  undefined setCustomValidity(DOMString error);
};
```

Normative owner definition

HTMLFontElement

interface / inherits HTMLElement

WebIDL / HTMLFontElement

```
[Exposed=Window]
interface HTMLFontElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, Reflect] attribute [LegacyNullToEmptyString] DOMString color;
  [CEReactions, Reflect] attribute DOMString face;
  [CEReactions, Reflect] attribute DOMString size;
};
```

Normative owner definition

HTMLFormControlsCollection

interface / inherits HTMLCollection

WebIDL / HTMLFormControlsCollection

```
[Exposed=Window]
interface HTMLFormControlsCollection : HTMLCollection {
  // inherits length and item()
  getter (RadioNodeList or Element)? namedItem(DOMString name); // shadows inherited namedIt
em()
};
```

Normative owner definition

HTMLFormElement

interface / inherits HTMLElement

WebIDL / HTMLFormElement

```
[Exposed=Window,
 LegacyOverrideBuiltIns,
 LegacyUnenumerableNamedProperties]
interface HTMLFormElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, Reflect="accept-charset"] attribute DOMString acceptCharset;
  [CEReactions, ReflectSetter] attribute USVString action;
  [CEReactions] attribute DOMString autocomplete;
  [CEReactions] attribute DOMString enctype;
  [CEReactions] attribute DOMString encoding;
  [CEReactions] attribute DOMString method;
  [CEReactions, Reflect] attribute DOMString name;
  [CEReactions, Reflect] attribute boolean noValidate;
  [CEReactions, Reflect] attribute DOMString target;
  [CEReactions, Reflect] attribute DOMString rel;
  [SameObject, PutForwards=value, Reflect="rel"] readonly attribute DOMTokenList relList;

  [SameObject] readonly attribute HTMLFormControlsCollection elements;
  readonly attribute unsigned long length;
  getter Element (unsigned long index);
  getter (RadioNodeList or Element) (DOMString name);

  undefined submit();
  undefined requestSubmit(optional HTMLElement? submitter = null);
  [CEReactions] undefined reset();
  boolean checkValidity();
  boolean reportValidity();
};
```

Normative owner definition

HTMLFrameElement

interface / inherits HTMLElement

WebIDL / HTMLFrameElement

```
[Exposed=Window]
interface HTMLFrameElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, Reflect] attribute DOMString name;
  [CEReactions, Reflect] attribute DOMString scrolling;
  [CEReactions, ReflectURL] attribute USVString src;
  [CEReactions, Reflect] attribute DOMString frameBorder;
  [CEReactions, ReflectURL] attribute USVString longDesc;
  [CEReactions, Reflect] attribute boolean noResize;
  readonly attribute Document? contentDocument;
  readonly attribute WindowProxy? contentWindow;

  [CEReactions, Reflect] attribute [LegacyNullToEmptyString] DOMString marginHeight;
  [CEReactions, Reflect] attribute [LegacyNullToEmptyString] DOMString marginWidth;
};
```

Normative owner definition

HTMLFrameSetElement

interface / inherits HTMLElement / includes WindowEventHandlers

WebIDL / HTMLFrameSetElement

```
[Exposed=Window]
interface HTMLFrameSetElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, Reflect] attribute DOMString cols;
  [CEReactions, Reflect] attribute DOMString rows;
};
```

Normative owner definition

HTMLHeadElement

interface / inherits HTMLElement

WebIDL / HTMLHeadElement

```
[Exposed=Window]
interface HTMLHeadElement : HTMLElement {
  [HTMLConstructor] constructor();
};
```

Normative owner definition

HTMLHeadingElement

interface / inherits HTMLElement

WebIDL / HTMLHeadingElement

```
[Exposed=Window]
interface HTMLHeadingElement : HTMLElement {
  [HTMLConstructor] constructor();

  // also has obsolete members
};
```

WebIDL / HTMLHeadingElement

```
partial interface HTMLHeadingElement {
  [CEReactions, Reflect] attribute DOMString align;
};
```

Normative owner definition

HTMLHRElement

interface / inherits HTMLElement

WebIDL / HTMLHRElement

```
[Exposed=Window]
interface HTMLHRElement : HTMLElement {
  [HTMLConstructor] constructor();

  // also has obsolete members
};
```

WebIDL / HTMLHRElement

```
partial interface HTMLHRElement {
  [CEReactions, Reflect] attribute DOMString align;
  [CEReactions, Reflect] attribute DOMString color;
  [CEReactions, Reflect] attribute boolean noShade;
  [CEReactions, Reflect] attribute DOMString size;
  [CEReactions, Reflect] attribute DOMString width;
};
```

Normative owner definition

HTMLHtmlElement

interface / inherits HTMLElement

WebIDL / HTMLHtmlElement

```
[Exposed=Window]
interface HTMLHtmlElement : HTMLElement {
  [HTMLConstructor] constructor();

  // also has obsolete members
};
```

WebIDL / HTMLHtmlElement

```
partial interface HTMLHtmlElement {
  [CEReactions, Reflect] attribute DOMString version;
};
```

Normative owner definition

HTMLHyperlinkElementUtils

interface mixin

WebIDL / HTMLHyperlinkElementUtils

```
interface mixin HTMLHyperlinkElementUtils {
  [CEReactions, ReflectSetter] stringifier attribute USVString href;
  [CEReactions, Reflect] attribute DOMString target;
};
```

Normative owner definition

HTMLIFrameElement

interface / inherits HTMLElement

WebIDL / HTMLIFrameElement

```
[Exposed=Window]
interface HTMLIFrameElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, ReflectURL] attribute USVString src;
  [CEReactions] attribute (TrustedHTML or DOMString) srcdoc;
  [CEReactions, Reflect] attribute DOMString name;
```

Code continues on the next page

WebIDL / HTMLIFrameElement / continued

```
  [SameObject, PutForwards=value, Reflect] readonly attribute DOMTokenList sandbox;
  [CEReactions, Reflect] attribute DOMString allow;
  [CEReactions, Reflect] attribute boolean allowFullscreen;
  [CEReactions, Reflect] attribute DOMString width;
  [CEReactions, Reflect] attribute DOMString height;
  [CEReactions] attribute DOMString referrerPolicy;
  [CEReactions] attribute DOMString loading;
  readonly attribute Document? contentDocument;
  readonly attribute WindowProxy? contentWindow;
  Document? getSVGDocument();

  // also has obsolete members
};
```

WebIDL / HTMLIFrameElement

```
partial interface HTMLIFrameElement {
  [CEReactions, Reflect] attribute DOMString align;
  [CEReactions, Reflect] attribute DOMString scrolling;
  [CEReactions, Reflect] attribute DOMString frameBorder;
  [CEReactions, ReflectURL] attribute USVString longDesc;

  [CEReactions, Reflect] attribute [LegacyNullToEmptyString] DOMString marginHeight;
  [CEReactions, Reflect] attribute [LegacyNullToEmptyString] DOMString marginWidth;
};
```

Normative owner definition

HTMLImageElement

interface / inherits HTMLElement

WebIDL / HTMLImageElement

```
[Exposed=Window,
 LegacyFactoryFunction=Image(optional unsigned long width, optional unsigned long height)]
interface HTMLImageElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, Reflect] attribute DOMString alt;
  [CEReactions, ReflectURL] attribute USVString src;
  [CEReactions, Reflect] attribute USVString srcset;
  [CEReactions, Reflect] attribute DOMString sizes;
  [CEReactions] attribute DOMString? crossOrigin;
  [CEReactions, Reflect] attribute DOMString useMap;
  [CEReactions, Reflect] attribute boolean isMap;
  [CEReactions, Reflect] attribute boolean controls;
  [CEReactions, ReflectSetter] attribute unsigned long width;
```

Code continues on the next page

WebIDL / HTMLImageElement / continued

```
  [CEReactions, ReflectSetter] attribute unsigned long height;
  readonly attribute unsigned long naturalWidth;
  readonly attribute unsigned long naturalHeight;
  readonly attribute boolean complete;
  readonly attribute USVString currentSrc;
  [CEReactions] attribute DOMString referrerPolicy;
  [CEReactions] attribute DOMString decoding;
  [CEReactions] attribute DOMString loading;
  [CEReactions] attribute DOMString fetchPriority;

  Promise<undefined> decode();

  // also has obsolete members
};
```

WebIDL / HTMLImageElement

```
partial interface HTMLImageElement {
  [CEReactions, Reflect] attribute DOMString name;
  [CEReactions, ReflectURL] attribute USVString lowsrc;
  [CEReactions, Reflect] attribute DOMString align;
  [CEReactions, Reflect] attribute unsigned long hspace;
  [CEReactions, Reflect] attribute unsigned long vspace;
  [CEReactions, ReflectURL] attribute USVString longDesc;

  [CEReactions, Reflect] attribute [LegacyNullToEmptyString] DOMString border;
};
```

Normative owner definition

HTMLInputElement

interface / inherits HTMLElement / includes PopoverTargetAttributes

WebIDL / HTMLInputElement

```
[Exposed=Window]
interface HTMLInputElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, Reflect] attribute DOMString accept;
  [CEReactions, Reflect] attribute boolean alpha;
  [CEReactions, Reflect] attribute DOMString alt;
  [CEReactions, ReflectSetter] attribute DOMString autocomplete;
  [CEReactions, Reflect="checked"] attribute boolean defaultChecked;
  attribute boolean checked;
  [CEReactions] attribute DOMString colorSpace;
  [CEReactions, Reflect] attribute DOMString dirName;
```

Code continues on the next page

WebIDL / HTMLInputElement / continued

```
  [CEReactions, Reflect] attribute boolean disabled;
  readonly attribute HTMLFormElement? form;
  attribute FileList? files;
  [CEReactions, ReflectSetter] attribute USVString formAction;
  [CEReactions] attribute DOMString formEnctype;
  [CEReactions] attribute DOMString formMethod;
  [CEReactions, Reflect] attribute boolean formNoValidate;
  [CEReactions, Reflect] attribute DOMString formTarget;
  [CEReactions, ReflectSetter] attribute unsigned long height;
  attribute boolean indeterminate;
  readonly attribute HTMLDataListElement? list;
  [CEReactions, Reflect] attribute DOMString max;
  [CEReactions, ReflectNonNegative] attribute long maxLength;
  [CEReactions, Reflect] attribute DOMString min;
  [CEReactions, ReflectNonNegative] attribute long minLength;
  [CEReactions, Reflect] attribute boolean multiple;
  [CEReactions, Reflect] attribute DOMString name;
  [CEReactions, Reflect] attribute DOMString pattern;
  [CEReactions, Reflect] attribute DOMString placeholder;
  [CEReactions, Reflect] attribute boolean readOnly;
  [CEReactions, Reflect] attribute boolean required;
  [CEReactions, Reflect] attribute unsigned long size;
  [CEReactions, ReflectURL] attribute USVString src;
  [CEReactions, Reflect] attribute DOMString step;
  [CEReactions] attribute DOMString type;
  [CEReactions, Reflect="value"] attribute DOMString defaultValue;
  [CEReactions] attribute [LegacyNullToEmptyString] DOMString value;
  attribute object? valueAsDate;
  attribute unrestricted double valueAsNumber;
  [CEReactions, ReflectSetter] attribute unsigned long width;

  undefined stepUp(optional long n = 1);
  undefined stepDown(optional long n = 1);

  readonly attribute boolean willValidate;
  readonly attribute ValidityState validity;
  readonly attribute DOMString validationMessage;
  boolean checkValidity();
  boolean reportValidity();
  undefined setCustomValidity(DOMString error);

  readonly attribute NodeList? labels;

  undefined select();
  attribute unsigned long? selectionStart;
  attribute unsigned long? selectionEnd;
  attribute DOMString? selectionDirection;
  undefined setRangeText(DOMString replacement);
  undefined setRangeText(DOMString replacement, unsigned long start, unsigned long end, opti
onal SelectionMode selectionMode = "preserve");
  undefined setSelectionRange(unsigned long start, unsigned long end, optional DOMString dir
ection);

  undefined showPicker();
```

Code continues on the next page

WebIDL / HTMLInputElement / continued

```
  // also has obsolete members
};
```

WebIDL / HTMLInputElement

```
partial interface HTMLInputElement {
  [CEReactions, Reflect] attribute DOMString align;
  [CEReactions, Reflect] attribute DOMString useMap;
};
```

Normative owner definition

HTMLLabelElement

interface / inherits HTMLElement

WebIDL / HTMLLabelElement

```
[Exposed=Window]
interface HTMLLabelElement : HTMLElement {
  [HTMLConstructor] constructor();

  readonly attribute HTMLFormElement? form;
  [CEReactions, Reflect="for"] attribute DOMString htmlFor;
  readonly attribute HTMLElement? control;
};
```

Normative owner definition

HTMLLegendElement

interface / inherits HTMLElement

WebIDL / HTMLLegendElement

```
[Exposed=Window]
interface HTMLLegendElement : HTMLElement {
  [HTMLConstructor] constructor();

  readonly attribute HTMLFormElement? form;

  // also has obsolete members
};
```

WebIDL / HTMLLegendElement

```
partial interface HTMLLegendElement {
  [CEReactions, Reflect] attribute DOMString align;
};
```

Normative owner definition

HTMLLIElement

interface / inherits HTMLElement

WebIDL / HTMLLIElement

```
[Exposed=Window]
interface HTMLLIElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, Reflect] attribute long value;

  // also has obsolete members
};
```

WebIDL / HTMLLIElement

```
partial interface HTMLLIElement {
  [CEReactions, Reflect] attribute DOMString type;
};
```

Normative owner definition

HTMLLinkElement

interface / inherits HTMLElement / includes LinkStyle

WebIDL / HTMLLinkElement

```
[Exposed=Window]
interface HTMLLinkElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, ReflectURL] attribute USVString href;
  [CEReactions] attribute DOMString? crossOrigin;
  [CEReactions, Reflect] attribute DOMString rel;
  [CEReactions] attribute DOMString as;
  [SameObject, PutForwards=value, Reflect="rel"] readonly attribute DOMTokenList relList;
  [CEReactions, Reflect] attribute DOMString media;
```

Code continues on the next page

WebIDL / HTMLLinkElement / continued

```
  [CEReactions, Reflect] attribute DOMString integrity;
  [CEReactions, Reflect] attribute DOMString hreflang;
  [CEReactions, Reflect] attribute DOMString type;
  [SameObject, PutForwards=value, Reflect] readonly attribute DOMTokenList sizes;
  [CEReactions, Reflect] attribute USVString imageSrcset;
  [CEReactions, Reflect] attribute DOMString imageSizes;
  [CEReactions] attribute DOMString referrerPolicy;
  [SameObject, PutForwards=value, Reflect] readonly attribute DOMTokenList blocking;
  [CEReactions, Reflect] attribute boolean disabled;
  [CEReactions] attribute DOMString fetchPriority;

  // also has obsolete members
};
```

WebIDL / HTMLLinkElement

```
partial interface HTMLLinkElement {
  [CEReactions, Reflect] attribute DOMString charset;
  [CEReactions, Reflect] attribute DOMString rev;
  [CEReactions, Reflect] attribute DOMString target;
};
```

Normative owner definition

HTMLMapElement

interface / inherits HTMLElement

WebIDL / HTMLMapElement

```
[Exposed=Window]
interface HTMLMapElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, Reflect] attribute DOMString name;
  [SameObject] readonly attribute HTMLCollection areas;
};
```

Normative owner definition

HTMLMarqueeElement

interface / inherits HTMLElement

WebIDL / HTMLMarqueeElement

```
[Exposed=Window]
interface HTMLMarqueeElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, Reflect] attribute DOMString behavior;
  [CEReactions, Reflect] attribute DOMString bgColor;
  [CEReactions, Reflect] attribute DOMString direction;
  [CEReactions, Reflect] attribute DOMString height;
  [CEReactions, Reflect] attribute unsigned long hspace;
  [CEReactions] attribute long loop;
  [CEReactions, Reflect, ReflectDefault=6] attribute unsigned long scrollAmount;
  [CEReactions, Reflect, ReflectDefault=85] attribute unsigned long scrollDelay;
  [CEReactions, Reflect] attribute boolean trueSpeed;
  [CEReactions, Reflect] attribute unsigned long vspace;
  [CEReactions, Reflect] attribute DOMString width;

  undefined start();
  undefined stop();
};
```

Normative owner definition

HTMLMediaElement

interface / inherits HTMLElement

WebIDL / HTMLMediaElement

```
[Exposed=Window]
interface HTMLMediaElement : HTMLElement {

  // error state
  readonly attribute MediaError? error;

  // network state
  [CEReactions, ReflectURL] attribute USVString src;
  attribute MediaProvider? srcObject;
  readonly attribute USVString currentSrc;
  [CEReactions] attribute DOMString? crossOrigin;
  const unsigned short NETWORK_EMPTY = 0;
  const unsigned short NETWORK_IDLE = 1;
  const unsigned short NETWORK_LOADING = 2;
  const unsigned short NETWORK_NO_SOURCE = 3;
  readonly attribute unsigned short networkState;
  [CEReactions] attribute DOMString preload;
  readonly attribute TimeRanges buffered;
  undefined load();
  CanPlayTypeResult canPlayType(DOMString type);

  // ready state
  const unsigned short HAVE_NOTHING = 0;
```

Code continues on the next page

WebIDL / HTMLMediaElement / continued

```
  const unsigned short HAVE_METADATA = 1;
  const unsigned short HAVE_CURRENT_DATA = 2;
  const unsigned short HAVE_FUTURE_DATA = 3;
  const unsigned short HAVE_ENOUGH_DATA = 4;
  readonly attribute unsigned short readyState;
  readonly attribute boolean seeking;

  // playback state
  attribute double currentTime;
  undefined fastSeek(double time);
  readonly attribute unrestricted double duration;
  object getStartDate();
  readonly attribute boolean paused;
  attribute double defaultPlaybackRate;
  attribute double playbackRate;
  attribute boolean preservesPitch;
  readonly attribute TimeRanges played;
  readonly attribute TimeRanges seekable;
  readonly attribute boolean ended;
  [CEReactions, Reflect] attribute boolean autoplay;
  [CEReactions, Reflect] attribute boolean loop;
  Promise<undefined> play();
  undefined pause();

  // controls
  [CEReactions, Reflect] attribute boolean controls;
  attribute double volume;
  attribute boolean muted;
  [CEReactions, Reflect="muted"] attribute boolean defaultMuted;
  [CEReactions] attribute DOMString loading;

  // tracks
  [SameObject] readonly attribute AudioTrackList audioTracks;
  [SameObject] readonly attribute VideoTrackList videoTracks;
  [SameObject] readonly attribute TextTrackList textTracks;
  TextTrack addTextTrack(TextTrackKind kind, optional DOMString label = "", optional DOMStri
ng language = "");
};
```

Normative owner definition

HTMLMenuElement

interface / inherits HTMLElement

WebIDL / HTMLMenuElement

```
[Exposed=Window]
interface HTMLMenuElement : HTMLElement {
  [HTMLConstructor] constructor();
```

Code continues on the next page

WebIDL / HTMLMenuElement / continued

```
  // also has obsolete members
};
```

WebIDL / HTMLMenuElement

```
partial interface HTMLMenuElement {
  [CEReactions, Reflect] attribute boolean compact;
};
```

Normative owner definition

HTMLMetaElement

interface / inherits HTMLElement

WebIDL / HTMLMetaElement

```
[Exposed=Window]
interface HTMLMetaElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, Reflect] attribute DOMString name;
  [CEReactions, Reflect="http-equiv"] attribute DOMString httpEquiv;
  [CEReactions, Reflect] attribute DOMString content;
  [CEReactions, Reflect] attribute DOMString media;

  // also has obsolete members
};
```

WebIDL / HTMLMetaElement

```
partial interface HTMLMetaElement {
  [CEReactions, Reflect] attribute DOMString scheme;
};
```

Normative owner definition

HTMLMeterElement

interface / inherits HTMLElement

WebIDL / HTMLMeterElement

```
[Exposed=Window]
interface HTMLMeterElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, ReflectSetter] attribute double value;
  [CEReactions, ReflectSetter] attribute double min;
  [CEReactions, ReflectSetter] attribute double max;
  [CEReactions, ReflectSetter] attribute double low;
  [CEReactions, ReflectSetter] attribute double high;
  [CEReactions, ReflectSetter] attribute double optimum;
  readonly attribute NodeList labels;
};
```

Normative owner definition

HTMLModElement

interface / inherits HTMLElement

WebIDL / HTMLModElement

```
[Exposed=Window]
interface HTMLModElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, ReflectURL] attribute USVString cite;
  [CEReactions, Reflect] attribute DOMString dateTime;
};
```

Normative owner definition

HTMLObjectElement

interface / inherits HTMLElement

WebIDL / HTMLObjectElement

```
[Exposed=Window]
interface HTMLObjectElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, ReflectURL] attribute USVString data;
  [CEReactions, Reflect] attribute DOMString type;
  [CEReactions, Reflect] attribute DOMString name;
  readonly attribute HTMLFormElement? form;
  [CEReactions, Reflect] attribute DOMString width;
  [CEReactions, Reflect] attribute DOMString height;
  readonly attribute Document? contentDocument;
```

Code continues on the next page

WebIDL / HTMLObjectElement / continued

```
  readonly attribute WindowProxy? contentWindow;
  Document? getSVGDocument();

  readonly attribute boolean willValidate;
  readonly attribute ValidityState validity;
  readonly attribute DOMString validationMessage;
  boolean checkValidity();
  boolean reportValidity();
  undefined setCustomValidity(DOMString error);

  // also has obsolete members
};
```

WebIDL / HTMLObjectElement

```
partial interface HTMLObjectElement {
  [CEReactions, Reflect] attribute DOMString align;
  [CEReactions, Reflect] attribute DOMString archive;
  [CEReactions, Reflect] attribute DOMString code;
  [CEReactions, Reflect] attribute boolean declare;
  [CEReactions, Reflect] attribute unsigned long hspace;
  [CEReactions, Reflect] attribute DOMString standby;
  [CEReactions, Reflect] attribute unsigned long vspace;
  [CEReactions, ReflectURL] attribute DOMString codeBase;
  [CEReactions, Reflect] attribute DOMString codeType;
  [CEReactions, Reflect] attribute DOMString useMap;

  [CEReactions, Reflect] attribute [LegacyNullToEmptyString] DOMString border;
};
```

Normative owner definition

HTMLOListElement

interface / inherits HTMLElement

WebIDL / HTMLOListElement

```
[Exposed=Window]
interface HTMLOListElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, Reflect] attribute boolean reversed;
  [CEReactions, Reflect, ReflectDefault=1] attribute long start;
  [CEReactions, Reflect] attribute DOMString type;

  // also has obsolete members
};
```

WebIDL / HTMLOListElement

```
partial interface HTMLOListElement {
  [CEReactions, Reflect] attribute boolean compact;
};
```

Normative owner definition

HTMLOptGroupElement

interface / inherits HTMLElement

WebIDL / HTMLOptGroupElement

```
[Exposed=Window]
interface HTMLOptGroupElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, Reflect] attribute boolean disabled;
  [CEReactions, Reflect] attribute DOMString label;
};
```

Normative owner definition

HTMLOptionElement

interface / inherits HTMLElement

WebIDL / HTMLOptionElement

```
[Exposed=Window,
 LegacyFactoryFunction=Option(optional DOMString text = "", optional DOMString value, option
al boolean defaultSelected = false, optional boolean selected = false)]
interface HTMLOptionElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, Reflect] attribute boolean disabled;
  readonly attribute HTMLFormElement? form;
  [CEReactions, ReflectSetter] attribute DOMString label;
  [CEReactions, Reflect="selected"] attribute boolean defaultSelected;
  [CEReactions] attribute boolean selected;
  [CEReactions, ReflectSetter] attribute DOMString value;

  [CEReactions] attribute DOMString text;
  readonly attribute long index;
};
```

Normative owner definition

HTMLOptionsCollection

interface / inherits HTMLCollection

WebIDL / HTMLOptionsCollection

```
[Exposed=Window]
interface HTMLOptionsCollection : HTMLCollection {
  // inherits item(), namedItem()
  [CEReactions] attribute unsigned long length; // shadows inherited length
  [CEReactions] setter undefined (unsigned long index, HTMLOptionElement? option);
  [CEReactions] undefined add((HTMLOptionElement or HTMLOptGroupElement) element, optional (
HTMLElement or long)? before = null);
  [CEReactions] undefined remove(long index);
  [CEReactions] attribute long selectedIndex;
};
```

Normative owner definition

HTMLOrSVGImageElement

typedef

WebIDL / HTMLOrSVGImageElement

```
typedef (HTMLImageElement or
         SVGImageElement) HTMLOrSVGImageElement;
```

Normative owner definition

HTMLOrSVGOrMathMLElement

interface mixin

WebIDL / HTMLOrSVGOrMathMLElement

```
interface mixin HTMLOrSVGOrMathMLElement {
  [SameObject] readonly attribute DOMStringMap dataset;
  attribute DOMString nonce; // intentionally no [CEReactions]

  [CEReactions, Reflect] attribute boolean autofocus;
  [CEReactions, ReflectSetter] attribute long tabIndex;
  undefined focus(optional FocusOptions options = {});
  undefined blur();
};
```

Normative owner definition

HTMLOrSVGScriptElement

typedef

WebIDL / HTMLOrSVGScriptElement

```
typedef (HTMLScriptElement or SVGScriptElement) HTMLOrSVGScriptElement;
```

Normative owner definition

HTMLOutputElement

interface / inherits HTMLElement

WebIDL / HTMLOutputElement

```
[Exposed=Window]
interface HTMLOutputElement : HTMLElement {
  [HTMLConstructor] constructor();

  [SameObject, PutForwards=value, Reflect="for"] readonly attribute DOMTokenList htmlFor;
  readonly attribute HTMLFormElement? form;
  [CEReactions, Reflect] attribute DOMString name;

  readonly attribute DOMString type;
  [CEReactions] attribute DOMString defaultValue;
  [CEReactions] attribute DOMString value;

  readonly attribute boolean willValidate;
  readonly attribute ValidityState validity;
  readonly attribute DOMString validationMessage;
  boolean checkValidity();
  boolean reportValidity();
  undefined setCustomValidity(DOMString error);

  readonly attribute NodeList labels;
};
```

Normative owner definition

HTMLParagraphElement

interface / inherits HTMLElement

WebIDL / HTMLParagraphElement

```
[Exposed=Window]
interface HTMLParagraphElement : HTMLElement {
  [HTMLConstructor] constructor();

  // also has obsolete members
};
```

WebIDL / HTMLParagraphElement

```
partial interface HTMLParagraphElement {
  [CEReactions, Reflect] attribute DOMString align;
};
```

Normative owner definition

HTMLParamElement

interface / inherits HTMLElement

WebIDL / HTMLParamElement

```
[Exposed=Window]
interface HTMLParamElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, Reflect] attribute DOMString name;
  [CEReactions, Reflect] attribute DOMString value;
  [CEReactions, Reflect] attribute DOMString type;
  [CEReactions, Reflect] attribute DOMString valueType;
};
```

Normative owner definition

HTMLPictureElement

interface / inherits HTMLElement

WebIDL / HTMLPictureElement

```
[Exposed=Window]
interface HTMLPictureElement : HTMLElement {
  [HTMLConstructor] constructor();
};
```

Normative owner definition

HTMLPreElement

interface / inherits HTMLElement

WebIDL / HTMLPreElement

```
[Exposed=Window]
interface HTMLPreElement : HTMLElement {
  [HTMLConstructor] constructor();

  // also has obsolete members
};
```

WebIDL / HTMLPreElement

```
partial interface HTMLPreElement {
  [CEReactions, Reflect] attribute long width;
};
```

Normative owner definition

HTMLProgressElement

interface / inherits HTMLElement

WebIDL / HTMLProgressElement

```
[Exposed=Window]
interface HTMLProgressElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, ReflectSetter] attribute double value;
  [CEReactions, ReflectPositive, ReflectDefault=1.0] attribute double max;
  readonly attribute double position;
  readonly attribute NodeList labels;
};
```

Normative owner definition

HTMLQuoteElement

interface / inherits HTMLElement

WebIDL / HTMLQuoteElement

```
[Exposed=Window]
interface HTMLQuoteElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, ReflectURL] attribute USVString cite;
};
```

Normative owner definition

HTMLScriptElement

interface / inherits HTMLElement

WebIDL / HTMLScriptElement

```
[Exposed=Window]
interface HTMLScriptElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, Reflect] attribute DOMString type;
  [CEReactions] attribute (TrustedScriptURL or USVString) src;
  [CEReactions, Reflect] attribute boolean noModule;
  [CEReactions] attribute boolean async;
  [CEReactions, Reflect] attribute boolean defer;
  [SameObject, PutForwards=value, Reflect] readonly attribute DOMTokenList blocking;
  [CEReactions] attribute DOMString? crossOrigin;
  [CEReactions] attribute DOMString referrerPolicy;
  [CEReactions, Reflect] attribute DOMString integrity;
  [CEReactions] attribute DOMString fetchPriority;

  // innerText and textContent shadow HTMLElement's and Node's, to add Trusted Types checks
  [CEReactions] attribute (TrustedScript or [LegacyNullToEmptyString] DOMString) innerText;
  [CEReactions] attribute (TrustedScript or DOMString)? textContent;
  [CEReactions] attribute (TrustedScript or DOMString) text;

  static boolean supports(DOMString type);

  // also has obsolete members
};
```

WebIDL / HTMLScriptElement

```
partial interface HTMLScriptElement {
  [CEReactions, Reflect] attribute DOMString charset;
  [CEReactions, Reflect] attribute DOMString event;
  [CEReactions, Reflect="for"] attribute DOMString htmlFor;
};
```

Normative owner definition

HTMLSelectedContentElement

interface / inherits HTMLElement

WebIDL / HTMLSelectedContentElement

```
[Exposed=Window]
interface HTMLSelectedContentElement : HTMLElement {
  [HTMLConstructor] constructor();
};
```

Normative owner definition

HTMLSelectElement

interface / inherits HTMLElement

WebIDL / HTMLSelectElement

```
[Exposed=Window]
interface HTMLSelectElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, ReflectSetter] attribute DOMString autocomplete;
  [CEReactions, Reflect] attribute boolean disabled;
  readonly attribute HTMLFormElement? form;
  [CEReactions, Reflect] attribute boolean multiple;
  [CEReactions, Reflect] attribute DOMString name;
  [CEReactions, Reflect] attribute boolean required;
  [CEReactions, Reflect, ReflectDefault=0] attribute unsigned long size;

  readonly attribute DOMString type;

  [SameObject] readonly attribute HTMLOptionsCollection options;
  [CEReactions] attribute unsigned long length;
  getter HTMLOptionElement? item(unsigned long index);
  HTMLOptionElement? namedItem(DOMString name);
  [CEReactions] undefined add((HTMLOptionElement or HTMLOptGroupElement) element, optional (
HTMLElement or long)? before = null);
  [CEReactions] undefined remove(); // ChildNode overload
  [CEReactions] undefined remove(long index);
  [CEReactions] setter undefined (unsigned long index, HTMLOptionElement? option);

  [SameObject] readonly attribute HTMLCollection selectedOptions;
  [CEReactions] attribute long selectedIndex;
  [CEReactions] attribute DOMString value;

  readonly attribute boolean willValidate;
  readonly attribute ValidityState validity;
  readonly attribute DOMString validationMessage;
  boolean checkValidity();
  boolean reportValidity();
  undefined setCustomValidity(DOMString error);

  undefined showPicker();

  readonly attribute NodeList labels;
};
```

Normative owner definition

HTMLSlotElement

interface / inherits HTMLElement

WebIDL / HTMLSlotElement

```
[Exposed=Window]
interface HTMLSlotElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, Reflect] attribute DOMString name;
  sequence<Node> assignedNodes(optional AssignedNodesOptions options = {});
  sequence<Element> assignedElements(optional AssignedNodesOptions options = {});
  undefined assign((Element or Text)... nodes);
};
```

Normative owner definition

HTMLSourceElement

interface / inherits HTMLElement

WebIDL / HTMLSourceElement

```
[Exposed=Window]
interface HTMLSourceElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, ReflectURL] attribute USVString src;
  [CEReactions, Reflect] attribute DOMString type;
  [CEReactions, Reflect] attribute USVString srcset;
  [CEReactions, Reflect] attribute DOMString sizes;
  [CEReactions, Reflect] attribute DOMString media;
  [CEReactions, Reflect] attribute unsigned long width;
  [CEReactions, Reflect] attribute unsigned long height;
};
```

Normative owner definition

HTMLSpanElement

interface / inherits HTMLElement

WebIDL / HTMLSpanElement

```
[Exposed=Window]
interface HTMLSpanElement : HTMLElement {
  [HTMLConstructor] constructor();
};
```

Normative owner definition

HTMLStyleElement

interface / inherits HTMLElement / includes LinkStyle

WebIDL / HTMLStyleElement

```
[Exposed=Window]
interface HTMLStyleElement : HTMLElement {
  [HTMLConstructor] constructor();

  attribute boolean disabled;
  [CEReactions, Reflect] attribute DOMString media;
  [SameObject, PutForwards=value, Reflect] readonly attribute DOMTokenList blocking;

  // also has obsolete members
};
```

WebIDL / HTMLStyleElement

```
partial interface HTMLStyleElement {
  [CEReactions, Reflect] attribute DOMString type;
};
```

Normative owner definition

HTMLTableCaptionElement

interface / inherits HTMLElement

WebIDL / HTMLTableCaptionElement

```
[Exposed=Window]
interface HTMLTableCaptionElement : HTMLElement {
  [HTMLConstructor] constructor();

  // also has obsolete members
};
```

WebIDL / HTMLTableCaptionElement

```
partial interface HTMLTableCaptionElement {
  [CEReactions, Reflect] attribute DOMString align;
};
```

Normative owner definition

HTMLTableCellElement

interface / inherits HTMLElement

WebIDL / HTMLTableCellElement

```
[Exposed=Window]
interface HTMLTableCellElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, Reflect, ReflectDefault=1, ReflectRange=(1, 1000)] attribute unsigned long c
olSpan;
  [CEReactions, Reflect, ReflectDefault=1, ReflectRange=(0, 65534)] attribute unsigned long
rowSpan;
  [CEReactions, Reflect] attribute DOMString headers;
  readonly attribute long cellIndex;

  [CEReactions] attribute DOMString scope; // only conforming for th elements
  [CEReactions, Reflect] attribute DOMString abbr;  // only conforming for th elements

  // also has obsolete members
};
```

WebIDL / HTMLTableCellElement

```
partial interface HTMLTableCellElement {
  [CEReactions, Reflect] attribute DOMString align;
  [CEReactions, Reflect] attribute DOMString axis;
  [CEReactions, Reflect] attribute DOMString height;
  [CEReactions, Reflect] attribute DOMString width;

  [CEReactions, Reflect="char"] attribute DOMString ch;
  [CEReactions, Reflect="charoff"] attribute DOMString chOff;
  [CEReactions, Reflect] attribute boolean noWrap;
  [CEReactions, Reflect] attribute DOMString vAlign;

  [CEReactions, Reflect] attribute [LegacyNullToEmptyString] DOMString bgColor;
};
```

Normative owner definition

HTMLTableColElement

interface / inherits HTMLElement

WebIDL / HTMLTableColElement

```
[Exposed=Window]
interface HTMLTableColElement : HTMLElement {
  [HTMLConstructor] constructor();
```

Code continues on the next page

WebIDL / HTMLTableColElement / continued

```
  [CEReactions, Reflect, ReflectDefault=1, ReflectRange=(1, 1000)] attribute unsigned long s
pan;

  // also has obsolete members
};
```

WebIDL / HTMLTableColElement

```
partial interface HTMLTableColElement {
  [CEReactions, Reflect] attribute DOMString align;
  [CEReactions, Reflect="char"] attribute DOMString ch;
  [CEReactions, Reflect="charoff"] attribute DOMString chOff;
  [CEReactions, Reflect] attribute DOMString vAlign;
  [CEReactions, Reflect] attribute DOMString width;
};
```

Normative owner definition

HTMLTableElement

interface / inherits HTMLElement

WebIDL / HTMLTableElement

```
[Exposed=Window]
interface HTMLTableElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions] attribute HTMLTableCaptionElement? caption;
  HTMLTableCaptionElement createCaption();
  [CEReactions] undefined deleteCaption();

  [CEReactions] attribute HTMLTableSectionElement? tHead;
  HTMLTableSectionElement createTHead();
  [CEReactions] undefined deleteTHead();

  [CEReactions] attribute HTMLTableSectionElement? tFoot;
  HTMLTableSectionElement createTFoot();
  [CEReactions] undefined deleteTFoot();

  [SameObject] readonly attribute HTMLCollection tBodies;
  HTMLTableSectionElement createTBody();

  [SameObject] readonly attribute HTMLCollection rows;
  HTMLTableRowElement insertRow(optional long index = -1);
  [CEReactions] undefined deleteRow(long index);
```

Code continues on the next page

WebIDL / HTMLTableElement / continued

```
  // also has obsolete members
};
```

WebIDL / HTMLTableElement

```
partial interface HTMLTableElement {
  [CEReactions, Reflect] attribute DOMString align;
  [CEReactions, Reflect] attribute DOMString border;
  [CEReactions, Reflect] attribute DOMString frame;
  [CEReactions, Reflect] attribute DOMString rules;
  [CEReactions, Reflect] attribute DOMString summary;
  [CEReactions, Reflect] attribute DOMString width;

  [CEReactions, Reflect] attribute [LegacyNullToEmptyString] DOMString bgColor;
  [CEReactions, Reflect] attribute [LegacyNullToEmptyString] DOMString cellPadding;
  [CEReactions, Reflect] attribute [LegacyNullToEmptyString] DOMString cellSpacing;
};
```

Normative owner definition

HTMLTableRowElement

interface / inherits HTMLElement

WebIDL / HTMLTableRowElement

```
[Exposed=Window]
interface HTMLTableRowElement : HTMLElement {
  [HTMLConstructor] constructor();

  readonly attribute long rowIndex;
  readonly attribute long sectionRowIndex;
  [SameObject] readonly attribute HTMLCollection cells;
  HTMLTableCellElement insertCell(optional long index = -1);
  [CEReactions] undefined deleteCell(long index);

  // also has obsolete members
};
```

WebIDL / HTMLTableRowElement

```
partial interface HTMLTableRowElement {
  [CEReactions, Reflect] attribute DOMString align;
  [CEReactions, Reflect="char"] attribute DOMString ch;
  [CEReactions, Reflect="charoff"] attribute DOMString chOff;
  [CEReactions, Reflect] attribute DOMString vAlign;
```

Code continues on the next page

WebIDL / HTMLTableRowElement / continued

```
  [CEReactions, Reflect] attribute [LegacyNullToEmptyString] DOMString bgColor;
};
```

Normative owner definition

HTMLTableSectionElement

interface / inherits HTMLElement

WebIDL / HTMLTableSectionElement

```
[Exposed=Window]
interface HTMLTableSectionElement : HTMLElement {
  [HTMLConstructor] constructor();

  [SameObject] readonly attribute HTMLCollection rows;
  HTMLTableRowElement insertRow(optional long index = -1);
  [CEReactions] undefined deleteRow(long index);

  // also has obsolete members
};
```

WebIDL / HTMLTableSectionElement

```
partial interface HTMLTableSectionElement {
  [CEReactions, Reflect] attribute DOMString align;
  [CEReactions, Reflect="char"] attribute DOMString ch;
  [CEReactions, Reflect="charoff"] attribute DOMString chOff;
  [CEReactions, Reflect] attribute DOMString vAlign;
};
```

Normative owner definition

HTMLTemplateElement

interface / inherits HTMLElement

WebIDL / HTMLTemplateElement

```
[Exposed=Window]
interface HTMLTemplateElement : HTMLElement {
  [HTMLConstructor] constructor();

  readonly attribute DocumentFragment content;
```

Code continues on the next page

WebIDL / HTMLTemplateElement / continued

```
  [CEReactions, Reflect="for"] attribute DOMString htmlFor;
  [CEReactions] attribute DOMString shadowRootMode;
  [CEReactions, Reflect] attribute boolean shadowRootDelegatesFocus;
  [CEReactions, Reflect] attribute boolean shadowRootSerializable;
  [CEReactions] attribute DOMString shadowRootSlotAssignment;
  [CEReactions, Reflect] attribute boolean shadowRootClonable;
  [CEReactions, Reflect] attribute DOMString shadowRootCustomElementRegistry;
};
```

Normative owner definition

HTMLTextAreaElement

interface / inherits HTMLElement

WebIDL / HTMLTextAreaElement

```
[Exposed=Window]
interface HTMLTextAreaElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, ReflectSetter] attribute DOMString autocomplete;
  [CEReactions, ReflectPositiveWithFallback, ReflectDefault=20] attribute unsigned long cols
;
  [CEReactions, Reflect] attribute DOMString dirName;
  [CEReactions, Reflect] attribute boolean disabled;
  readonly attribute HTMLFormElement? form;
  [CEReactions, ReflectNonNegative] attribute long maxLength;
  [CEReactions, ReflectNonNegative] attribute long minLength;
  [CEReactions, Reflect] attribute DOMString name;
  [CEReactions, Reflect] attribute DOMString placeholder;
  [CEReactions, Reflect] attribute boolean readOnly;
  [CEReactions, Reflect] attribute boolean required;
  [CEReactions, ReflectPositiveWithFallback, ReflectDefault=2] attribute unsigned long rows;
  [CEReactions, Reflect] attribute DOMString wrap;

  readonly attribute DOMString type;
  [CEReactions] attribute DOMString defaultValue;
  attribute [LegacyNullToEmptyString] DOMString value;
  readonly attribute unsigned long textLength;

  readonly attribute boolean willValidate;
  readonly attribute ValidityState validity;
  readonly attribute DOMString validationMessage;
  boolean checkValidity();
  boolean reportValidity();
  undefined setCustomValidity(DOMString error);

  readonly attribute NodeList labels;

  undefined select();
```

Code continues on the next page

WebIDL / HTMLTextAreaElement / continued

```
  attribute unsigned long selectionStart;
  attribute unsigned long selectionEnd;
  attribute DOMString selectionDirection;
  undefined setRangeText(DOMString replacement);
  undefined setRangeText(DOMString replacement, unsigned long start, unsigned long end, opti
onal SelectionMode selectionMode = "preserve");
  undefined setSelectionRange(unsigned long start, unsigned long end, optional DOMString dir
ection);
};
```

Normative owner definition

HTMLTimeElement

interface / inherits HTMLElement

WebIDL / HTMLTimeElement

```
[Exposed=Window]
interface HTMLTimeElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions, Reflect] attribute DOMString dateTime;
};
```

Normative owner definition

HTMLTitleElement

interface / inherits HTMLElement

WebIDL / HTMLTitleElement

```
[Exposed=Window]
interface HTMLTitleElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions] attribute DOMString text;
};
```

Normative owner definition

HTMLTrackElement

interface / inherits HTMLElement

WebIDL / HTMLTrackElement

```
[Exposed=Window]
interface HTMLTrackElement : HTMLElement {
  [HTMLConstructor] constructor();

  [CEReactions] attribute DOMString kind;
  [CEReactions, ReflectURL] attribute USVString src;
  [CEReactions, Reflect] attribute DOMString srclang;
  [CEReactions, Reflect] attribute DOMString label;
  [CEReactions, Reflect] attribute boolean default;

  const unsigned short NONE = 0;
  const unsigned short LOADING = 1;
  const unsigned short LOADED = 2;
  const unsigned short ERROR = 3;
  readonly attribute unsigned short readyState;

  readonly attribute TextTrack track;
};
```

Normative owner definition

HTMLUListElement

interface / inherits HTMLElement

WebIDL / HTMLUListElement

```
[Exposed=Window]
interface HTMLUListElement : HTMLElement {
  [HTMLConstructor] constructor();

  // also has obsolete members
};
```

WebIDL / HTMLUListElement

```
partial interface HTMLUListElement {
  [CEReactions, Reflect] attribute boolean compact;
  [CEReactions, Reflect] attribute DOMString type;
};
```

Normative owner definition

HTMLUnknownElement

interface / inherits HTMLElement

WebIDL / HTMLUnknownElement

```
[Exposed=Window]
interface HTMLUnknownElement : HTMLElement {
  // Note: intentionally no [HTMLConstructor]
};
```

Normative owner definition

HTMLVideoElement

interface / inherits HTMLMediaElement

WebIDL / HTMLVideoElement

```
[Exposed=Window]
interface HTMLVideoElement : HTMLMediaElement {
  [HTMLConstructor] constructor();

  [CEReactions, Reflect] attribute unsigned long width;
  [CEReactions, Reflect] attribute unsigned long height;
  readonly attribute unsigned long videoWidth;
  readonly attribute unsigned long videoHeight;
  [CEReactions, ReflectURL] attribute USVString poster;
  [CEReactions, Reflect] attribute boolean playsInline;

};
```

Normative owner definition

HyperlinkElementUtils

interface mixin

WebIDL / HyperlinkElementUtils

```
interface mixin HyperlinkElementUtils {
  readonly attribute USVString origin;
  [CEReactions] attribute USVString protocol;
  [CEReactions] attribute USVString username;
  [CEReactions] attribute USVString password;
  [CEReactions] attribute USVString host;
  [CEReactions] attribute USVString hostname;
  [CEReactions] attribute USVString port;
  [CEReactions] attribute USVString pathname;
  [CEReactions] attribute USVString search;
  [CEReactions] attribute USVString hash;

  [CEReactions, Reflect] attribute DOMString hreflang;
  [CEReactions, Reflect] attribute DOMString type;
};
```

Normative owner definition

ImageBitmap

interface

WebIDL / ImageBitmap

```
[Exposed=(Window,Worker), Serializable, Transferable]
interface ImageBitmap {
  readonly attribute unsigned long width;
  readonly attribute unsigned long height;
  undefined close();
};
```

Normative owner definition

ImageBitmapOptions

dictionary

WebIDL / ImageBitmapOptions

```
dictionary ImageBitmapOptions {
  ImageOrientation imageOrientation = "from-image";
  PremultiplyAlpha premultiplyAlpha = "default";
  ColorSpaceConversion colorSpaceConversion = "default";
  [EnforceRange] unsigned long resizeWidth;
  [EnforceRange] unsigned long resizeHeight;
  ResizeQuality resizeQuality = "low";
};
```

Normative owner definition

ImageBitmapRenderingContext

interface

WebIDL / ImageBitmapRenderingContext

```
[Exposed=(Window,Worker)]
interface ImageBitmapRenderingContext {
  readonly attribute (HTMLCanvasElement or OffscreenCanvas) canvas;
  undefined transferFromImageBitmap(ImageBitmap? bitmap);
};
```

Normative owner definition

ImageBitmapRenderingContextSettings

dictionary

WebIDL / ImageBitmapRenderingContextSettings

```
dictionary ImageBitmapRenderingContextSettings {
  boolean alpha = true;
};
```

Normative owner definition

ImageBitmapSource

typedef

WebIDL / ImageBitmapSource

```
typedef (CanvasImageSource or
         Blob or
         ImageData) ImageBitmapSource;
```

Normative owner definition

ImageData

interface

WebIDL / ImageData

```
[Exposed=(Window,Worker),
 Serializable]
interface ImageData {
  constructor(unsigned long sw, unsigned long sh, optional ImageDataSettings settings = {});
  constructor(ImageDataArray data, unsigned long sw, optional unsigned long sh, optional Ima
geDataSettings settings = {});

  readonly attribute unsigned long width;
  readonly attribute unsigned long height;
  readonly attribute ImageDataArray data;
  readonly attribute ImageDataPixelFormat pixelFormat;
  readonly attribute PredefinedColorSpace colorSpace;
};
```

Normative owner definition

ImageDataArray

typedef

WebIDL / ImageDataArray

```
typedef (Uint8ClampedArray or Float16Array) ImageDataArray;
```

Normative owner definition

ImageDataPixelFormat

enum

WebIDL / ImageDataPixelFormat

```
enum ImageDataPixelFormat { "rgba-unorm8", "rgba-float16" };
```

Normative owner definition

ImageDataSettings

dictionary

WebIDL / ImageDataSettings

```
dictionary ImageDataSettings {
  PredefinedColorSpace colorSpace;
  ImageDataPixelFormat pixelFormat = "rgba-unorm8";
};
```

Normative owner definition

ImageEncodeOptions

dictionary

WebIDL / ImageEncodeOptions

```
dictionary ImageEncodeOptions {
  DOMString type = "image/png";
  unrestricted double quality;
};
```

Normative owner definition

ImageOrientation

enum

WebIDL / ImageOrientation

```
enum ImageOrientation { "from-image", "flipY" };
```

Normative owner definition

ImageSmoothingQuality

enum

WebIDL / ImageSmoothingQuality

```
enum ImageSmoothingQuality { "low", "medium", "high" };
```

Normative owner definition

Location

interface

WebIDL / Location

```
[Exposed=Window]
interface Location { // but see also additional creation steps and overridden internal metho
ds
  [LegacyUnforgeable] stringifier attribute USVString href;
  [LegacyUnforgeable] readonly attribute USVString origin;
  [LegacyUnforgeable] attribute USVString protocol;
  [LegacyUnforgeable] attribute USVString host;
  [LegacyUnforgeable] attribute USVString hostname;
  [LegacyUnforgeable] attribute USVString port;
  [LegacyUnforgeable] attribute USVString pathname;
  [LegacyUnforgeable] attribute USVString search;
  [LegacyUnforgeable] attribute USVString hash;

  [LegacyUnforgeable] undefined assign(USVString url);
  [LegacyUnforgeable] undefined replace(USVString url);
  [LegacyUnforgeable] undefined reload();

  [LegacyUnforgeable] readonly attribute DOMStringList ancestorOrigins;
};
```

Normative owner definition

MediaError

interface

WebIDL / MediaError

```
[Exposed=Window]
interface MediaError {
  const unsigned short MEDIA_ERR_ABORTED = 1;
  const unsigned short MEDIA_ERR_NETWORK = 2;
  const unsigned short MEDIA_ERR_DECODE = 3;
  const unsigned short MEDIA_ERR_SRC_NOT_SUPPORTED = 4;

  readonly attribute unsigned short code;
  readonly attribute DOMString message;
};
```

Normative owner definition

MediaProvider

typedef

WebIDL / MediaProvider

```
typedef (MediaStream or MediaSource or Blob) MediaProvider;
```

Normative owner definition

MessageChannel

interface

WebIDL / MessageChannel

```
[Exposed=(Window,Worker)]
interface MessageChannel {
  constructor();

  readonly attribute MessagePort port1;
  readonly attribute MessagePort port2;
};
```

Normative owner definition

MessageEvent

interface / inherits Event

WebIDL / MessageEvent

```
[Exposed=(Window,Worker,AudioWorklet)]
interface MessageEvent : Event {
  constructor(DOMString type, optional MessageEventInit eventInitDict = {});

  readonly attribute any data;
  readonly attribute USVString origin;
  readonly attribute DOMString lastEventId;
  readonly attribute MessageEventSource? source;
  readonly attribute FrozenArray<MessagePort> ports;

  undefined initMessageEvent(DOMString type, optional boolean bubbles = false, optional bool
ean cancelable = false, optional any data = null, optional USVString origin = "", optional D
OMString lastEventId = "", optional MessageEventSource? source = null, optional sequence<Mes
sagePort> ports = []);
};
```

Normative owner definition

MessageEventInit

dictionary / inherits EventInit

WebIDL / MessageEventInit

```
dictionary MessageEventInit : EventInit {
  any data = null;
  USVString origin = "";
  DOMString lastEventId = "";
  MessageEventSource? source = null;
  sequence<MessagePort> ports = [];
};
```

Normative owner definition

MessageEventSource

typedef

WebIDL / MessageEventSource

```
typedef (WindowProxy or MessagePort or ServiceWorker) MessageEventSource;
```

Normative owner definition

MessageEventTarget

interface mixin

WebIDL / MessageEventTarget

```
interface mixin MessageEventTarget {
  attribute EventHandler onmessage;
  attribute EventHandler onmessageerror;
};
```

Normative owner definition

MessagePort

interface / inherits EventTarget / includes MessageEventTarget

WebIDL / MessagePort

```
[Exposed=(Window,Worker,AudioWorklet), Transferable]
interface MessagePort : EventTarget {
  undefined postMessage(any message, sequence<object> transfer);
  undefined postMessage(any message, optional StructuredSerializeOptions options = {});
  undefined start();
  undefined close();
};
```

Normative owner definition

MimeType

interface

WebIDL / MimeType

```
[Exposed=Window]
interface MimeType {
  readonly attribute DOMString type;
  readonly attribute DOMString description;
  readonly attribute DOMString suffixes;
  readonly attribute Plugin enabledPlugin;
};
```

Normative owner definition

MimeTypeArray

interface

WebIDL / MimeTypeArray

```
[Exposed=Window,
 LegacyUnenumerableNamedProperties]
interface MimeTypeArray {
  readonly attribute unsigned long length;
  getter MimeType? item(unsigned long index);
  getter MimeType? namedItem(DOMString name);
};
```

Normative owner definition

NavigateEvent

interface / inherits Event

WebIDL / NavigateEvent

```
[Exposed=Window]
interface NavigateEvent : Event {
  constructor(DOMString type, NavigateEventInit eventInitDict);

  readonly attribute NavigationType navigationType;
  readonly attribute NavigationDestination destination;
  readonly attribute boolean canIntercept;
  readonly attribute boolean userInitiated;
  readonly attribute boolean hashChange;
  readonly attribute AbortSignal signal;
  readonly attribute FormData? formData;
  readonly attribute DOMString? downloadRequest;
  readonly attribute any info;
  readonly attribute boolean hasUAVisualTransition;
  readonly attribute Element? sourceElement;

  undefined intercept(optional NavigationInterceptOptions options = {});
  undefined scroll();
};
```

Normative owner definition

NavigateEventInit

dictionary / inherits EventInit

WebIDL / NavigateEventInit

```
dictionary NavigateEventInit : EventInit {
  NavigationType navigationType = "push";
  required NavigationDestination destination;
  boolean canIntercept = false;
```

Code continues on the next page

WebIDL / NavigateEventInit / continued

```
  boolean userInitiated = false;
  boolean hashChange = false;
  required AbortSignal signal;
  FormData? formData = null;
  DOMString? downloadRequest = null;
  any info;
  boolean hasUAVisualTransition = false;
  Element? sourceElement = null;
};
```

Normative owner definition

Navigation

interface / inherits EventTarget

WebIDL / Navigation

```
[Exposed=Window]
interface Navigation : EventTarget {
  sequence<NavigationHistoryEntry> entries();
  readonly attribute NavigationHistoryEntry? currentEntry;
  undefined updateCurrentEntry(NavigationUpdateCurrentEntryOptions options);
  readonly attribute NavigationTransition? transition;
  readonly attribute NavigationActivation? activation;

  readonly attribute boolean canGoBack;
  readonly attribute boolean canGoForward;

  NavigationResult navigate(USVString url, optional NavigationNavigateOptions options = {});
  NavigationResult reload(optional NavigationReloadOptions options = {});

  NavigationResult traverseTo(DOMString key, optional NavigationOptions options = {});
  NavigationResult back(optional NavigationOptions options = {});
  NavigationResult forward(optional NavigationOptions options = {});

  attribute EventHandler onnavigate;
  attribute EventHandler onnavigatesuccess;
  attribute EventHandler onnavigateerror;
  attribute EventHandler oncurrententrychange;
};
```

Normative owner definition

NavigationActivation

interface

WebIDL / NavigationActivation

```
[Exposed=Window]
interface NavigationActivation {
  readonly attribute NavigationHistoryEntry? from;
  readonly attribute NavigationHistoryEntry entry;
  readonly attribute NavigationType navigationType;
};
```

Normative owner definition

NavigationCurrentEntryChangeEvent

interface / inherits Event

WebIDL / NavigationCurrentEntryChangeEvent

```
[Exposed=Window]
interface NavigationCurrentEntryChangeEvent : Event {
  constructor(DOMString type, NavigationCurrentEntryChangeEventInit eventInitDict);

  readonly attribute NavigationType? navigationType;
  readonly attribute NavigationHistoryEntry from;
};
```

Normative owner definition

NavigationCurrentEntryChangeEventInit

dictionary / inherits EventInit

WebIDL / NavigationCurrentEntryChangeEventInit

```
dictionary NavigationCurrentEntryChangeEventInit : EventInit {
  NavigationType? navigationType = null;
  required NavigationHistoryEntry from;
};
```

Normative owner definition

NavigationDestination

interface

WebIDL / NavigationDestination

```
[Exposed=Window]
interface NavigationDestination {
  readonly attribute USVString url;
  readonly attribute DOMString key;
  readonly attribute DOMString id;
  readonly attribute long long index;
  readonly attribute boolean sameDocument;

  any getState();
};
```

Normative owner definition

NavigationFocusReset

enum

WebIDL / NavigationFocusReset

```
enum NavigationFocusReset {
  "after-transition",
  "manual"
};
```

Normative owner definition

NavigationHistoryBehavior

enum

WebIDL / NavigationHistoryBehavior

```
enum NavigationHistoryBehavior {
  "auto",
  "push",
  "replace"
};
```

Normative owner definition

NavigationHistoryEntry

interface / inherits EventTarget

WebIDL / NavigationHistoryEntry

```
[Exposed=Window]
interface NavigationHistoryEntry : EventTarget {
  readonly attribute USVString? url;
  readonly attribute DOMString key;
  readonly attribute DOMString id;
  readonly attribute long long index;
  readonly attribute boolean sameDocument;

  any getState();

  attribute EventHandler ondispose;
};
```

Normative owner definition

NavigationInterceptHandler

callback

WebIDL / NavigationInterceptHandler

```
callback NavigationInterceptHandler = Promise<undefined> ();
```

Normative owner definition

NavigationInterceptOptions

dictionary

WebIDL / NavigationInterceptOptions

```
dictionary NavigationInterceptOptions {
  NavigationPrecommitHandler precommitHandler;
  NavigationInterceptHandler handler;
  NavigationFocusReset focusReset;
  NavigationScrollBehavior scroll;
};
```

Normative owner definition

NavigationNavigateOptions

dictionary / inherits NavigationOptions

WebIDL / NavigationNavigateOptions

```
dictionary NavigationNavigateOptions : NavigationOptions {
  any state;
  NavigationHistoryBehavior history = "auto";
};
```

Normative owner definition

NavigationOptions

dictionary

WebIDL / NavigationOptions

```
dictionary NavigationOptions {
  any info;
};
```

Normative owner definition

NavigationPrecommitController

interface

WebIDL / NavigationPrecommitController

```
[Exposed=Window]
interface NavigationPrecommitController {
  undefined redirect(USVString url, optional NavigationNavigateOptions options = {});
  undefined addHandler(NavigationInterceptHandler handler);
};
```

Normative owner definition

NavigationPrecommitHandler

callback

WebIDL / NavigationPrecommitHandler

```
callback NavigationPrecommitHandler = Promise<undefined> (NavigationPrecommitController cont
roller);
```

Normative owner definition

NavigationReloadOptions

dictionary / inherits NavigationOptions

WebIDL / NavigationReloadOptions

```
dictionary NavigationReloadOptions : NavigationOptions {
  any state;
};
```

Normative owner definition

NavigationResult

dictionary

WebIDL / NavigationResult

```
dictionary NavigationResult {
  Promise<NavigationHistoryEntry> committed;
  Promise<NavigationHistoryEntry> finished;
};
```

Normative owner definition

NavigationScrollBehavior

enum

WebIDL / NavigationScrollBehavior

```
enum NavigationScrollBehavior {
  "after-transition",
  "manual"
};
```

Normative owner definition

NavigationTransition

interface

WebIDL / NavigationTransition

```
[Exposed=Window]
interface NavigationTransition {
  readonly attribute NavigationType navigationType;
```

Code continues on the next page

WebIDL / NavigationTransition / continued

```
  readonly attribute NavigationHistoryEntry from;
  readonly attribute NavigationDestination to;
  readonly attribute Promise<undefined> committed;
  readonly attribute Promise<undefined> finished;
};
```

Normative owner definition

NavigationType

enum

WebIDL / NavigationType

```
enum NavigationType {
 "push",
 "replace",
 "reload",
 "traverse"
};
```

Normative owner definition

NavigationUpdateCurrentEntryOptions

dictionary

WebIDL / NavigationUpdateCurrentEntryOptions

```
dictionary NavigationUpdateCurrentEntryOptions {
  required any state;
};
```

Normative owner definition

Navigator

interface / includes NavigatorID, NavigatorLanguage, NavigatorOnLine, NavigatorContentUtils, NavigatorCookies, NavigatorPlugins, NavigatorConcurrentHardware

WebIDL / Navigator

```
partial interface Navigator {
  [SameObject] readonly attribute UserActivation userActivation;
};
```

WebIDL / Navigator

```
[Exposed=Window]
interface Navigator {
  // objects implementing this interface also implement the interfaces given below
};
```

Normative owner definition

NavigatorConcurrentHardware

interface mixin

WebIDL / NavigatorConcurrentHardware

```
interface mixin NavigatorConcurrentHardware {
  readonly attribute unsigned long long hardwareConcurrency;
};
```

Normative owner definition

NavigatorContentUtils

interface mixin

WebIDL / NavigatorContentUtils

```
interface mixin NavigatorContentUtils {
  [SecureContext] undefined registerProtocolHandler(DOMString scheme, USVString url);
  [SecureContext] undefined unregisterProtocolHandler(DOMString scheme, USVString url);
};
```

Normative owner definition

NavigatorCookies

interface mixin

WebIDL / NavigatorCookies

```
interface mixin NavigatorCookies {
  readonly attribute boolean cookieEnabled;
};
```

Normative owner definition

NavigatorID

interface mixin

WebIDL / NavigatorID

```
interface mixin NavigatorID {
  readonly attribute DOMString appCodeName; // constant "Mozilla"
  readonly attribute DOMString appName; // constant "Netscape"
  readonly attribute DOMString appVersion;
  readonly attribute DOMString platform;
  readonly attribute DOMString product; // constant "Gecko"
  [Exposed=Window] readonly attribute DOMString productSub;
  readonly attribute DOMString userAgent;
  [Exposed=Window] readonly attribute DOMString vendor;
  [Exposed=Window] readonly attribute DOMString vendorSub; // constant ""
};
```

WebIDL / NavigatorID

```
partial interface mixin NavigatorID {
  [Exposed=Window] boolean taintEnabled(); // constant false
  [Exposed=Window] readonly attribute DOMString oscpu;
};
```

Normative owner definition

NavigatorLanguage

interface mixin

WebIDL / NavigatorLanguage

```
interface mixin NavigatorLanguage {
  readonly attribute DOMString language;
  readonly attribute FrozenArray<DOMString> languages;
};
```

Normative owner definition

NavigatorOnLine

interface mixin

WebIDL / NavigatorOnLine

```
interface mixin NavigatorOnLine {
  readonly attribute boolean onLine;
};
```

Normative owner definition

NavigatorPlugins

interface mixin

WebIDL / NavigatorPlugins

```
interface mixin NavigatorPlugins {
  [SameObject] readonly attribute PluginArray plugins;
  [SameObject] readonly attribute MimeTypeArray mimeTypes;
  boolean javaEnabled();
  readonly attribute boolean pdfViewerEnabled;
};
```

Normative owner definition

NonDocumentTypeChildNode

interface mixin

WebIDL / NonDocumentTypeChildNode

```
partial interface mixin NonDocumentTypeChildNode {
  [CEReactions] undefined beforeHTML(DOMString html, optional SetHTMLOptions options = {});
  [CEReactions] undefined afterHTML(DOMString html, optional SetHTMLOptions options = {});
  [CEReactions] undefined replaceWithHTML(DOMString html, optional SetHTMLOptions options =
{});
  [CEReactions] undefined beforeHTMLUnsafe((TrustedHTML or DOMString) html, optional (SetHTM
LUnsafeOptions or TrustedHTMLParserOptions) options = {});
  [CEReactions] undefined afterHTMLUnsafe((TrustedHTML or DOMString) html, optional (SetHTML
UnsafeOptions or TrustedHTMLParserOptions) options = {});
  [CEReactions] undefined replaceWithHTMLUnsafe((TrustedHTML or DOMString) html, optional (S
etHTMLUnsafeOptions or TrustedHTMLParserOptions) options = {});
};
```

Normative owner definition

NotRestoredReasonDetails

interface

WebIDL / NotRestoredReasonDetails

```
[Exposed=Window]
interface NotRestoredReasonDetails {
  readonly attribute DOMString reason;
  [Default] object toJSON();
};
```

Normative owner definition

NotRestoredReasons

interface

WebIDL / NotRestoredReasons

```
[Exposed=Window]
interface NotRestoredReasons {
  readonly attribute USVString? src;
  readonly attribute DOMString? id;
  readonly attribute DOMString? name;
  readonly attribute USVString? url;
  readonly attribute FrozenArray<NotRestoredReasonDetails>? reasons;
  readonly attribute FrozenArray<NotRestoredReasons>? children;
  [Default] object toJSON();
};
```

Normative owner definition

OffscreenCanvas

interface / inherits EventTarget

WebIDL / OffscreenCanvas

```
[Exposed=(Window,Worker), Transferable]
interface OffscreenCanvas : EventTarget {
  constructor([EnforceRange] unsigned long long width, [EnforceRange] unsigned long long hei
ght);

  attribute [EnforceRange] unsigned long long width;
  attribute [EnforceRange] unsigned long long height;

  OffscreenRenderingContext? getContext(OffscreenRenderingContextId contextId, optional any
options = null);
  ImageBitmap transferToImageBitmap();
  Promise<Blob> convertToBlob(optional ImageEncodeOptions options = {});

  attribute EventHandler oncontextlost;
  attribute EventHandler oncontextrestored;
};
```

Normative owner definition

OffscreenCanvasRenderingContext2D

interface / includes CanvasSettings, CanvasState, CanvasTransform, CanvasCompositing, CanvasImageSmoothing, CanvasFillStrokeStyles, CanvasShadowStyles, CanvasFilters, CanvasRect, CanvasDrawPath, CanvasText, CanvasDrawImage, CanvasImageData, CanvasPathDrawingStyles, CanvasTextDrawingStyles, CanvasPath

WebIDL / OffscreenCanvasRenderingContext2D

```
[Exposed=(Window,Worker)]
interface OffscreenCanvasRenderingContext2D {
  readonly attribute OffscreenCanvas canvas;
};
```

Normative owner definition

OffscreenRenderingContext

typedef

WebIDL / OffscreenRenderingContext

```
typedef (OffscreenCanvasRenderingContext2D or ImageBitmapRenderingContext or WebGLRenderingC
ontext or WebGL2RenderingContext or GPUCanvasContext) OffscreenRenderingContext;
```

Normative owner definition

OffscreenRenderingContextId

enum

WebIDL / OffscreenRenderingContextId

```
enum OffscreenRenderingContextId { "2d", "bitmaprenderer", "webgl", "webgl2", "webgpu" };
```

Normative owner definition

OnBeforeUnloadEventHandler

typedef

WebIDL / OnBeforeUnloadEventHandler

```
typedef OnBeforeUnloadEventHandlerNonNull? OnBeforeUnloadEventHandler;
```

Normative owner definition

OnBeforeUnloadEventHandlerNonNull

callback

WebIDL / OnBeforeUnloadEventHandlerNonNull

```
[LegacyTreatNonObjectAsNull]
callback OnBeforeUnloadEventHandlerNonNull = DOMString? (Event event);
```

Normative owner definition

OnErrorEventHandler

typedef

WebIDL / OnErrorEventHandler

```
typedef OnErrorEventHandlerNonNull? OnErrorEventHandler;
```

Normative owner definition

OnErrorEventHandlerNonNull

callback

WebIDL / OnErrorEventHandlerNonNull

```
[LegacyTreatNonObjectAsNull]
callback OnErrorEventHandlerNonNull = any ((Event or DOMString) event, optional DOMString so
urce, optional unsigned long lineno, optional unsigned long colno, optional any error);
```

Normative owner definition

Origin

interface

WebIDL / Origin

```
[Exposed=*]
interface Origin {
  constructor();

  static Origin from(any value);

  readonly attribute boolean opaque;
```

Code continues on the next page

WebIDL / Origin / continued

```
  boolean isSameOrigin(Origin other);
  boolean isSameSite(Origin other);
};
```

Normative owner definition

PageRevealEvent

interface / inherits Event

WebIDL / PageRevealEvent

```
[Exposed=Window]
interface PageRevealEvent : Event {
  constructor(DOMString type, optional PageRevealEventInit eventInitDict = {});
  readonly attribute ViewTransition? viewTransition;
};
```

Normative owner definition

PageRevealEventInit

dictionary / inherits EventInit

WebIDL / PageRevealEventInit

```
dictionary PageRevealEventInit : EventInit {
  ViewTransition? viewTransition = null;
};
```

Normative owner definition

PageSwapEvent

interface / inherits Event

WebIDL / PageSwapEvent

```
[Exposed=Window]
interface PageSwapEvent : Event {
  constructor(DOMString type, optional PageSwapEventInit eventInitDict = {});
  readonly attribute NavigationActivation? activation;
  readonly attribute ViewTransition? viewTransition;
};
```

Normative owner definition

PageSwapEventInit

dictionary / inherits EventInit

WebIDL / PageSwapEventInit

```
dictionary PageSwapEventInit : EventInit {
  NavigationActivation? activation = null;
  ViewTransition? viewTransition = null;
};
```

Normative owner definition

PageTransitionEvent

interface / inherits Event

WebIDL / PageTransitionEvent

```
[Exposed=Window]
interface PageTransitionEvent : Event {
  constructor(DOMString type, optional PageTransitionEventInit eventInitDict = {});

  readonly attribute boolean persisted;
};
```

Normative owner definition

PageTransitionEventInit

dictionary / inherits EventInit

WebIDL / PageTransitionEventInit

```
dictionary PageTransitionEventInit : EventInit {
  boolean persisted = false;
};
```

Normative owner definition

ParseHTMLUnsafeOptions

dictionary

WebIDL / ParseHTMLUnsafeOptions

```
dictionary ParseHTMLUnsafeOptions {
  (Sanitizer or SanitizerConfig or SanitizerPresets) sanitizer;
};
```

Normative owner definition

Path2D

interface / includes CanvasPath

WebIDL / Path2D

```
[Exposed=(Window,Worker)]
interface Path2D {
  constructor(optional (Path2D or DOMString) path);

  undefined addPath(Path2D path, optional DOMMatrix2DInit transform = {});
};
```

Normative owner definition

Plugin

interface

WebIDL / Plugin

```
[Exposed=Window,
 LegacyUnenumerableNamedProperties]
interface Plugin {
  readonly attribute DOMString name;
  readonly attribute DOMString description;
  readonly attribute DOMString filename;
  readonly attribute unsigned long length;
  getter MimeType? item(unsigned long index);
  getter MimeType? namedItem(DOMString name);
};
```

Normative owner definition

PluginArray

interface

WebIDL / PluginArray

```
[Exposed=Window,
 LegacyUnenumerableNamedProperties]
interface PluginArray {
  undefined refresh();
  readonly attribute unsigned long length;
  getter Plugin? item(unsigned long index);
  getter Plugin? namedItem(DOMString name);
};
```

Normative owner definition

PopoverTargetAttributes

interface mixin

WebIDL / PopoverTargetAttributes

```
interface mixin PopoverTargetAttributes {
  [CEReactions, Reflect="popovertarget"] attribute Element? popoverTargetElement;
  [CEReactions] attribute DOMString popoverTargetAction;
};
```

Normative owner definition

PopStateEvent

interface / inherits Event

WebIDL / PopStateEvent

```
[Exposed=Window]
interface PopStateEvent : Event {
  constructor(DOMString type, optional PopStateEventInit eventInitDict = {});

  readonly attribute any state;
  readonly attribute boolean hasUAVisualTransition;
};
```

Normative owner definition

PopStateEventInit

dictionary / inherits EventInit

WebIDL / PopStateEventInit

```
dictionary PopStateEventInit : EventInit {
  any state = null;
  boolean hasUAVisualTransition = false;
};
```

Normative owner definition

PredefinedColorSpace

enum

WebIDL / PredefinedColorSpace

```
enum PredefinedColorSpace { "srgb", "srgb-linear", "display-p3", "display-p3-linear" };
```

Normative owner definition

PremultiplyAlpha

enum

WebIDL / PremultiplyAlpha

```
enum PremultiplyAlpha { "none", "premultiply", "default" };
```

Normative owner definition

PromiseRejectionEvent

interface / inherits Event

WebIDL / PromiseRejectionEvent

```
[Exposed=*]
interface PromiseRejectionEvent : Event {
  constructor(DOMString type, PromiseRejectionEventInit eventInitDict);

  readonly attribute object promise;
  readonly attribute any reason;
};
```

Normative owner definition

PromiseRejectionEventInit

dictionary / inherits EventInit

WebIDL / PromiseRejectionEventInit

```
dictionary PromiseRejectionEventInit : EventInit {
  required object promise;
  any reason;
};
```

Normative owner definition

RadioNodeList

interface / inherits NodeList

WebIDL / RadioNodeList

```
[Exposed=Window]
interface RadioNodeList : NodeList {
  attribute DOMString value;
};
```

Normative owner definition

Range

interface

WebIDL / Range

```
partial interface Range {
  [CEReactions, NewObject] DocumentFragment createContextualFragment((TrustedHTML or DOMStri
ng) string);
};
```

Normative owner definition

RenderingContext

typedef

WebIDL / RenderingContext

```
typedef (CanvasRenderingContext2D or ImageBitmapRenderingContext or WebGLRenderingContext or
 WebGL2RenderingContext or GPUCanvasContext) RenderingContext;
```

Normative owner definition

ResizeQuality

enum

WebIDL / ResizeQuality

```
enum ResizeQuality { "pixelated", "low", "medium", "high" };
```

Normative owner definition

Sanitizer

interface

WebIDL / Sanitizer

```
[Exposed=Window]
interface Sanitizer {
  constructor(optional (SanitizerConfig or SanitizerPresets) configuration = "default");

  // Query configuration:
  SanitizerConfig get();

  // Modify a Sanitizer's lists and fields:
  boolean allowElement(SanitizerElementWithAttributes element);
  boolean removeElement(SanitizerElement element);
  boolean replaceElementWithChildren(SanitizerElement element);
  boolean allowProcessingInstruction(SanitizerPI pi);
  boolean removeProcessingInstruction(SanitizerPI pi);
  boolean allowAttribute(SanitizerAttribute attribute);
  boolean removeAttribute(SanitizerAttribute attribute);
  boolean setComments(boolean allow);
  boolean setDataAttributes(boolean allow);
  boolean setJavascriptURLs(boolean allow);

  // Remove markup that executes script.
  boolean removeUnsafe();
};
```

Normative owner definition

SanitizerAttribute

typedef

WebIDL / SanitizerAttribute

```
typedef (DOMString or SanitizerAttributeNamespace) SanitizerAttribute;
```

Normative owner definition

SanitizerAttributeNamespace

dictionary

WebIDL / SanitizerAttributeNamespace

```
dictionary SanitizerAttributeNamespace {
  required DOMString name;
  DOMString? _namespace = null;
};
```

Normative owner definition

SanitizerConfig

dictionary

WebIDL / SanitizerConfig

```
dictionary SanitizerConfig {
  sequence<SanitizerElementWithAttributes> elements;
  sequence<SanitizerElement> removeElements;
  sequence<SanitizerElement> replaceWithChildrenElements;

  sequence<SanitizerPI> processingInstructions;
  sequence<SanitizerPI> removeProcessingInstructions;

  sequence<SanitizerAttribute> attributes;
  sequence<SanitizerAttribute> removeAttributes;

  boolean comments;
  boolean dataAttributes;
  boolean javascriptURLs;
};
```

Normative owner definition

SanitizerElement

typedef

WebIDL / SanitizerElement

```
typedef (DOMString or SanitizerElementNamespace) SanitizerElement;
```

Normative owner definition

SanitizerElementNamespace

dictionary

WebIDL / SanitizerElementNamespace

```
dictionary SanitizerElementNamespace {
  required DOMString name;
  DOMString? _namespace = "http://www.w3.org/1999/xhtml";
};
```

Normative owner definition

SanitizerElementNamespaceWithAttributes

dictionary / inherits SanitizerElementNamespace

WebIDL / SanitizerElementNamespaceWithAttributes

```
// Used by "elements"
dictionary SanitizerElementNamespaceWithAttributes : SanitizerElementNamespace {
  sequence<SanitizerAttribute> attributes;
  sequence<SanitizerAttribute> removeAttributes;
};
```

Normative owner definition

SanitizerElementWithAttributes

typedef

WebIDL / SanitizerElementWithAttributes

```
typedef (DOMString or SanitizerElementNamespaceWithAttributes) SanitizerElementWithAttribute
s;
```

Normative owner definition

SanitizerPI

typedef

WebIDL / SanitizerPI

```
typedef (DOMString or SanitizerProcessingInstruction) SanitizerPI;
```

Normative owner definition

SanitizerPresets

enum

```
WebIDL / SanitizerPresets
```

```
enum SanitizerPresets { "default" };
```

Normative owner definition

SanitizerProcessingInstruction

dictionary

```
WebIDL / SanitizerProcessingInstruction
```

```
dictionary SanitizerProcessingInstruction {
  required DOMString target;
};
```

Normative owner definition

ScrollRestoration

enum

```
WebIDL / ScrollRestoration
```

```
enum ScrollRestoration { "auto", "manual" };
```

Normative owner definition

SelectionMode

enum

```
WebIDL / SelectionMode
```

```
enum SelectionMode {
  "select",
  "start",
  "end",
  "preserve" // default
};
```

Normative owner definition

SetHTMLOptions

dictionary

WebIDL / SetHTMLOptions

```
dictionary SetHTMLOptions {
  (Sanitizer or SanitizerConfig or SanitizerPresets) sanitizer = "default";
};
```

Normative owner definition

SetHTMLUnsafeOptions

dictionary

WebIDL / SetHTMLUnsafeOptions

```
dictionary SetHTMLUnsafeOptions {
  (Sanitizer or SanitizerConfig or SanitizerPresets) sanitizer;
  boolean runScripts = false;
};
```

Normative owner definition

ShadowRoot

interface

WebIDL / ShadowRoot

```
partial interface ShadowRoot {
  [CEReactions] undefined setHTML(DOMString html, optional SetHTMLOptions options = {});
  [CEReactions] undefined appendHTML(DOMString html, optional SetHTMLOptions options = {});
  [CEReactions] undefined prependHTML(DOMString html, optional SetHTMLOptions options = {});
  [CEReactions] undefined setHTMLUnsafe((TrustedHTML or DOMString) html, optional (SetHTMLUn
safeOptions or TrustedHTMLParserOptions) options = {});
  [CEReactions] undefined appendHTMLUnsafe((TrustedHTML or DOMString) html, optional (SetHTM
LUnsafeOptions or TrustedHTMLParserOptions) options = {});
  [CEReactions] undefined prependHTMLUnsafe((TrustedHTML or DOMString) html, optional (SetHT
MLUnsafeOptions or TrustedHTMLParserOptions) options = {});
  DOMString getHTML(optional GetHTMLOptions options = {});

  [CEReactions] attribute (TrustedHTML or [LegacyNullToEmptyString] DOMString) innerHTML;
};
```

Normative owner definition

SharedWorker

interface / inherits EventTarget / includes AbstractWorker

WebIDL / SharedWorker

```
[Exposed=Window]
interface SharedWorker : EventTarget {
  constructor((TrustedScriptURL or USVString) scriptURL, optional (DOMString or SharedWorker
Options) options = {});

  readonly attribute MessagePort port;
};
```

Normative owner definition

SharedWorkerGlobalScope

interface / inherits WorkerGlobalScope

WebIDL / SharedWorkerGlobalScope

```
[Global=(Worker,SharedWorker),Exposed=SharedWorker]
interface SharedWorkerGlobalScope : WorkerGlobalScope {
  [Replaceable] readonly attribute DOMString name;

  undefined close();

  attribute EventHandler onconnect;
};
```

Normative owner definition

SharedWorkerOptions

dictionary / inherits WorkerOptions

WebIDL / SharedWorkerOptions

```
dictionary SharedWorkerOptions : WorkerOptions {
  boolean extendedLifetime = false;
};
```

Normative owner definition

ShowPopoverOptions

dictionary

WebIDL / ShowPopoverOptions

```
dictionary ShowPopoverOptions {
  HTMLElement source;
};
```

Normative owner definition

Storage

interface

WebIDL / Storage

```
[Exposed=Window]
interface Storage {
  readonly attribute unsigned long length;
  DOMString? key(unsigned long index);
  getter DOMString? getItem(DOMString key);
  setter undefined setItem(DOMString key, DOMString value);
  deleter undefined removeItem(DOMString key);
  undefined clear();
};
```

Normative owner definition

StorageEvent

interface / inherits Event

WebIDL / StorageEvent

```
[Exposed=Window]
interface StorageEvent : Event {
  constructor(DOMString type, optional StorageEventInit eventInitDict = {});

  readonly attribute DOMString? key;
  readonly attribute DOMString? oldValue;
  readonly attribute DOMString? newValue;
  readonly attribute USVString url;
  readonly attribute Storage? storageArea;

  undefined initStorageEvent(DOMString type, optional boolean bubbles = false, optional bool
ean cancelable = false, optional DOMString? key = null, optional DOMString? oldValue = null,
 optional DOMString? newValue = null, optional USVString url = "", optional Storage? storage
Area = null);
};
```

Normative owner definition

StorageEventInit

dictionary / inherits EventInit

WebIDL / StorageEventInit

```
dictionary StorageEventInit : EventInit {
  DOMString? key = null;
  DOMString? oldValue = null;
  DOMString? newValue = null;
  USVString url = "";
  Storage? storageArea = null;
};
```

Normative owner definition

StructuredSerializeOptions

dictionary

WebIDL / StructuredSerializeOptions

```
dictionary StructuredSerializeOptions {
  sequence<object> transfer = [];
};
```

Normative owner definition

SubmitEvent

interface / inherits Event

WebIDL / SubmitEvent

```
[Exposed=Window]
interface SubmitEvent : Event {
  constructor(DOMString type, optional SubmitEventInit eventInitDict = {});

  readonly attribute HTMLElement? submitter;
};
```

Normative owner definition

SubmitEventInit

dictionary / inherits EventInit

WebIDL / SubmitEventInit

```
dictionary SubmitEventInit : EventInit {
  HTMLElement? submitter = null;
};
```

Normative owner definition

TextMetrics

interface

WebIDL / TextMetrics

```
[Exposed=(Window,Worker)]
interface TextMetrics {
  // x-direction
  readonly attribute double width; // advance width
  readonly attribute double actualBoundingBoxLeft;
  readonly attribute double actualBoundingBoxRight;

  // y-direction
  readonly attribute double fontBoundingBoxAscent;
  readonly attribute double fontBoundingBoxDescent;
  readonly attribute double actualBoundingBoxAscent;
  readonly attribute double actualBoundingBoxDescent;
  readonly attribute double emHeightAscent;
  readonly attribute double emHeightDescent;
  readonly attribute double hangingBaseline;
  readonly attribute double alphabeticBaseline;
  readonly attribute double ideographicBaseline;
};
```

Normative owner definition

TextTrack

interface / inherits EventTarget

WebIDL / TextTrack

```
[Exposed=Window]
interface TextTrack : EventTarget {
  readonly attribute TextTrackKind kind;
  readonly attribute DOMString label;
  readonly attribute DOMString language;

  readonly attribute DOMString id;
  readonly attribute DOMString inBandMetadataTrackDispatchType;
```

Code continues on the next page

WebIDL / TextTrack / continued

```
  attribute TextTrackMode mode;

  readonly attribute TextTrackCueList? cues;
  readonly attribute TextTrackCueList? activeCues;

  undefined addCue(TextTrackCue cue);
  undefined removeCue(TextTrackCue cue);

  attribute EventHandler oncuechange;
};
```

Normative owner definition

TextTrackCue

interface / inherits EventTarget

WebIDL / TextTrackCue

```
[Exposed=Window]
interface TextTrackCue : EventTarget {
  readonly attribute TextTrack? track;

  attribute DOMString id;
  attribute double startTime;
  attribute unrestricted double endTime;
  attribute boolean pauseOnExit;

  attribute EventHandler onenter;
  attribute EventHandler onexit;
};
```

Normative owner definition

TextTrackCueList

interface

WebIDL / TextTrackCueList

```
[Exposed=Window]
interface TextTrackCueList {
  readonly attribute unsigned long length;
  getter TextTrackCue (unsigned long index);
  TextTrackCue? getCueById(DOMString id);
};
```

Normative owner definition

TextTrackKind

enum

WebIDL / TextTrackKind

```
enum TextTrackKind { "subtitles", "captions", "descriptions", "chapters", "metadata" };
```

Normative owner definition

TextTrackList

interface / inherits EventTarget

WebIDL / TextTrackList

```
[Exposed=Window]
interface TextTrackList : EventTarget {
  readonly attribute unsigned long length;
  getter TextTrack (unsigned long index);
  TextTrack? getTrackById(DOMString id);

  attribute EventHandler onchange;
  attribute EventHandler onaddtrack;
  attribute EventHandler onremovetrack;
};
```

Normative owner definition

TextTrackMode

enum

WebIDL / TextTrackMode

```
enum TextTrackMode { "disabled", "hidden", "showing" };
```

Normative owner definition

TimeRanges

interface

WebIDL / TimeRanges

```
[Exposed=Window]
interface TimeRanges {
  readonly attribute unsigned long length;
  double start(unsigned long index);
  double end(unsigned long index);
};
```

Normative owner definition

TimerHandler

typedef

WebIDL / TimerHandler

```
typedef (DOMString or Function or TrustedScript) TimerHandler;
```

Normative owner definition

ToggleEvent

interface / inherits Event

WebIDL / ToggleEvent

```
[Exposed=Window]
interface ToggleEvent : Event {
  constructor(DOMString type, optional ToggleEventInit eventInitDict = {});
  readonly attribute DOMString oldState;
  readonly attribute DOMString newState;
  readonly attribute Element? source;
};
```

Normative owner definition

ToggleEventInit

dictionary / inherits EventInit

WebIDL / ToggleEventInit

```
dictionary ToggleEventInit : EventInit {
  DOMString oldState = "";
  DOMString newState = "";
  Element? source = null;
};
```

Normative owner definition

TogglePopoverOptions

dictionary / inherits ShowPopoverOptions

WebIDL / TogglePopoverOptions

```
dictionary TogglePopoverOptions : ShowPopoverOptions {
  boolean force;
};
```

Normative owner definition

TrackEvent

interface / inherits Event

WebIDL / TrackEvent

```
[Exposed=Window]
interface TrackEvent : Event {
  constructor(DOMString type, optional TrackEventInit eventInitDict = {});

  readonly attribute (VideoTrack or AudioTrack or TextTrack)? track;
};
```

Normative owner definition

TrackEventInit

dictionary / inherits EventInit

WebIDL / TrackEventInit

```
dictionary TrackEventInit : EventInit {
  (VideoTrack or AudioTrack or TextTrack)? track = null;
};
```

Normative owner definition

UserActivation

interface

WebIDL / UserActivation

```
[Exposed=Window]
interface UserActivation {
  readonly attribute boolean hasBeenActive;
  readonly attribute boolean isActive;
};
```

Normative owner definition

ValidityState

interface

WebIDL / ValidityState

```
[Exposed=Window]
interface ValidityState {
  readonly attribute boolean valueMissing;
  readonly attribute boolean typeMismatch;
  readonly attribute boolean patternMismatch;
  readonly attribute boolean tooLong;
  readonly attribute boolean tooShort;
  readonly attribute boolean rangeUnderflow;
  readonly attribute boolean rangeOverflow;
  readonly attribute boolean stepMismatch;
  readonly attribute boolean badInput;
  readonly attribute boolean customError;
  readonly attribute boolean valid;
};
```

Normative owner definition

ValidityStateFlags

dictionary

WebIDL / ValidityStateFlags

```
dictionary ValidityStateFlags {
  boolean valueMissing = false;
  boolean typeMismatch = false;
  boolean patternMismatch = false;
  boolean tooLong = false;
  boolean tooShort = false;
  boolean rangeUnderflow = false;
  boolean rangeOverflow = false;
  boolean stepMismatch = false;
  boolean badInput = false;
  boolean customError = false;
};
```

Normative owner definition

VideoTrack

interface

WebIDL / VideoTrack

```
[Exposed=Window]
interface VideoTrack {
  readonly attribute DOMString id;
  readonly attribute DOMString kind;
  readonly attribute DOMString label;
  readonly attribute DOMString language;
  attribute boolean selected;
};
```

Normative owner definition

VideoTrackList

interface / inherits EventTarget

WebIDL / VideoTrackList

```
[Exposed=Window]
interface VideoTrackList : EventTarget {
  readonly attribute unsigned long length;
  getter VideoTrack (unsigned long index);
  VideoTrack? getTrackById(DOMString id);
  readonly attribute long selectedIndex;

  attribute EventHandler onchange;
  attribute EventHandler onaddtrack;
  attribute EventHandler onremovetrack;
};
```

Normative owner definition

VisibilityStateEntry

interface / inherits PerformanceEntry

WebIDL / VisibilityStateEntry

```
[Exposed=(Window)]
interface VisibilityStateEntry : PerformanceEntry {
  readonly attribute DOMString name;                 // shadows inherited name
  readonly attribute DOMString entryType;            // shadows inherited entryType
  readonly attribute DOMHighResTimeStamp startTime;  // shadows inherited startTime
```

Code continues on the next page

WebIDL / VisibilityStateEntry / continued

```
  readonly attribute unsigned long duration;         // shadows inherited duration
};
```

Normative owner definition

Window

interface / inherits EventTarget / includes GlobalEventHandlers, WindowEventHandlers, WindowOrWorkerGlobalScope, AnimationFrameProvider, WindowSessionStorage, WindowLocalStorage

WebIDL / Window

```
[Global=Window,
 Exposed=Window,
 LegacyUnenumerableNamedProperties]
interface Window : EventTarget {
  // the current browsing context
  [LegacyUnforgeable] readonly attribute WindowProxy window;
  [Replaceable] readonly attribute WindowProxy self;
  [LegacyUnforgeable] readonly attribute Document document;
  attribute DOMString name;
  [PutForwards=href, LegacyUnforgeable] readonly attribute Location location;
  readonly attribute History history;
  [Replaceable] readonly attribute Navigation navigation;
  readonly attribute CustomElementRegistry customElements;
  [Replaceable] readonly attribute BarProp locationbar;
  [Replaceable] readonly attribute BarProp menubar;
  [Replaceable] readonly attribute BarProp personalbar;
  [Replaceable] readonly attribute BarProp scrollbars;
  [Replaceable] readonly attribute BarProp statusbar;
  [Replaceable] readonly attribute BarProp toolbar;
  attribute DOMString status;
  undefined close();
  readonly attribute boolean closed;
  undefined stop();
  undefined focus();
  undefined blur();

  // other browsing contexts
  [Replaceable] readonly attribute WindowProxy frames;
  [Replaceable] readonly attribute unsigned long length;
  [LegacyUnforgeable] readonly attribute WindowProxy? top;
  attribute any opener;
  [Replaceable] readonly attribute WindowProxy? parent;
  readonly attribute Element? frameElement;
  WindowProxy? open(optional USVString url = "", optional DOMString target = "_blank", optio
nal [LegacyNullToEmptyString] DOMString features = "");

  // Since this is the global object, the IDL named getter adds a NamedPropertiesObject exot
ic
  // object on the prototype chain. Indeed, this does not make the global object an exotic o
```

Code continues on the next page

WebIDL / Window / continued

```
bject.
  // Indexed access is taken care of by the WindowProxy exotic object.
  getter object (DOMString name);

  // the user agent
  readonly attribute Navigator navigator;
  [Replaceable] readonly attribute Navigator clientInformation; // legacy alias of .navigato
r
  readonly attribute boolean originAgentCluster;

  // user prompts
  undefined alert();
  undefined alert(DOMString message);
  boolean confirm(optional DOMString message = "");
  DOMString? prompt(optional DOMString message = "", optional DOMString default = "");
  undefined print();

  undefined postMessage(any message, USVString targetOrigin, optional sequence<object> trans
fer = []);
  undefined postMessage(any message, optional WindowPostMessageOptions options = {});

  // also has obsolete members
};
```

WebIDL / Window

```
partial interface Window {
  undefined captureEvents();
  undefined releaseEvents();

  [Replaceable, SameObject] readonly attribute External external;
};
```

Normative owner definition

WindowEventHandlers

interface mixin

WebIDL / WindowEventHandlers

```
interface mixin WindowEventHandlers {
  attribute EventHandler onafterprint;
  attribute EventHandler onbeforeprint;
  attribute OnBeforeUnloadEventHandler onbeforeunload;
  attribute EventHandler onhashchange;
  attribute EventHandler onlanguagechange;
  attribute EventHandler onmessage;
```

Code continues on the next page

WebIDL / WindowEventHandlers / continued

```
  attribute EventHandler onmessageerror;
  attribute EventHandler onoffline;
  attribute EventHandler ononline;
  attribute EventHandler onpagehide;
  attribute EventHandler onpagereveal;
  attribute EventHandler onpageshow;
  attribute EventHandler onpageswap;
  attribute EventHandler onpopstate;
  attribute EventHandler onrejectionhandled;
  attribute EventHandler onstorage;
  attribute EventHandler onunhandledrejection;
  attribute EventHandler onunload;
};
```

Normative owner definition

WindowLocalStorage

interface mixin

WebIDL / WindowLocalStorage

```
interface mixin WindowLocalStorage {
  readonly attribute Storage localStorage;
};
```

Normative owner definition

WindowOrWorkerGlobalScope

interface mixin

WebIDL / WindowOrWorkerGlobalScope

```
interface mixin WindowOrWorkerGlobalScope {
  [Replaceable] readonly attribute USVString origin;
  readonly attribute boolean isSecureContext;
  readonly attribute boolean crossOriginIsolated;

  undefined reportError(any e);

  // base64 utility methods
  DOMString btoa(DOMString data);
  ByteString atob(DOMString data);

  // timers
  long setTimeout(TimerHandler handler, optional long timeout = 0, any... arguments);
  undefined clearTimeout(optional long id = 0);
```

Code continues on the next page

WebIDL / WindowOrWorkerGlobalScope / continued

```
  long setInterval(TimerHandler handler, optional long timeout = 0, any... arguments);
  undefined clearInterval(optional long id = 0);

  // microtask queuing
  undefined queueMicrotask(VoidFunction callback);

  // ImageBitmap
  Promise<ImageBitmap> createImageBitmap(ImageBitmapSource image, optional ImageBitmapOption
s options = {});
  Promise<ImageBitmap> createImageBitmap(ImageBitmapSource image, long sx, long sy, long sw,
 long sh, optional ImageBitmapOptions options = {});

  // structured cloning
  any structuredClone(any value, optional StructuredSerializeOptions options = {});
};
```

Normative owner definition

WindowPostMessageOptions

dictionary / inherits StructuredSerializeOptions

WebIDL / WindowPostMessageOptions

```
dictionary WindowPostMessageOptions : StructuredSerializeOptions {
  USVString targetOrigin = "/";
};
```

Normative owner definition

WindowSessionStorage

interface mixin

WebIDL / WindowSessionStorage

```
interface mixin WindowSessionStorage {
  readonly attribute Storage sessionStorage;
};
```

Normative owner definition

Worker

interface / inherits EventTarget / includes AbstractWorker, MessageEventTarget

WebIDL / Worker

```
[Exposed=(Window,DedicatedWorker,SharedWorker)]
interface Worker : EventTarget {
  constructor((TrustedScriptURL or USVString) scriptURL, optional WorkerOptions options = {}
);

  undefined terminate();

  undefined postMessage(any message, sequence<object> transfer);
  undefined postMessage(any message, optional StructuredSerializeOptions options = {});
};
```

Normative owner definition

WorkerGlobalScope

interface / inherits EventTarget / includes WindowOrWorkerGlobalScope

WebIDL / WorkerGlobalScope

```
[Exposed=Worker]
interface WorkerGlobalScope : EventTarget {
  readonly attribute WorkerGlobalScope self;
  readonly attribute WorkerLocation location;
  readonly attribute WorkerNavigator navigator;
  undefined importScripts((TrustedScriptURL or USVString)... urls);

  attribute OnErrorEventHandler onerror;
  attribute EventHandler onlanguagechange;
  attribute EventHandler onoffline;
  attribute EventHandler ononline;
  attribute EventHandler onrejectionhandled;
  attribute EventHandler onunhandledrejection;
};
```

Normative owner definition

WorkerLocation

interface

WebIDL / WorkerLocation

```
[Exposed=Worker]
interface WorkerLocation {
  stringifier readonly attribute USVString href;
  readonly attribute USVString origin;
  readonly attribute USVString protocol;
  readonly attribute USVString host;
```

Code continues on the next page

WebIDL / WorkerLocation / continued

```
  readonly attribute USVString hostname;
  readonly attribute USVString port;
  readonly attribute USVString pathname;
  readonly attribute USVString search;
  readonly attribute USVString hash;
};
```

Normative owner definition

WorkerNavigator

interface / includes NavigatorID, NavigatorLanguage, NavigatorOnLine, NavigatorConcurrentHardware

WebIDL / WorkerNavigator

```
[Exposed=Worker]
interface WorkerNavigator {};
```

Normative owner definition

WorkerOptions

dictionary

WebIDL / WorkerOptions

```
dictionary WorkerOptions {
  DOMString name = "";
  WorkerType type = "classic";
  RequestCredentials credentials = "same-origin"; // credentials is only used if type is "mo
dule"
};
```

Normative owner definition

WorkerType

enum

WebIDL / WorkerType

```
enum WorkerType { "classic", "module" };
```

Normative owner definition

Worklet

interface

WebIDL / Worklet

```
[Exposed=Window, SecureContext]
interface Worklet {
  [NewObject] Promise<undefined> addModule(USVString moduleURL, optional WorkletOptions opti
ons = {});
};
```

Normative owner definition

WorkletGlobalScope

interface

WebIDL / WorkletGlobalScope

```
[Exposed=Worklet, SecureContext]
interface WorkletGlobalScope {};
```

Normative owner definition

WorkletOptions

dictionary

WebIDL / WorkletOptions

```
dictionary WorkletOptions {
  RequestCredentials credentials = "same-origin";
};
```

Normative owner definition

XMLSerializer

interface

WebIDL / XMLSerializer

```
[Exposed=Window]
interface XMLSerializer {
  constructor();
```

Code continues on the next page

WebIDL / XMLSerializer / continued

```
  DOMString serializeToString(Node root);
};
```

Normative owner definition

Keyword, state and event routes

437 captured public definition records. These routes distinguish HTML attribute keywords, input states, WebIDL enumeration values and events. They do not form a complete missing-value, invalid-value or conditional applicability table.

anonymous

attr-value / audio/crossorigin, video/crossorigin, img/crossorigin, link/crossorigin, script/crossorigin

Normative definition

use-credentials

attr-value / audio/crossorigin, video/crossorigin, img/crossorigin, link/crossorigin, script/crossorigin

Normative definition

lazy

attr-value / img/loading, audio/loading, video/loading, iframe/loading

Normative definition

eager

attr-value / img/loading, audio/loading, video/loading, iframe/loading

Normative definition

high

attr-value / img/fetchpriority, script/fetchpriority, link/fetchpriority

Normative definition

low

attr-value / img/fetchpriority, script/fetchpriority, link/fetchpriority

Normative definition

auto

attr-value / img/fetchpriority, script/fetchpriority, link/fetchpriority

Normative definition

yes

attr-value / html-global/translate

Normative definition

no

attr-value / html-global/translate

Normative definition

ltr

attr-value / html-global/dir

Normative definition

rtl

attr-value / html-global/dir

Normative definition

auto

attr-value / html-global/dir

Normative definition

application-name

attr-value / meta/name

Normative definition

author

attr-value / meta/name

Normative definition

description

attr-value / meta/name

Normative definition

generator

attr-value / meta/name

Normative definition

keywords

attr-value / meta/name

Normative definition

referrer

attr-value / meta/name

Normative definition

theme-color

attr-value / meta/name

Normative definition

color-scheme

attr-value / meta/name

Normative definition

content-language

attr-value / meta/http-equiv

Normative definition

content-type

attr-value / meta/http-equiv

Normative definition

default-style

attr-value / meta/http-equiv

Normative definition

refresh

attr-value / meta/http-equiv

Normative definition

set-cookie

attr-value / meta/http-equiv

Normative definition

x-ua-compatible

attr-value / meta/http-equiv

Normative definition

content-security-policy

attr-value / meta/http-equiv

Normative definition

l

attr-value / ol/type

Normative definition

a

attr-value / ol/type

Normative definition

A

attr-value / ol/type

Normative definition

i

attr-value / ol/type

Normative definition

I

attr-value / ol/type

Normative definition

alternate

attr-value / link/rel, a/rel, area/rel

Normative definition

author

attr-value / link/rel, a/rel, area/rel

Normative definition

bookmark

attr-value / a/rel, area/rel

Normative definition

canonical

attr-value / link/rel

Normative definition

dns-prefetch

attr-value / link/rel

Normative definition

expect

attr-value / link/rel

Normative definition

external

attr-value / a/rel, area/rel, form/rel

Normative definition

help

attr-value / link/rel, a/rel, area/rel, form/rel

Normative definition

icon

attr-value / link/rel

Normative definition

license

attr-value / link/rel, a/rel, area/rel, form/rel

Normative definition

manifest

attr-value / link/rel

Normative definition

modulepreload

attr-value / link/rel

Normative definition

nofollow

attr-value / a/rel, area/rel, form/rel

Normative definition

noopener

attr-value / a/rel, area/rel, form/rel

Normative definition

noreferrer

attr-value / a/rel, area/rel, form/rel

Normative definition

opener

attr-value / a/rel, area/rel, form/rel

Normative definition

pingback

attr-value / link/rel

Normative definition

preconnect

attr-value / link/rel

Normative definition

prefetch

attr-value / link/rel

Normative definition

preload

attr-value / link/rel

Normative definition

privacy-policy

attr-value / link/rel, a/rel, area/rel

Normative definition

search

attr-value / link/rel, a/rel, area/rel, form/rel

Normative definition

stylesheet

attr-value / link/rel

Normative definition

tag

attr-value / a/rel, area/rel

Normative definition

terms-of-service

attr-value / link/rel, a/rel, area/rel

Normative definition

next

attr-value / link/rel, a/rel, area/rel, form/rel

Normative definition

prev

attr-value / link/rel, a/rel, area/rel, form/rel

Normative definition

sync

attr-value / img/decoding

Normative definition

async

attr-value / img/decoding

Normative definition

auto

attr-value / img/decoding

Normative definition

subtitles

attr-value / track/kind

Normative definition

captions

attr-value / track/kind

Normative definition

descriptions

attr-value / track/kind

Normative definition

chapters

attr-value / track/kind

Normative definition

metadata

attr-value / track/kind

Normative definition

probably

enum-value / CanPlayTypeResult

Normative definition

maybe

enum-value / CanPlayTypeResult

Normative definition

auto

attr-value / audio/preload, video/preload

Normative definition

none

attr-value / audio/preload, video/preload

Normative definition

metadata

attr-value / audio/preload, video/preload

Normative definition

disabled

enum-value / TextTrackMode

Normative definition

hidden

enum-value / TextTrackMode

Normative definition

showing

enum-value / TextTrackMode

Normative definition

loadstart

event / HTMLMediaElement

Normative definition

progress

event / HTMLMediaElement

Normative definition

suspend

event / HTMLMediaElement

Normative definition

abort

event / HTMLMediaElement

Normative definition

error

event / HTMLMediaElement

Normative definition

emptied

event / HTMLMediaElement

Normative definition

stalled

event / HTMLMediaElement

Normative definition

loadedmetadata

event / HTMLMediaElement

Normative definition

loadeddata

event / HTMLMediaElement

Normative definition

canplay

event / HTMLMediaElement

Normative definition

canplaythrough

event / HTMLMediaElement

Normative definition

playing

event / HTMLMediaElement

Normative definition

waiting

event / HTMLMediaElement

Normative definition

seeking

event / HTMLMediaElement

Normative definition

seeked

event / HTMLMediaElement

Normative definition

ended

event / HTMLMediaElement

Normative definition

durationchange

event / HTMLMediaElement

Normative definition

timeupdate

event / HTMLMediaElement

Normative definition

play

event / HTMLMediaElement

Normative definition

pause

event / HTMLMediaElement

Normative definition

ratechange

event / HTMLMediaElement

Normative definition

resize

event / HTMLMediaElement

Normative definition

volumechange

event / HTMLMediaElement

Normative definition

error

event / HTMLSourceElement

Normative definition

change

event / AudioTrackList, VideoTrackList, TextTrackList

Normative definition

addtrack

event / AudioTrackList, VideoTrackList, TextTrackList

Normative definition

removetrack

event / AudioTrackList, VideoTrackList, TextTrackList

Normative definition

cuechange

event / TextTrack, HTMLTrackElement

Normative definition

error

event / HTMLTrackElement

Normative definition

load

event / HTMLTrackElement

Normative definition

enter

event / TextTrackCue

Normative definition

exit

event / TextTrackCue

Normative definition

circle state

attr-value / area/shape

Normative definition

default state

attr-value / area/shape

Normative definition

polygon state

attr-value / area/shape

Normative definition

rectangle state

attr-value / area/shape

Normative definition

row

attr-value / th/scope

Normative definition

col

attr-value / th/scope

Normative definition

rowgroup

attr-value / th/scope

Normative definition

colgroup

attr-value / th/scope

Normative definition

on

attr-value / form/autocomplete

Normative definition

off

attr-value / form/autocomplete

Normative definition

hidden

attr-value / input/type

Normative definition

text

attr-value / input/type

Normative definition

search

attr-value / input/type

Normative definition

tel

attr-value / input/type

Normative definition

url

attr-value / input/type

Normative definition

email

attr-value / input/type

Normative definition

password

attr-value / input/type

Normative definition

date

attr-value / input/type

Normative definition

month

attr-value / input/type

Normative definition

week

attr-value / input/type

Normative definition

time

attr-value / input/type

Normative definition

datetime-local

attr-value / input/type

Normative definition

number

attr-value / input/type

Normative definition

range

attr-value / input/type

Normative definition

color

attr-value / input/type

Normative definition

checkbox

attr-value / input/type

Normative definition

radio

attr-value / input/type

Normative definition

file

attr-value / input/type

Normative definition

submit

attr-value / input/type

Normative definition

image

attr-value / input/type

Normative definition

reset

attr-value / input/type

Normative definition

button

attr-value / input/type

Normative definition

Hidden

element-state / input

Normative definition

Text

element-state / input

Normative definition

Search

element-state / input

Normative definition

Telephone

element-state / input

Normative definition

URL

element-state / input

Normative definition

Email

element-state / input

Normative definition

Password

element-state / input

Normative definition

Date

element-state / input

Normative definition

Month

element-state / input

Normative definition

Week

element-state / input

Normative definition

Time

element-state / input

Normative definition

Local Date and Time

element-state / input

Normative definition

Number

element-state / input

Normative definition

Range

element-state / input

Normative definition

Color

element-state / input

Normative definition

limited-srgb

attr-value / input/colorspace

Normative definition

display-p3

attr-value / input/colorspace

Normative definition

Checkbox

element-state / input

Normative definition

Radio Button

element-state / input

Normative definition

File Upload

element-state / input

Normative definition

Submit Button

element-state / input

Normative definition

Image Button

element-state / input

Normative definition

Reset Button

element-state / input

Normative definition

Button

element-state / input

Normative definition

submit

attr-value / button/type

Normative definition

reset

attr-value / button/type

Normative definition

button

attr-value / button/type

Normative definition

toggle-popover

attr-value / html-global/command

Normative definition

show-popover

attr-value / html-global/command

Normative definition

hide-popover

attr-value / html-global/command

Normative definition

close

attr-value / html-global/command

Normative definition

request-close

attr-value / html-global/command

Normative definition

show-modal

attr-value / html-global/command

Normative definition

soft

attr-value / textarea/wrap

Normative definition

hard

attr-value / textarea/wrap

Normative definition

get

attr-value / form/method, button/formmethod, input/formmethod

Normative definition

post

attr-value / form/method, button/formmethod, input/formmethod

Normative definition

dialog

attr-value / form/method, button/formmethod, input/formmethod

Normative definition

application/x-www-form-urlencoded

attr-value / form/enctype, button/enctype

Normative definition

multipart/form-data

attr-value / form/enctype, button/enctype

Normative definition

text/plain

attr-value / form/enctype, button/enctype

Normative definition

section-

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

shipping

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

billing

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

home

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

work

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

mobile

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

fax

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

pager

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

off

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

on

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

name

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

honorific-prefix

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

given-name

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

additional-name

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

family-name

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

honorific-suffix

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

nickname

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

organization-title

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

username

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

new-password

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

current-password

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

one-time-code

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

organization

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

street-address

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

address-linel

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

address-line2

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

address-line3

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

address-level4

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

address-level3

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

address-level2

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

address-levell

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

country

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

country-name

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

postal-code

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

cc-name

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

cc-given-name

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

cc-additional-name

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

cc-family-name

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

cc-number

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

cc-exp

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

cc-exp-month

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

cc-exp-year

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

cc-csc

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

cc-type

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

transaction-currency

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

transaction-amount

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

language

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

bday

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

bday-day

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

bday-month

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

bday-year

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

sex

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

url

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

photo

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

tel

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

tel-country-code

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

tel-national

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

tel-area-code

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

tel-local

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

tel-local-prefix

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

tel-local-suffix

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

tel-extension

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

email

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

impp

attr-value / button/autocomplete, fieldset/autocomplete, input/autocomplete, object/autocomplete, output/autocomplete, select/autocomplete, textarea/autocomplete

Normative definition

select

enum-value / SelectionMode

Normative definition

start

enum-value / SelectionMode

Normative definition

end

enum-value / SelectionMode

Normative definition

preserve

enum-value / SelectionMode

Normative definition

any

attr-value / dialog/closedby

Normative definition

closerequest

attr-value / dialog/closedby

Normative definition

none

attr-value / dialog/closedby

Normative definition

open

attr-value / template/shadowrootmode

Normative definition

closed

attr-value / template/shadowrootmode

Normative definition

named

attr-value / template/shadowrootslotassignment

Normative definition

manual

attr-value / template/shadowrootslotassignment

Normative definition

unorm8

enum-value / CanvasColorType

Normative definition

floatl6

enum-value / CanvasColorType

Normative definition

nonzero

enum-value / CanvasFillRule

Normative definition

evenodd

enum-value / CanvasFillRule

Normative definition

low

enum-value / ImageSmoothingQuality

Normative definition

medium

enum-value / ImageSmoothingQuality

Normative definition

high

enum-value / ImageSmoothingQuality

Normative definition

start

enum-value / CanvasTextAlign

Normative definition

end

enum-value / CanvasTextAlign

Normative definition

left

enum-value / CanvasTextAlign

Normative definition

right

enum-value / CanvasTextAlign

Normative definition

center

enum-value / CanvasTextAlign

Normative definition

top

enum-value / CanvasTextBaseline

Normative definition

hanging

enum-value / CanvasTextBaseline

Normative definition

middle

enum-value / CanvasTextBaseline

Normative definition

alphabetic

enum-value / CanvasTextBaseline

Normative definition

ideographic

enum-value / CanvasTextBaseline

Normative definition

bottom

enum-value / CanvasTextBaseline

Normative definition

ltr

enum-value / CanvasDirection

Normative definition

rtl

enum-value / CanvasDirection

Normative definition

inherit

enum-value / CanvasDirection

Normative definition

auto

enum-value / CanvasFontKerning

Normative definition

normal

enum-value / CanvasFontKerning

Normative definition

none

enum-value / CanvasFontKerning

Normative definition

ultra-condensed

enum-value / CanvasFontStretch

Normative definition

extra-condensed

enum-value / CanvasFontStretch

Normative definition

condensed

enum-value / CanvasFontStretch

Normative definition

semi-condensed

enum-value / CanvasFontStretch

Normative definition

normal

enum-value / CanvasFontStretch

Normative definition

semi-expanded

enum-value / CanvasFontStretch

Normative definition

expanded

enum-value / CanvasFontStretch

Normative definition

extra-expanded

enum-value / CanvasFontStretch

Normative definition

ultra-expanded

enum-value / CanvasFontStretch

Normative definition

normal

enum-value / CanvasFontVariantCaps

Normative definition

small-caps

enum-value / CanvasFontVariantCaps

Normative definition

all-small-caps

enum-value / CanvasFontVariantCaps

Normative definition

petite-caps

enum-value / CanvasFontVariantCaps

Normative definition

all-petite-caps

enum-value / CanvasFontVariantCaps

Normative definition

unicase

enum-value / CanvasFontVariantCaps

Normative definition

titling-caps

enum-value / CanvasFontVariantCaps

Normative definition

auto

enum-value / CanvasTextRendering

Normative definition

optimizeSpeed

enum-value / CanvasTextRendering

Normative definition

optimizeLegibility

enum-value / CanvasTextRendering

Normative definition

geometricPrecision

enum-value / CanvasTextRendering

Normative definition

srgb

enum-value / PredefinedColorSpace

Normative definition

srgb-linear

enum-value / PredefinedColorSpace

Normative definition

display-p3

enum-value / PredefinedColorSpace

Normative definition

display-p3-linear

enum-value / PredefinedColorSpace

Normative definition

hidden

attr-value / html-global/hidden

Normative definition

until-found

attr-value / html-global/hidden

Normative definition

true

attr-value / html-global/contenteditable

Normative definition

false

attr-value / html-global/contenteditable

Normative definition

plaintext-only

attr-value / html-global/contenteditable

Normative definition

true

attr-value / html-global/spellcheck

Normative definition

false

attr-value / html-global/spellcheck

Normative definition

true

attr-value / html-global/writingsuggestions

Normative definition

false

attr-value / html-global/writingsuggestions

Normative definition

off

attr-value / html-global/autocapitalize

Normative definition

none

attr-value / html-global/autocapitalize

Normative definition

on

attr-value / html-global/autocapitalize

Normative definition

sentences

attr-value / html-global/autocapitalize

Normative definition

words

attr-value / html-global/autocapitalize

Normative definition

characters

attr-value / html-global/autocapitalize

Normative definition

on

attr-value / html-global/autocorrect

Normative definition

off

attr-value / html-global/autocorrect

Normative definition

none

attr-value / html-global/inputmode

Normative definition

text

attr-value / html-global/inputmode

Normative definition

tel

attr-value / html-global/inputmode

Normative definition

url

attr-value / html-global/inputmode

Normative definition

email

attr-value / html-global/inputmode

Normative definition

numeric

attr-value / html-global/inputmode

Normative definition

decimal

attr-value / html-global/inputmode

Normative definition

search

attr-value / html-global/inputmode

Normative definition

enter

attr-value / html-global/enterkeyhint

Normative definition

done

attr-value / html-global/enterkeyhint

Normative definition

go

attr-value / html-global/enterkeyhint

Normative definition

next

attr-value / html-global/enterkeyhint

Normative definition

previous

attr-value / html-global/enterkeyhint

Normative definition

search

attr-value / html-global/enterkeyhint

Normative definition

send

attr-value / html-global/enterkeyhint

Normative definition

dragstart

event / GlobalEventHandlers, Text

Normative definition

drag

event / GlobalEventHandlers, Text

Normative definition

dragenter

event / GlobalEventHandlers

Normative definition

dragleave

event / GlobalEventHandlers

Normative definition

dragover

event / GlobalEventHandlers

Normative definition

drop

event / GlobalEventHandlers

Normative definition

dragend

event / GlobalEventHandlers, Text

Normative definition

true

attr-value / html-global/draggable

Normative definition

false

attr-value / html-global/draggable

Normative definition

auto

attr-value / html-global/popover

Normative definition

manual

attr-value / html-global/popover

Normative definition

hint

attr-value / html-global/popover

Normative definition

toggle

attr-value / html-global/popovertargetaction

Normative definition

show

attr-value / html-global/popovertargetaction

Normative definition

hide

attr-value / html-global/popovertargetaction

Normative definition

allow-popups

attr-value / iframe/sandbox

Normative definition

allow-top-navigation

attr-value / iframe/sandbox

Normative definition

allow-top-navigation-by-user-activation

attr-value / iframe/sandbox

Normative definition

allow-same-origin

attr-value / iframe/sandbox

Normative definition

allow-forms

attr-value / iframe/sandbox

Normative definition

allow-pointer-lock

attr-value / iframe/sandbox

Normative definition

allow-scripts

attr-value / iframe/sandbox

Normative definition

allow-popups-to-escape-sandbox

attr-value / iframe/sandbox

Normative definition

allow-modals

attr-value / iframe/sandbox

Normative definition

allow-orientation-lock

attr-value / iframe/sandbox

Normative definition

allow-presentation

attr-value / iframe/sandbox

Normative definition

allow-downloads

attr-value / iframe/sandbox

Normative definition

allow-top-navigation-to-custom-protocols

attr-value / iframe/sandbox

Normative definition

push

enum-value / NavigationType

Normative definition

replace

enum-value / NavigationType

Normative definition

reload

enum-value / NavigationType

Normative definition

traverse

enum-value / NavigationType

Normative definition

after-transition

enum-value / NavigationFocusReset

Normative definition

manual

enum-value / NavigationFocusReset

Normative definition

after-transition

enum-value / NavigationScrollBehavior

Normative definition

manual

enum-value / NavigationScrollBehavior

Normative definition

auto

enum-value / ScrollRestoration

Normative definition

manual

enum-value / ScrollRestoration

Normative definition

push

enum-value / NavigationHistoryBehavior

Normative definition

replace

enum-value / NavigationHistoryBehavior

Normative definition

auto

enum-value / NavigationHistoryBehavior

Normative definition

text/html

enum-value / DOMParserSupportedType

Normative definition

default

enum-value / SanitizerPresets

Normative definition

rgba-unorm8

enum-value / ImageDataPixelFormat

Normative definition

rgba-float16

enum-value / ImageDataPixelFormat

Normative definition

flipY

enum-value / ImageOrientation

Normative definition

from-image

enum-value / ImageOrientation

Normative definition

default

enum-value / ColorSpaceConversion

Normative definition

none

enum-value / ColorSpaceConversion

Normative definition

default

enum-value / PremultiplyAlpha

Normative definition

premultiply

enum-value / PremultiplyAlpha

Normative definition

none

enum-value / PremultiplyAlpha

Normative definition

pixelated

enum-value / ResizeQuality

Normative definition

low

enum-value / ResizeQuality

Normative definition

medium

enum-value / ResizeQuality

Normative definition

high

enum-value / ResizeQuality

Normative definition

DOMContentLoaded

event / Document

Normative definition

afterprint

event / Window

Normative definition

beforeprint

event / Window

Normative definition

beforematch

event / HTMLElement

Normative definition

beforetoggle

event / HTMLElement

Normative definition

beforeunload

event / Window

Normative definition

blur

event / Window, HTMLElement

Normative definition

cancel

event / CloseWatcher, HTMLElement

Normative definition

change

event / HTMLElement

Normative definition

close

event / CloseWatcher, HTMLElement

Normative definition

command

event / HTMLElement

Normative definition

connect

event / SharedWorkerGlobalScope

Normative definition

contextlost

event / HTMLElement, OffscreenCanvas

Normative definition

contextrestored

event / HTMLElement, OffscreenCanvas

Normative definition

currententrychange

event / Navigation

Normative definition

dispose

event / NavigationHistoryEntry

Normative definition

error

event / Window, WorkerGlobalScope, HTMLElement, EventSource, Worker, SharedWorker, AbstractWorker

Normative definition

focus

event / Window, HTMLElement

Normative definition

formdata

event / HTMLElement

Normative definition

hashchange

event / Window

Normative definition

invalid

event / HTMLElement

Normative definition

languagechange

event / Window, WorkerGlobalScope

Normative definition

load

event / Window, HTMLElement

Normative definition

message

event / Window, EventSource, MessagePort, BroadcastChannel, DedicatedWorkerGlobalScope, Worker, ServiceWorkerContainer

Normative definition

messageerror

event / Window, MessagePort, BroadcastChannel, DedicatedWorkerGlobalScope, Worker, ServiceWorkerContainer

Normative definition

navigate

event / Navigation

Normative definition

navigateerror

event / Navigation

Normative definition

navigatesuccess

event / Navigation

Normative definition

offline

event / Window, WorkerGlobalScope

Normative definition

online

event / Window, WorkerGlobalScope

Normative definition

open

event / EventSource

Normative definition

pageswap

event / Window

Normative definition

pagehide

event / Window

Normative definition

pagereveal

event / Window

Normative definition

pageshow

event / Window

Normative definition

popstate

event / Window

Normative definition

readystatechange

event / Document

Normative definition

rejectionhandled

event / Window, WorkerGlobalScope

Normative definition

reset

event / HTMLElement

Normative definition

select

event / HTMLElement

Normative definition

storage

event / Window

Normative definition

submit

event / HTMLElement

Normative definition

toggle

event / HTMLElement

Normative definition

unhandledrejection

event / Window, WorkerGlobalScope

Normative definition

unload

event / Window

Normative definition

visibilitychange

event / Document

Normative definition

Glossary

Accessible name

The text alternative used to identify an interface element in accessibility APIs, computed from the relevant naming rules and relationships.

Attribute

A name and value associated with an element. Its authoring syntax and behavior depend on its definition and context.

Boolean attribute

An attribute whose defined true state is represented by presence and false state by absence. The string false does not turn it off.

Browsing context

A context in which a document can be presented, such as a top-level page or an embedded iframe, with further terminology defined by the Standard.

Character reference

HTML source syntax that represents a character, such as < for a less-than sign in a relevant parsing context.

Conformance

Satisfaction of the applicable requirements of a specification. A checker result has a version and scope; it is not a general accessibility or security certificate.

Content model

The defined rules for an element's contents, including categories and additional restrictions.

Content category

A classification used by the Standard to describe elements and content rules. Categories can overlap or have conditional membership.

Content attribute

An attribute represented in the markup/DOM attribute layer, distinguished from a corresponding IDL property and current control state.

Custom element

An element defined through the custom-elements machinery. A valid autonomous custom-element name has a required hyphen and other naming restrictions.

Descendant

A node reached by following child relationships one or more levels down the tree. It need not be a direct child.

Document element

The element child of a document; for an ordinary HTML document it is html.

DOM

The document object model: nodes, relationships and APIs through which software can inspect and modify documents.

Enumerated attribute

An attribute defined in terms of keywords and states, potentially with distinct missing-value and invalid-value defaults.

Fallback

An alternative path or representation designed for the actual failure or unsupported case. Its presence does not prove that users can reach or use it.

Flow content

A content category covering much of the material used in a document body. Membership still depends on the individual element definition.

Focus

The platform's designation of an interaction target for relevant input. Focusable and sequentially focusable are related but distinct concepts.

Fragment

The part of a URL introduced by #, commonly used to navigate to a matching id within a document.

Global attribute

An attribute applicable across HTML elements under its defined rules. This does not remove contextual constraints or cross-specification ownership.

Hyperlink

A link to another resource or location, commonly created by an a element with href.

IDL property

A property exposed through a web interface. Reflection can parse or normalize an attribute, and some properties represent live state rather than raw markup.

Inert

A state that suppresses certain interaction and focus behavior and affects accessibility exposure under the relevant platform rules.

Landmark

An accessibility role identifying a useful region for navigation, such as main content or a named navigation area.

Living Standard

A specification maintained over time rather than a single frozen feature release. Its current presence and browser support are separate questions.

Metadata

Information about the document or its relationships, commonly placed in head using elements such as title, meta and link.

Namespace

A mechanism distinguishing vocabularies and names, important at HTML, SVG, MathML and XML integration boundaries.

Origin

A security-relevant identity derived under platform rules, ordinarily involving scheme, host and port for a conventional URL. Opaque origins and special cases exist.

Phrasing content

A category used for text and markup that participates at the phrasing level of a document. Extra element-specific restrictions still apply.

Progressive enhancement

Designing an appropriate useful baseline and adding capabilities without needlessly destroying that baseline when an enhancement fails.

Reflection

The defined relationship between a content attribute and an interface property. It is not a universal raw-string equivalence.

Sandbox

A set of restrictions for an embedded browsing context. Added permission tokens restore capabilities and must be reviewed for the actual trust boundary.

Serialization

A textual representation of a document or fragment. HTML and XML serialization have different rules and processing consequences.

Shadow tree

A DOM tree attached through a shadow root, used for encapsulation and composition. It is not a general security sandbox.

Slot

A placeholder for assigned content in a shadow tree. A slot element and a slot attribute have different jobs.

Text alternative

A textual replacement or representation suited to non-text content's actual purpose and context.

Tokenization

The parsing stage that recognizes tokens according to the tokenizer's state before tree-construction rules process them.

Transparent content model

A model that derives permitted content from the surrounding context while preserving the element's own additional restrictions.

Tree construction

The parsing rules that turn tokens into nodes and relationships, including specified recovery behavior.

Void element

An HTML element that has no contents and no end tag in HTML syntax, such as img or input.

WebVTT

A text format commonly used for timed text tracks. A valid file still needs content review against the actual media.

Source and release notes

Reference summaries derive from the WHATWG HTML source snapshot dated 14 October 2026. See https://whatwg.org/ipr-policy#copyright-license-grant for source terms. MDN prose and Browser Compatibility Data have separate roles and licenses. Instrument Serif is included under the SIL Open Font License. The supplied source and font notices retain upstream terms. The original editorial copyright notice does not override upstream rights.

indices

dom

obsolete

contents

input

mdn-elements

mdn-global-attributes

bcd-selectedcontent

bcd-global-attributes

bcd-button

bcd-geolocation

form-elements

sections

Still open

Independent chapter and answer review; expanded API behavior, cross-specification reconciliation and complete value/default inventories; assistive-technology review; full security review; tagged-PDF accessibility workflow. Executed browser and deployment-package checks are recorded separately. The public site is https://ctrobook.vercel.app/. The author confirmed successful delivery on 14 October 2026. Later repository updates require their own deployment; hosting success is not content approval.